

Servizi digitali inclusi

Questo volume (VOL-08) è collegato ai servizi digitali Capitale Personale: aggiornamenti operativi, schede compilabili, planner, Bando Decoder e materiali di lavoro collegati ai moduli interni.

COSA AGGIUNGE IL DIGITALE

Servizio	Funzione
Bando Decoder	Trasforma il bando in piano di studio e priorità.
Planner	Organizza tempi, prove, ripassi e simulazioni.
Servizio	Funzione
Schede modulo	Adatta il Metodo BANDO alla famiglia concorsuale del volume.
Aggiornamenti	Aiuta a controllare sulle fonti ufficiali le novità rilevanti per il bando e per la prova.

BANDO IN PRATICA

Il volume resta utilizzabile anche senza piattaforma: il digitale accelera compilazione, verifica e aggiornamento, ma non sostituisce il libro.

VOL-08

ICT, DIGITALE, CYBERSECURITY E DATI

Modulo verticale tecnico per profili digitali non
coperti dal base.

CAPITALE PERSONALE

Volume operativo Metodo BANDO per Profili ICT,
data, cyber e trasformazione digitale nella PA.

Copyright e note editoriali

Questo volume appartiene alla linea ConcorsoBook OS / Capitale Personale ed è costruito come libro-workbook per la preparazione ai concorsi pubblici italiani.

Per la tua candidatura controlla sempre il bando, gli allegati e gli avvisi ufficiali della procedura. Per l'uso professionale verifica le disposizioni vigenti e le regole applicabili al caso concreto.

Il volume non promette copertura totale di ogni bando, né aggiornamento automatico: offre un metodo riusabile, una struttura modulare e strumenti di lavoro collegati al perimetro editoriale dichiarato.

Perimetro volume: ICT, digitale, cybersecurity e dati.

Sommario

Modulo verticale tecnico per profili digitali non coperti dal base.

Modulo	Percorso interno	Stato
M-TR01	ICT e trasformazione digitale	13 sezioni

Verticali del volume

- Cybersecurity
- Dati e interoperabilità
- Servizi digitali PA

Premessa

Questo volume accompagna la preparazione per ict, digitale, cybersecurity e dati. I percorsi M-TR01 collegano le materie specialistiche alle prove e agli strumenti di lavoro.

Parti dal bando: individua requisiti, materie, prove e criteri di valutazione. Usa l'indice per scegliere il percorso pertinente e le schede per trasformare lo studio in esercizi, ripassi e simulazioni.

Il volume specialistico si affianca al Metodo BANDO per le basi comuni. I rinvii indicano quali argomenti riprendere; gli strumenti cartacei permettono di lavorare anche senza piattaforma digitale.

Indice completo

M-TR01

ICT e trasformazione digitale

Lavorare come ICT nella PA: ruoli, enti e prove

Cap. 1		10
1.1	Il profilo ICT pubblico	10
1.2	Le famiglie di profilo	12
1.3	Enti e contesti di lavoro	14
1.4	La Mappa BANDO del profilo ICT	16
1.5	Requisiti, materie e prove	18
1.6	Il Bando Decoder ICT	20

Informatica specialistica: cosa serve oltre il VOL-01

Cap. 2		26
2.1	L'elaboratore come sistema	27
2.2	Rappresentare dati e istruzioni	29
2.3	La gerarchia di memoria	31
2.4	Input/output e gestione degli eventi	33
2.5	Capire le prestazioni	34
2.6	Il sistema operativo come gestore delle risorse	36

Cap. 3	Programmazione, algoritmi e strutture dati	42
3.1	Problema, algoritmo e correttezza	43
3.2	Paradigmi e flusso di controllo	45
3.3	Funzioni e procedure	47
3.4	Strutture lineari e associative	49
3.5	Alberi, grafi, ricerca e ordinamento	51
3.6	Complessità e output concorsuale	53

Cap. 4	Basi dati, SQL/NoSQL e qualità del dato	60
4.1	Dai requisiti al modello relazionale	61
4.2	Vincoli, NULL e normalizzazione	64
4.3	SQL per interrogare e modificare	65
4.4	Transazioni, concorrenza e indici	68
4.5	Famiglie NoSQL e criteri di scelta	71
4.6	Qualità, accessi e output concorsuale	73

Cap. 5	Reti, sistemi operativi e infrastrutture	79
5.1	Livelli, incapsulamento e rete locale	80
5.2	Indirizzamento, subnetting e routing	82
5.3	Trasporto e servizi di rete	83
5.4	Sistema operativo, processi e servizi	85
5.5	Memoria virtuale, file system e risorse	87
5.6	Disponibilità e troubleshooting infrastrutturale	89

Ingegneria software, API e interoperabilità

Cap. 6	PA	96
6.1	Requisiti e ciclo di vita verificabile	97
6.2	Architettura e qualità del software	99

CAPITALE PERSONALE

6.3	Verifica, validazione, test e configurazione	101
6.4	API come contratto	103
6.5	Evoluzione e gestione delle API	106
6.6	Interoperabilità PA ed e-service	108
Cap. 7	Cloud PA, virtualizzazione, container e DevOps	115
7.1	Quadro, modelli cloud e responsabilità	116
7.2	Virtualizzazione, container e orchestrazione	118
7.3	Cloud PA e percorso di migrazione	120
7.4	DevOps, CI/CD e Infrastructure as Code	123
7.5	Operabilità, osservabilità, capacità e costi	125
7.6	Resilienza, backup, disaster recovery e continuità operativa	126
Cap. 8	Cybersecurity operativa: rischio, controlli e vulnerabilità	133
8.1	Obiettivi di sicurezza e asset	134
8.2	Minacce, vulnerabilità e scenari	136
8.3	Valutazione e trattamento del rischio	138
8.4	Controlli e difesa per livelli	140
8.5	Threat modeling e gestione delle vulnerabilità	142
8.6	Secure SDLC e software supply chain	144
Cap. 9	IAM, crittografia, logging e incident response	151
9.1	Identità, account e ciclo di vita	151
9.2	Autenticazione, autorizzazione e privilegi	153
9.3	Crittografia e gestione delle chiavi	154
9.4	Logging, rilevazione e triage	157
9.5	Incident response e continuità del servizio	158
9.6	PA, ACN, CSIRT, NIS2 e privacy	160
Cap. 10	Data governance, open data, interoperabilità e qualità	167
10.1	Il dato come asset informativo e le decisioni di governo	167
10.2	Ruoli, decisioni, RACI ed evidenze	169
10.3	Ciclo di vita, classificazione e lineage	171
10.4	Inventario, catalogo, glossario e metadati	172
10.5	Qualità del dato: regole, metriche, anomalie e rimedio	174
10.6	Open data, riuso, distribuzioni e limiti	176
10.7	Interoperabilità, privacy, sicurezza e responsabilizzazione	177
Cap. 11	AI/ML nella PA: modelli, rischi e compliance	183
11.1	Algoritmo, sistema AI, modello e inferenza	183
11.2	Paradigmi, partizioni e generalizzazione	185
11.3	Dati, provenienza, etichette e rappresentatività	187
11.4	Baseline, metriche e conseguenze degli errori	188
11.5	Bias, fairness, spiegabilità e controllo umano	190

CAPITALE PERSONALE

11.6	Lifecycle, MLOps, rischio e governance	191
11.7	Quadro UE e italiano: ruoli, rischio e confini	194
Cap. 12	Procurement ICT e gestione dei fornitori	199
12.1	Fabbisogno, baseline e strategia di acquisizione	200
12.2	Requisiti verificabili e capitolato tecnico	203
12.3	SLA, SLI, KPI e misurazione del servizio	206
12.4	Ruoli, evidenze e governo dell'esecuzione	208
12.5	Sicurezza, dati e supply chain contrattuale	210
12.6	Modifiche, rilasci e continuità del servizio	212
12.7	Lock-in, portabilità, reversibilità ed exit strategy	214
Cap. 13	Laboratorio prove ICT: quiz, scritto tecnico, orale e casi	222
	Mappa BANDO: decodifica della prova e della consegna ICT	222
13.1		222
13.2	Quiz tecnico: decisione, distrattori e correzione	225
13.3	Risposta breve e domanda aperta	227
13.4	Elaborato tecnico: progetto, rischi ed evidenze	229
13.5	Prova orale ICT: struttura e controllo	231
13.6	Caso tecnico: diagnosi, decisione e verifica	233
13.7	Simulazione, autocorrezione e diario degli errori	236

M-TR01

ICT e trasformazione digitale

Sezione interna del volume VOL-08 — ICT, digitale, cybersecurity e dati.

Blocco	Sommario del modulo
Avvio	1 Lavorare come ICT nella PA: ruoli, enti e prove; 2 Informatica specialistica: cosa serve oltre il VOL-01; 3 Programmazione, algoritmi e strutture dati; 4 Basi dati, SQL/NoSQL e qualità del dato
Sviluppo	5 Reti, sistemi operativi e infrastrutture; 6 Ingegneria software, API e interoperabilità PA; 7 Cloud PA, virtualizzazione, container e DevOps; 8 Cybersecurity operativa: rischio, controlli e vulnerabilità
Allenamento	9 IAM, crittografia, logging e incident response; 10 Data governance, open data, interoperabilità e qualità; 11 AI/ML nella PA: modelli, rischi e compliance; 12 Procurement ICT e gestione dei fornitori
Strumenti	13 Laboratorio prove ICT: quiz, scritto tecnico, orale e casi

Lavorare come ICT nella PA: ruoli, enti e prove

B
BANDO**A**
AREE**N**
NUCLEI**D**
DIARIO**O**
OUTPUT

Apertura editoriale

Un bando che contiene la parola “informatico” non identifica automaticamente un unico mestiere. Dietro denominazioni simili possono trovarsi sviluppo applicativo, gestione delle infrastrutture, sicurezza, governo dei dati, continuità dei servizi o controllo dei fornitori. Il primo compito del candidato è capire quale lavoro descrive davvero il bando.

Il campione ufficiale consolidato per questo volume comprende enti locali, università, un ente regionale e un’agenzia digitale. Mostra profili generalisti, specialisti ICT, cybersecurity e alta professionalità tecnico-informatica. Non permette di calcolare frequenze statistiche sull’intero mercato; dimostra però che titolo del profilo, attività, materie e prove devono essere letti insieme.

Obiettivo del capitolo

Al termine saprai:

- distinguere le principali famiglie operative dei profili ICT;
- riconoscere gli elementi che definiscono il contesto di lavoro;
- separare requisiti di accesso, materie d’esame e abilità valutate;
- trasformare il programma in una mappa di studio;
- associare ogni nucleo all’output richiesto dalla prova.

1.1 Il profilo ICT pubblico

Un profilo ICT pubblico è una posizione professionale nella quale competenze tecnologiche, responsabilità organizzative e finalità dell’amministrazione si combinano per produrre o presidiare servizi. La tecnologia permette all’ente di gestire dati, procedimenti, comunicazioni, infrastrutture e servizi rivolti a utenti interni o alla collettività.

La funzione del profilo è assicurare che sistemi e servizi siano adeguati allo scopo, utilizzabili, governabili e protetti. Gli elementi che lo definiscono sono cinque:

1. oggetto del lavoro, come applicazioni, dati, reti o servizi cloud;
2. attività, come analisi, progettazione, gestione, controllo o assistenza;
3. contesto, cioè ente, ufficio e utenti serviti;
4. livello di responsabilità, operativo, specialistico, progettuale o di coordinamento;
5. output atteso, come servizio funzionante, progetto, controllo, diagnosi o decisione tecnica.

Per questo due bandi con la stessa denominazione possono richiedere piani di studio differenti.

LEGGERE IL PROFILO COME UN SISTEMA

I cinque elementi non vanno osservati separatamente. L’oggetto dice su che cosa si lavora; l’attività indica che cosa si fa; il contesto chiarisce per chi e con quali vincoli; la responsabilità stabilisce fino a dove arriva la decisione; l’output rende verificabile il risultato. Una descrizione che cita “gestione di applicativi” resta incompleta finché non si capisce se il candidato dovrà assistere utenti, analizzare requisiti, controllare un fornitore, progettare integrazioni oppure assumere decisioni sul servizio.

Questa lettura evita di confondere la competenza con il prodotto commerciale. Conoscere un ambiente tecnologico può essere utile, ma il concorso valuta soprattutto la capacità di spiegare principi, scegliere in base ai requisiti e motivare controlli. Se il bando nomina un prodotto, occorre verificare quanto quella indicazione pesi davvero nel programma; non si può trasformare ogni marchio citato in un corso autonomo.

La responsabilità merita un controllo specifico. Un profilo operativo applica procedure, esegue controlli e segnala anomalie. Un funzionario può analizzare esigenze, coordinare attività, verificare risultati e proporre decisioni. Un’elevata professionalità può richiedere maggiore autonomia progettuale, esperienza e capacità di governo. Queste sono chiavi di lettura, non equivalenze automatiche: il testo del singolo bando resta decisivo.

CONSEGUENZE SUL PIANO DI STUDIO

Dal profilo derivano profondità e forma dell’allenamento. Chi presidia reti e sistemi deve saper diagnosticare un disservizio e distinguere le cause possibili; chi governa dati deve valutare qualità, accessi e riuso; chi segue fornitori deve trasformare esigenze in requisiti ed evidenze; chi opera nella sicurezza deve collegare rischio, controllo e risposta. Lo stesso concetto può quindi essere studiato con un livello diverso.

Usa questa griglia prima di aprire il manuale:

Elemento	Domanda di controllo	Effetto sullo studio
Oggetto	applicazioni, dati, infrastrutture o servizi?	individua i capitoli tecnici
Attività	analisi, sviluppo, gestione, controllo o assistenza?	determina esercizi e profondità
Contesto	quali utenti, procedimenti e servizi sono coinvolti?	costruisce esempi plausibili
Responsabilità	esecuzione, proposta, coordinamento o decisione?	regola il livello della risposta

Elemento	Domanda di controllo	Effetto sullo studio
Output	quiz, diagnosi, progetto, controllo o esposizione?	seleziona la forma di allenamento

Come lo chiede la commissione. Può presentare una breve descrizione di attività e chiedere quale competenza sia centrale, oppure domandare come un funzionario traduce un bisogno dell'ufficio in un servizio verificabile. Una risposta solida non elenca tecnologie: identifica oggetto, attività, interlocutori, vincoli, responsabilità e risultato atteso.

Errore da evitare. Trattare “informatico” come sinonimo di programmatore. Lo sviluppo è una possibile attività; molti profili riguardano anche infrastrutture, dati, sicurezza, interoperabilità, assistenza, continuità e controllo dei fornitori.

In prova, chiudi il ragionamento con un'evidenza: requisito verificato, servizio disponibile, anomalia diagnosticata, rischio ridotto o decisione motivata. L'evidenza trasforma una descrizione generica del ruolo in una risposta professionale.

Una buona sintesi può seguire questa sequenza: finalità del servizio, oggetto tecnico, attività affidate, responsabilità esercitata e risultato controllabile. La sequenza resta valida anche quando il bando usa denominazioni poco uniformi.

1.2 Le famiglie di profilo

FUNZIONARIO ICT GENERALISTA

Il funzionario generalista collega più componenti del sistema informativo. Può analizzare esigenze, seguire applicazioni e infrastrutture, coordinare interventi, controllare forniture e dialogare con uffici non tecnici.

Il candidato deve quindi saper spiegare relazioni: come un requisito diventa servizio; come il servizio usa dati e infrastrutture; come viene testato, monitorato e protetto. La prova può richiedere ampiezza più che specializzazione estrema.

SPECIALISTA RETI, SISTEMI, CLOUD E DEVOPS

Questa famiglia presidia disponibilità, configurazione, automazione, osservabilità, backup e continuità. Memorizzare marchi commerciali serve poco. Occorre comprendere principi e conseguenze delle scelte: isolamento, scalabilità, ridondanza, aggiornamento, ripristino e controllo delle configurazioni.

Un caso tecnico può chiedere di diagnosticare un disservizio o proporre un'architettura coerente con requisiti di continuità e sicurezza.

CYBERSECURITY SPECIALIST

Qui il lavoro parte dal rischio: quali risorse proteggere, da quali minacce, con quali controlli e come reagire a un incidente. Il profilo cyber non coincide con la sicurezza informatica elementare del volume base.

La prova richiede spesso di collegare un evento a una misura concreta: autenticazione, segregazione dei privilegi, logging, gestione delle vulnerabilità, protezione dei dati, rilevazione e risposta. L'Università di Firenze, nel campione ufficiale, presenta un profilo dedicato alla cybersecurity inserito nel contesto di sistemi, cloud e continuità operativa: è un esempio della natura trasversale della specializzazione.

DATA E AI SPECIALIST

Il profilo lavora sul ciclo di vita del dato: acquisizione, modellazione, qualità, condivisione, analisi e governo. Quando sono coinvolti sistemi di AI, entrano anche valutazione dei modelli, rischio, bias, spiegabilità e controllo umano.

Capacità tecnica e responsabilità di governo, però, sono piani diversi. Un risultato analitico va valutato per qualità, limiti e uso nel processo amministrativo.

ALTA PROFESSIONALITÀ TECNICO-INFORMATICA

Le procedure per elevate professionalità, come quella AgID inclusa nel campione, segnalano un livello più alto di autonomia e specializzazione. Non costituiscono un semplice “funzionario con più materie”: richiedono lettura attenta delle attività, delle competenze e dei criteri specifici del bando.

FAMIGLIE CHE SI SOVRAPPONGONO SENZA COINCIDERE

Le famiglie orientano il piano senza chiudere ogni profilo in una casella. Un servizio cloud richiede infrastruttura, sicurezza, identità, monitoraggio e controllo contrattuale; un progetto data usa database, integrazione, qualità e protezione; un'applicazione espone API, produce log e dipende dalla continuità operativa. Queste sovrapposizioni sono normali. Bisogna riconoscere la competenza che costituisce il baricentro e separarla dalle dipendenze.

Il generalista deve costruire collegamenti affidabili fra domini. Non gli basta nominare tutti gli argomenti: deve spiegare, per esempio, perché un requisito applicativo genera scelte sui dati, sui test, sugli accessi e sul monitoraggio. Lo specialista di infrastrutture approfondisce disponibilità, configurazione e diagnosi; lo specialista cybersecurity parte dagli asset e dal rischio; il profilo data/AI parte dal ciclo di vita del dato e dalla valutazione dei risultati. L'alta professionalità aggiunge soprattutto autonomia, esperienza e responsabilità progettuale, secondo quanto stabilito dalla procedura.

Famiglia	Domanda centrale	Output tipico da allenare
ICT generalista	come funzionano insieme servizio, dati, infrastruttura e fornitore?	analisi trasversale e priorità
Reti, sistemi, cloud	come garantire operabilità, osservabilità e ripristino?	diagnosi o proposta architetturale

Famiglia	Domanda centrale	Output tipico da allenare
Cybersecurity	quale rischio proteggerò e con quali controlli verificabili?	valutazione del rischio e piano di trattamento
Data e AI	il dato e il modello sono adeguati, governati e controllabili?	schema, analisi di qualità o valutazione
Alta professionalità	quale decisione tecnica e organizzativa regge i vincoli?	proposta motivata e governo dell'esecuzione

COME SCEGLIERE IL BARICENTRO

Segna i verbi presenti nelle attività: sviluppare, amministrare, progettare, monitorare, analizzare, coordinare, verificare. Poi collega ciascun verbo all'oggetto. "Monitorare infrastrutture" spinge verso sistemi e operazioni; "valutare modelli e dati" verso data/AI; "governare servizi digitali" può indicare un profilo trasversale o di elevata professionalità. Infine controlla materie e prove: devono confermare o correggere la prima ipotesi.

Un profilo non diventa cyber perché compare la parola sicurezza in una materia. La sicurezza attraversa quasi ogni servizio ICT. Diventa baricentro quando attività, profondità del programma e output richiedono analisi del rischio, vulnerabilità, identità, logging o risposta. Analogamente, la presenza di "cloud" non elimina reti, sistemi e continuità: ne cambia l'applicazione.

Mini-esercizio. Prendi tre attività del tuo bando. Per ciascuna scrivi verbo, oggetto, responsabilità e prova più coerente. Se le righe convergono sulla stessa famiglia, hai un baricentro plausibile. Se divergono, probabilmente il profilo è generalista o ibrido e richiede un piano per aree, non l'acquisto indiscriminato di più specialistici.

1.3 Enti e contesti di lavoro

Il campione comprende comuni, provincia, università, ente regionale e agenzia digitale. Questa varietà permette di distinguere almeno quattro contesti.

Negli enti locali il profilo può essere vicino al CED e ai servizi trasversali dell'amministrazione. Nelle università può operare in direzioni ICT con infrastrutture, servizi alla comunità accademica e specializzazioni cyber. In enti regionali o agenzie il lavoro può concentrarsi su sistemi di scala più ampia, politiche digitali, interoperabilità o alta specializzazione.

Il nome dell'ente non basta per dedurre le mansioni. Fanno fede attività e programma del singolo bando. Questa regola impedisce due errori: studiare tutto ciò che l'ente fa oppure ridurre la preparazione a ciò che suggerisce il titolo del profilo.

CONTESTO, UTENTI E VINCOLI

Il contesto modifica gli esempi e le priorità. In un ente locale i servizi ICT possono sostenere uffici molto diversi e un rapporto diretto con cittadini e imprese. In un'università gli utenti comprendono personale, studenti, docenti e ricerca, con servizi e infrastrutture eterogenei. In una regione o in un'agenzia digitale possono emergere piattaforme condivise, interoperabilità e coordinamento tra amministrazioni. In un'autorità o in un ente di ricerca possono pesare dati specialistici, autonomia tecnica e requisiti propri del settore.

Queste differenze non autorizzano a inventare mansioni. Servono a comprendere interlocutori, impatto e vincoli. Una stessa indisponibilità applicativa ha conseguenze diverse se interrompe un servizio al cittadino, una procedura interna o un'attività di ricerca; il metodo di analisi, però, resta riconoscibile: servizio interessato, utenti, dipendenze, rischio, priorità, comunicazione e criterio di ripristino.

Per preparare casi plausibili, estrai quattro dati:

1. utenti serviti, interni, esterni o entrambi;
2. servizio o processo, senza studiare l'intera missione dell'ente;
3. dipendenze, come dati, identità, rete, fornitore e uffici;
4. vincoli, ricavati dal bando e dalle regole effettivamente pertinenti.

DAL CONTESTO ALLA RISPOSTA

La risposta concorsuale deve mostrare capacità di contestualizzare senza perdersi nei dettagli. Se il caso riguarda un portale comunale, non basta dire che “il sistema va ripristinato”: occorre distinguere contenimento del disservizio, raccolta delle evidenze, priorità degli utenti, comunicazione e verifica del ritorno in esercizio. Se riguarda una piattaforma universitaria, cambiano utenti e calendario operativo, non i principi di diagnosi e controllo.

Contesto	Interlocutori possibili	Domanda utile
ente locale	uffici, cittadini, gestore esterno	quale servizio essenziale è impattato?
università	studenti, docenti, personale, ricerca	quali utenze e periodi critici considero?
regione o agenzia	enti aderenti, fornitori, strutture interne	quali dipendenze e responsabilità sono condivise?
autorità o ricerca	specialisti, analisti, infrastrutture dedicate	quale requisito tecnico distingue il profilo?

Microcaso. Due bandi cercano un funzionario informatico. Il primo cita assistenza agli uffici, applicativi, rete locale e fornitori; il secondo piattaforme dati, integrazione e analisi. La denominazione è uguale, ma il primo richiede una preparazione trasversale con forte componente di servizio, il secondo sposta il baricentro su dati e interoperabilità. Il contesto conferma la lettura soltanto dopo l'esame delle attività.

MICRO-VERIFICA: LEGGERE IL CONTESTO

Un bando comunale cita assistenza agli uffici, applicativi e rete locale; uno di un'agenzia regionale cita e-service e piattaforme condivise. Quale differenza deve comparire per prima nel piano di studio?

Soluzione: si parte sempre da attività e materie del singolo bando. Nel primo caso il piano darà più spazio all'esercizio dei servizi interni e alla rete; nel secondo, a interoperabilità e piattaforme. Il nome dell'ente orienta il caso, ma non sostituisce il programma.

Errore tipico. Studiare l'ordinamento completo dell'ente per dedurre il lavoro ICT. La preparazione deve restare ancorata alle attività e alle materie del profilo; il nucleo amministrativo comune si studia nel volume base e secondo il programma reale.

Il contesto rende concreta la risposta, ma non sostituisce il programma. Prima si identifica ciò che il bando assegna; poi si costruisce un esempio coerente con utenti, servizi e vincoli dell'amministrazione.

Il controllo finale consiste nel verificare che ogni dettaglio del contesto cambi davvero una decisione, una priorità o un'evidenza della risposta.

Se non cambia nulla, quel dettaglio può essere eliminato senza perdita.

Nell'orale, esplicita sempre il passaggio dal contesto alla conseguenza: "poiché gli utenti e il servizio sono questi, considero queste dipendenze e verifico questi risultati". In questo modo l'esempio dimostra ragionamento e non diventa una descrizione ornamentale dell'ente.

1.4 La Mappa BANDO del profilo ICT

BANDO

Leggi avviso, allegati e comunicazioni ufficiali. Estrai denominazione, attività, requisiti, materie, prove e criteri. Mantieni separato ciò che permette di partecipare da ciò che sarà valutato.

AREE

Raggruppa le materie in aree: sviluppo; dati; reti e sistemi; cloud; cybersecurity; governo digitale; procurement ICT; nucleo comune. La riclassificazione trasforma una lista frammentata in una mappa.

NUCLEI

Scomponi ogni area in concetti verificabili. “Basi di dati” può comprendere modellazione, normalizzazione, SQL, transazioni, indici, NoSQL e qualità. “Cybersecurity” può comprendere rischio, identità, vulnerabilità, controlli, logging e incident response.

DIARIO

Registra l'errore per tipo: definizione, distinzione, procedura, applicazione o esposizione. Aggiungi l'azione correttiva: ripetere, confrontare, svolgere un esercizio o costruire una risposta orale.

OUTPUT

Associa ogni nucleo a un prodotto osservabile: risposta a quiz, spiegazione, pseudocodice, query, schema, diagnosi, valutazione del rischio o soluzione di un caso.

DALLA LISTA AL PIANO

La Mappa BANDO funziona quando ogni passaggio produce una decisione. Dal bando non si copia soltanto la lista delle materie: si conserva la formula originale, la si colloca in un'area e la si scompone in nuclei. Ogni nucleo riceve poi una profondità, una destinazione e un output. Se manca uno di questi elementi, la voce non è ancora pronta per lo studio.

Esempio: “basi di dati” è troppo ampia per diventare una sessione. Nell'area dati può essere scomposta in modellazione, chiavi e vincoli, normalizzazione, SQL, transazioni, indici, NoSQL, qualità e sicurezza. Il candidato non deve presumere che ogni bando richieda la stessa profondità: attività, programma, tracce e criteri disponibili aiutano a graduarla.

Passaggio	Decisione	Segnale di incompletezza
Bando	quali parole e vincoli sono ufficiali?	formula ricostruita a memoria
Aree	quale famiglia raccoglie la voce?	elenco senza gerarchia
Nuclei	quali concetti sono davvero verificabili?	materia lasciata come etichetta
Diario	quale errore e quale correzione registro?	conteggio degli sbagli senza causa
Output	che cosa devo saper produrre?	sola lettura o sottolineatura

PRIORITÀ E LACUNE

La mappa deve distinguere ciò che è assegnato da ciò che è soltanto plausibile. Una materia esplicita ha priorità diversa da una tecnologia incontrata in un esempio. Anche all'interno della stessa materia conviene separare prerequisiti, nucleo essenziale e approfondimenti. Questo impedisce di consumare tempo su dettagli non richiesti mentre resta scoperta una distinzione fondamentale.

Quando un nucleo non trova una destinazione, non va nascosto. Segnalo “mancante” se non possiedo ancora una copertura; “parziale” se la spiegazione non raggiunge tutte le dimensioni; “rinviato” solo se esiste una destinazione precisa e sufficiente. Lo stato guida il lavoro successivo e impedisce false sensazioni di completezza.

Il Diario completa il ciclo. Per un errore di definizione serve richiamo attivo; per una distinzione confusa serve confronto; per una procedura serve ricostruire la sequenza; per un'applicazione serve un caso; per un'esposizione debole serve una risposta orale cronometrata. Scrivere soltanto “ripassare database” non indica alcuna azione.

BANDO in pratica. Scegli una materia del tuo programma e crea tre righe: formula ufficiale, nuclei, output. Aggiungi per ogni nucleo una destinazione e uno stato. Se non sai quale prodotto allenare, torna alle prove; se non sai quali concetti includere, torna alle attività e alla profondità richiesta.

Errore tipico. Usare la mappa come indice definitivo del manuale. È invece una traduzione del bando target: cambia quando cambiano profilo, attività o prove, mentre il capitale di studio già costruito può essere riutilizzato.

La mappa va riesaminata a ogni nuova comunicazione ufficiale. Un calendario più breve modifica le priorità; una precisazione sulle prove modifica gli output; la pubblicazione di criteri o tracce può chiarire la profondità. Aggiornare non significa ricominciare: significa correggere le righe interessate mantenendo il lavoro valido.

Conserva la data dell'ultimo controllo e la formula ufficiale.

Così ogni revisione resta motivata e facilmente ricostruibile.

1.5 Requisiti, materie e prove

REQUISITI DI ACCESSO

Stabiliscono chi può partecipare. Titolo di studio ed eventuale esperienza non diventano automaticamente materie d'esame. Copiali in una sezione autonoma del decoder.

MATERIE

Sono gli oggetti della preparazione. Conserva la formula originale, poi normalizzala in area e nuclei. Non ampliare la materia fino a trasformarla in un intero corso universitario; non ridurla a una sigla.

PROVE

La prova scritta può assumere forme diverse: quesiti, domande aperte, elaborato o caso. L'orale verifica padronanza, collegamenti e chiarezza. Alcune procedure pubblicano tracce, questionari o criteri: DiSCo, Provincia di Belluno e Comune di Firenze offrono nel campione evidenze utili per osservare gli output reali.

Per lo studio cambia molto: conoscere una definizione non significa ancora saper motivare una soluzione. Lo stesso nucleo va allenato nel formato richiesto.

ATTIVITÀ E COMPETENZE NON SONO REQUISITI

Le attività descrivono il lavoro; le competenze indicano capacità necessarie per svolgerlo; i requisiti stabiliscono l'accesso; le materie definiscono ciò che può essere valutato. Le categorie si collegano, ma non coincidono. Un'attività di controllo dei fornitori può richiedere capacità di analisi e comunicazione; non ne deriva automaticamente uno specifico titolo di studio né l'intero diritto dei contratti come materia.

Per ogni voce del bando, applica quattro domande:

1. serve a essere ammesso?
2. descrive ciò che farò?
3. indica ciò che devo studiare?
4. chiarisce come sarò valutato?

Una stessa frase può offrire più indizi, ma la classificazione evita salti logici. Il titolo richiesto non incorpora implicitamente tutti gli esami universitari; l'attività non sostituisce il programma; la materia non rivela da sola il formato della prova.

FORMATI DI PROVA E ALLENAMENTO

Nel quiz occorre riconoscere rapidamente definizioni, relazioni e conseguenze, gestendo alternative vicine. La risposta breve richiede selezione: tesi iniziale, elementi necessari, distinzione decisiva e conclusione. L'elaborato tecnico chiede struttura, assunzioni e motivazione. Il caso richiede leggere vincoli, formulare ipotesi, scegliere controlli ed evidenze. L'orale aggiunge ordine, linguaggio preciso e capacità di collegamento.

Prova	Prestazione centrale	Allenamento coerente
quiz	riconoscere e discriminare	domande commentate e diario degli errori
risposta breve	selezionare e ordinare	schema definizione-funzione-elementi-conseguenza
elaborato	sviluppare una soluzione motivata	scaletta, assunzioni, alternative ed evidenze
caso	diagnosticare e decidere	contesto-vincoli-rischi-scelta-verifica

Prova	Prestazione centrale	Allenamento coerente
orale	spiegare e collegare	risposta cronometrata con esempio e chiusura

I materiali pubblicati da alcune amministrazioni, come questionari, tracce o criteri, mostrano prodotti reali. Non vanno però estesi a tutti i concorsi. Se il bando target non chiarisce ancora un dettaglio, il piano deve mantenere più forme di allenamento essenziali e aggiornarsi quando escono comunicazioni ufficiali.

PROFONDITÀ E CRITERI

La profondità si ricava dall'incrocio di profilo, attività, formulazione della materia e prova. “Nozioni di reti” e “progettazione e gestione di reti complesse” non promettono lo stesso livello. Anche i verbi aiutano: conoscere, distinguere, applicare, progettare, valutare e governare richiedono prestazioni crescenti. Non sono scale automatiche, ma segnali da leggere nel contesto.

Mini-esercizio. Classifica queste voci ipotetiche: “laurea in informatica”; “gestione delle infrastrutture”; “reti e protocolli”; “prova orale”. Le risposte sono, rispettivamente, requisito, attività, materia e formato. Poi aggiungi l'output: per reti e protocolli potresti dover risolvere un quiz, spiegare una distinzione o diagnosticare un caso, secondo la prova.

Errore tipico. Preparare tutte le materie allo stesso livello. Il risultato è un piano uniforme ma inefficiente. La profondità deve essere motivata e riesaminata quando cambiano calendario, avvisi, tracce o criteri.

Quando l'informazione è incerta, annota l'assunzione e prepara il nucleo essenziale. Non colmare il vuoto inventando dettagli. Appena l'amministrazione pubblica un chiarimento, aggiorna classificazione, profondità e allenamento.

Questa disciplina protegge sia dall'eccesso di studio sia da una preparazione troppo stretta costruita su supposizioni non verificate.

Le assunzioni dichiarate possono essere corrette; quelle nascoste producono errori difficili da diagnosticare.

1.6 Il Bando Decoder ICT

Compila una riga per ogni voce rilevante:

Campo	Domanda
Profilo	Quale famiglia descrive meglio attività e responsabilità?
Contesto	Quali servizi, dati e utenti sono coinvolti?

Campo	Domanda
Requisito	Serve per partecipare o sarà oggetto di prova?
Materia	Qual è la formula esatta?
Nuclei	Quali concetti la compongono?
Output	Quiz, spiegazione, progetto, diagnosi o caso?
Fonte	Quale riferimento consolidato sostiene lo studio?
Destinazione	Quale capitolo o risorsa verificata copre il nucleo?
Stato	Completo, parziale, rinviato o mancante?

Il decoder è pronto per essere usato solo quando ogni promessa di studio ha una destinazione e una verifica.

Compilalo dall'alto verso il basso, senza saltare direttamente alla materia preferita. Prima identifica profilo e contesto; poi separa requisiti, attività, materie e prove. Solo dopo assegna nuclei, fonti, destinazioni e stato. Infine controlla il pacchetto: per un profilo ICT ordinario il percorso minimo è il volume base più lo specialistico ICT; un'altra famiglia entra soltanto se il programma la richiede davvero.

La colonna "Stato" non misura quante pagine hai letto. "Completo" significa che sai definire, distinguere, applicare e verificare il nucleo nel perimetro richiesto. "Parziale" segnala una dimensione scoperta. "Rinviato" richiede una destinazione precisa. "Mancante" apre un'azione, non una colpa.

Prima di considerare chiuso il decoder, leggilo come farebbe un'altra persona: deve capire perché una voce è stata classificata, dove si studia e quale prova ne dimostra l'apprendimento. Se la decisione dipende da informazioni rimaste soltanto nella tua memoria, la riga non è ancora autosufficiente.

CASO GUIDATO

Un bando ipotetico ricerca un funzionario informatico addetto a servizi applicativi e fornitori. Il programma cita basi dati, reti, cloud e sicurezza; la selezione prevede scritto e orale.

Dal solo titolo si potrebbe pensare a un profilo generalista. Sono le attività, però, a confermare che il candidato deve collegare applicazioni, infrastrutture e controllo del servizio. Il piano assegna basi dati al capitolo 4, reti al 5, software e API al 6, cloud al 7, rischio cyber all'8 e risposta agli incidenti al 9. Il capitolo 13 allena scritto e orale.

Le materie non sviluppate nel piano del volume vanno annotate nel decoder come lacune da colmare tramite una risorsa verificata. Finché non è disponibile una destinazione editoriale precisa, il rinvio non vale come copertura acquisita.

DA SAPERE IN 5 RIGHE

Un profilo ICT si interpreta attraverso attività, contesto, materie e prove. La denominazione non basta a determinare il piano. Requisiti di accesso e materie d'esame sono distinti. Ogni materia va scomposta in nuclei e collegata a un output. Il campione dei bandi verifica la mappa, ma non autorizza percentuali.

DOMANDA DA COMMISSARIO

Come imposterebbe la preparazione per un profilo ICT trasversale?

Partirei dalle attività e dalle prove. Classificherei le materie in sviluppo, dati, infrastrutture, cloud, sicurezza e governo digitale; le scomporrei poi in nuclei. Per ogni nucleo definirei fonte, profondità, output e stato, separando il nucleo comune dagli approfondimenti specialistici.

DOMANDA-TRAPPOLA

Se il bando richiede una laurea informatica, tutto il corso di laurea è materia d'esame?

No. Il titolo è un requisito. Materie e profondità derivano dal programma, dalle attività e dalle prove. Ampliare senza criterio sottrae tempo ai nuclei effettivamente assegnati.

ERRORE TIPICO

Costruire il piano da un indice universitario generico. Si ottiene molto materiale, ma nessuna relazione verificabile con il bando. Ogni blocco di studio deve rispondere a una voce, a un'attività o a un output della selezione.

MINI-ESERCIZIO

Compila cinque righe del decoder sul tuo bando. Controlla:

- requisiti e materie sono separati?
- la formula originale è conservata?
- ogni materia è scomposta?
- ogni nucleo ha fonte, destinazione e output?
- le eventuali frequenze provengono da un campione documentato?

Se una risposta è negativa, annota nel diario che cosa devi correggere.

▣ Verifica — Profilo, bando e piano specialistico**QUIZ 1**

Quale elemento distingue meglio un requisito di accesso da una materia?

- A. Il requisito stabilisce chi può partecipare; la materia indica un oggetto della valutazione.
- B. Il requisito è sempre tecnico; la materia è sempre giuridica.
- C. Il requisito compare nel bando; la materia compare soltanto nel diario.
- D. Non esiste alcuna differenza.

Risposta corretta: A. Titolo, esperienza e altre condizioni di accesso non diventano automaticamente programma d'esame. B e C introducono distinzioni inesistenti; D elimina un passaggio essenziale del decoder.

QUIZ 2

Due bandi usano la denominazione “funzionario informatico”. Qual è la prima verifica utile?

- A. Comprare lo stesso manuale per entrambi senza leggere altro.
- B. Confrontare attività, responsabilità, materie e prove.
- C. Studiare l'intera missione istituzionale dei due enti.
- D. Presumere che entrambi riguardino programmazione.

Risposta corretta: B. La denominazione è un indizio, non una descrizione completa. C disperde il piano; D riduce arbitrariamente il perimetro; A impedisce qualsiasi adattamento.

QUIZ 3

Quando una materia può essere considerata correttamente “rinvia”?

- A. Quando il candidato pensa di conoscerla già.
- B. Quando è citata in un altro indice, anche incompleto.
- C. Quando esiste una destinazione precisa, sufficiente e verificata.
- D. Quando non resta tempo.

Risposta corretta: C. Il rinvio è una copertura solo se risolve davvero la promessa formativa. A e D descrivono decisioni personali; B non garantisce né completezza né precisione.

QUIZ 4

Quale output è più coerente con un caso tecnico?

- A. Ripetere una lista di sigle senza collegamenti.
- B. Diagnosticare il contesto, dichiarare vincoli, scegliere e indicare evidenze.
- C. Citare un prodotto commerciale come soluzione universale.
- D. Ricopiare il titolo della materia.

Risposta corretta: B. Il caso valuta ragionamento applicato e motivazione. A e D non producono una decisione; C ignora requisiti, alternative e limiti.

QUIZ 5

Un'attività cita sicurezza, ma il programma la tratta solo come requisito trasversale dei servizi. Che cosa ne segue?

- A. Il profilo è certamente cybersecurity.
- B. La sicurezza va ignorata.
- C. Occorre valutarne profondità e baricentro insieme alle altre attività e prove.
- D. Serve automaticamente un secondo volume specialistico.

Risposta corretta: C. La sicurezza attraversa molti profili ICT. A e D generalizzano; B elimina una competenza pertinente. Il baricentro emerge dall'incrocio delle evidenze.

QUIZ 6

Quale voce del Diario è più operativa?

- A. "Ripassare informatica".
- B. "Ho sbagliato".
- C. "Confondo requisito e materia: riclassifico cinque voci e spiego il criterio ad alta voce".
- D. "Leggere altre cento pagine".

Risposta corretta: C. Identifica tipo di errore e azione correttiva osservabile. A, B e D non indicano quale distinzione recuperare né come verificarla.

CASO RAGIONATO

Un'amministrazione ricerca un funzionario ICT. Le attività comprendono analisi delle esigenze degli uffici, gestione di servizi applicativi, raccordo con il fornitore e presidio di continuità e sicurezza. Il programma cita basi dati, reti, cloud, sicurezza e nozioni sui contratti; sono previste una prova scritta e una prova orale.

1. Profilo. Il baricentro è generalista: il candidato deve collegare applicazioni, infrastrutture, controlli e fornitore. Sicurezza e cloud sono componenti importanti, ma il testo non autorizza a ridurre il profilo a una sola specializzazione.

2. Classificazione. Eventuale titolo di studio resta requisito. Basi dati, reti, cloud, sicurezza e nozioni contrattuali sono materie. Analisi delle esigenze e raccordo con il fornitore sono attività. Scritto e orale sono formati di prova.

3. Aree e nuclei. Dati: modellazione, SQL, qualità e accessi. Infrastrutture: reti, cloud, disponibilità e ripristino. Cyber: rischio, identità, logging e incidenti. Fornitore: requisiti, livelli di servizio, evidenze e continuità. La disciplina generale dei contratti resta nel nucleo comune; qui interessa il governo tecnico del servizio.

4. Output. Per lo scritto si allenano risposta breve e caso: definizione del problema, vincoli, scelta, controlli ed evidenze. Per l'orale si prepara una spiegazione ordinata che colleghi requisito, servizio, rischio e verifica.

5. Controllo del piano. Ogni nucleo riceve destinazione e stato. Se manca una spiegazione completa, resta parziale; un rinvio vale solo se preciso. Il piano evita sia di studiare un intero corso universitario sia di fermarsi a definizioni isolate.

CHECKLIST FINALE

- ☐ Ho separato requisiti, attività, materie e prove.
- ☐ Ho identificato il baricentro del profilo senza cancellare le dipendenze.
- ☐ Ogni materia è scomposta in nuclei verificabili.
- ☐ Ogni nucleo ha profondità, destinazione, stato e output.
- ☐ Le lacune sono dichiarate, non nascoste dietro rinvii generici.
- ☐ Il Diario contiene un'azione correttiva osservabile.
- ☐ Il pacchetto di studio è il minimo coerente con il bando.

Informatica specialistica: cosa serve oltre il VOL-01

B BANDO	A AREE	N NUCLEI	D DIARIO	O OUTPUT
-------------------	------------------	--------------------	--------------------	--------------------

In una prova da funzionario ICT, riconoscere i componenti è solo il punto di partenza. Le domande specialistiche chiedono di collegare le parti del sistema: dove si trova un dato, chi esegue un'istruzione, perché le memorie hanno tempi diversi, quale risorsa limita le prestazioni e come interviene il sistema operativo quando più programmi competono per CPU e memoria.

Office, gestione elementare dei file e classificazioni di base restano fuori dal perimetro. CPU, memoria, dispositivi e software di sistema vengono qui studiati nelle loro relazioni, così da poterli usare nei quiz, nell'orale e nei casi tecnici.

Obiettivo e confine con il VOL-01

Il VOL-01, capitolo 10, § 1, presenta i prerequisiti: hardware, software, dati, bit e byte, CPU, RAM, ROM, memoria di massa e periferiche. Il § 2 introduce sistema operativo, file e cartelle. Qui quei concetti sono dati per acquisiti.

La preparazione specialistica richiede di:

- descrivere l'elaboratore come un insieme coordinato di unità;
- ricostruire, a livello concettuale, l'esecuzione di un'istruzione;
- collegare la rappresentazione dei dati a intervallo e precisione;
- spiegare la gerarchia di memoria;
- distinguere latenza, throughput e tempo di risposta;
- individuare un collo di bottiglia;
- leggere il sistema operativo come gestore e astrattore delle risorse.

Programmazione, reti, cloud e cybersecurity saranno sviluppati nei rispettivi capitoli. Qui compaiono soltanto quando servono a chiarire l'architettura del sistema.

Mappa BANDO dell'informatica specialistica

Passaggio	Domanda operativa	Azione del candidato
B — Bando	Il programma richiede semplice conoscenza informatica o architetture e sistemi?	Evidenziare verbi e nuclei: conoscere, descrivere, confrontare, progettare, diagnosticare.
A — Aree	Il nucleo riguarda elaborazione, memoria, I/O, prestazioni o sistema operativo?	Separare le aree senza confonderle con reti, sviluppo o cloud.

Passaggio	Domanda operativa	Azione del candidato
N — Nuclei	Quali relazioni devo saper spiegare?	Costruire coppie causa-effetto: cache e latenza, RAM e processi, I/O e throughput.
D — Diario	Quale confusione ha prodotto l'errore?	Registrare l'opposizione corretta: capacità/velocità, latenza/throughput, processo/programma.
O — Output	Che cosa richiede la prova?	Preparare una risposta orale, risolvere un quiz o diagnosticare un caso.

La profondità non si deduce dal solo titolo della materia. “Informatica” può indicare competenze operative di base; “architettura degli elaboratori”, “sistemi operativi”, “prestazioni” o “gestione delle risorse” richiedono invece una preparazione specialistica. Il singolo bando resta decisivo per delimitare il programma.

2.1 L’elaboratore come sistema

Un elaboratore riceve dati e istruzioni, li conserva, li trasferisce, li elabora e produce risultati. Queste funzioni coinvolgono quattro gruppi essenziali:

- la CPU, che interpreta ed esegue le istruzioni;
- la memoria, che conserva istruzioni e dati in uso;
- i dispositivi di input/output, che scambiano informazioni con l'esterno;
- i collegamenti di comunicazione, che trasportano dati, indirizzi e segnali di controllo.

Contano soprattutto le relazioni fra le parti. Una CPU rapida lavora al di sotto delle proprie possibilità se deve attendere di continuo dati provenienti da una memoria o da un dispositivo più lento. Una maggiore quantità di RAM consente di mantenere più dati e programmi in uso, ma non accelera per questo ogni elaborazione.

CPU, REGISTRI, UNITÀ DI CONTROLLO E ALU

La CPU contiene unità con funzioni diverse. L'unità di controllo coordina l'esecuzione delle istruzioni e genera i segnali necessari. L'unità aritmetico-logica, o ALU, svolge operazioni aritmetiche e logiche. I registri conservano temporaneamente operandi, indirizzi, istruzioni e risultati immediatamente necessari.

I registri sono pochi e molto vicini alle unità di esecuzione. Non sostituiscono la RAM: servono a rendere disponibili alla CPU le informazioni richieste nell'istante di lavoro.

BUS E DISPOSITIVI DI I/O

Un bus è un insieme di linee e regole che permette ai componenti di comunicare. I segnali hanno compiti diversi: trasportare dati, identificare una posizione o un dispositivo, coordinare lettura, scrittura e temporizzazione.

I dispositivi di I/O non dialogano necessariamente con la CPU come se fossero semplici celle di memoria. Controller e interfacce traducono richieste, gestiscono differenze di velocità e segnalano il completamento delle operazioni.

IL CICLO DI ESECUZIONE DI UN'ISTRUZIONE

Un modello didattico efficace divide l'esecuzione in passaggi:

1. la CPU preleva l'istruzione dalla memoria;
2. l'istruzione viene interpretata;
3. sono acquisiti gli operandi necessari;
4. l'operazione viene eseguita;
5. il risultato viene scritto nella destinazione prevista;
6. il flusso prosegue con l'istruzione successiva o con quella indicata da un salto.

Questa sequenza, spesso riassunta come prelievo, decodifica ed esecuzione, è un modello concettuale. Le CPU reali possono sovrapporre fasi, eseguire più operazioni e adottare strategie complesse. In prova, il modello serve a spiegare le funzioni, non a descrivere ogni dettaglio di una specifica architettura.

ARCHITETTURA, STATO E COORDINAMENTO

La CPU non opera isolatamente. Il clock fornisce un riferimento temporale alle attività interne, ma la sola frequenza non misura il lavoro utile completato: architetture diverse possono eseguire quantità differenti di lavoro nello stesso numero di cicli. Anche il numero di unità di esecuzione, la disponibilità dei dati e le dipendenze fra istruzioni incidono sul risultato.

I registri comprendono sia valori manipolati dal programma sia informazioni di controllo. Il contatore di programma identifica il punto del flusso da eseguire; altri registri conservano lo stato necessario a proseguire correttamente. Quando avviene un'interruzione o il sistema operativo assegna la CPU a un altro processo, una parte di questo contesto deve essere salvata e poi ripristinata.

Il bus è un canale governato da regole. Le linee dati trasportano contenuti, quelle di indirizzo individuano destinazioni, quelle di controllo specificano operazioni e sincronizzazione. Quando più componenti richiedono lo stesso collegamento, un meccanismo di arbitraggio assegna gli accessi. La capacità del canale può diventare il limite del sistema.

APPLICAZIONE AL PROFILO

In una prova orale, una risposta chiara parte dalle funzioni: elaborare, conservare, trasferire e coordinare. Poi colloca CPU, memoria, controller e bus, e segue un'istruzione dal prelievo alla scrittura del risultato. Così i componenti non restano un elenco mnemonico: ciascuno trova posto nella catena causa-effetto.

Mini-caso. Una CPU più rapida non migliora sensibilmente un'elaborazione. Se il processore trascorre molto tempo in attesa dei dati, il limite può trovarsi nella memoria, nel bus o nell'I/O. La risposta corretta non sceglie subito un componente: identifica il percorso dei dati, misura le attese e localizza la risorsa satura.

2.2 Rappresentare dati e istruzioni

Un sistema digitale opera su configurazioni di bit. Il significato non è contenuto nei bit in sé: dipende dalla codifica e dall'interpretazione scelta. La stessa sequenza può rappresentare un numero, un carattere, una parte di un'immagine o un'istruzione.

INTERVALLO E CODIFICA

Un numero finito di bit produce un insieme finito di configurazioni. Disporre di più bit amplia l'intervallo rappresentabile oppure consente una descrizione più precisa.

Per gli interi va distinta almeno la rappresentazione senza segno da quella con segno. Nel primo caso tutte le configurazioni possono indicare valori non negativi; nel secondo serve una convenzione che comprenda anche i negativi. La stessa sequenza binaria può quindi assumere valori diversi secondo il formato adottato.

I caratteri sono associati a codici numerici definiti da uno standard. La codifica rende possibile memorizzare e scambiare testo, ma mittente e destinatario devono interpretare i byte secondo regole compatibili.

PRECISIONE, ARROTONDAMENTO E OVERFLOW

I numeri reali non possono essere tutti rappresentati esattamente con una quantità finita di bit. Una rappresentazione in virgola mobile descrive un insieme limitato di valori e deve approssimarne altri. Per questo alcuni calcoli possono produrre piccoli errori di arrotondamento.

L'overflow si verifica quando un risultato supera l'intervallo previsto dal formato. La perdita di precisione riguarda invece l'impossibilità di conservare tutte le cifre o differenze rilevanti. Sono problemi distinti: un valore può rientrare nell'intervallo ma essere rappresentato con precisione insufficiente.

DAL BIT ALL'INFORMAZIONE

Il bit ammette due stati, convenzionalmente indicati con 0 e 1. Raggruppare i bit consente di costruire configurazioni più numerose: con n bit si ottengono 2^n combinazioni. Questa relazione serve a ragionare sull'intervallo, non a memorizzare elenchi. Se un campo senza segno dispone di otto bit, per esempio, può distinguere 256 configurazioni. Il valore massimo dipende poi dalla convenzione scelta e dal fatto che una configurazione possa essere riservata.

Il byte è normalmente un gruppo di otto bit. Le unità maggiori descrivono capacità di memoria o quantità di dati. Il contesto conta: una misura decimale e una basata su potenze di due non sono automaticamente equivalenti. Nei quiz si identifica prima l'unità richiesta, poi la codifica e, per ultima, l'operazione.

Una codifica è un accordo fra chi produce e chi interpreta i dati. Nel testo, un codice associa numeri a caratteri; nelle immagini, i bit rappresentano campioni o componenti di colore; nell'audio, descrivono campioni del segnale. Aumentare il numero di bit disponibili può ampliare i valori distinguibili, ma non garantisce da solo qualità: contano anche il metodo di acquisizione, la codifica, la compressione e l'uso previsto.

INTERI, VALORI REALI E ISTRUZIONI

Gli interi con segno sono comunemente rappresentati in complemento a due. Con n bit l'intervallo è da $-2^{(n-1)}$ a $2^{(n-1)}-1$; senza segno è da 0 a 2^n-1 . Su otto bit, +5 è 00000101: invertendo i bit e aggiungendo uno si ottiene -5, 11111011. La stessa sequenza letta senza segno vale 251. Il formato decide il significato. La somma $127+1$ non è rappresentabile su otto bit con segno: nell'aritmetica modulare i bit diventano 10000000, cioè -128, ma il comportamento concreto del linguaggio (errore, wraparound o altro) va distinto dal calcolo sui bit.

I valori in virgola mobile separano, in forma codificata, segno, esponente e significando. Per un numero binario normalizzato il modello è $(-1)^s \times 1.f \times 2^e$: $1,5 = 1,12 \times 2^0$. Esponente e significando hanno un numero finito di bit; valori subnormali, zeri e valori speciali seguono regole proprie del formato. Il modello consente di rappresentare numeri molto grandi e molto piccoli, ma non tutti i reali. Ne deriva che il confronto diretto fra risultati calcolati può richiedere una tolleranza. In un procedimento amministrativo informatizzato, un importo monetario non dovrebbe essere trattato con leggerezza come un'approssimazione binaria: formato, regole di arrotondamento e controlli devono essere espliciti.

Anche un'istruzione è una configurazione codificata. Una parte indica l'operazione, altre parti possono identificare registri, dati o modalità per reperirli. La CPU non attribuisce significato linguistico all'istruzione: riconosce un formato previsto dalla propria architettura e attiva le unità necessarie.

APPLICAZIONE AL PROFILO

Il funzionario ICT deve saper collegare un'anomalia al livello corretto. Un carattere illeggibile suggerisce un problema di codifica; un contatore che supera il massimo riguarda l'intervallo; un risultato decimale leggermente diverso dall'atteso può dipendere dalla precisione; un file interpretato con il formato sbagliato non è necessariamente corrotto. La diagnosi parte quindi dalla domanda: «quale significato attribuisce il sistema a questi bit?».

Mini-esercizio. Un campo dispone di quattro bit senza segno. Quante configurazioni distingue? La risposta è sedici, perché $2^4 = 16$. Questo non significa che possa rappresentare sedici valori arbitrari: il significato delle configurazioni dipende dalla codifica adottata.

2.3 La gerarchia di memoria

Non esiste una memoria ideale che sia contemporaneamente vastissima, velocissima, persistente ed economica. I sistemi combinano livelli diversi.

Livello	Funzione prevalente	Caratteristica da ricordare
Registri	Operandi e stato immediato della CPU	Capacità molto ridotta, accesso rapidissimo
Cache	Copia temporanea di dati e istruzioni usati con maggiore probabilità	Riduce gli accessi ai livelli più lenti
RAM	Programmi e dati attivi	Volatile, più ampia della cache
Memoria persistente	File, programmi e dati conservati	Grande capacità, permanenza senza alimentazione

La gerarchia funziona perché i programmi tendono a riutilizzare dati e istruzioni già usati e ad accedere a posizioni vicine. È il principio di località: temporale quando un elemento viene riutilizzato a breve; spaziale quando è probabile l'accesso a elementi vicini.

Una cache non aumenta la capacità complessiva come farebbe l'aggiunta di storage. Cerca invece di ridurre il tempo medio di accesso mantenendo vicino alla CPU ciò che potrebbe servire presto. Se il dato richiesto è presente si ha un hit; se manca, un miss impone il recupero da un livello più lento.

PERCHÉ SERVONO PIÙ LIVELLI

La CPU può eseguire operazioni molto più rapidamente di quanto i livelli lontani riescano a fornire dati. La gerarchia riduce questa distanza combinando memorie con caratteristiche diverse. I livelli prossimi al processore privilegiano rapidità e accesso immediato; quelli più lontani privilegiano capacità, persistenza e costo per unità di informazione. Non esiste quindi un livello «migliore» in assoluto: ciascuno risponde a una funzione.

Quando la CPU richiede un dato, il sistema cerca di soddisfare la richiesta al livello più vicino utile. Un hit evita l'accesso al livello successivo; un miss introduce una penalità. Il tempo medio dipende sia dalla rapidità del livello sia dalla frequenza con cui i dati richiesti vi sono presenti. Per questo una cache molto veloce ma usata male non elimina le attese.

La località temporale emerge, per esempio, quando un ciclo riusa la stessa variabile. La località spaziale compare quando il programma percorre in ordine gli elementi contigui di un vettore. Strutture dati e modalità di accesso possono quindi incidere sulle prestazioni anche senza cambiare la frequenza della CPU.

RAM, MEMORIA VIRTUALE E PERSISTENZA

La RAM contiene codice e dati dei programmi attivi ed è volatile: perdendo alimentazione, non conserva stabilmente il contenuto. Lo storage mantiene file e programmi, ma ha finalità e tempi diversi. Confondere RAM e spazio su disco porta a diagnosi sbagliate: liberare storage non equivale ad aggiungere memoria di lavoro.

Il sistema operativo offre a ciascun processo uno spazio di indirizzamento e traduce gli indirizzi usati dal programma nelle posizioni effettivamente disponibili. La memoria virtuale è anzitutto questa astrazione organizzativa e protettiva; non coincide con il semplice uso del disco come estensione della RAM. Se parti non immediatamente necessarie vengono spostate verso un livello persistente, l'accesso successivo può però diventare molto più lento.

La persistenza richiede regole di organizzazione e affidabilità. Il salvataggio attraversa buffer, controller e file system; la conferma dell'applicazione non garantisce sempre che ogni livello abbia già completato la scrittura fisica. Nei sistemi critici servono politiche coerenti di sincronizzazione, ridondanza e backup. Sono temi distinti dalla gerarchia di memoria.

APPLICAZIONE AL PROFILO

Di fronte a un'applicazione lenta, il candidato deve chiedersi se il working set entra in RAM, se si verificano molti miss di cache, se il sistema ricorre frequentemente allo storage e se più processi competono per la memoria. Aggiungere RAM aiuta quando la pressione di memoria è reale; incide poco se il limite è un algoritmo inefficiente o un dispositivo di I/O saturo.

Errore tipico. Ordinare i livelli solo per capacità e concludere che il più grande sia il più efficace. La risposta completa espone il compromesso fra vicinanza, latenza, capacità, costo e persistenza.

2.4 Input/output e gestione degli eventi

Le periferiche hanno tempi e modalità di funzionamento molto diversi dalla CPU. Costringere il processore a controllare senza sosta se un dispositivo ha terminato sprecherebbe capacità di calcolo.

Con un interrupt, un dispositivo o un evento segnala che richiede attenzione. La CPU interrompe il flusso corrente, salva lo stato necessario, esegue la routine prevista e riprende il lavoro. Questo non rende istantanea la gestione: il sistema deve riconoscere l'evento, stabilirne la priorità e servirlo.

Il buffering usa un'area temporanea per assorbire differenze di velocità o di dimensione dei trasferimenti. Un buffer non elimina il limite del dispositivo: può rendere più regolare il flusso e ridurre le attese immediate, ma se i dati arrivano stabilmente più in fretta di quanto possano essere elaborati il collo di bottiglia rimane.

CONTROLLER, DRIVER E TRASFERIMENTI

Un dispositivo presenta registri, comandi e vincoli propri. Il controller governa gli aspetti elettronici e temporali del trasferimento; il driver è il componente software con cui il sistema operativo controlla quella famiglia di dispositivi. Questa separazione consente alle applicazioni di usare interfacce relativamente uniformi senza conoscere ogni dettaglio dell'hardware.

Il trasferimento può coinvolgere la CPU in misura diversa. Nel controllo programmato, o polling, il processore interroga ripetutamente lo stato del dispositivo. La soluzione è semplice e può essere adeguata per attese brevissime o sistemi elementari, ma spreca tempo se l'evento è raro. Con gli interrupt, invece, il dispositivo segnala la necessità di servizio. Per blocchi consistenti di dati, meccanismi dedicati possono trasferire informazioni fra dispositivo e memoria riducendo il lavoro puntuale della CPU.

Un interrupt ha un costo. Il sistema salva lo stato, individua la sorgente, esegue il gestore e riprende l'attività interrotta. Troppi eventi consumano risorse; per contenerne l'effetto, il sistema può aggregarli o regolarne le priorità. La scelta dipende dalla frequenza degli eventi, dalla loro urgenza e dal volume dei dati.

SINCRONIA, ASINCRONIA E CODE

Un'operazione sincrona trattiene il flusso chiamante finché non raggiunge il punto di completamento previsto. Un'operazione asincrona consente al programma di proseguire e ricevere in seguito una notifica o controllare l'esito. Asincrono non vuol dire automaticamente più rapido: permette soprattutto di organizzare meglio l'attesa.

Buffer e code separano produttore e consumatore. Se una stampante riceve dati a raffiche, una coda consente all'applicazione di consegnare il lavoro e continuare, mentre il dispositivo procede alla propria velocità. Se però i lavori arrivano più rapidamente di quanto siano smaltiti, la coda cresce: il ritardo viene differito, non eliminato.

Perdita, duplicazione e ordine sono problemi diversi. Un sistema di acquisizione deve stabilire che cosa accade quando il buffer è pieno, come riconoscere un trasferimento completato e come reagire a un errore. Queste decisioni incidono sulla correttezza del servizio oltre che sulle prestazioni.

APPLICAZIONE AL PROFILO

In un caso d'esame, il candidato può confrontare polling e interrupt indicando funzione, vantaggio, costo e contesto. Per diagnosticare un I/O lento deve osservare profondità delle code, tempi di servizio, quantità trasferite, errori e utilizzo del dispositivo. Dire soltanto «il disco è lento» non distingue un limite fisico, richieste casuali, contesa, driver o buffering inadeguato.

Mini-caso. Un servizio riceve documenti a raffiche e li protocolla più lentamente. Un buffer assorbe i picchi brevi; se il tasso medio di arrivo supera stabilmente quello di lavorazione, servono capacità aggiuntiva, regolazione del flusso o una diversa organizzazione. Aumentare indefinitamente il buffer rinvia la saturazione e può peggiorare il tempo di risposta.

Nell'orale, il confronto deve arrivare a una conseguenza pratica. Polling, interrupt, buffer e trasferimenti dedicati possono convivere nello stesso sistema. La scelta considera frequenza degli eventi, volume, urgenza, costo di gestione e rischio di perdita. Una definizione tecnica diventa così una decisione motivata.

2.5 Capire le prestazioni

La parola “veloce” è troppo generica. Prima del confronto va definito il risultato che interessa.

- Latenza: tempo necessario perché un'operazione o una risposta si completi.
- Throughput: quantità di lavoro completata nell'unità di tempo.
- Tempo di risposta: intervallo percepito fra richiesta e risultato, che può includere attese, elaborazione e trasferimenti.

Una minore latenza della singola operazione non garantisce il massimo throughput complessivo. Può accadere anche l'opposto: il sistema elabora molte richieste al secondo, ma una richiesta specifica riceve risposta troppo tardi.

COLLI DI BOTTIGLIA E PARALLELISMO

Il collo di bottiglia è la risorsa o la fase che limita il risultato osservato. Può essere CPU, memoria, storage, rete, dispositivo esterno, software o contesa fra attività. La diagnosi corretta parte dalla metrica e dal carico: non dall'acquisto automatico del componente con il numero commerciale più alto.

Il parallelismo può aumentare il lavoro svolto quando le attività possono procedere in modo indipendente. Non rende però parallela una parte intrinsecamente sequenziale e introduce costi di coordinamento, sincronizzazione e scambio dei dati.

MISURARE PRIMA DI INTERVENIRE

Una misura ha senso solo se specifica carico, intervallo e punto di osservazione. Il tempo della CPU, il tempo trascorso percepito dall'utente e il tempo passato in attesa di I/O descrivono fenomeni diversi. Anche una media può nascondere richieste molto lente: per un servizio al pubblico interessano spesso distribuzione e valori di coda, non soltanto il valore medio.

Il benchmark riproduce un carico definito per confrontare sistemi o configurazioni. Deve essere rappresentativo: un test che usa dati già in cache non descrive necessariamente un'importazione reale; un test con un solo utente non misura la contesa di cento accessi concorrenti. Modificare più componenti insieme rende inoltre difficile attribuire il risultato.

La capacità indica quanto lavoro il sistema può sostenere entro livelli di servizio accettabili. Quando il carico cresce, inizialmente il throughput può aumentare; vicino alla saturazione, le code e il tempo di risposta possono crescere rapidamente. Questo spiega perché un servizio apparentemente stabile degradi in modo improvviso nelle ore di punta.

DIAGNOSI DEL COLLO DI BOTTIGLIA

Un metodo ordinato comprende cinque mosse: definire il sintomo; scegliere la metrica; raccogliere evidenze su CPU, memoria, I/O e code; formulare un'ipotesi; modificare una variabile e misurare di nuovo. La correlazione temporale aiuta, ma non prova da sola la causa. Una CPU molto utilizzata può essere il limite oppure svolgere lavoro inutile provocato da un altro difetto.

Il parallelismo distingue almeno concorrenza e simultaneità. Più attività possono avanzare alternandosi su una CPU; con più unità di esecuzione alcune possono procedere nello stesso istante. Il beneficio massimo resta limitato dalle parti sequenziali e dai costi di coordinamento. Se molti lavoratori accedono alla stessa risorsa protetta, la contesa può annullare il vantaggio.

Scalabilità significa mantenere un comportamento utile al crescere del carico aggiungendo o organizzando risorse. Non coincide con una singola prestazione elevata. Un sistema può essere rapido per dieci utenti e degradare male al centesimo; un altro può avere una latenza iniziale maggiore ma sostenere meglio l'aumento.

APPLICAZIONE AL PROFILO

Una risposta tecnica non parte dalla prescrizione. «Aggiungere CPU» è una soluzione solo dopo aver dimostrato che il lavoro è limitato dal calcolo e può sfruttare la risorsa. Se il tempo è dominato da letture casuali, attese di rete o lock, l'investimento può incidere poco.

Domanda-trappola. Un throughput doppio implica tempo di risposta dimezzato? No. Le metriche descrivono aspetti diversi e dipendono dal carico. Il candidato deve dichiarare che cosa viene misurato e in quali condizioni.

Va considerato anche il costo dell'ottimizzazione. Ridurre di poco la latenza di un'operazione rara può essere meno utile che eliminare una coda frequente. Il funzionario ICT collega la misura all'obiettivo del servizio: tempi per l'utente, capacità nelle scadenze, affidabilità o uso efficiente delle risorse.

2.6 Il sistema operativo come gestore delle risorse

Il sistema operativo offre ai programmi un ambiente di esecuzione e governa l'accesso alle risorse. Non è soltanto l'interfaccia grafica vista dall'utente.

Un programma è un insieme di istruzioni memorizzate; un processo è un'istanza in esecuzione, con stato e risorse. Il sistema operativo decide quando assegnare tempo di CPU ai processi, ne conserva il contesto durante gli avvicendamenti e coordina le attese.

Nella gestione della memoria assegna spazi ai processi, protegge aree che non devono essere usate indebitamente e offre astrazioni che separano la visione del programma dalla disposizione fisica. Nella gestione dell'I/O usa driver e meccanismi di controllo per fornire interfacce più uniformi a dispositivi differenti.

Il file system organizza dati persistenti attraverso file, directory, metadati e permessi. Qui interessa la funzione di organizzazione, accesso e protezione; le operazioni pratiche su file e cartelle restano nel VOL-01.

La protezione tra processi impedisce che un programma possa leggere o modificare liberamente lo spazio di un altro. Il sistema operativo combina separazione degli indirizzi, modalità di esecuzione e controlli sugli accessi. Questa funzione non elimina ogni vulnerabilità, ma crea il confine entro cui applicazioni e utenti esercitano soltanto le autorizzazioni previste.

CASO GUIDATO: IL GESTIONALE RALLENTA DURANTE L'IMPORTAZIONE

Un ente importa un archivio molto grande. Durante l'operazione il gestionale risponde lentamente. Un collega propone di sostituire subito la CPU.

La diagnosi deve procedere per ipotesi verificabili:

1. definire la metrica: tempo totale dell'importazione o risposta delle altre funzioni;
2. osservare quali risorse sono sature;
3. distinguere elaborazione, lettura da storage, scrittura, memoria e attese;
4. verificare se più processi competono per la stessa risorsa;
5. provare l'intervento sulla risorsa limitante e misurare di nuovo.

Se la CPU attende lo storage, sostituirla con un modello più rapido può incidere poco. Se la trasformazione dei dati occupa stabilmente tutta la capacità di calcolo, la CPU è invece un candidato plausibile. Per formulare la diagnosi non basta una specifica tecnica isolata.

DOMANDA DA COMMISSARIO

Descriva il ciclo di esecuzione di un'istruzione e il ruolo della memoria.

Una risposta ordinata parte dal prelievo dell'istruzione dalla memoria, prosegue con decodifica, acquisizione degli operandi, esecuzione e scrittura del risultato. L'unità di controllo coordina le fasi, l'ALU svolge le operazioni previste e i registri conservano le informazioni immediatamente necessarie. Cache e RAM appartengono alla gerarchia che alimenta la CPU, con compromessi diversi fra capacità e tempo di accesso.

DOMANDA-TRAPPOLA

Aumentare la RAM rende sempre più veloce il processore?

No. Più RAM può evitare la carenza di memoria e ridurre il ricorso a livelli persistenti più lenti. In determinati carichi, il sistema ne trae beneficio. La velocità interna della CPU però non cambia e un collo di bottiglia collocato altrove rimane.

ERRORE TIPICO

Confrontare componenti usando una sola cifra. Frequenza, capacità o numero di unità non descrivono da soli le prestazioni osservate. Prima si identifica l'output richiesto, poi la metrica, infine la risorsa che limita quel carico.

CHECKLIST DELLE COMPETENZE SPECIALISTICHE

Per ogni riga segnare: R se riconosci il concetto, S se sai spiegarlo, A se sai applicarlo.

Competenza	R	S	A
Collegare CPU, memoria, bus e I/O	☐	☐	☐
Ricostruire il ciclo di un'istruzione	☐	☐	☐
Distinguere codifica, intervallo e precisione	☐	☐	☐
Spiegare overflow e perdita di precisione	☐	☐	☐

Competenza	R	S	A
Ordinare e confrontare i livelli di memoria	☐	☐	☐
Spiegare località, hit e miss	☐	☐	☐
Distinguere latenza, throughput e tempo di risposta	☐	☐	☐
Individuare un possibile collo di bottiglia	☐	☐	☐
Distinguere programma e processo	☐	☐	☐
Spiegare il sistema operativo come gestore di risorse	☐	☐	☐

Una riga segnata soltanto con R non è ancora pronta per l'orale. Per passare ad A, formulare un esempio o risolvere un caso.

Il sistema operativo realizza dunque una mediazione continua. Offre astrazioni stabili ai programmi, assegna risorse scarse, isola attività che non devono interferire e registra gli eventi necessari alla gestione. In una risposta concorsuale, questa sintesi va accompagnata da almeno un meccanismo concreto — scheduling, memoria virtuale, driver o permessi — e dalla relativa conseguenza operativa.

▣ Verifica

ESERCIZIO 1

Ordina dal livello normalmente più vicino alla CPU a quello destinato alla conservazione persistente: RAM, registri, storage, cache.

Soluzione: registri, cache, RAM, storage. L'ordine descrive la gerarchia funzionale generale, non una garanzia assoluta sulle prestazioni di ogni prodotto.

ESERCIZIO 2

Un servizio completa 500 richieste al secondo, ma una singola richiesta attende molto prima della risposta. Quale distinzione devi applicare?

Soluzione: throughput e latenza o tempo di risposta. Un throughput elevato non dimostra che ogni richiesta riceva risposta rapidamente.

QUIZ 1

Quale affermazione è corretta?

- A. La cache serve principalmente ad aumentare la capacità persistente.
- B. Un buffer elimina qualunque collo di bottiglia di I/O.
- C. Un processo è un programma in esecuzione con stato e risorse.
- D. La RAM modifica sempre la frequenza della CPU.

Risposta corretta: C. Le altre opzioni attribuiscono a cache, buffer e RAM effetti che non hanno necessariamente.

QUIZ 2

Quale situazione descrive meglio un cache miss?

- A. Il dato richiesto è già in un registro.
- B. Il dato non è nel livello di cache consultato e va cercato più lontano.
- C. Lo storage ha esaurito lo spazio.
- D. Il processo ha terminato l'esecuzione.

Risposta corretta: B. Il miss riguarda l'assenza del dato nella cache interrogata e comporta l'accesso a un livello successivo, normalmente più lento.

QUIZ 3

Quale affermazione distingue correttamente polling e interrupt?

- A. Il polling interroga lo stato; l'interrupt segnala un evento.
- B. L'interrupt elimina ogni costo di gestione.
- C. Il polling trasferisce sempre più dati.
- D. Sono due nomi dello stesso meccanismo.

Risposta corretta: A. Nel polling la CPU controlla ripetutamente; con l'interrupt riceve una segnalazione. Entrambe le modalità hanno costi e contesti appropriati.

QUIZ 4

Un sistema completa molte richieste al secondo, ma alcune attendono a lungo. Quale conclusione è corretta?

- A. Il throughput elevato esclude problemi.
- B. La latenza delle singole richieste deve essere misurata separatamente.
- C. Occorre aumentare subito la RAM.
- D. Il processore è necessariamente guasto.

Risposta corretta: B. Throughput e latenza non sono intercambiabili; una misura aggregata può nascondere code o richieste lente.

QUIZ 5

Che cosa distingue un programma da un processo?

- A. Il programma è il codice memorizzato; il processo è un'istanza in esecuzione con stato e risorse.
- B. Il processo è sempre un file.
- C. Il programma dispone sempre di una CPU dedicata.
- D. Non esiste alcuna distinzione.

Risposta corretta: A. Più processi possono derivare dallo stesso programma e possedere contesti di esecuzione distinti.

QUIZ 6

Qual è il primo passo corretto davanti a un rallentamento?

- A. Sostituire il componente con la frequenza più bassa.
- B. Definire sintomo, carico e metrica osservabile.
- C. Aumentare tutti i buffer.
- D. Disattivare lo scheduling.

Risposta corretta: B. Senza una metrica e condizioni definite non è possibile individuare il collo di bottiglia né verificare l'effetto dell'intervento.

Da sapere in 5 righe

- L'architettura si comprende collegando CPU, memoria e I/O.
- I bit acquistano significato attraverso una codifica.
- La gerarchia di memoria bilancia capacità, latenza, costo e persistenza.
- Le prestazioni richiedono una metrica e l'individuazione del collo di bottiglia.
- Il sistema operativo gestisce e astrae le risorse per programmi e utenti.

Riferimenti professionali essenziali

- Il Metodo BANDO, capitolo 10, paragrafi 1-2, per i prerequisiti di informatica di base.
- The RISC-V Instruction Set Manual, Volume I — Unprivileged Architecture, per la distinzione fra istruzioni, architettura e dettagli di implementazione.
- IEEE 754-2019, Standard for Floating-Point Arithmetic, per intervallo, precisione e arrotondamento.
- The Unicode Standard e Unicode Character Encoding Model (UTR #17), per caratteri e forme di codifica.
- Documentazione ufficiale del kernel Linux su gestione della memoria e interrupt, usata qui a sostegno dei concetti generali e non delle API specifiche.
- Documentazione SPEC CPU sulle metriche orientate alla velocità e al throughput.
- Programma del singolo bando, che resta decisivo per stabilire profondità e lessico richiesti.

Programmazione, algoritmi e strutture dati

B BANDO	A AREE	N NUCLEI	D DIARIO	O OUTPUT
-------------------	------------------	--------------------	--------------------	--------------------

Nelle prove tecniche, conoscere una parola chiave non è sufficiente. Può essere necessario leggere un algoritmo, seguirne l'esecuzione, trovare un errore, scegliere una struttura dati o spiegare perché una soluzione rallenta al crescere dell'input.

La sintassi cambia da un linguaggio all'altro. Il metodo, invece, parte dal problema, rappresenta i dati, ordina i passi, controlla i casi limite e valuta il costo della soluzione. Per questo il capitolo usa soprattutto lo pseudocodice.

Obiettivo e confine con il VOL-01

Il VOL-01, capitolo 10, § 8, definisce programma, linguaggio, algoritmo, variabile, funzione, condizione e ciclo. Presenta inoltre compilatore, interprete e linguaggi di marcatura. Questi sono i prerequisiti.

Questo capitolo sviluppa quei prerequisiti. Al termine dovrai saper:

- trasformare un problema in input, vincoli e output;
- leggere e scrivere pseudocodice;
- scegliere tipi e strutture dati coerenti con le operazioni;
- usare selezione, iterazione e funzioni;
- confrontare ricerca lineare e binaria;
- spiegare due strategie di ordinamento;
- stimare tempo e spazio al crescere dell'input;
- verificare correttezza e casi limite.

Le basi dati sono trattate nel capitolo 4; ciclo di vita, test di sistema e API nel capitolo 6; algoritmi di machine learning nel capitolo 11.

Mappa BANDO della programmazione

Passaggio	Domanda da porsi	Azione
B — Bando	Sono richiesti concetti, pseudocodice o un linguaggio specifico?	Separare il nucleo comune dalle sintassi nominate.
A — Aree	La richiesta riguarda controllo, funzioni, strutture, algoritmi o complessità?	Collocare ogni voce nell'area corretta.

Passaggio	Domanda da porsi	Azione
N — Nuclei	Quali operazioni devo saper spiegare o eseguire?	Preparare micro-esempi e confronti.
D — Diario	L'errore dipende da concetto, traccia, caso limite o costo?	Registrare causa e correzione, non solo la risposta.
O — Output	La prova chiede quiz, orale o esercizio?	Allenare riconoscimento, spiegazione e produzione.

Se il bando cita Java, Python, C# o un altro linguaggio, va studiata anche la sintassi richiesta. Lo pseudocodice non sostituisce quella preparazione: separa la logica dalle regole del linguaggio.

3.1 Problema, algoritmo e correttezza

Un problema computazionale descrive una relazione fra dati in ingresso e risultato atteso. Un'istanza è un caso concreto del problema. “Trovare una pratica dato un identificativo” è il problema; cercare la pratica A-1042 è un'istanza.

Un algoritmo è una procedura finita e non ambigua che, ricevuto un input ammesso, produce l'output previsto. Un programma è l'implementazione dell'algoritmo in un linguaggio eseguibile in un determinato ambiente.

Prima di scrivere i passi, chiarisci:

- input e loro formato;
- output atteso;
- vincoli sui dati;
- comportamento sui casi limite;
- criterio con cui giudicare corretta la risposta.

Supponiamo di dover contare le pratiche con stato DA_VERIFICARE. L'input è una collezione di pratiche; l'output è un intero non negativo. Il caso della collezione vuota deve restituire zero. Un valore di stato non riconosciuto richiede una regola: ignorarlo, segnalarlo o interrompere l'elaborazione. Senza questa scelta, il problema è incompleto.

CORRETTEZZA E TERMINAZIONE

Un algoritmo è corretto se produce l'output previsto per tutti gli input ammessi, non soltanto per un esempio favorevole. Deve inoltre terminare: un ciclo che non modifica mai la propria condizione può proseguire indefinitamente.

I test aiutano a trovare errori, ma non dimostrano da soli la correttezza per ogni input possibile. In una prova, la soluzione può essere motivata con un'invariante semplice: una proprietà che resta vera durante l'esecuzione. Nel conteggio delle pratiche, dopo aver esaminato i primi k elementi, il contatore deve coincidere con il numero di stati cercati fra quei k elementi.

PSEUDOCODICE E TRACCIAMENTO

Il pseudocodice descrive un algoritmo con istruzioni leggibili e convenzioni dichiarate, senza adottare tutta la sintassi di un linguaggio. Deve essere abbastanza preciso da permettere a un'altra persona di eseguire i passi.

Le convenzioni adottate sono:

- $\leftarrow$ per l'assegnazione;
- $=$ per il confronto;
- SE, ALTRIMENTI, FINE SE per la selezione;
- PER OGNI e MENTRE per i cicli;
- RESTITUISCI per l'output di una funzione.

```
FUNZIONE contaDaVerificare(pratiche)
    conteggio  $\leftarrow$  0
    PER OGNI pratica IN pratiche
        SE pratica.stato = "DA_VERIFICARE"
            conteggio  $\leftarrow$  conteggio + 1
        FINE SE
    FINE PER
    RESTITUISCI conteggio
FINE FUNZIONE
```

La trace table registra lo stato dopo ogni iterazione:

Iterazione	Stato letto	Conteggio
iniziale	—	0
1	COMPLETA	0
2	DA_VERIFICARE	1
3	DA_VERIFICARE	2
4	ARCHIVIATA	2

Tracciare l'esecuzione fa emergere assegnazioni mancanti, condizioni invertite e limiti errati. È particolarmente utile quando un ciclo usa un indice.

PRINCIPI DI SPECIFICA, PRECONDIZIONI E CASI LIMITE

Una specifica separa ciò che la soluzione deve fare dal modo in cui lo farà. La preconditione descrive ciò che deve essere vero prima dell'esecuzione; la postcondizione descrive il risultato garantito al termine. Per una ricerca binaria, la preconditione essenziale è che la collezione sia ordinata secondo lo stesso criterio usato nei confronti.

I casi limite incidono sul contratto dell'algoritmo. Una collezione vuota, un solo elemento, una chiave assente, valori duplicati o un input non valido possono cambiarne il comportamento. Prima del codice si decide se l'input non valido viene rifiutato, corretto o gestito come risultato speciale.

Nel colloquio tecnico, la correttezza si argomenta collegando ogni passo alla specifica. La trace table dimostra il comportamento su istanze scelte; un'invariante spiega perché il ragionamento continua a valere durante tutte le iterazioni. Test e argomentazione svolgono ruoli diversi e complementari.

Per dimostrare un ciclo si controllano tre momenti. L'invariante deve essere vera prima della prima iterazione, conservarsi dopo ogni esecuzione del corpo e, insieme alla condizione di uscita, implicare la postcondizione. La terminazione richiede inoltre una quantità che progredisca verso un limite. Questo schema rende la risposta più solida del semplice «funziona nei test».

Quando il problema ammette più risultati corretti, la specifica deve chiarire il criterio. Cercare una pratica duplicata può significare restituire la prima occorrenza, tutte le occorrenze oppure segnalare l'anomalia. Algoritmi diversi possono essere corretti rispetto a specifiche diverse.

3.2 Paradigmi e flusso di controllo

Un paradigma è un modo generale di organizzare il calcolo.

- Nel paradigma imperativo il programma modifica uno stato attraverso istruzioni.
- La programmazione procedurale organizza tali istruzioni in procedure e funzioni.
- La programmazione orientata agli oggetti combina dati e comportamenti in oggetti appartenenti a classi o tipi.
- La programmazione funzionale privilegia composizione di funzioni e controllo degli effetti sullo stato.
- La programmazione dichiarativa descrive il risultato o le regole, lasciando al sistema parte della scelta del procedimento.

I linguaggi reali possono sostenere più paradigmi. Un singolo esempio non basta quindi ad assegnare un linguaggio in modo esclusivo a una categoria.

TIPI, ESPRESSIONI E STRUTTURE DI CONTROLLO

Un valore è un dato concreto. Una variabile associa un nome a un valore o a una posizione che lo contiene. Una costante esprime un valore che il programma non dovrebbe modificare nel contesto previsto.

Il tipo stabilisce l'insieme dei valori ammessi e le operazioni disponibili. Numeri, valori booleani, caratteri e stringhe hanno comportamenti diversi. Una conversione può essere esplicita o implicita; in entrambi i casi può fallire o perdere informazione.

Un'espressione combina valori, variabili e operatori e produce un risultato. L'assegnazione modifica lo stato associando un valore calcolato a una variabile.

SEQUENZA, SELEZIONE E ITERAZIONE

La sequenza esegue istruzioni nell'ordine indicato. La selezione sceglie un percorso in base a una condizione. L'iterazione ripete un blocco.

```
SE punteggio >= soglia
    esito ← "AMMESSO"
ALTRIMENTI
    esito ← "NON AMMESSO"
FINE SE
```

Un ciclo PER OGNI è adatto quando si visita una collezione. Un ciclo MENTRE è utile quando il numero di ripetizioni dipende da una condizione:

```
i ← 0
MENTRE i < lunghezza(pratiche)
    elabora(pratiche[i])
    i ← i + 1
FINE MENTRE
```

L'errore classico è dimenticare $i \leftarrow i + 1$, oppure usare $i \leq \text{lunghezza}(\text{pratiche})$ quando gli indici validi terminano prima. Il risultato può essere un ciclo infinito o un accesso fuori limite.

SCEGLIERE IL COSTRUTTO

Il paradigma orienta la soluzione, ma non sostituisce l'analisi del problema. Un flusso procedurale può essere chiaro per elaborare una sequenza di pratiche; gli oggetti aiutano quando dati e comportamenti evolvono insieme; una trasformazione funzionale è utile quando si vuole limitare lo stato modificabile. Nei linguaggi multiparadigma queste scelte possono convivere.

La selezione è appropriata quando il programma sceglie fra alternative. Le condizioni devono coprire tutti i casi previsti e, quando necessario, essere mutuamente esclusive. Un ramo finale generico può intercettare dati inattesi, ma non deve nasconderli senza una decisione esplicita.

L'iterazione si sceglie in base a ciò che è noto. PER OGNI esprime bene la visita di tutti gli elementi; MENTRE è adatto quando la ripetizione termina al verificarsi di una condizione. Il controllo deve progredire verso la terminazione: indice, elemento corrente o stato devono cambiare in modo coerente.

Applicazione. Per validare una lista di identificativi, il candidato può usare un ciclo, una selezione per distinguere valori validi e non validi e un accumulatore per contare gli errori. La risposta deve dichiarare tipo dei dati, condizione di validità e comportamento sull'input vuoto.

Gli operatori logici combinano condizioni. AND richiede che entrambe siano vere, OR che ne sia vera almeno una, NOT nega il risultato. Le parentesi rendono esplicita la precedenza e riducono ambiguità. Nei quiz, negare una condizione composta è una fonte frequente di errore.

Una conversione di tipo può fallire. Prima di trasformare una stringa in numero, il programma verifica il formato oppure gestisce l'errore. Assumere che ogni input sia corretto sposta il difetto dal requisito all'esecuzione.

Nel tracciamento, ogni espressione va valutata con i valori correnti, prima dell'eventuale assegnazione. Confondere il vecchio e il nuovo stato produce errori soprattutto quando più variabili vengono aggiornate nello stesso ciclo. Una tabella con una colonna per variabile rende visibile l'ordine.

MICRO-VERIFICA: CONTROLLO DEL FLUSSO

Devi esaminare tutte le pratiche e contare quelle con importo negativo. Quale struttura di controllo scegli e quale paradigma stai usando?

Soluzione: un ciclo PER OGNI visita le pratiche; al suo interno una selezione controlla l'importo e un contatore viene aggiornato. È una soluzione imperativa o procedurale: modifica uno stato secondo istruzioni ordinate.

3.3 Funzioni e procedure

Una funzione incapsula un calcolo e restituisce un valore. Una procedura, nel lessico didattico, esegue un'azione senza che il valore restituito sia il suo scopo principale. I linguaggi adottano terminologie diverse.

I parametri compaiono nella definizione; gli argomenti sono i valori forniti nella chiamata.

```
FUNZIONE massimo(a, b)
  SE a >= b
    RESTITUISCI a
  ALTRIMENTI
    RESTITUISCI b
  FINE SE
FINE FUNZIONE
```

L'ambito stabilisce dove un nome è visibile. Una variabile locale appartiene alla funzione o al blocco che la dichiara; una variabile condivisa amplia le dipendenze e può rendere più difficile capire chi modifica lo stato.

La ricorsione si ha quando una funzione richiama sé stessa su un problema più piccolo. Richiede un caso base che interrompa le chiamate. Senza caso base, o senza avvicinamento a esso, la computazione non termina correttamente.

CONTRATTO E SCOMPOSIZIONE

Una buona funzione ha uno scopo riconoscibile. Il nome, i parametri e il valore restituito formano un contratto leggibile: chi la usa deve sapere quali dati fornire, quale risultato attendersi e quali condizioni di errore gestire. Se una funzione modifica anche stato esterno, l'effetto deve essere esplicito.

Scomporre un algoritmo riduce il numero di dettagli da considerare nello stesso momento. Invece di mescolare lettura, validazione, classificazione e conteggio, si possono definire funzioni separate e comporle. La scomposizione aiuta test, riuso e manutenzione, ma frammentare ogni istruzione in una funzione distinta rende il flusso difficile da seguire.

Una funzione di validazione può restituire un valore booleano, mentre una funzione di conteggio governa l'iterazione e usa la prima. Il chiamante non deve conoscere i dettagli della validazione, ma dipende dal suo contratto. Cambiare la regola in un solo punto evita duplicazioni e risultati incoerenti.

PARAMETRI, PASSAGGIO E STATO

I linguaggi differiscono nel modo in cui trasferiscono argomenti e riferimenti. In prova va distinto il valore fornito alla chiamata dall'oggetto eventualmente condiviso. Modificare una struttura ricevuta può produrre effetti visibili al chiamante; creare e restituire una nuova struttura rende il flusso più esplicito, ma può richiedere memoria aggiuntiva.

Le variabili globali facilitano l'accesso condiviso, ma aumentano l'accoppiamento. Una funzione che dipende da uno stato nascosto è più difficile da testare perché lo stesso input esplicito può produrre risultati diversi. Passare le dipendenze come parametri rende il comportamento più leggibile.

La ricorsione usa lo stack delle chiamate. Ogni invocazione conserva parametri e stato locale finché la chiamata più interna termina. Una soluzione ricorsiva può rispecchiare bene strutture come alberi, ma non è automaticamente più efficiente di un ciclo: vanno considerati profondità, memoria e chiarezza.

Errore tipico. Confondere parametro e argomento, oppure descrivere una procedura come se fosse priva di effetti solo perché non restituisce un valore. Nell'orale va chiarito sia il risultato sia lo stato eventualmente modificato.

COESIONE, DIPENDENZE E TEST

Una funzione è coesa quando le sue istruzioni collaborano a un solo compito comprensibile. Una funzione che valida dati, scrive file e aggiorna statistiche contiene responsabilità diverse: separarle permette di verificare ogni parte con input mirati. La separazione non impone un numero fisso di righe; dipende dal contratto.

Le dipendenze esplicite facilitano il test. Se una funzione riceve i dati e restituisce un risultato, si possono preparare casi ordinari, vuoti e non validi. Se legge direttamente una risorsa globale, il test deve anche predisporre e ripristinare quello stato.

Per una funzione ricorsiva si provano almeno il caso base, il primo caso ricorsivo e un input abbastanza grande da rivelare errori nella progressione. La risposta concorsuale deve spiegare quale parte del problema si riduce a ogni chiamata.

Una funzione troppo dipendente dal contesto è difficile da riusare. Parametri essenziali, risultato chiaro ed effetti dichiarati permettono invece di sostituire l'implementazione senza cambiare chi la chiama. Questo è il valore pratico dell'astrazione, non soltanto una scelta di stile.

3.4 Strutture lineari e associative

Una struttura dati organizza valori e relazioni per rendere possibili determinate operazioni. La scelta va rapportata alle esigenze di accesso, ricerca, inserimento, cancellazione, ordine e memoria.

ARRAY E LISTE

Un array memorizza una sequenza indicizzata. Nel modello più comune consente accesso diretto tramite indice. Inserire un elemento nel mezzo può richiedere lo spostamento di quelli successivi.

Una lista rappresenta una sequenza e può essere realizzata in modi diversi. In una lista concatenata, ogni nodo collega il successivo; raggiungere l'elemento in posizione k richiede normalmente di attraversare i precedenti. Inserimenti e cancellazioni possono essere convenienti quando il punto è già noto.

Dire che “la lista è più veloce dell'array” non ha senso senza specificare operazione e rappresentazione.

PILE E CODE

La pila segue la regola LIFO: l'ultimo elemento inserito è il primo rimosso. Operazioni tipiche sono push, pop e lettura della cima. È utile per annullamento, chiamate di funzione e attraversamenti.

La coda segue la regola FIFO: il primo elemento inserito è il primo servito. È adatta a richieste in attesa, messaggi e visite per livelli. Una coda di priorità, invece, estrae secondo una priorità e non secondo il solo ordine di arrivo.

INSIEMI E MAPPE

Un insieme conserva elementi senza duplicati secondo la nozione di uguaglianza adottata. Serve per appartenenza, eliminazione dei duplicati e operazioni insiemistiche.

Una mappa o dizionario associa chiavi a valori. Può collegare l'identificativo di una pratica al suo stato. Le chiavi devono rispettare i requisiti della rappresentazione scelta; i costi delle operazioni dipendono dall'implementazione, per esempio tabella hash o albero.

OPERAZIONI E CRITERI DI SCELTA

La struttura si sceglie partendo dalle operazioni dominanti. Un array è adatto quando serve spesso l'accesso per posizione; una lista concatenata può favorire inserimenti quando il punto è già noto; una coda mantiene l'ordine di arrivo; una pila recupera per primo l'ultimo elemento inserito; una mappa cerca per chiave; un insieme controlla appartenenza e duplicati.

Esigenza	Struttura candidata	Domanda da verificare
Accesso per indice	array	la dimensione cambia spesso?
Ordine di arrivo	coda	esistono priorità diverse?
Annullamento dell'ultima azione	pila	quante operazioni vanno conservate?
Ricerca per identificativo	mappa	come sono gestite chiavi e collisioni?
Valori unici	insieme	quale regola stabilisce l'uguaglianza?

La tabella propone strutture candidate. L'implementazione concreta ne determina costi e vincoli. Una mappa basata su hash e una basata su albero offrono la stessa relazione chiave-valore, ma hanno proprietà differenti su ordinamento e prestazioni.

MUTABILITÀ, DUPLICATI E ORDINE

Una struttura mutabile può essere modificata dopo la creazione; una struttura immutabile produce una nuova versione a ogni trasformazione. La mutabilità può ridurre copie, ma richiede attenzione quando più parti del programma condividono lo stesso oggetto. L'immutabilità semplifica il ragionamento sugli effetti, con possibili costi di rappresentazione.

Duplicati e ordine sono proprietà separate. Una lista può contenere duplicati e conservarne la sequenza; un insieme elimina duplicati ma non garantisce necessariamente un ordine significativo; una mappa richiede chiavi uniche, mentre i valori possono ripetersi. Il candidato deve dichiarare quali proprietà servono al problema.

Mini-caso. Un ufficio deve evitare di elaborare due volte lo stesso identificativo, conservare l'ordine di arrivo e recuperare rapidamente lo stato. Una singola struttura potrebbe non soddisfare tutto. Una lista o coda conserva l'ordine, un insieme segnala duplicati e una mappa associa identificativi e stati. La soluzione va motivata anche per il costo di mantenere più rappresentazioni coerenti.

Anche inserimento e cancellazione richiedono una domanda precisa: si conosce già la posizione o bisogna prima cercarla? In una lista concatenata modificare i collegamenti può essere economico quando il nodo è noto, ma trovarlo può richiedere una scansione. Il costo complessivo include entrambe le fasi.

Alcune strutture dinamiche riservano capacità aggiuntiva per evitare una riallocazione a ogni inserimento. Una singola espansione può essere costosa, mentre il costo medio distribuito su molte operazioni resta contenuto. Anche qui va dichiarata l'implementazione considerata.

3.5 Alberi, grafi, ricerca e ordinamento

ALBERI E GRAFI

Un albero descrive una struttura gerarchica senza cicli nel modello radicato. Ogni nodo, tranne la radice, ha un genitore. Alberi di ricerca e heap aggiungono proprietà specifiche.

Un grafo è formato da vertici e archi e rappresenta relazioni più generali: collegamenti fra sistemi, dipendenze o percorsi. Può essere orientato o non orientato, pesato o non pesato. Il capitolo introduce la struttura; gli algoritmi avanzati sui grafi restano fuori perimetro.

RICERCA

La ricerca lineare esamina gli elementi uno dopo l'altro finché trova la chiave o termina la collezione.

```
FUNZIONE cercaLineare(elementi, chiave)
  PER i DA 0 A lunghezza(elementi) - 1
    SE elementi[i] = chiave
      RESTITUISCI i
    FINE SE
  FINE PER
  RESTITUISCI -1
FINE FUNZIONE
```

Nel caso peggiore visita tutti gli n elementi: il tempo cresce linearmente, $O(n)$.

La ricerca binaria usa una collezione ordinata e confronta la chiave con l'elemento centrale. A ogni passo elimina circa metà dell'intervallo.

```
FUNZIONE cercaBinaria(elementi, chiave)
    basso ← 0
    alto ← lunghezza(elementi) - 1
    MENTRE basso ≤ alto
        medio ← parteIntera((basso + alto) / 2)
        SE elementi[medio] = chiave
            RESTITUISCI medio
        ALTRIMENTI SE elementi[medio] < chiave
            basso ← medio + 1
        ALTRIMENTI
            alto ← medio - 1
    FINE SE
    FINE MENTRE
    RESTITUISCI -1
FINE FUNZIONE
```

Il numero di confronti cresce come $O(\log n)$. Applicare questo algoritmo a dati non ordinati rende il risultato inaffidabile: l'ordinamento è un prerequisito, non un dettaglio.

ORDINAMENTO

L'ordinamento dispone gli elementi secondo una relazione coerente. Due algoritmi possono produrre lo stesso ordine ma avere costi e proprietà differenti.

L'insertion sort costruisce una parte ordinata inserendo ogni nuovo elemento nella posizione corretta. È semplice e può funzionare bene su collezioni piccole o quasi ordinate. Nel caso peggiore, confronti e spostamenti crescono quadraticamente: $O(n^2)$.

Il merge sort divide la collezione, ordina ricorsivamente le parti e le fonde. Il suo tempo cresce come $O(n \log n)$ nei casi tipicamente analizzati, ma la fusione richiede memoria aggiuntiva nella rappresentazione comune.

L'ordine di crescita è uno dei criteri di scelta. Vanno considerati anche la dimensione, la memoria disponibile, lo stato iniziale dei dati, la stabilità richiesta e il costo effettivo delle operazioni.

COLLEGARE STRUTTURA E ALGORITMO

Ricerca e ordinamento non sono indipendenti dalla rappresentazione. La ricerca binaria richiede accesso efficiente all'elemento centrale e dati ordinati; su una lista concatenata, raggiungere il centro può costare attraversamenti ripetuti. Un albero di ricerca conserva una relazione d'ordine durante gli inserimenti, ma le prestazioni dipendono dalla sua forma e dalle proprietà dell'implementazione.

Nei grafi, la ricerca diventa visita delle relazioni. Una coda sostiene una visita per livelli, mentre una pila o la ricorsione sostengono una visita in profondità. Il capitolo non richiede gli algoritmi avanzati, ma questa connessione mostra perché la struttura dati influenza direttamente il procedimento.

Domanda-trappola. Ordinare prima rende sempre migliore la ricerca? No. Se la raccolta viene interrogata una sola volta, il costo dell'ordinamento può superare il risparmio. Se invece molte ricerche seguono e i dati cambiano poco, mantenere un ordine o un indice può essere conveniente.

Un ordinamento è stabile se conserva l'ordine relativo degli elementi con la stessa chiave. La proprietà conta quando esistono criteri successivi: dopo aver ordinato pratiche per data, un ordinamento stabile per stato può preservare l'ordine cronologico all'interno di ogni stato.

La scelta considera anche il caso peggiore, la memoria aggiuntiva e lo stato iniziale dei dati. Insertion sort può essere adatto a pochi elementi quasi ordinati; merge sort offre crescita $O(n \log n)$ ma usa spazio per la fusione nella versione comune. Nessuna etichetta sostituisce il profilo del carico.

Nel confronto orale è utile dichiarare anche l'output in caso di insuccesso. Una ricerca può restituire una posizione speciale, un valore assente o un errore controllato. La convenzione deve evitare di confondere «non trovato» con una posizione valida.

3.6 Complessità e output concorsuale

La complessità temporale descrive come cresce il lavoro al crescere della dimensione n dell'input. La complessità spaziale totale comprende input e memoria usata dall'algoritmo; lo spazio ausiliario esclude l'input. In questi esercizi, quando si valuta memoria aggiuntiva, si dichiara espressamente lo spazio ausiliario.

La notazione O grande esprime un limite superiore asintotico. Tralascia costanti e termini di ordine inferiore per mettere in evidenza la crescita dominante. Non misura direttamente i secondi impiegati su una macchina.

Crescita	Esempio intuitivo
$O(1)$	accesso diretto a una posizione nota
$O(\log n)$	ricerca binaria su dati ordinati
$O(n)$	scansione completa
$O(n \log n)$	merge sort
$O(n^2)$	doppio confronto fra molte coppie

Nel modello $T(n)=a \cdot n$, $a>0$, o $T(n)=a \cdot n^2$, considerando il termine dominante, se n raddoppia, un algoritmo lineare tende a raddoppiare il lavoro dominante; uno quadratico tende a quadruplicarlo. L'analisi va però riferita a un modello e a un'operazione dichiarati.

La notazione asintotica confronta la crescita per input grandi e trascura fattori costanti. Due algoritmi entrambi $O(n)$ possono avere tempi diversi su input reali; il solo limite $O(n)$ non dice che il lavoro cresca esattamente in modo lineare: anche un costo costante è $O(n)$. Per un ordine stretto si usa $\Theta(n)$, con limite superiore e inferiore dello stesso ordine, non che le implementazioni siano equivalenti. Per input piccoli, semplicità e costi fissi possono pesare più dell'ordine di crescita.

Si distinguono caso migliore, medio e peggiore quando hanno significato. La ricerca lineare trova subito una chiave posta all'inizio, ma nel caso peggiore visita tutta la collezione. Dichiarare soltanto $O(n)$ senza specificare scenario e operazione può nascondere l'ipotesi usata.

Lo spazio ausiliario considera la memoria aggiuntiva rispetto all'input. Una scansione di un array di n elementi con un solo contatore usa $\Theta(n)$ spazio totale e $\Theta(1)$ ausiliario. Un algoritmo può ridurre il tempo creando una struttura ausiliaria, oppure risparmiare memoria accettando più passaggi. Il compromesso va motivato rispetto ai vincoli del problema.

DALLA TRACCIA ALLA RISPOSTA

In un esercizio tecnico, il candidato procede in ordine: definisce input e output, dichiara precondizioni, sceglie struttura e algoritmo, tratta i casi limite, stima tempo e spazio, quindi verifica con una trace table. Nell'orale può esporre la stessa sequenza in forma breve.

Nel debug, cambiare istruzioni a tentativi rende difficile capire la causa. Si riproduce invece il difetto con un input minimo, si confrontano stato atteso e stato osservato, si localizza il primo passaggio divergente e si corregge la causa. Un nuovo test conserva l'evidenza del caso scoperto.

La stima asintotica non sostituisce la misurazione, e la misurazione non sostituisce l'analisi. La prima anticipa la crescita, la seconda verifica una specifica implementazione su dati e ambiente dichiarati. Una risposta completa tiene separati i due piani.

CASO GUIDATO: CLASSIFICARE LE PRATICHE

Un ufficio deve separare pratiche complete, incomplete e con stato non riconosciuto. Ogni pratica ha identificativo e stato.

Una soluzione usa tre liste:

```
complete ← lista vuota
incomplete ← lista vuota
anomalie ← lista vuota

PER OGNI pratica IN pratiche
  SE pratica.stato = "COMPLETA"
    aggiungi pratica A complete
  ALTRIMENTI SE pratica.stato = "INCOMPLETA"
    aggiungi pratica A incomplete
  ALTRIMENTI
    aggiungi pratica A anomalie
  FINE SE
FINE PER
```

L'algoritmo visita ogni pratica una volta: il tempo cresce come $O(n)$. Le tre liste occupano complessivamente spazio proporzionale al numero di pratiche. La categoria anomalie evita di perdere silenziosamente dati inattesi.

DOMANDA DA COMMISSARIO

Come sceglie una struttura dati per risolvere un problema?

Parto dalle operazioni richieste: accesso per posizione, ricerca per chiave, inserimento, cancellazione, mantenimento dell'ordine o gestione FIFO/LIFO. Considero poi dimensione, frequenza delle operazioni, memoria e implementazione disponibile. La scelta va motivata sul carico: una struttura efficace per accessi indicizzati può non esserlo per inserimenti frequenti nel mezzo.

DOMANDA-TRAPPOLA

La ricerca binaria è sempre preferibile alla ricerca lineare?

No. Richiede dati ordinati e un accesso compatibile alla posizione centrale. Se la collezione non è ordinata e deve essere cercata una sola volta, il costo dell'ordinamento può superare il beneficio. La ricerca lineare resta inoltre adatta a raccolte piccole o prive di accesso diretto.

ERRORE TIPICO

Associare una complessità alla struttura senza indicare operazione e rappresentazione. "La mappa è $O(1)$ " è incompleto: alcune implementazioni offrono costo medio costante per certe operazioni, altre hanno costi logaritmici, e il caso peggiore può differire.

▣ Verifica

ESERCIZIO 1

Traccia il valore di somma:

```
somma ← 0
PER OGNI x IN [3, 1, 4]
    somma ← somma + x
FINE PER
```

Soluzione: i valori successivi sono 3, 4 e 8. L'output finale è 8.

ESERCIZIO 2

Quale struttura useresti per servire richieste nell'ordine di arrivo?

Soluzione: una coda FIFO. Una pila invertirebbe l'ordine; una coda di priorità seguirebbe un criterio diverso dall'arrivo.

QUIZ 1

Quale affermazione è corretta?

- A. La ricerca binaria funziona correttamente su qualunque sequenza.
- B. Merge sort ha crescita quadratica in ogni caso.
- C. Una funzione ricorsiva richiede un caso base raggiungibile.
- D. Un insieme conserva necessariamente duplicati e ordine di inserimento.

Risposta corretta: C. A ignora l'ordinamento, B attribuisce una crescita errata, D contraddice la funzione dell'insieme.

QUIZ 2

Quale affermazione descrive una preconditione della ricerca binaria?

- A. La collezione deve essere ordinata secondo il criterio di confronto.
- B. La collezione deve contenere duplicati.
- C. L'algoritmo deve usare una pila.
- D. La chiave deve trovarsi nella prima metà.

Risposta corretta: A. Senza ordinamento coerente, eliminare metà dell'intervallo dopo un confronto non è giustificato.

QUIZ 3

Quale struttura segue la regola FIFO?

- A. Pila.
- B. Coda.
- C. Insieme.
- D. Albero binario.

Risposta corretta: B. Nella coda il primo elemento inserito è il primo servito; la pila segue invece la regola LIFO.

QUIZ 4

Che cosa distingue un parametro da un argomento?

- A. Il parametro compare nella definizione; l'argomento è fornito nella chiamata.
- B. Il parametro è sempre globale.
- C. L'argomento è sempre una stringa.
- D. Non esiste alcuna differenza.

Risposta corretta: A. La distinzione descrive due ruoli nello stesso meccanismo di chiamata.

QUIZ 5

Nel modello indicato, con a positivo e costante, quale termine dominante quadruplica quando n raddoppia?

- A. a .
- B. $a \cdot \log_2(n)$.
- C. $a \cdot n$.
- D. $a \cdot n^2$.

Risposta corretta: D. $a \cdot (2n)^2 = 4a \cdot n^2$. Il modello quadratico è $\Theta(n^2)$, ma il solo limite $O(n^2)$ non dimostra questo rapporto fra costi: qui è specificata la funzione dominante.

QUIZ 6

Quale affermazione sulla ricorsione è corretta?

- A. È sempre più veloce di un ciclo.
- B. Richiede un caso base raggiungibile.
- C. Non usa memoria per le chiamate.
- D. Può risolvere soltanto problemi numerici.

Risposta corretta: B. Il caso base e il progresso verso di esso impediscono una sequenza indefinita di chiamate.

Checklist finale

Competenza	Riconosco	Spiego	Applico
Distinguere problema, algoritmo e programma	☐	☐	☐
Tracciare pseudocodice	☐	☐	☐
Usare selezione e iterazione	☐	☐	☐
Spiegare parametri, ambito e ricorsione	☐	☐	☐
Scegliere una struttura dati	☐	☐	☐
Confrontare ricerca lineare e binaria	☐	☐	☐
Confrontare insertion sort e merge sort	☐	☐	☐
Stimare tempo e spazio	☐	☐	☐
Individuare casi limite	☐	☐	☐

Da sapere in 5 righe

- Un algoritmo collega input, vincoli, passi e output.
- Lo pseudocodice rende verificabile la logica senza imporre una sintassi.
- La struttura dati si sceglie in base alle operazioni.
- Ricerca binaria e complessità richiedono prerequisiti e modello dichiarati.
- Test, trace table e casi limite riducono gli errori di ragionamento.

Riferimenti professionali essenziali

- Il Metodo BANDO, capitolo 10, paragrafo 8, per i prerequisiti di programmazione.
- Documentazione ufficiale Python per tipi, controllo, funzioni e strutture fondamentali, usata soltanto per concetti trasferibili.
- Pat Morin, Open Data Structures, versione Python, § 1.3 e capitoli sulle strutture: <https://opendatastructures.org/ods-python/>.
- Programma del singolo bando, decisivo per linguaggi, profondità e formato della prova.

Basi dati, SQL/NoSQL e qualità del dato

B
BANDO

A
AREE

N
NUCLEI

D
DIARIO

O
OUTPUT

La struttura di una base dati stabilisce quali informazioni si possono registrare, quali errori vengono bloccati e con quale difficoltà si ottiene una risposta. Nelle prove tecniche il candidato può dover leggere uno schema, scegliere una chiave, completare una query, riconoscere un'anomalia o motivare l'uso di un modello relazionale o NoSQL.

Il caso di riferimento segue la gestione delle pratiche di un ente. Lo stesso scenario collega requisiti, schema, query e controlli di qualità lungo tutto il capitolo.

Obiettivo e confine con il VOL-01

Il VOL-01, capitolo 10, § 7, definisce database, DBMS, tabella, riga, colonna, chiave, relazione e query. Introduce anche i principali comandi SQL. Queste nozioni costituiscono i prerequisiti.

Dopo lo studio del capitolo dovrai saper:

- tradurre un requisito informativo in entità, attributi e relazioni;
- riconoscere chiavi, cardinalità e vincoli;
- diagnosticare anomalie e applicare la normalizzazione di base;
- leggere e scrivere query con filtri, join e aggregazioni;
- spiegare transazioni, proprietà ACID e isolamento;
- distinguere indice, chiave e vincolo;
- confrontare modello relazionale e principali famiglie NoSQL;
- definire controlli misurabili di qualità del dato;
- motivare ruoli e privilegi di accesso.

Le API e l'interoperabilità applicativa sono sviluppate nel capitolo 6. Cybersecurity, IAM e logging sono trattati nei capitoli 8 e 9. Ruoli di data governance, metadati, cataloghi e open data appartengono al capitolo 10.

Mappa BANDO delle basi dati

Passaggio	Domanda da porsi	Azione
B — Bando	Sono richiesti concetti, SQL, modellazione o uno specifico DBMS?	Separare il nucleo generale dalla sintassi di prodotto.
A — Aree	La voce riguarda modello, query, transazioni, prestazioni, NoSQL, qualità o sicurezza?	Collocarla nell'area corretta.

Passaggio	Domanda da porsi	Azione
N — Nuclei	Quale operazione devo spiegare o realizzare?	Preparare uno schema minimo e query verificabili.
D — Diario	L'errore nasce da modello, sintassi, cardinalità, dati o permessi?	Registrare causa, effetto e correzione.
O — Output	La prova chiede quiz, SQL, schema, orale o diagnosi?	Allenare il formato richiesto, non soltanto le definizioni.

Se il bando nomina PostgreSQL, Oracle, SQL Server, MySQL o un prodotto NoSQL, affianca la relativa documentazione. Gli esempi del capitolo usano un sottoinsieme semplice di SQL e non presumono che ogni DBMS implementi nello stesso modo tutte le funzioni.

4.1 Dai requisiti al modello relazionale

Un modello dei dati stabilisce come rappresentare informazioni, relazioni, vincoli e operazioni. Lo schema descrive la struttura prevista; l'istanza è l'insieme dei dati presenti in un determinato momento.

Consideriamo questo requisito:

NOTA

Ogni pratica ha un identificativo, una data di presentazione, uno stato e un ufficio responsabile. Un ufficio gestisce più pratiche. Una pratica può riguardare più soggetti e uno stesso soggetto può comparire in più pratiche.

Le entità principali sono Pratica, Ufficio e Soggetto. Gli attributi descrivono le entità: per Pratica, per esempio, identificativo, data e stato. Le relazioni esprimono i collegamenti.

CARDINALITÀ

La cardinalità indica quante istanze possono partecipare a una relazione:

- uno-a-uno: a un'istanza corrisponde al massimo un'altra istanza;
- uno-a-molti: un ufficio può gestire molte pratiche, mentre ogni pratica ha un ufficio responsabile;
- multi-a-molti: una pratica può coinvolgere più soggetti e un soggetto può comparire in più pratiche.

Nel modello relazionale, una relazione multi-a-molti richiede normalmente una tabella associativa. PraticaSoggetto può contenere le chiavi di Pratica e Soggetto, oltre a eventuali attributi del collegamento, come il ruolo del soggetto nella pratica.

IL MODELLO RELAZIONALE

Il modello relazionale organizza i dati in relazioni, rappresentate operativamente come tabelle. Una riga, o tupla, descrive un'occorrenza; una colonna, o attributo, rappresenta una proprietà; il dominio delimita i valori ammessi.

Uno schema essenziale può essere descritto così:

```
Ufficio(  
    id_ufficio PK,  
    nome UNIQUE  
)  
  
Pratica(  
    id_pratica PK,  
    data_presentazione,  
    stato,  
    id_ufficio NOT NULL FK → Ufficio.id_ufficio  
)  
  
Soggetto(  
    id_soggetto PK,  
    denominazione  
)  
  
PraticaSoggetto(  
    id_pratica FK → Pratica.id_pratica,  
    id_soggetto FK → Soggetto.id_soggetto,  
    ruolo,  
    PK(id_pratica, id_soggetto)  
)
```

CHIAVI E VINCOLI

Una chiave candidata è un insieme minimo di attributi capace di identificare univocamente una riga. Fra le chiavi candidate se ne sceglie una come chiave primaria. Una chiave formata da più attributi è composta.

La chiave esterna collega una tabella a una chiave della tabella referenziata. Il vincolo impedisce, secondo le regole dichiarate, che una pratica faccia riferimento a un ufficio inesistente.

I vincoli hanno funzioni diverse:

Vincolo	Funzione
PRIMARY KEY	identifica univocamente la riga e non ammette valori nulli
FOREIGN KEY	tutela il collegamento fra tabelle
UNIQUE	impedisce duplicati nella chiave dichiarata
NOT NULL	rende obbligatorio un valore
CHECK	impone una condizione sui valori

NULL indica l'assenza o la non disponibilità di un valore, non il numero zero e non una stringa vuota. I confronti con valori nulli seguono una logica specifica: per verificarli si usa normalmente IS NULL o IS NOT NULL, non = NULL.

DAL REQUISITO ALLO SCHEMA

La modellazione parte dalle domande alle quali il sistema deve rispondere e dalle regole che non può violare. Da qui emergono le entità, cioè gli oggetti di interesse, e i loro attributi pertinenti. Relazioni e cardinalità diventano quindi tabelle, chiavi e vincoli. Partire direttamente dalle colonne rischia di incorporare nello schema abitudini dell'ufficio non dichiarate o dati che appartengono a concetti diversi.

La cardinalità esprime quante istanze possono partecipare a una relazione. Se un ufficio gestisce molte pratiche e ogni pratica appartiene a un solo ufficio, la relazione è uno-a-molti e la chiave esterna può stare in Pratica. Se una pratica coinvolge molti soggetti e un soggetto può comparire in molte pratiche, la relazione è molti-a-molti e richiede una tabella associativa. Questa tabella può avere attributi propri, come ruolo, data di partecipazione o ordine di firma: non è un semplice espediente tecnico.

Scegli la chiave valutando stabilità, unicità e semplicità d'uso. Un identificativo artificiale può evitare che cambiamenti di denominazione modifichino i collegamenti; una chiave naturale può invece esprimere una regola del dominio. La scelta non elimina la necessità di proteggere con UNIQUE gli altri identificatori che devono restare univoci. Nella prova, uno schema convincente dichiara anche l'opzionalità: una relazione obbligatoria richiede un vincolo diverso da una relazione che ammette temporaneamente l'assenza del collegamento.

4.2 Vincoli, NULL e normalizzazione

Una tabella unica con dati della pratica, nome dell'ufficio e tutti i soggetti ripetuti può sembrare comoda. Produce però ridondanza e anomalie:

- inserimento: non si può registrare un nuovo ufficio finché non esiste una pratica;
- aggiornamento: cambiare il nome di un ufficio richiede modifiche in molte righe;
- cancellazione: eliminando l'ultima pratica si può perdere anche l'unica informazione sull'ufficio.

La normalizzazione organizza le relazioni per ridurre dipendenze indesiderate e anomalie, preservando i collegamenti necessari.

PRIMA FORMA NORMALE

In prima forma normale ogni posizione contiene un valore appartenente al dominio previsto, non un elenco ripetuto nello stesso campo. Un campo soggetti = "12, 18, 31" rende difficili vincoli, join e aggiornamenti. La tabella PraticaSoggetto rappresenta ogni collegamento con una riga.

SECONDA FORMA NORMALE

La seconda forma normale riguarda le tabelle con chiave composta: ogni attributo non chiave deve dipendere dall'intera chiave. In PraticaSoggetto(id_pratica, id_soggetto, denominazione_soggetto, ruolo), la denominazione dipende soltanto da id_soggetto. Va quindi collocata in Soggetto.

TERZA FORMA NORMALE

In terza forma normale, gli attributi non chiave non devono dipendere transitivamente dalla chiave attraverso un altro attributo non chiave. Se Pratica contenesse sia id_ufficio sia nome_ufficio, il nome dipenderebbe dall'identificativo dell'ufficio. Separare Ufficio evita ripetizioni.

Il progetto determina fin dove spingere la normalizzazione. Requisiti, carico e DBMS possono giustificare una denormalizzazione, purché sia motivata e accompagnata da controlli che mantengano coerenti le copie.

DIPENDENZE E PERCORSO DI NORMALIZZAZIONE

Una dipendenza funzionale esprime che il valore di uno o più attributi determina il valore di altri. Nell'insieme delle pratiche, `id_ufficio` determina il nome dell'ufficio se ogni identificativo designa un solo ufficio. Riconoscere questa dipendenza spiega perché ripetere il nome in ogni pratica crea ridondanza: il dato dipende dall'ufficio, non dalla singola pratica.

Normalizzare significa analizzare le dipendenze, non dividere tabelle finché diventano piccole. Per ogni attributo si individua la chiave dalla quale dipende. La 1NF elimina gruppi ripetuti e valori multipli nella stessa posizione. La 2NF rimuove dipendenze soltanto da una parte di una chiave composta. La 3NF separa dipendenze transitive fra attributi non chiave. Ogni separazione conserva il collegamento mediante una chiave e deve permettere di ricostruire le informazioni necessarie con join corrette.

Considera `Assegnazione(id_pratica, id_dipendente, nome_dipendente, ruolo, id_ufficio, nome_ufficio)`. Se la chiave è composta da `pratica` e `dipendente`, `nome_dipendente` dipende solo da `id_dipendente`: è un problema di 2NF. Se il dipendente appartiene a un ufficio e il nome dell'ufficio dipende dall'identificativo dell'ufficio, conservarlo nella stessa tabella introduce una dipendenza transitiva. Le tabelle `Dipendente`, `Ufficio` e `Assegnazione` rendono esplicite le responsabilità dei dati.

VINCOLI E DATI GIÀ ESISTENTI

Aggiungere un vincolo a una base popolata richiede una ricognizione. Se esistono chiavi duplicate, riferimenti orfani o stati fuori dominio, il DBMS può rifiutare il vincolo oppure l'amministratore può essere tentato di cancellare dati senza comprenderli. Il lavoro parte dalla misura delle anomalie, ne individua la causa, definisce una regola di correzione e conserva evidenza delle trasformazioni.

Anche `NULL` va deciso nel modello. Può significare dato non ancora noto, non applicabile o non acquisito: significati diversi che una sola rappresentazione rischia di confondere. Rendere un campo obbligatorio impedisce assenze future, ma non garantisce che il valore inserito sia vero. Vincoli strutturali, controlli applicativi e verifiche sul processo si completano a vicenda.

La denormalizzazione può ridurre join o sostenere letture frequenti, ma introduce copie da sincronizzare. La decisione richiede query critiche, volume, frequenza degli aggiornamenti, meccanismo di allineamento e controllo delle divergenze. “È più veloce” senza carico e misura non è una motivazione tecnica.

Dopo la trasformazione, collauda lo schema con inserimenti, aggiornamenti e cancellazioni rappresentativi. Il controllo verifica che i vincoli blocchino gli stati impossibili, che le join ricostruiscano l'informazione richiesta e che nessuna dipendenza significativa sia stata persa.

4.3 SQL per interrogare e modificare

SQL è un linguaggio dichiarativo: la query specifica il risultato richiesto, mentre il DBMS sceglie un piano di esecuzione compatibile.

SELEZIONE, FILTRO E ORDINAMENTO

```
SELECT id_pratica, data_presentazione, stato
FROM Pratica
WHERE stato = 'DA_VERIFICARE'
ORDER BY data_presentazione;
```

SELECT sceglie le colonne del risultato, FROM la sorgente, WHERE filtra le righe e ORDER BY ordina l'output. Senza ORDER BY, il risultato non ha un ordine stabile garantito.

Per cercare pratiche senza ufficio associato, se lo schema lo consente:

```
SELECT id_pratica
FROM Pratica
WHERE id_ufficio IS NULL;
```

JOIN

Una join combina righe sulla base di una condizione:

```
SELECT p.id_pratica, p.stato, u.nome AS ufficio
FROM Pratica AS p
JOIN Ufficio AS u
  ON u.id_ufficio = p.id_ufficio
WHERE p.stato = 'DA_VERIFICARE';
```

La INNER JOIN restituisce le combinazioni che soddisfano la condizione. Una LEFT JOIN conserva tutte le righe della tabella a sinistra e completa con valori nulli quando non trova corrispondenza.

Una join errata o incompleta può moltiplicare le righe. Se una pratica è collegata a tre soggetti, la join con PraticaSoggetto produce tre righe per quella pratica. DISTINCT può eliminare duplicati dal risultato, ma non corregge automaticamente un modello o una condizione di join sbagliati.

AGGREGAZIONI

```
SELECT u.nome, COUNT(*) AS numero_pratiche
FROM Ufficio AS u
JOIN Pratica AS p
  ON p.id_ufficio = u.id_ufficio
GROUP BY u.nome
HAVING COUNT(*) >= 10
ORDER BY numero_pratiche DESC;
```

GROUP BY forma gruppi; COUNT calcola un valore per gruppo; HAVING filtra i gruppi dopo l'aggregazione. WHERE, invece, filtra le righe prima del raggruppamento.

SOTTOQUERY

Una sottoquery produce un risultato usato da un'altra query. Per individuare pratiche appartenenti a uffici selezionati:

```
SELECT id_pratica, stato
FROM Pratica
WHERE id_ufficio IN (
    SELECT id_ufficio
    FROM Ufficio
    WHERE nome LIKE 'Servizio%'
);
```

Join e sottoquery possono talvolta esprimere richieste equivalenti, ma leggibilità e piano di esecuzione dipendono dal caso e dal DBMS.

MODIFICA DEI DATI

```
UPDATE Pratica
SET stato = 'ARCHIVIATA'
WHERE id_pratica = 1042;
```

```
DELETE FROM Pratica
WHERE id_pratica = 1042;
```

Omettere WHERE estende l'operazione a tutte le righe della tabella. Una modifica massiva richiede quindi il controllo preventivo della condizione, delle autorizzazioni, degli effetti dei vincoli e delle possibilità di ripristino.

SEMANTICA DELLA QUERY E CARDINALITÀ

Scrivere SQL significa prevedere il significato del risultato, oltre a produrre una sintassi valida. Prima della query si stabilisce quale unità rappresenta ogni riga attesa. Se l'output deve contenere una riga per ufficio, una join con pratiche e soggetti può moltiplicare le righe prima dell'aggregazione. Il conteggio dipende allora dalla domanda: pratiche, soggetti, collegamenti o valori distinti non sono la stessa misura.

L'ordine logico aiuta a leggere una query: si individuano sorgenti e join, si applicano i filtri sulle righe, si formano i gruppi, si calcolano le aggregazioni, si filtrano i gruppi e infine si ordina il risultato. Questa sequenza spiega perché una condizione su COUNT(*) appartiene normalmente a HAVING, mentre una condizione sullo stato della singola pratica appartiene a WHERE.

I valori nulli incidono sulle query. Una INNER JOIN esclude le righe prive di corrispondenza; una LEFT JOIN le conserva dal lato sinistro. Molte funzioni aggregate ignorano i valori nulli, mentre COUNT() conta le righe. Perciò COUNT(colonna) e COUNT() possono produrre risultati diversi. La risposta concorsuale deve dichiarare quale popolazione si vuole misurare.

MODIFICHE CONTROLLATE E PORTABILITÀ

Prima di un UPDATE o DELETE significativo si può eseguire una SELECT con la stessa condizione, controllare il numero e il contenuto delle righe, quindi operare dentro una transazione quando il DBMS e il contesto lo consentono. I vincoli possono impedire una modifica oppure propagare effetti secondo regole configurate: queste conseguenze vanno comprese prima dell'esecuzione.

Il nucleo SQL del capitolo usa costrutti ampiamente condivisi, ma tipi, funzioni, gestione delle date, sintassi di limitazione dei risultati e dettagli delle transazioni possono variare. Se il bando nomina un prodotto, il candidato deve studiarne il dialetto e gli strumenti. La conoscenza generale resta utile per spiegare il procedimento e riconoscere ciò che dipende dall'implementazione.

4.4 Transazioni, concorrenza e indici

Una transazione raggruppa operazioni che devono essere trattate come un'unità. Se l'archiviazione di una pratica richiede sia l'aggiornamento dello stato sia la registrazione di un evento, non è accettabile salvare una sola delle due modifiche.

```
BEGIN;  
  
UPDATE Pratica  
SET stato = 'ARCHIVIATA'  
WHERE id_pratica = 1042;  
  
INSERT INTO EventoPratica(id_pratica, tipo_evento)  
VALUES (1042, 'ARCHIVIAZIONE');  
  
COMMIT;
```

Se una condizione impedisce di completare il lavoro, ROLLBACK annulla le modifiche della transazione non confermata.

Le proprietà ACID esprimono quattro obiettivi:

- atomicità: le operazioni riescono come unità oppure non producono effetti parziali;
- consistenza: la transazione porta la base dati da uno stato valido a un altro, nel rispetto dei vincoli dichiarati;
- isolamento: le transazioni concorrenti non devono interferire in modo incompatibile con il livello scelto;
- durabilità: dopo il commit, gli effetti devono sopravvivere ai guasti contemplati dal sistema.

I DBMS realizzano l'isolamento con livelli e meccanismi diversi, bilanciando concorrenza e fenomeni osservabili. In prova, la risposta parte dal problema: due operazioni simultanee possono leggere o modificare dati sovrapposti e richiedono un controllo adeguato al rischio.

INDICI E PRESTAZIONI

Un indice è una struttura ausiliaria che può permettere al DBMS di trovare determinate righe senza esaminare l'intera tabella. Può sostenere filtri, join o ordinamenti compatibili con la sua struttura.

Ogni indice comporta costi:

- occupa spazio;
- deve essere aggiornato quando cambiano i dati;
- può rallentare inserimenti e modifiche;
- non viene necessariamente usato per ogni query.

Una chiave primaria identifica una riga; un vincolo tutela una regola; un indice serve principalmente all'accesso. Un DBMS può creare indici per attuare alcuni vincoli, ma i concetti non sono sinonimi.

Le query reali guidano la progettazione degli indici. Un indice su Pratica(stato) può essere poco selettivo se quasi tutte le righe hanno lo stesso stato; un indice composto è utile solo per pattern compatibili. Le misure sul DBMS, sui dati e sul carico effettivi ne mostrano le prestazioni.

CONCORRENZA E FENOMENI OSSERVABILI

L'isolamento serve quando più transazioni lavorano sugli stessi dati. Senza garanzie adeguate, una transazione può leggere modifiche non ancora confermate, ottenere due risultati diversi ripetendo la stessa lettura oppure vedere comparire righe che prima non soddisfacevano una condizione. I DBMS descrivono questi fenomeni e offrono livelli di isolamento, ma il comportamento concreto dipende dal prodotto e dal meccanismo di concorrenza.

Un isolamento più forte non è gratuito: può aumentare attese, conflitti o annullamenti. La scelta parte dal rischio del processo. Per assegnare una risorsa unica, perdere un aggiornamento può essere inaccettabile; per una statistica esplorativa può essere tollerabile una vista meno rigida. Il candidato deve collegare la garanzia al requisito, evitando di presentare il livello massimo come soluzione automatica per ogni carico.

Anche una transazione ben delimitata deve gestire gli errori. Dopo un'eccezione, l'applicazione deve sapere se annullare, riprovare o segnalare l'operazione. I tentativi ripetuti richiedono attenzione: un comando non progettato per essere ripetuto può duplicare effetti. La registrazione dell'esito e l'identificazione univoca dell'operazione aiutano a distinguere un nuovo evento dalla ripetizione dello stesso.

SCEGLIERE E VERIFICARE UN INDICE

La scelta di un indice nasce da query concrete: colonne usate nei filtri, nelle join, negli ordinamenti e nelle aggregazioni; selettività dei valori; frequenza di letture e scritture. In un indice composto conta anche l'ordine delle colonne, perché non tutte le combinazioni di ricerca sfruttano allo stesso modo la struttura. Inserire molti indici "per sicurezza" aumenta spazio e lavoro di manutenzione e può confondere l'ottimizzatore.

Il piano di esecuzione mostra come il DBMS intende accedere ai dati: scansioni, indici, join e stime. È una previsione legata a statistiche, parametri, distribuzione dei valori e volume, tutti fattori che possono cambiare la scelta. La verifica professionale confronta piano previsto, misure e carico rappresentativo. In una prova basta spiegare che un indice è candidato quando sostiene un pattern frequente e selettivo; il contesto reale ne dimostrerà l'utilità.

LIVELLI DI ISOLAMENTO E ANOMALIE

Una lettura sporca usa un dato non ancora confermato da un'altra transazione. Una lettura non ripetibile trova cambiato un valore già letto, dopo il commit altrui. Un phantom cambia l'insieme delle righe che soddisfano una query. Un'anomalia di serializzazione produce un risultato incompatibile con qualsiasi esecuzione seriale delle transazioni concluse.

Livello SQL	Garanzie minime rispetto alle letture	Serializzazione
Read Uncommitted	Può consentire tutte e tre le anomalie	Non garantita
Read Committed	Esclude letture sporche	Non garantita
Repeatable Read	Esclude sporche e non ripetibili; phantom ammessi dal minimo SQL	Non garantita
Serializable	Esclude tutte e tre	Equivalenza a un ordine seriale

PostgreSQL 18 implementa Read Uncommitted come Read Committed; il suo Repeatable Read usa snapshot e impedisce anche i phantom. Può comunque richiedere retry, e non equivale a Serializable. A quest’ultimo livello l’applicazione deve saper ripetere l’intera transazione abortita per un conflitto di serializzazione, senza duplicare effetti esterni.

Sequenza svolta. Saldo iniziale 100. T1, in Read Committed, legge 100. T2 aggiorna a 80 e fa commit. T1 legge di nuovo e ottiene 80: lettura non ripetibile, senza lettura sporca. In Repeatable Read di PostgreSQL, con snapshot già fissato dalla prima lettura, T1 continua a vedere 100. Se invece T2 avesse solo aggiornato senza commit, la lettura di 80 da T1 sarebbe sporca, esclusa già da Read Committed. Non confondere la stabilità delle letture con la correttezza di una regola che coinvolge più righe e transazioni.

4.5 Famiglie NoSQL e criteri di scelta

NoSQL raccoglie modelli non riconducibili a un’unica struttura relazionale. Non significa “senza query” né “senza schema”: anche un modello flessibile richiede decisioni su struttura, identificazione, validazione e accesso.

Famiglia	Rappresentazione prevalente	Uso concettuale
chiave-valore	associazione chiave → valore	accesso diretto mediante una chiave
documentale	documenti con campi e strutture annidate	dati aggregati letti spesso insieme
wide-column	righe con famiglie di colonne	grandi volumi distribuiti e accessi orientati alla chiave
grafo	nodi, archi e proprietà	relazioni e percorsi complessi

Nel modello documentale, dati collegati possono essere incorporati nello stesso documento oppure referenziati. L’incorporamento riduce alcune letture multiple e può rendere atomico l’aggiornamento di un aggregato, ma può anche duplicare informazioni o produrre documenti difficili da gestire. I pattern di accesso e aggiornamento determinano quale soluzione è adatta.

RELAZIONALE O NOSQL?

Il confronto può partire da queste domande:

- le relazioni e i vincoli sono centrali?
- i dati hanno struttura stabile o molto variabile?
- quali elementi vengono letti e aggiornati insieme?
- servono join articolate o attraversamenti di relazioni?

- quali garanzie di consistenza richiede il processo?
- quali volumi, distribuzione e tempi di risposta sono previsti?

La velocità o la capacità di scala non derivano dall'etichetta “documentale” o “relazionale”. Vanno valutate rispetto al modello, all'implementazione, ai dati e al carico.

AGGREGATI, RIFERIMENTI E AGGIORNAMENTI

Nel modello documentale conviene individuare l'aggregato, cioè l'insieme di dati che viene letto e aggiornato come unità. Incorporare nello stesso documento informazioni usate insieme può semplificare alcune operazioni. Se però lo stesso dato è condiviso da molti documenti e cambia spesso, la duplicazione rende più difficile mantenere coerenza. Il riferimento separa le informazioni, ma può richiedere più letture o elaborazioni applicative.

La scelta fra incorporare e referenziare dipende quindi dalla direzione delle relazioni, dalla frequenza degli aggiornamenti e dalle garanzie richieste. Una relazione uno-a-pochi stabile può adattarsi all'incorporamento; una relazione multi-a-molti con entità autonome suggerisce spesso riferimenti. Non è una regola assoluta: il modello va provato sui casi d'uso principali.

Le altre famiglie rispondono a esigenze diverse. Un archivio chiave-valore privilegia l'accesso mediante chiave e non offre automaticamente interrogazioni ricche sul contenuto. Un sistema wide-column organizza grandi insiemi distribuiti attorno a chiavi e famiglie di colonne. Una base a grafo rende centrali relazioni e attraversamenti. Chiamarle tutte “NoSQL” non cancella le differenze di modello, query e consistenza.

CONSISTENZA E DISTRIBUZIONE

In un sistema distribuito, replica e partizionamento introducono decisioni su dove si trovano i dati, come vengono aggiornate le copie e che cosa può osservare un client durante un guasto o un ritardo. Formule come “NoSQL è eventualmente consistente” sono troppo generiche: prodotti e configurazioni possono offrire garanzie differenti, talvolta selezionabili per operazione.

Stabilisci quale anomalia il processo può tollerare. Una vista statistica può accettare un breve ritardo; l'assegnazione esclusiva di una pratica richiede regole più forti. Anche il modello relazionale può essere distribuito, e un database documentale può offrire transazioni: le etichette non sostituiscono l'analisi delle garanzie effettive. La decisione dipende dal comportamento richiesto in lettura, scrittura e durante i guasti.

UNA DECISIONE VERIFICABILE

Per motivare la tecnologia, il candidato può costruire una matrice: struttura dei dati, relazioni, operazioni dominanti, volumi, distribuzione, consistenza, latenza, evoluzione dello schema, competenze e gestione. Ogni requisito deve collegarsi a una conseguenza progettuale. Se servono vincoli forti e join articolate, il modello relazionale è un candidato naturale; se l'unità di lettura è un documento variabile e autonomo, un modello documentale può ridurre trasformazioni.

Un sistema può affiancare al database relazionale un motore specializzato per ricerca, cache o relazioni complesse. L'architettura ibrida aumenta però integrazione, monitoraggio, backup e competenze necessarie. La pluralità di strumenti ha senso quando un requisito misurabile giustifica il costo operativo aggiuntivo e quando il gruppo sa gestire consistenza, errori e ripristino fra componenti diversi.

4.6 Qualità, accessi e output concorsuale

Un dato è di qualità quando è adeguato allo scopo dichiarato. Non esiste una qualità astratta e perfetta: occorrono dimensioni, regole, metriche e soglie motivate.

Dimensione	Domanda	Controllo possibile
accuratezza	il valore rappresenta correttamente il fenomeno?	confronto con fonte autorevole o campionamento
completezza	i valori necessari sono presenti?	percentuale di campi obbligatori valorizzati
coerenza	valori collegati rispettano le stesse regole?	controlli incrociati e integrità referenziale
validità	il dato rispetta formato, dominio e regole?	tipo, CHECK, elenco ammesso
unicità	la stessa entità è registrata più volte?	chiavi, regole di matching e deduplicazione
tempestività	il dato è aggiornato entro il tempo utile?	differenza fra evento e aggiornamento

Una regola operativa deve essere verificabile. “Lo stato deve essere corretto” è vago; “lo stato appartiene all’insieme PRESENTATA, IN_ISTRUTTORIA, CHIUSA” è controllabile.

I vincoli del DBMS prevengono alcuni errori, ma non garantiscono l’accuratezza del mondo reale. Una data formalmente valida può essere associata alla pratica sbagliata. Servono quindi controlli all’ingresso, riconciliazioni, gestione delle anomalie e responsabilità definite. Il capitolo 10 sviluppa questi aspetti nel ciclo di governance.

Per ogni regola di qualità specifica popolazione osservata, formula, frequenza di misura e azione conseguente. Per esempio, la completezza dei campi obbligatori può essere espressa come rapporto fra pratiche complete e pratiche esaminate in un intervallo dichiarato. Il valore acquista significato quando sono definiti i campi obbligatori, il momento della rilevazione e la soglia che attiva una verifica.

Collega la misura alla causa. Un'anomalia può derivare dal modello, da una validazione assente, da un'importazione, da un processo manuale o da una fonte esterna. Correggere le sole righe senza intervenire sull'origine produce recidive. Il ciclo tecnico comprende rilevazione, classificazione, correzione controllata, prevenzione e nuova misura. Ogni passaggio deve lasciare evidenza sufficiente a spiegare che cosa è cambiato e perché.

SICUREZZA, ACCESSI E RIPRISTINO

Il DBMS deve permettere soltanto le operazioni necessarie al ruolo. Un utente che produce statistiche può ricevere SELECT senza poter cancellare righe; un servizio applicativo può ottenere privilegi su un insieme circoscritto di oggetti.

Il principio del privilegio minimo riduce l'impatto di errori e abusi. Proprietà degli oggetti, ruoli, concessione e revoca dei privilegi devono essere documentati. Autenticazione, autorizzazione, cifratura, audit, backup e ripristino concorrono alla protezione, ma rispondono a problemi distinti.

Il backup non equivale alla disponibilità immediata e non dimostra da solo la possibilità di recupero. Un ripristino controllato mette alla prova le procedure. I capitoli 8 e 9 approfondiscono rischio, IAM e logging; il capitolo 7 tratta continuità e infrastruttura.

CASO GUIDATO: DIAGNOSTICARE UNA BASE DATI DELLE PRATICHE

Un ufficio conserva una tabella con queste colonne:

```
id_pratica | stato | id_ufficio | nome_ufficio | soggetti
```

La colonna soggetti contiene identificativi separati da virgole. Il nome dell'ufficio è ripetuto in ogni pratica. Alcune righe hanno uno stato non previsto.

La diagnosi procede così:

1. separare Ufficio da Pratica, collegandole con una chiave esterna;
2. creare Soggetto e PraticaSoggetto per la relazione molti-a-molti;
3. definire un dominio o un vincolo per gli stati ammessi;
4. scegliere chiavi che impediscano duplicati;
5. migrare e validare i dati, senza eliminare silenziosamente le anomalie;
6. aggiungere indici soltanto dopo aver esaminato le query;
7. misurare completezza, validità e unicità dopo la migrazione.

La normalizzazione corregge la struttura, ma non rende automaticamente esatti i dati già presenti. Le anomalie devono essere classificate, corrette quando possibile e tracciate.

DOMANDA DA COMMISSARIO

Come progetterebbe una base dati per gestire pratiche e uffici?

Partirei dai requisiti, individuando entità, attributi, relazioni e cardinalità. Userei chiavi stabili, una chiave esterna per il rapporto uno-a-molti e una tabella associativa per le relazioni molti-a-molti. Dopo la verifica delle anomalie definirei vincoli e transazioni; query e indici verrebbero progettati sul carico reale. Infine aggiungerei regole di qualità e privilegi coerenti con i ruoli.

DOMANDA-TRAPPOLA

Una base NoSQL non richiede uno schema?

No. Può consentire una struttura più flessibile o demandare parte della validazione all'applicazione, ma campi, identificatori, relazioni, documenti incorporati, regole di accesso e pattern di aggiornamento devono comunque essere progettati. “Schema flessibile” non significa “assenza di modello”.

ERRORE TIPICO

Usare `DISTINCT` per nascondere righe duplicate prodotte da una join errata. Prima di eliminare duplicati dal risultato, controlla cardinalità, condizione di join e significato delle righe. Più righe possono essere corrette se rappresentano collegamenti diversi.

▣ Verifica

ESERCIZIO 1 — SCHEMA

Un dipendente può partecipare a più progetti e ogni progetto può coinvolgere più dipendenti. Proponi le tabelle minime.

Soluzione: Dipendente, Progetto e una tabella associativa `DipendenteProgetto`, con chiave composta dalle due chiavi esterne. Eventuali attributi come ruolo o data di assegnazione appartengono alla relazione.

ESERCIZIO 2 — QUERY

Scrivi una query che restituisca il numero di pratiche per stato, mostrando soltanto gli stati con almeno cinque pratiche.

```
SELECT stato, COUNT(*) AS numero_pratiche
FROM Pratica
GROUP BY stato
HAVING COUNT(*) >= 5
ORDER BY numero_pratiche DESC;
```

`HAVING` filtra i gruppi prodotti da `GROUP BY`.

ESERCIZIO 3 — DIAGNOSI

Una transazione aggiorna lo stato della pratica, poi fallisce prima di registrare l'evento associato. Quale proprietà è a rischio?

Soluzione: l'atomicità. Le operazioni devono essere confermate insieme oppure annullate.

QUIZ 1

Quale affermazione è corretta?

- A. Una chiave esterna accelera sempre ogni query.
- B. WHERE filtra i gruppi dopo GROUP BY.
- C. Un indice può accelerare letture ma introduce costi di manutenzione.
- D. NULL equivale sempre a zero.

Risposta corretta: C. A confonde vincolo e prestazione; B descrive HAVING; D confonde assenza del valore e valore numerico.

QUIZ 2

Quale situazione indica una relazione multi-a-molti?

- A. Ogni pratica appartiene a un solo ufficio e un ufficio gestisce molte pratiche.
- B. Ogni dipendente può lavorare su più progetti e ogni progetto può coinvolgere più dipendenti.
- C. Ogni ufficio ha un solo identificativo.
- D. Ogni pratica ha una sola data di presentazione.

Risposta corretta: B. La partecipazione è multipla in entrambe le direzioni e richiede normalmente una tabella associativa. A descrive una relazione uno-a-molti; C e D descrivono attributi o vincoli, non una relazione multi-a-molti.

QUIZ 3

Quale dipendenza segnala un problema di seconda forma normale in una tabella con chiave composta?

- A. Un attributo non chiave dipende dall'intera chiave.
- B. Un attributo non chiave dipende soltanto da una parte della chiave.
- C. La chiave primaria non ammette valori nulli.
- D. Una chiave esterna riferisce un'altra tabella.

Risposta corretta: B. La 2NF richiede che gli attributi non chiave dipendano dall'intera chiave composta. A è la condizione desiderata; C e D riguardano vincoli diversi.

QUIZ 4

Quale clausola filtra i gruppi dopo il calcolo di un'aggregazione?

- A. WHERE.
- B. ORDER BY.
- C. HAVING.
- D. FROM.

Risposta corretta: C. HAVING applica condizioni ai gruppi e può usare risultati aggregati. WHERE filtra le righe prima del raggruppamento; ORDER BY ordina il risultato; FROM indica le sorgenti.

QUIZ 5

Una procedura aggiorna lo stato di una pratica e registra un evento, ma deve evitare che resti salvata una sola operazione. Quale proprietà è centrale?

- A. Atomicità.
- B. Selettività.
- C. Normalizzazione.
- D. Indicizzazione.

Risposta corretta: A. L'atomicità tratta le operazioni come un'unità: tutte confermate oppure tutte annullate. Le altre risposte riguardano prestazioni o struttura dei dati.

QUIZ 6

Quale affermazione sui modelli NoSQL è corretta?

- A. Tutti eliminano la necessità di progettare lo schema.
- B. Tutti offrono le stesse query e garanzie di consistenza.
- C. Comprendono famiglie diverse, da valutare rispetto a dati e pattern di accesso.
- D. Sono sempre preferibili quando il volume dei dati cresce.

Risposta corretta: C. Chiave-valore, documentale, wide-column e grafo adottano modelli e operazioni differenti. Flessibilità, scala o consistenza non derivano automaticamente dall'etichetta NoSQL.

Checklist finale

Competenza	Riconosco	Spiego	Applico
Distinguere modello, schema e istanza	☐	☐	☐
Individuare entità, attributi e cardinalità	☐	☐	☐
Usare chiavi e vincoli	☐	☐	☐
Diagnosticare anomalie di normalizzazione	☐	☐	☐

Competenza	Riconosco	Spiego	Applico
Scrivere filtri, join e aggregazioni	☐	☐	☐
Spiegare transazioni e ACID	☐	☐	☐
Distinguere indice, chiave e vincolo	☐	☐	☐
Confrontare relazionale e NoSQL	☐	☐	☐
Definire una regola di qualità misurabile	☐	☐	☐
Motivare ruoli e privilegi	☐	☐	☐

Da sapere in 5 righe

- Il modello traduce requisiti in strutture, relazioni e vincoli.
- La normalizzazione riduce ridondanza e anomalie.
- SQL interroga e modifica dati; join e aggregazioni richiedono cardinalità chiare.
- Transazioni e indici risolvono problemi diversi e comportano compromessi.
- NoSQL e qualità si valutano rispetto a scopo, dati, accessi e controlli.

Riferimenti professionali essenziali

- Il Metodo BANDO, capitolo 10, paragrafo 7, per i prerequisiti su database, tabelle, chiavi e SQL elementare.
- Documentazione ufficiale PostgreSQL per query, vincoli, transazioni, controllo della concorrenza, indici, ruoli e privilegi.
- Documentazione ufficiale MongoDB per modello documentale, incorporamento, riferimenti e operazioni sui documenti.
- NIST Dictionary of Algorithms and Data Structures e materiali universitari di basi dati per terminologia, modelli e strutture.
- Programma del singolo bando e documentazione del DBMS nominato, necessari per dialetto, profondità e strumenti specifici della prova.

Reti, sistemi operativi e infrastrutture

B
BANDO

A
AREE

N
NUCLEI

D
DIARIO

O
OUTPUT

Di fronte a un servizio che non risponde si accusa spesso la rete. Il guasto può invece trovarsi nel collegamento, nell'indirizzamento, nel DNS, nella porta, nel processo, nei permessi oppure nelle risorse del sistema. La diagnosi parte dalla separazione di questi livelli e dalla raccolta di evidenze.

Il troubleshooting è il punto di incontro fra reti e sistemi operativi: il modello tecnico serve a localizzare il guasto.

Obiettivo, inquadramento e confine con il VOL-01

Il VOL-01, capitolo 10, §§ 2, 4 e 5, introduce sistema operativo, file, directory, LAN/WAN, client/server, IP, TCP/IP, DNS, HTTP e dispositivi di rete. Qui tali concetti diventano strumenti di analisi.

Dopo lo studio del capitolo dovrai saper:

- leggere una comunicazione per livelli;
- distinguere MAC, IP, porta e nome DNS;
- interpretare un prefisso IPv4 e una sottorete semplice;
- confrontare switching, routing, TCP e UDP;
- collocare i principali servizi di rete;
- spiegare processi, thread, memoria virtuale e file system;
- distinguere ridondanza, disponibilità e backup;
- costruire una sequenza di troubleshooting basata su test.

Architettura della CPU e gerarchia hardware della memoria sono nel capitolo 2. API e interoperabilità sono nel capitolo 6; virtualizzazione, cloud e continuità nel capitolo 7; sicurezza operativa, IAM e incident response nei capitoli 8-9.

Mappa BANDO di reti e sistemi

Passaggio	Domanda da porsi	Azione
B — Bando	Sono richiesti concetti, comandi, apparati o uno specifico sistema operativo?	Separare il modello comune dai dettagli di prodotto.
A — Aree	Il problema riguarda collegamento, rete, trasporto, servizio, processo o risorsa?	Collocare sintomo e test al livello corretto.

Passaggio	Domanda da porsi	Azione
N — Nuclei	Quali protocolli, servizi e risorse devo saper distinguere?	Costruire una mappa funzione-livello-evidenza.
D — Diario	Quale ipotesi era errata e quale test l'ha esclusa?	Registrare sintomo, prova, risultato e correzione.
O — Output	La prova chiede quiz, orale, mappa o caso tecnico?	Allenare la diagnosi nel formato richiesto.

5.1 Livelli, incapsulamento e rete locale

Una rete collega interfacce e sistemi affinché possano scambiare dati secondo regole condivise. I protocolli definiscono formato dei messaggi, significato dei campi e comportamento dei partecipanti.

La divisione in livelli riduce la complessità: ogni livello offre servizi a quello superiore e usa il livello inferiore. Il modello OSI descrive sette livelli concettuali; l'architettura TCP/IP raggruppa funzioni in un numero minore di livelli. Le due strutture aiutano a ragionare, ma non corrispondono perfettamente voce per voce.

Funzione	Esempi	Domanda diagnostica
accesso alla rete	Ethernet, Wi-Fi, frame, MAC	il collegamento locale funziona?
rete	IPv4, IPv6, routing	esiste un percorso verso la destinazione?
trasporto	TCP, UDP, porte	il servizio è raggiungibile sulla porta prevista?
applicazione	DNS, HTTP, SMTP, SSH	il protocollo applicativo risponde correttamente?

INCAPSULAMENTO

Durante l'invio, ogni livello aggiunge informazioni necessarie alla propria funzione. I dati applicativi diventano unità del trasporto, poi pacchetti IP e infine frame sul collegamento locale. Il destinatario compie il percorso inverso.

MAC, IP, porta e nome DNS rispondono a domande diverse e non sono intercambiabili: ciascuno descrive un livello distinto della comunicazione.

ETHERNET, SWITCHING E RETI LOCALI

Ethernet è una famiglia di tecnologie per reti locali. Sul collegamento, le interfacce usano indirizzi MAC. Uno switch inoltra frame fra porte della rete locale sulla base delle informazioni apprese; un router inoltra pacchetti fra reti IP differenti.

Una VLAN separa logicamente domini di rete sulla stessa infrastruttura di switching. La comunicazione fra VLAN richiede una funzione di livello rete. La segmentazione può ridurre il dominio di broadcast e separare gruppi di sistemi, ma non sostituisce da sola autorizzazioni e controlli di sicurezza.

Il dispositivo deve essere descritto per funzione, non per forma. Un apparato reale può integrare switching, routing, accesso wireless e filtraggio, ma le funzioni restano concettualmente distinte.

Per leggere una comunicazione si segue il dato lungo il percorso. Un browser produce una richiesta applicativa, il trasporto la associa a due estremità, IP individua il percorso fra le reti ed Ethernet o Wi-Fi gestiscono il passaggio sul collegamento locale. A ogni tratto possono cambiare il frame e gli indirizzi di collegamento; la destinazione IP resta quella prevista, salvo funzioni intermedie come la traduzione degli indirizzi. Il “pacchetto”, dunque, non è un oggetto identico a ogni livello.

Queste differenze contano quando si raccolgono le evidenze. Il collegamento attivo prova che l'interfaccia vede il mezzo, ma non dice nulla sulla configurazione IP. Un indirizzo assegnato non garantisce l'esistenza di una rotta. Anche una porta raggiungibile lascia aperta una domanda: l'applicazione restituisce il contenuto atteso? Ogni test copre un tratto preciso.

Evidenza osservata	Che cosa sostiene	Che cosa non dimostra
collegamento attivo	interfaccia e mezzo sono operativi a livello locale	indirizzamento, routing e servizio
MAC appreso dallo switch	è avvenuto traffico sul segmento locale	raggiungibilità di una rete remota
risposta dal gateway	il percorso locale fino al router funziona	raggiungibilità della destinazione finale
connessione alla porta	trasporto e listener rispondono	correttezza della risposta applicativa

Nella prova. Elencare OSI, switch e router serve a poco senza collegare ogni livello a una funzione, a un sintomo possibile e a un test discriminante. Più uffici comunicano nella stessa VLAN ma non raggiungono altre reti? Il collegamento locale è meno sospetto del gateway o del routing. È isolato un solo ufficio? Il confronto parte da link, VLAN e configurazione locale.

Il traffico broadcast resta confinato nel relativo dominio di collegamento; una VLAN può separare logicamente tali domini anche quando gli apparati fisici sono condivisi. Questa separazione migliora ordine e controllo del traffico, ma la comunicazione fra VLAN richiede routing e regole coerenti. In un caso d'esame bisogna perciò distinguere isolamento di livello 2, percorso di livello 3 e autorizzazione: sono decisioni collegate, non sinonimi.

5.2 Indirizzamento, subnetting e routing

Un indirizzo IP identifica un'interfaccia nel contesto della rete. Il prefisso indica la parte usata per individuare la rete; i bit restanti distinguono le interfacce all'interno di quella rete.

In IPv4 un indirizzo contiene 32 bit. La notazione CIDR associa un prefisso, per esempio 192.0.2.64/26. Un /26 lascia 6 bit per la parte host: il blocco contiene 64 indirizzi, da 192.0.2.64 a 192.0.2.127. Nel modello IPv4 tradizionale dell'esercizio:

- indirizzo di rete: 192.0.2.64;
- indirizzo di broadcast: 192.0.2.127;
- intervallo host: 192.0.2.65–192.0.2.126.

Il calcolo va sempre riferito al prefisso. Non basta confrontare le prime cifre scritte in decimale.

IPv6 usa indirizzi di 128 bit e una rappresentazione esadecimale. Introduce un formato di intestazione diverso e non usa il broadcast IPv4. In una prova generale conta distinguere ampiezza, notazione e principi; configurazioni avanzate ed extension header richiedono uno studio specifico.

GATEWAY E ROUTING

Se la destinazione appartiene alla rete locale, l'host cerca il recapito sul collegamento. Se appartiene a un'altra rete, invia il pacchetto al gateway previsto dalla tabella di routing. Il router sceglie un percorso usando prefissi e regole.

La tabella di routing può contenere rotte specifiche e una rotta predefinita. La presenza di un indirizzo IP corretto non garantisce quindi la raggiungibilità: possono mancare gateway, rotta di ritorno o instradamento intermedio.

Nelle reti IPv4, ARP consente di associare un indirizzo IPv4 locale a un indirizzo di collegamento. IPv6 usa meccanismi propri di neighbor discovery. I dettagli operativi differiscono, ma l'esigenza diagnostica resta la stessa: raggiungere il prossimo nodo sul collegamento locale.

VERIFICARE UNA SOTTORETE SENZA INTUZIONI DECIMALI

Il prefisso stabilisce la dimensione dei blocchi. Con /26, i primi 26 bit appartengono al prefisso e restano 6 bit: ogni blocco contiene quindi 2^6 elevato alla sesta, cioè 64 indirizzi. Nell'ultimo ottetto i blocchi iniziano a 0, 64, 128 e 192. L'indirizzo 192.0.2.70 cade nel blocco 64-127; nel modello IPv4 tradizionale dell'esercizio, 64 è l'indirizzo di rete e 127 quello di broadcast.

La stessa tecnica permette di confrontare due host. 192.0.2.70/26 e 192.0.2.120/26 appartengono allo stesso blocco; 192.0.2.130/26 appartiene al blocco successivo. Nel primo caso la consegna locale avviene sul collegamento; nel secondo serve un dispositivo capace di instradare fra le reti. La verifica riguarda indirizzi e prefisso di ciascuna interfaccia, non la somiglianza visiva delle prime tre cifre.

Nella tabella di routing si cerca la rotta più specifica compatibile con la destinazione. La rotta predefinita raccoglie il traffico che non trova una corrispondenza più precisa. La diagnosi comprende anche il percorso di ritorno e gli eventuali filtri: il pacchetto può arrivare a destinazione mentre la risposta non riesce a tornare al mittente.

Domanda	Dato da controllare	Errore ricorrente
La destinazione è locale?	indirizzo e prefisso di entrambe le interfacce	confrontare solo la notazione decimale
Qual è il prossimo nodo?	tabella di routing e gateway	ritenere sufficiente la presenza di un gateway
Il recapito locale è possibile?	risoluzione del vicino sul collegamento	confondere ARP con DNS
La risposta può tornare?	rotte e filtri nel verso inverso	osservare soltanto il percorso di andata

Nella prova. Una postazione raggiunge i sistemi della propria sottorete ma non un servizio remoto. Si verificano prefisso e gateway, scelta della rotta, percorso e ritorno. Cambiare DNS non risolve un errore di instradamento già dimostrato con una prova diretta sull'indirizzo IP.

Quando il problema interessa soltanto una direzione, il controllo del ritorno è decisivo. Rotte asimmetriche possono essere legittime, ma ogni verso deve restare praticabile e compatibile con i controlli intermedi. L'assenza della risposta non prova quindi, da sola, che la richiesta non sia arrivata.

5.3 Trasporto e servizi di rete

TCP e UDP operano sopra IP e usano numeri di porta per distinguere le comunicazioni applicative.

TCP stabilisce una connessione logica e fornisce un flusso ordinato, con controllo degli errori e ritrasmissione secondo il protocollo. UDP invia datagrammi senza offrire le stesse garanzie di consegna e ordinamento. L'applicazione può aggiungere i meccanismi di cui ha bisogno.

L'etichetta "più veloce" non basta a descrivere UDP: contano rete, carico, dimensione dei messaggi e comportamento applicativo. TCP è adatto quando serve un flusso affidabile; UDP quando l'applicazione preferisce datagrammi, latenza contenuta o una gestione autonoma delle perdite.

Una comunicazione è descritta, in forma semplificata, da indirizzi e porte delle due estremità. Un socket è l'astrazione usata dal sistema operativo per accedere alla comunicazione.

SERVIZI PRINCIPALI

Servizio o protocollo	Funzione
DNS	risolve nomi e pubblica informazioni di dominio
DHCP	assegna parametri di configurazione di rete
HTTP/HTTPS	trasferisce richieste e risposte per risorse e servizi web
SMTP	trasferisce messaggi di posta
IMAP	consente accesso e gestione della casella sul server
SSH	offre accesso remoto protetto e altri servizi sicuri
NTP	sincronizza gli orologi dei sistemi

Il DNS è gerarchico e distribuito. La risoluzione può coinvolgere cache e più server. Record differenti svolgono funzioni diverse: A e AAAA associano nomi a indirizzi, CNAME indica un alias, MX riguarda la posta e NS i server autorevoli della zona.

Se un servizio è raggiungibile mediante indirizzo IP ma non mediante nome, DNS diventa un'ipotesi concreta. Se il nome si risolve correttamente ma la porta non risponde, la diagnosi deve spostarsi verso trasporto, processo, ascolto o filtraggio.

Il NAT modifica informazioni di indirizzamento durante l'attraversamento di un dispositivo; un proxy agisce per conto di un client o di un server a livello applicativo o intermedio. Né NAT né proxy è, per definizione, sinonimo di firewall.

DALLA RISOLUZIONE DEL NOME ALLA RISPOSTA APPLICATIVA

Quando un utente apre un servizio web, più passaggi devono riuscire. Il client ricava dal nome una destinazione tramite DNS; sceglie una rotta; apre o usa una comunicazione di trasporto verso la porta prevista; infine scambia messaggi applicativi. HTTPS aggiunge la protezione della sessione e la verifica del certificato secondo il contesto. Un errore mostrato dal browser può quindi derivare da livelli diversi.

La diagnosi separa quattro esiti. Un nome che non produce indirizzi porta a controllare resolver, record e cache. Un indirizzo esistente ma irraggiungibile riporta a configurazione e routing. Quando l'host risponde ma la porta rifiuta o non completa la connessione, si verificano listener, servizio e filtraggio. Se invece la connessione riesce ma la risposta è errata, l'attenzione passa al protocollo applicativo, alla configurazione e ai log correlati.

Sintomo	Prima ipotesi utile	Evidenza discriminante
nome non risolto	DNS o configurazione del resolver	risposta alla richiesta per il record atteso
timeout verso l'indirizzo	percorso, ritorno o filtro	test progressivi di raggiungibilità e rotta
connessione rifiutata	nessun listener o rifiuto esplicito	stato della porta sul server
risposta HTTP inattesa	applicazione, proxy o configurazione	codice, intestazioni, contenuto e log

TCP non rende automaticamente corretta l'applicazione: garantisce proprietà del flusso, non il significato dei dati scambiati. Analogamente, l'uso di UDP non implica assenza di affidabilità complessiva, perché un protocollo applicativo può introdurre conferme, ripetizioni o controllo degli errori. In sede concorsuale la scelta si motiva partendo dai requisiti: ordine, perdita tollerabile, latenza, dimensione e gestione applicativa.

Nella prova. Davanti a un portale intermittente, la risposta distingue la risoluzione DNS dalla connessione alla porta e dalla risposta HTTP. Orari, client coinvolti, indirizzi restituiti e codici osservati impediscono di attribuire al DNS un errore applicativo o al server un problema di percorso.

Le prove vanno eseguite nello stesso intervallo del sintomo: cache, indirizzi e stato del servizio possono cambiare, rendendo poco attendibile un confronto raccolto troppo tardi.

5.4 Sistema operativo, processi e servizi

Il sistema operativo gestisce CPU, memoria, dispositivi, file, identità e comunicazioni. Il kernel opera con privilegi elevati e media l'accesso alle risorse. Le applicazioni lavorano nello spazio utente e richiedono servizi al kernel attraverso interfacce definite.

Un programma è un insieme di istruzioni e dati conservati; un processo è un'istanza in esecuzione, con stato e risorse. Un thread è un flusso di esecuzione all'interno di un processo e condivide parte delle sue risorse con gli altri thread dello stesso processo.

Il sistema operativo assegna tempo di CPU tramite lo scheduling. I processi possono essere in esecuzione, pronti o in attesa; le denominazioni precise cambiano fra sistemi. Un processo che non usa CPU può essere in attesa di I/O, di un lock o di un evento: il basso consumo non prova da solo che sia guasto.

SERVIZI E DIPENDENZE

Un servizio o demone esegue una funzione in background, per esempio un server web o un resolver locale. Per diagnosticare un servizio occorre distinguere:

- configurazione presente;
- processo avviato;
- porta in ascolto;
- dipendenze disponibili;
- permessi sufficienti;
- log privi di errori bloccanti;
- risposta effettiva alla richiesta.

Riavviare il servizio può rimuovere temporaneamente il sintomo e cancellare indizi utili. Prima dell'intervento conviene acquisire stato, messaggi e condizioni riproducibili.

DALLO STATO DEL PROCESSO ALL'EROGAZIONE DEL SERVIZIO

Lo stato “attivo” descrive solo una parte della situazione. L'istanza può essere ancora in inizializzazione, bloccata su una dipendenza, priva dei permessi necessari oppure configurata per ascoltare su un'interfaccia diversa. Il controllo prosegue dalla supervisione del servizio fino alla risorsa effettivamente offerta.

Un thread può attendere I/O mentre altri thread dello stesso processo continuano a lavorare. Lo scheduler assegna la CPU alle unità eseguibili, ma una coda di richieste può crescere anche con CPU moderata se il collo di bottiglia è altrove. Per questo stato, utilizzo e latenza vanno letti insieme.

Controllo	Esito utile	Passo successivo
configurazione caricata	file e parametri sono leggibili e coerenti	verificare avvio e dipendenze
processo presente	esiste un'istanza	controllare stato, thread e messaggi
socket in ascolto	il processo espone l'estremità prevista	provare localmente e poi da remoto
richiesta locale riuscita	applicazione e percorso locale rispondono	verificare rete e filtri esterni

Controllo	Esito utile	Passo successivo
log correlato	l'evento è associato all'orario e alla richiesta	formulare una modifica mirata

I log hanno valore quando sono correlati a una prova precisa. Una lunga sequenza di messaggi senza riferimento temporale o identificativo può confondere causa ed effetti. Prima di modificare il sistema si annotano ora, richiesta, nodo, configurazione rilevante e risultato. Dopo la modifica si ripete la stessa prova: il confronto rende verificabile il ripristino.

Caso breve. Un servizio risulta “running”, ma la porta prevista non appare in ascolto. Il tecnico controlla il log di avvio e trova un errore di binding dovuto a un indirizzo non più assegnato. Corregge un solo parametro, riavvia in modo controllato, verifica il listener, esegue una richiesta locale e poi una remota. L'evidenza collega sintomo, causa e soluzione; un riavvio iniziale senza raccolta dati non avrebbe dimostrato nulla.

Le dipendenze richiedono lo stesso rigore. Un servizio applicativo può dipendere da un database, da un resolver, da uno storage o da un altro processo locale. La presenza del processo principale non dimostra che tali componenti siano disponibili né che le credenziali e i percorsi configurati siano validi. Il controllo segue la catena fino alla prima evidenza incoerente.

Anche la terminazione di un processo va qualificata. Può essere volontaria, causata da un errore non gestito oppure conseguire a un limite di risorse. Stato di uscita, messaggi e condizioni del sistema nello stesso intervallo aiutano a separare queste ipotesi. Avviare ripetutamente l'istanza senza capire perché termina aumenta il rumore e può aggravare il carico.

Schema per l'orale. Il servizio si descrive come una catena: configurazione, processo, dipendenze, socket, richiesta e risposta. A ogni passaggio corrisponde un'evidenza osservabile. La struttura mostra la comprensione del sistema operativo e non riduce la diagnosi al comando usato.

5.5 Memoria virtuale, file system e risorse

La memoria virtuale assegna a ogni processo uno spazio di indirizzamento. Il sistema traduce indirizzi virtuali in memoria fisica mediante strutture come le tabelle delle pagine. Le pagine possono contenere memoria anonima, codice o dati di file.

La page cache conserva in memoria dati letti dai file per ridurre accessi più costosi allo storage. La swap può ospitare pagine espulse dalla RAM secondo le politiche del sistema. La sua presenza non garantisce prestazioni adeguate: una pressione di memoria elevata può aumentare l'I/O e la latenza.

Una diagnosi distingue memoria libera, memoria disponibile, cache recuperabile, swap usata e processi responsabili. Un singolo numero, isolato dal carico, non basta.

FILE SYSTEM

Il file system organizza nomi, directory, contenuti e metadati. Un mount rende un file system accessibile in un punto della gerarchia. I metadati includono, secondo il sistema, proprietà, permessi, dimensione e tempi.

Nei sistemi di tipo Unix, proprietario, gruppo e permessi concorrono a determinare le operazioni consentite. Per attraversare una directory serve un'autorizzazione diversa dalla sola lettura del suo elenco. Il controllo deve coprire l'intero percorso, oltre al file finale.

Un errore di scrittura può dipendere da:

- permessi insufficienti;
- file system montato in sola lettura;
- spazio o metadati esauriti;
- quota raggiunta;
- percorso errato;
- errore del dispositivo o del file system.

DIAGNOSI DELLE RISORSE SENZA AFFIDARSI A UN SOLO NUMERO

La memoria “usata” comprende componenti con comportamento diverso. Parte può appartenere direttamente ai processi, parte può essere cache recuperabile, parte può essere impegnata dal kernel. La memoria disponibile è quindi più informativa della sola memoria libera, ma anch'essa va collegata a pressione, recupero delle pagine, swap, latenze e carico. Un picco breve non equivale a un esaurimento persistente.

Sul file system occorre distinguere capacità dei dati e disponibilità dei metadati. Un volume può avere spazio nominale ma non consentire nuove scritture per quota, esaurimento delle strutture necessarie, mount in sola lettura o permessi lungo il percorso. La prova deve riprodurre l'operazione con la stessa identità e sullo stesso percorso usati dal servizio.

Sintomo	Misure da correlare	Ipotesi da distinguere
servizio lento	memoria disponibile, swap, I/O, code e latenza	pressione di memoria, storage o lock
processo terminato	eventi di sistema, limiti e consumo per processo	esaurimento, limite o errore applicativo
scrittura fallita	spazio, quota, mount, permessi e percorso	capacità, autorizzazione o errore del supporto
file non trovato	percorso effettivo, mount e identità	nome errato, volume non montato o visibilità diversa

I permessi vanno letti come una catena. Per creare un file in una directory servono autorizzazioni sulla directory e accesso alle componenti del percorso; possedere il file finale non risolve un blocco su una directory superiore. Nei sistemi che applicano controlli aggiuntivi, i permessi tradizionali possono non essere l'unico fattore: la risposta concorsuale deve ammettere l'esistenza di più livelli senza inventare una configurazione specifica.

Nella prova. Se un'applicazione smette di produrre documenti, si confrontano identità del processo, percorso configurato, mount, spazio, quota e messaggi. Una prova minima controllata precede la modifica di un solo fattore. Attribuire tutto allo "spazio disco" porta a ignorare permessi e file system in sola lettura.

Una pressione di memoria persistente può manifestarsi come aumento della latenza prima che il servizio fallisca. Se il sistema recupera continuamente pagine e sposta dati fra RAM e storage, le richieste restano in attesa anche con CPU non satura. Occorre correlare andamento temporale delle metriche e tempi di risposta, evitando soglie universali prive di contesto.

La cache richiede una lettura altrettanto prudente: occupare memoria per conservare dati usati di recente può essere utile e quella memoria può diventare recuperabile. Liberarla indiscriminatamente non risolve la causa di una crescita applicativa e può peggiorare temporaneamente le prestazioni. La domanda corretta è se la risorsa rimane disponibile per il carico e con quale latenza.

Nel caso di un file system pieno, la correzione non coincide automaticamente con la cancellazione del primo file grande. Si identifica chi produce i dati, quali regole di conservazione valgono, se esistono file ancora aperti e quale misura impedisce la ricorrenza. L'intervento deve preservare integrità, autorizzazioni e tracciabilità.

5.6 Disponibilità e troubleshooting infrastrutturale

La disponibilità descrive la capacità di offrire il servizio quando richiesto. A determinarla concorre l'intera catena: alimentazione, rete, calcolo, storage, applicazione, dati e dipendenze esterne.

La ridondanza duplica componenti o percorsi per evitare che un singolo guasto interrompa il servizio. La fault tolerance mira a continuare l'erogazione nonostante determinati guasti. Un singolo punto di guasto è un elemento la cui indisponibilità interrompe la funzione senza alternativa.

Il RAID combina dischi secondo schemi che possono aumentare disponibilità o prestazioni rispetto al guasto contemplato. Non sostituisce il backup: una cancellazione, una corruzione logica o un errore applicativo può propagarsi anche sulle copie ridondanti.

L'affidabilità riguarda la probabilità di funzionare senza guasti in un intervallo; la manutenibilità riguarda la facilità e rapidità di ripristino. Entrambe incidono sulla disponibilità osservata.

CAPACITÀ E COLLI DI BOTTIGLIA

La capacità va valutata con misure:

- latenza delle operazioni;
- throughput o lavoro completato nell'unità di tempo;
- utilizzo e saturazione delle risorse;
- lunghezza delle code;
- errori, ritrasmissioni e timeout.

Una CPU poco utilizzata non esclude un collo di bottiglia: il sistema può attendere storage, rete, lock o servizio esterno. Le metriche devono essere correlate nel tempo con il sintomo.

TROUBLESHOOTING PER LIVELLI

Una diagnosi ripetibile segue questa sequenza:

1. descrivere il sintomo in termini osservabili;
2. definire perimetro e momento del problema;
3. raccogliere una configurazione di riferimento;
4. formulare ipotesi ordinate per livello e probabilità;
5. eseguire un test che distingua almeno due ipotesi;
6. registrare evidenza e risultato;
7. applicare una modifica controllata;
8. verificare servizio e assenza di effetti collaterali.

ORDINE DEI CONTROLLI

Per un servizio remoto non raggiungibile:

Controllo	Domanda	Evidenza
collegamento	l'interfaccia è attiva?	stato del link e contatori
configurazione	indirizzo, prefisso e gateway sono corretti?	configurazione locale
rete	la destinazione o il gateway risponde?	test di percorso e routing
nomi	il nome si risolve nel valore atteso?	risposta DNS

Controllo	Domanda	Evidenza
trasporto	la porta prevista è raggiungibile?	test di connessione
sistema	il processo è avviato e in ascolto?	stato servizio e socket
applicazione	la richiesta riceve risposta valida?	codice, contenuto e log

Gli strumenti cambiano fra sistemi. ping, traceroute o tracert, ip, ss, netstat, nslookup, dig, curl, visualizzatori di eventi e log sono esempi. Un comando ha valore solo se è chiaro quale ipotesi verifica e come interpretarne il risultato.

CASO GUIDATO: PORTALE INTERNO NON RAGGIUNGIBILE

Gli utenti segnalano che portale.ente.interno non risponde. Il server risulta acceso.

La diagnosi procede senza saltare subito alla configurazione applicativa:

1. un client ottiene un indirizzo e raggiunge il gateway;
2. il nome DNS restituisce un indirizzo non più assegnato al server;
3. collegandosi all’indirizzo corretto, la porta del servizio risponde;
4. i log applicativi non mostrano errori per le richieste di prova.

L’evidenza localizza il problema nel dato DNS. La correzione consiste nell’aggiornare il record previsto e verificare la propagazione nelle cache pertinenti. Riavviare il server non avrebbe corretto la causa.

DOMANDA DA COMMISSARIO

Come affronta il malfunzionamento di un servizio di rete?

Parto da un sintomo osservabile e ne delimito utenti, sistemi e momento di insorgenza. I controlli procedono dal collegamento alla risposta applicativa, passando per configurazione IP, percorso, DNS, porta e processo. Ogni prova deve discriminare un’ipotesi. Solo dopo raccolgo l’evidenza, applico una modifica controllata e verifico il ripristino senza effetti collaterali.

DOMANDA-TRAPPOLA

Un RAID rende inutile il backup?

No. Il RAID può mantenere il servizio o i dati disponibili in presenza di alcuni guasti dei dischi, secondo il livello adottato. Non crea automaticamente una copia storica indipendente e non protegge da ogni cancellazione, corruzione o errore applicativo.

ERRORE TIPICO

Cambiare più configurazioni insieme. Se il servizio torna disponibile, non si sa quale modifica abbia risolto il problema; se peggiora, diventa difficile tornare allo stato precedente. Una diagnosi riproducibile usa test mirati e varia una condizione alla volta.

▣ Verifica

MICRO-VERIFICA: LIVELLI E RETE LOCALE

Una workstation invia una richiesta HTTP/1.1 non cifrata a un server in un'altra rete; l'esempio isola l'incapsulamento TCP/IP. Ordina messaggio HTTP, segmento TCP, pacchetto IP e frame Ethernet; poi indica quale apparato inoltra il frame nella rete locale e quale instrada verso l'altra rete.

Soluzione: il messaggio HTTP è incapsulato in un segmento TCP, poi in un pacchetto IP e infine in un frame Ethernet. Lo switch inoltra il frame nella rete locale; il router instrada il pacchetto IP verso una rete diversa. HTTP/2 usa anch'esso TCP; HTTP/3 usa QUIC su UDP, perciò questa sequenza non si applica indistintamente a tutte le versioni.

ESERCIZIO 1 — SOTTORETE

Per 192.0.2.70/26, indica rete e broadcast nel modello IPv4 tradizionale.

Soluzione: il blocco /26 contiene 64 indirizzi. 192.0.2.70 appartiene al blocco 192.0.2.64–192.0.2.127; rete 192.0.2.64, broadcast 192.0.2.127.

ESERCIZIO 2 — DIAGNOSI

Il client raggiunge 192.0.2.10, ma servizio.ente.it non viene risolto. Quale controllo viene prima?

Soluzione: verificare la configurazione e la risposta DNS per il nome. Il test IP indica che almeno un percorso verso quell'indirizzo esiste, ma non dimostra che il nome sia configurato correttamente.

ESERCIZIO 3 — SISTEMA

Il processo è avviato, ma nessuna porta risulta in ascolto. Indica due ipotesi.

Soluzione: il servizio può aver fallito il binding per configurazione o conflitto di porta; può anche essere avviato in una modalità che non espone il listener previsto. Vanno controllati configurazione e log.

QUIZ 1

Quale affermazione è corretta?

- A. DNS assegna sempre gli indirizzi IP ai client.
- B. Uno switch e un router svolgono necessariamente la stessa funzione.
- C. Un processo può essere in attesa senza utilizzare molta CPU.
- D. Swap e RAM hanno identiche prestazioni.

Risposta corretta: C. A descrive una funzione tipica di DHCP; B confonde switching e routing; D ignora la diversa natura delle risorse.

QUIZ 2

Due host IPv4 hanno indirizzi 192.0.2.70/26 e 192.0.2.130/26. Quale affermazione è corretta?

- A. Appartengono necessariamente alla stessa sottorete perché i primi tre ottetti coincidono.
- B. Appartengono a blocchi /26 diversi e la comunicazione richiede instradamento.
- C. Il prefisso /26 indica 26 indirizzi utilizzabili.
- D. Il secondo indirizzo è sempre un broadcast.

Risposta corretta: B. Il primo indirizzo cade nel blocco 64-127, il secondo nel blocco 128-191. A ignora il prefisso; C confonde bit e indirizzi; D è falsa perché il broadcast del secondo blocco è .191 nel modello tradizionale.

QUIZ 3

Un nome DNS restituisce l'indirizzo previsto, ma la connessione alla porta del servizio viene rifiutata. Qual è il controllo più mirato?

- A. Sostituire immediatamente il server DNS.
- B. Verificare listener, stato del servizio e binding sul nodo destinatario.
- C. Cambiare il prefisso IP del client.
- D. Aumentare la swap del server.

Risposta corretta: B. La risoluzione del nome è già riuscita e il rifiuto orienta verso l'estremità di trasporto o il servizio. Le altre modifiche non discendono dall'evidenza raccolta.

QUIZ 4

Quale situazione dimostra da sola che un'applicazione web funziona correttamente?

- A. L'interfaccia di rete è attiva.
- B. Il server risponde al gateway.
- C. La porta TCP accetta una connessione.
- D. Nessuna delle precedenti.

Risposta corretta: D. Ogni evidenza copre solo una parte della catena. Per dimostrare il funzionamento applicativo serve una richiesta valida con risposta attesa, oltre ai controlli sottostanti.

QUIZ 5

Un servizio non riesce a creare un file, sebbene il volume mostri spazio disponibile. Quale spiegazione resta plausibile?

- A. Quota raggiunta, mount in sola lettura o permessi insufficienti.
- B. Il DNS non contiene un record MX.
- C. TCP usa ritrasmissioni.
- D. Lo switch ha appreso un indirizzo MAC.

Risposta corretta: A. La capacità libera non esclude altri vincoli del file system. B, C e D descrivono aspetti non pertinenti alla scrittura locale indicata.

QUIZ 6

Quale intervento rende un troubleshooting più verificabile?

- A. Cambiare contemporaneamente rete, DNS e configurazione applicativa.
- B. Riavviare ogni componente prima di raccogliere dati.
- C. Formulare un'ipotesi, eseguire un test discriminante e modificare una condizione alla volta.
- D. Considerare il primo messaggio di errore come causa certa.

Risposta corretta: C. La sequenza conserva il legame fra ipotesi, evidenza e risultato. Le altre opzioni cancellano informazioni o impediscono di attribuire l'effetto a una causa.

Checklist finale

Competenza	Riconosco	Spiego	Applico
Leggere una comunicazione per livelli	☐	☐	☐
Distinguere MAC, IP, porta e DNS	☐	☐	☐
Interpretare un prefisso IPv4	☐	☐	☐
Confrontare switch, router, TCP e UDP	☐	☐	☐
Collocare i principali servizi	☐	☐	☐
Spiegare processi, thread e servizi	☐	☐	☐
Diagnosticare memoria e file system	☐	☐	☐
Distinguere RAID e backup	☐	☐	☐

Competenza	Riconosco	Spiego	Applico
Individuare un singolo punto di guasto	☐	☐	☐
Costruire un piano di troubleshooting	☐	☐	☐

Da sapere in 5 righe

- I livelli separano funzioni e rendono diagnosticabile la comunicazione.
- MAC, IP, porta e nome DNS identificano elementi diversi.
- Processi, memoria e file system possono produrre sintomi simili a un guasto di rete.
- Ridondanza, disponibilità e backup rispondono a rischi differenti.
- Il troubleshooting procede per ipotesi, test, evidenza, modifica e verifica.

Riferimenti professionali essenziali

- IETF/RFC Editor, RFC 8200 per IPv6 e RFC 4632 per il Classless Inter-Domain Routing (CIDR).
- IETF/RFC Editor, RFC 9293 per TCP e RFC 768 per UDP.
- IETF/RFC Editor, RFC 1034 e RFC 1035 per l'architettura e il funzionamento del DNS.
- IETF/RFC Editor, RFC 2131 per DHCP e RFC 9110 per la semantica HTTP.
- Linux Kernel Documentation, sezioni dedicate alla gestione della memoria, ai file system e al file system /proc, consultate per i concetti trasferibili di memoria virtuale, processi e file system.
- Il Metodo BANDO, capitolo 10, §§ 2, 4 e 5, per i prerequisiti di informatica e reti richiamati nel confine iniziale.

Ingegneria software, API e interoperabilità PA

B
BANDO

A
AREE

N
NUCLEI

D
DIARIO

O
OUTPUT

Un servizio digitale nasce da un bisogno, non dal codice. Coinvolge persone e amministrazioni, tratta dati, dipende da altri sistemi e deve continuare a funzionare mentre requisiti e contesto cambiano. Perciò non basta realizzarlo: occorre dimostrare che risponde al bisogno, governarne l'evoluzione e consentire ad altri sistemi di usarlo senza conoscerne l'implementazione interna.

L'ingegneria software disciplina questo percorso. Le API rendono espliciti i contratti fra sistemi; l'interoperabilità li collega a regole organizzative, semantiche e tecniche. Nella PA il risultato atteso è un servizio affidabile, comprensibile, autorizzato e governabile. L'endpoint ne costituisce soltanto l'interfaccia tecnica.

Obiettivo, inquadramento e confine con il VOL-01

Il VOL-01, capitolo 10, § 16, introduce API, interoperabilità e PDND e chiarisce che scambio autorizzato e pubblicazione open data sono fenomeni diversi. Qui quei prerequisiti diventano strumenti di progettazione.

Dopo lo studio del capitolo dovrai saper:

- trasformare un bisogno in requisiti verificabili;
- confrontare modelli di ciclo di vita senza applicarli meccanicamente;
- leggere un'architettura per componenti, interfacce e dipendenze;
- distinguere verifica, validazione e livelli di test;
- spiegare controllo di versione e gestione delle modifiche;
- progettare il contratto essenziale di un'API;
- riconoscere compatibilità, idempotenza, errori e livelli di servizio;
- distinguere i livelli dell'interoperabilità;
- costruire un caso e-service con erogatore, fruitore, finalità ed evidenze.

Programmazione e strutture dati sono nel capitolo 3; basi dati nel capitolo 4; HTTP come protocollo e i servizi di rete nel capitolo 5. Cloud e CI/CD operativa sono nel capitolo 7; sicurezza, IAM e incident response nei capitoli 8-9; data governance e interoperabilità semantica nel capitolo 10; SLA contrattuali e fornitori nel capitolo 12.

Mappa BANDO dal requisito all'e-service

Passaggio	Domanda da porsi	Azione
B — Bando	Sono richiesti metodologie, testing, API, standard o piattaforme specifiche?	Separare il nucleo trasferibile dai dettagli di prodotto.
A — Aree	La domanda riguarda bisogno, progetto, test, contratto API o interoperabilità?	Collocare il problema nella fase e nel livello corretti.
N — Nuclei	Quali elementi devono restare verificabili ed evolvibili?	Collegare requisito, interfaccia, test, versione ed evidenza.
D — Diario	Quale assunzione ha prodotto un errore o una modifica incompatibile?	Registrare decisione, impatto, prova e correzione.
O — Output	Serve una risposta orale, un caso, una specifica o una matrice?	Produrre l'artefatto richiesto, non una lista di acronimi.

6.1 Requisiti e ciclo di vita verificabile

Un bisogno descrive il risultato atteso da un soggetto. Un requisito traduce quel bisogno in una proprietà o capacità che il sistema deve possedere. Un vincolo limita le soluzioni ammissibili: può derivare da norme, standard, tecnologie già presenti, tempi, budget o organizzazione. Il criterio di accettazione indica invece una condizione osservabile con cui stabilire se il requisito è soddisfatto.

Esempio: «ridurre le richieste di certificati già disponibili» è un bisogno. «Il servizio deve verificare, previa autorizzazione, il possesso del requisito anagrafico presso il sistema erogatore» è un requisito funzionale. «La risposta deve essere disponibile entro il tempo concordato e ogni richiesta deve essere tracciabile» introduce requisiti non funzionali. «Dato valido → esito coerente; dato inesistente → errore classificato; fruitore non abilitato → richiesta rifiutata» fornisce criteri verificabili.

REQUISITI FUNZIONALI E NON FUNZIONALI

I requisiti funzionali descrivono comportamenti, dati elaborati e risultati. I requisiti non funzionali qualificano il servizio: prestazioni, disponibilità, usabilità, accessibilità, sicurezza, manutenibilità, portabilità e osservabilità.

I requisiti non funzionali hanno effetti concreti. Un servizio può restituire il dato corretto e risultare comunque inadeguato perché non regge il carico, esclude parte degli utenti o rende impossibile ricostruire gli errori.

Per essere utilizzabile, un requisito deve rispondere a un bisogno riconoscibile, evitare ambiguità e restare coerente con gli altri. Deve inoltre risultare realizzabile, verificabile e collegabile a progetto, test e rilascio.

TRACCIABILITÀ

La tracciabilità consente di seguire un requisito lungo il ciclo di vita.

Requisito	Componente o API	Criterio	Test
verificare un requisito anagrafico	servizio di verifica	esito coerente per tre classi di input	integrazione e accettazione
registrare l'uso	componente di audit	evento presente e correlabile	sistema

Requisito	Evidenza
verificare un requisito anagrafico	esito di esecuzione registrato
registrare l'uso	evento con identificativo

Se una modifica interessa il requisito, la matrice aiuta a individuare contratto, test e documentazione da aggiornare. Se un test non è collegato a nessuna promessa, può essere ridondante; se una promessa non ha test, manca un'evidenza.

CICLO DI VITA E ORGANIZZAZIONE DELLO SVILUPPO

Il ciclo di vita del software comprende attività di concezione, analisi, progettazione, realizzazione, verifica, rilascio, esercizio, manutenzione e dismissione. Le attività possono sovrapporsi e ripetersi: la sequenza dipende da rischio, stabilità dei requisiti, dimensione, vincoli e capacità dell'organizzazione.

Un modello sequenziale rende esplicite fasi e approvazioni ed è utile quando requisiti e vincoli sono abbastanza stabili. Il limite emerge quando il feedback arriva tardi. Un modello iterativo rivede più volte la soluzione; uno incrementale consegna parti utilizzabili in passi successivi. Gli approcci Agile privilegiano feedback frequente, collaborazione e adattamento, ma non eliminano architettura, documentazione, controllo e responsabilità.

La scelta del modello dipende dalla capacità di produrre evidenze in tempo utile, governare i rischi e rispettare i vincoli del servizio. L'etichetta «Agile» o «tradizionale», da sola, dice poco.

RUOLI E RESPONSABILITÀ

Il committente definisce obiettivi e vincoli insieme agli utenti e agli altri portatori d'interesse. Gli analisti precisano i requisiti; progettisti e sviluppatori costruiscono la soluzione, mentre i tester raccolgono le evidenze e operations gestisce l'esercizio. Un responsabile del prodotto o del servizio ordina priorità e accettazione. Anche quando un fornitore riunisce più ruoli, responsabilità e criteri decisionali devono restare espliciti.

CONFIGURAZIONE E CONTROLLO DI VERSIONE

La gestione della configurazione identifica gli elementi controllati — codice, specifiche, schemi, configurazioni, test e documenti — e ne governa le modifiche. Il controllo di versione conserva la storia e permette di associare uno stato a una release.

Nel modello distribuito diffuso da Git, un repository conserva oggetti e riferimenti; un commit registra uno stato; un branch rappresenta una linea di sviluppo; un merge integra storie divergenti; un tag identifica un punto significativo. Il repository non garantisce qualità: servono regole su revisione, test, approvazione, tracciabilità e ripristino. L'integrazione continua automatizza controlli frequenti; deployment e infrastruttura della pipeline appartengono al capitolo 7.

6.2 Architettura e qualità del software

L'architettura software descrive elementi significativi, responsabilità, relazioni e decisioni che condizionano l'evoluzione del sistema. Un componente racchiude una responsabilità; un'interfaccia espone modalità di interazione; una dipendenza indica che un elemento necessita di un altro.

Alta coesione significa che le responsabilità di un componente sono correlate. Basso accoppiamento riduce le conoscenze e le dipendenze necessarie fra componenti. Non sono valori assoluti: l'obiettivo è evitare legami inutili e rendere visibili quelli inevitabili.

FORME ARCHITETTURALI

Un sistema monolitico distribuisce insieme più funzioni. Può essere semplice da avviare e operare, ma una crescita disordinata rende più difficili modifiche e rilasci. Un'architettura a livelli separa responsabilità. Il modello client-server distingue richiedente ed erogatore. Un'architettura a servizi scompone capacità dietro contratti, aumentando l'autonomia potenziale ma anche esigenze di rete, osservabilità e governo.

Scala, competenze, vincoli, frequenza delle modifiche e qualità richieste determinano quale forma architetturale sia più adatta al caso.

ATTRIBUTI DI QUALITÀ E TRADE-OFF

Fra gli attributi di qualità rientrano affidabilità, prestazioni, usabilità, accessibilità, sicurezza e manutenibilità. Portabilità, interoperabilità e osservabilità diventano decisive quando il sistema deve integrarsi ed evolvere. Ogni scelta comporta un costo: molti servizi acquistano autonomia ma introducono comunicazioni remote e gestione distribuita; una cache riduce la latenza ma richiede una politica di coerenza. Il progetto deve dichiarare il compromesso adottato.

DAL REQUISITO ALLA DECISIONE ARCHITETTURALE

Una decisione architettuale ha valore se collega una necessità a una conseguenza verificabile. Una funzione che deve evolvere senza imporre il rilascio dell'intero sistema richiede responsabilità e contratto separati; in cambio, introduce versioni, dipendenze e gestione operativa. Con un carico prevedibile e un gruppo piccolo, invece, una distribuzione unitaria può contenere complessità e costi. La forma architettuale dipende dal problema, non dalla moda del momento.

Esigenza	Decisione da valutare	Conseguenza da verificare
modifiche frequenti a una funzione	confine di componente più autonomo	impatto su dipendenze, test e rilascio
alta disponibilità	ridondanza dei componenti critici	coerenza dello stato e gestione dei guasti
integrazione con più fruitori	contratto esterno stabile	compatibilità, documentazione e versioni
dati sensibili	separazione delle responsabilità e accessi minimi	autorizzazioni, tracciabilità e superficie esposta
tempi di risposta contenuti	cache o elaborazione differita	consistenza, invalidazione e osservabilità

Ogni diagramma risponde a una domanda diversa. La vista di contesto mostra attori e sistemi esterni; quella dei componenti chiarisce responsabilità e dipendenze; la vista di distribuzione colloca gli elementi in esecuzione. Sono rappresentazioni complementari. In una prova va scelta quella pertinente alla domanda, dichiarandone il livello di astrazione.

La qualità va resa misurabile. “Manutenibile” può tradursi in componenti con responsabilità chiare, dipendenze controllate, test aggiornati e tempo sostenibile per introdurre una modifica. “Affidabile” richiede scenari di guasto, comportamento atteso e prove. L'attributo non coincide con una tecnologia: è una proprietà osservabile del sistema nel suo contesto.

Uso concorsuale. Una risposta architettuale solida muove dai requisiti più vincolanti, propone una struttura, considera almeno un'alternativa e motiva il compromesso. Va inoltre chiarito come verificare la decisione: con test, metriche, un prototipo o una revisione dell'architettura. Senza criteri di verifica, il disegno resta soltanto illustrativo.

Le decisioni significative vanno registrate con contesto, opzioni, scelta, motivazione e conseguenze. Questa traccia evita che un vincolo temporaneo diventi una regola incomprensibile e permette di riesaminare la scelta quando cambiano carico, requisiti o dipendenze. Non serve documentare ogni dettaglio: conta conservare ciò che limita le evoluzioni future.

Un confine architetturale è efficace se protegge una responsabilità e offre un’interfaccia comprensibile. Separare componenti senza autonomia reale moltiplica chiamate e versioni; accorpare responsabilità indipendenti rende invece ogni modifica più rischiosa. Coesione e accoppiamento si valutano osservando quali cambiamenti si propagano e quali dati ogni parte deve conoscere.

Errore ricorrente. Disegnare molti blocchi e chiamarli “microservizi” non dimostra modularità. Il candidato deve spiegare proprietà dei confini, gestione dei dati, dipendenze e costo operativo. Il nome della forma architetturale viene dopo la motivazione.

6.3 Verifica, validazione, test e configurazione

La verifica controlla che il prodotto sia costruito secondo specifiche e progetto. La validazione controlla che il prodotto risponda al bisogno d’uso.

Livello	Oggetto	Esempio
unitario	funzione o componente isolato	calcolo di una regola
integrazione	interazione fra elementi	chiamata fra servizio e database
sistema	comportamento complessivo	flusso end-to-end
accettazione	criteri concordati con stakeholder	esito del caso amministrativo

La regressione è una finalità trasversale, eseguibile a più livelli: riesamina comportamenti già validi dopo una modifica.

Un caso di test specifica identificativo, obiettivo, precondizioni, dati di ingresso, passi, risultato atteso e risultato ottenuto. Un difetto è una carenza nel prodotto o in un artefatto; una failure è il comportamento osservato che non soddisfa il risultato atteso. Gravità e priorità non coincidono: la prima misura l’impatto, la seconda ordina l’intervento.

STRATEGIA, COPERTURA ED EVIDENZE

La strategia di test stabilisce che cosa verificare, a quale livello, con quali dati e in quale ambiente. Una quantità elevata di test unitari non compensa l’assenza di prove sulle integrazioni; affidarsi a un solo test end-to-end, d’altra parte, rende la diagnosi lenta e poco precisa. I livelli si completano: vicino al codice si localizzano rapidamente gli errori, mentre le prove più ampie verificano contratti e flussi reali.

La copertura non si riduce alla percentuale di righe eseguite. Occorre coprire regole, classi di input, confini, errori e rischi. Per un servizio di verifica anagrafica servono almeno esito positivo, requisito assente, identificativo non valido, fruitore non autorizzato, dipendenza indisponibile e risposta oltre il tempo previsto. Ogni caso deve dichiarare l'oracolo, cioè la regola con cui si decide se l'esito è corretto.

Artefatto	Domanda di controllo	Evidenza
requisito	è chiaro e verificabile?	criterio di accettazione approvato
componente	rispetta responsabilità e contratto?	test unitario o di componente
integrazione	i sistemi scambiano dati ed errori corretti?	test con dipendenza controllata
sistema	il flusso soddisfa il caso d'uso?	esecuzione end-to-end
rilascio	le funzioni esistenti restano valide?	suite di regressione ed esiti registrati

I dati di test devono essere pertinenti e governati. Dati sintetici facilitano casi limite e ripetibilità; dati realistici aiutano a cogliere distribuzioni e formati, ma non autorizzano l'uso indiscriminato di dati personali. Ambiente, configurazione, versione e dipendenze vanno registrati, altrimenti un esito non è riproducibile.

La gestione della configurazione collega test e prodotto. Il difetto va riferito alla versione, all'ambiente e al caso che lo ha mostrato; la correzione modifica un elemento controllato e richiede prove mirate più regressione pertinente. Una release non è soltanto un pacchetto: è un insieme identificabile di codice, configurazioni, schemi, documentazione ed evidenze.

Costruzione della risposta. Un caso di test nasce da un criterio preciso. Deve riportare precondizioni e input, il risultato atteso e l'evidenza da conservare. Dire soltanto "testare l'API" non basta: occorre dichiarare operazione, classe di input, risposta attesa e versione del contratto.

I test positivi dimostrano il comportamento previsto con input ammessi; quelli negativi verificano rifiuti ed errori controllati. Servono anche valori limite, sequenze di operazioni e condizioni concorrenti quando pertinenti. Un esito inatteso non va subito classificato come difetto del codice: può rivelare requisito ambiguo, ambiente incoerente, dato errato o test costruito male.

La validazione coinvolge il contesto d'uso. Una funzione può rispettare la specifica e risultare inadatta al procedimento amministrativo perché il requisito iniziale era incompleto. Prototipi, dimostrazioni e prove di accettazione anticipano questo rischio, purché raccolgano osservazioni trasformabili in decisioni e criteri.

Esito	Prima domanda	Azione
test fallito	atteso, input e ambiente sono corretti?	riprodurre e classificare
test intermittente	esiste dipendenza da tempo, ordine o concorrenza?	isolare la condizione
test superato ma utente insoddisfatto	il criterio rappresenta davvero il bisogno?	riesaminare requisito e validazione
regressione dopo modifica	quale dipendenza è stata coinvolta?	tracciare impatto e correggere

Errore ricorrente. Accumulare test senza una strategia crea una suite lenta e poco informativa. Ogni prova deve proteggere un requisito, un rischio o un contratto; i duplicati privi di valore aumentano manutenzione e falsi allarmi.

6.4 API come contratto

Un'API definisce come un sistema può richiedere capacità o dati a un altro. Il contratto comprende operazioni, indirizzamento, parametri, rappresentazioni, precondizioni, risposte, errori e vincoli di utilizzo. L'implementazione interna può cambiare senza rompere i fruitori finché il comportamento promesso resta compatibile.

Un'interazione sincrona attende una risposta nel contesto della richiesta. In un'interazione asincrona, richiesta ed elaborazione o notifica possono essere separate nel tempo. Eventi e messaggi sono utili quando il produttore non deve attendere il completamento immediato, ma richiedono gestione di duplicati, ordine, errori e correlazione.

REST, SOAP E RAPPRESENTAZIONI

REST è uno stile architetturale; nelle API HTTP orienta l'uso di risorse, identificatori, rappresentazioni e semantica del protocollo. SOAP è un protocollo di messaggistica basato su XML. Dire «REST usa JSON, SOAP usa XML» è incompleto: JSON è frequente nelle API REST, ma non definisce REST; SOAP possiede regole proprie oltre al formato XML.

In una API HTTP, l'URI identifica la risorsa o il punto di interazione; il metodo esprime la semantica; gli header trasportano metadati; il corpo contiene una rappresentazione quando prevista; il codice di stato descrive la classe di esito; lo schema definisce struttura e vincoli.

GET recupera una rappresentazione senza richiedere una modifica dello stato; POST invia dati per un'elaborazione; PUT sostituisce o crea lo stato della risorsa identificata secondo il contratto; PATCH applica una modifica parziale; DELETE richiede la rimozione.

L'idempotenza riguarda l'effetto intenzionale di più richieste uguali: ripeterle produce lo stesso effetto previsto sul server della singola richiesta. Non significa che ogni risposta debba essere identica né che l'operazione sia priva di effetti interni, come il logging.

UN CONTRATTO HTTP CONCRETO

REST combina separazione client/server, richieste senza dipendere da sessione applicativa sul server, cache dichiarata, interfaccia uniforme e sistema a livelli; il code-on-demand è opzionale. L'interfaccia uniforme comprende identificazione delle risorse, manipolazione mediante rappresentazioni, messaggi autoesplicativi e collegamenti che guidano le transizioni. Usare JSON non basta a soddisfare questi vincoli. Il contratto ridotto seguente illustra una API HTTP orientata a risorse, senza pretendere di dimostrare da solo un'intera architettura REST.

Il fruitore autenticato chiede GET /v1/pratiche/42, con Accept: application/json e un token autorizzato alla lettura. Il servizio verifica anche che il soggetto possa consultare quella pratica: conoscere 42 non autorizza l'accesso.

```
{"id":42,"stato":"IN_ISTRUTTORIA","versione":3,
  "links":{"self":"/v1/pratiche/42"}}
```

Esito	Codice	Contratto
Lettura autorizzata	200	Oggetto con id intero positivo, stato enumerato, versione intera
Credenziali mancanti/non valide	401	Richiesta di autenticazione secondo lo schema
Identità valida, permesso assente	403	Accesso negato; nessun dato della pratica
Identificativo malformato	400	Errore di validazione, senza stack trace
Risorsa assente	404	Esito di assenza secondo la politica di visibilità
Dipendenza temporaneamente indisponibile	503	Errore temporaneo; retry limitato secondo contratto

Esempio di errore: {"codice":"ACCESSO_NEGATO","requestId":"r-17"}. Il campo di correlazione è registrato senza token o dati superflui. Il test confronta 200 per il soggetto assegnatario e 403 per un altro soggetto autenticato. Una politica che maschera l'esistenza con 404 va dichiarata e testata in modo uniforme. Rinominare stato rompe i fruitori: /v2 o una migrazione concordata conserva la compatibilità. La risposta non contiene campi sanitari o fiscali non necessari. Per aggiornamenti concorrenti si definisce separatamente il controllo della versione, evitando che una lettura precedente autorizzi a sovrascrivere modifiche successive.

ERRORI E DESCRIZIONE FORMALE

La risposta di errore deve permettere al fruitore di capire la classe del problema, correlare l'evento e decidere se correggere la richiesta o riprovarla. Input non valido, risorsa assente, autorizzazione mancante, conflitto e indisponibilità non possono confluire tutti in un generico errore interno.

Una descrizione OpenAPI rappresenta in modo indipendente dal linguaggio la superficie di un'API HTTP: percorsi, operazioni, parametri, corpi, risposte, schemi e requisiti di sicurezza descritti. Può alimentare documentazione, test e strumenti, ma non implementa da sola il comportamento e non prova che il servizio rispetti il contratto.

DAL CASO D'USO AL CONTRATTO OSSERVABILE

La progettazione muove dalle capacità richieste dal fruitore. Prima si chiariscono attori, precondizioni e risultato; da questi elementi discendono operazioni e rappresentazioni. Il contratto deve restare comprensibile a chi non conosce tabelle interne, classi o dettagli del codice. Esporre direttamente il modello di persistenza aumenta l'accoppiamento e ostacola l'evoluzione del servizio.

Per ogni operazione vanno precisati:

- scopo e semantica;
- dati obbligatori, opzionali e relativi vincoli;
- esito normale e classi di errore;
- effetti, idempotenza e comportamento in caso di ripetizione;
- autorizzazioni richieste e informazioni restituite;
- versione, limiti e identificativo di correlazione.

Un errore applicativo dovrebbe avere struttura stabile: codice riconoscibile dal programma, messaggio destinato alla diagnosi, riferimento alla richiesta ed eventuali dettagli consentiti. Il codice HTTP offre la classe dell'esito, ma non sostituisce la semantica del dominio. Due condizioni diverse non devono diventare indistinguibili se richiedono azioni diverse del fruitore.

Condizione	Esito contrattuale	Azione del fruitore
input non valido	errore di validazione con campo o regola	correggere la richiesta
risorsa assente	esito di assenza previsto	chiudere o modificare il flusso
conflitto di stato	errore correlato alla versione o allo stato	rileggere e decidere
indisponibilità temporanea	errore distinguibile e correlabile	applicare la politica di retry

Le interazioni asincrone richiedono un contratto anche per consegna, duplicati, ordine e correlazione. Chi pubblica un evento dichiara significato e schema; chi lo consuma deve tollerare l'evoluzione prevista. “Asincrono” non significa privo di risposta: può esistere una conferma di ricezione o un'operazione separata per conoscere lo stato.

Traccia operativa. Il disegno essenziale di un'API indica finalità, operazione, input minimo, output, errori, effetti e versione. Solo dopo queste decisioni un esempio di payload diventa davvero utile. OpenAPI consente di formalizzarle, ma la qualità dipende dal contratto e non dallo strumento scelto per descriverlo.

6.5 Evoluzione e gestione delle API

Una modifica è compatibile quando i fruitori conformi al contratto precedente possono continuare a funzionare. Aggiungere un campo opzionale può essere compatibile se i client ignorano elementi sconosciuti; rinominare un campo obbligatorio o cambiarne il tipo può rompere il contratto.

Il versionamento rende distinguibili contratti o comportamenti. La deprecazione annuncia che un elemento resta temporaneamente disponibile ma sarà sostituito; richiede documentazione, transizione e percorso di migrazione. Versione dell'API, versione dell'implementazione e versione della specifica OpenAPI sono concetti distinti.

REQUISITI OPERATIVI E API MANAGEMENT

Un contratto utile specifica limiti di dimensione, paginazione e filtri; timeout e retry; throttling; disponibilità e latenza attese; identificativi di correlazione; metriche, log e canale per modifiche e incidenti.

Uno SLO esprime un obiettivo misurabile di servizio; uno SLA formalizza impegni e conseguenze nel rapporto fra parti. Il capitolo 12 affronta il profilo contrattuale.

L'API management comprende catalogazione, pubblicazione, controllo degli accessi, politiche di utilizzo, versioni, documentazione, misurazione e dismissione. Un gateway può applicare alcune politiche, ma non sostituisce governance, qualità del contratto o autorizzazione applicativa.

PROGETTARE L'EVOLUZIONE PRIMA DELLA ROTTURA

La compatibilità si giudica dal punto di vista del fruitore. Il server può accettare nuovi input senza imporre modifiche ai client, così come un client può tollerare campi aggiuntivi; altre variazioni, invece, richiedono un aggiornamento coordinato. La compatibilità all'indietro preserva i fruitori del contratto precedente. Quella in avanti riguarda la capacità dei componenti meno recenti di gestire elementi prodotti da versioni nuove, nei limiti consentiti dal contratto.

Una politica di evoluzione definisce quali cambiamenti sono ammessi nella stessa versione, come viene annunciata una deprecazione, per quanto tempo convivono i contratti e quali evidenze autorizzano la dismissione. Il solo numero di versione non protegge i fruitori. Servono inventario degli utilizzatori, telemetria, documentazione, ambiente di prova e un canale per le comunicazioni.

Modifica	Rischio	Trattamento possibile
aggiunta di campo opzionale	client rigidi rifiutano elementi ignoti	verificare regole di tolleranza e testare i fruitori
rinomina di campo obbligatorio	rottura del parsing o della logica	nuova versione o transizione esplicita
nuovo valore di enumerazione	codice client non gestisce il caso	contratto estensibile e comportamento di fallback
modifica del significato	risposta formalmente valida ma semanticamente errata	nuovo contratto e migrazione documentata
riduzione di un limite	richieste prima valide vengono rifiutate	preavviso, misura dell'uso e adeguamento concordato

Retry e idempotenza vanno progettati insieme. Un timeout lascia incerto se l'operazione sia stata eseguita; ripetere una creazione non idempotente può produrre duplicati. Il contratto può prevedere una chiave idempotente, un identificativo di richiesta o un'operazione di consultazione dello stato. La scelta dipende dal processo e deve essere verificata con casi di perdita della risposta.

Paginazione e filtri proteggono risorse e rendono prevedibile la fruizione, ma devono avere ordinamento, limiti e semantica stabili. Il throttling controlla il tasso di richieste; non sostituisce il dimensionamento né il trattamento delle priorità. Metriche e identificativi di correlazione consentono di collegare esperienza del fruitore, log del servizio e versione del contratto.

Criterio di valutazione. Una proposta di versionamento deve indicare modifica, fruitori coinvolti, rischio di incompatibilità, periodo di transizione, test e criterio di dismissione. Limitarsi a “creare la v2” sposta il problema: senza un piano di migrazione, si rischia di mantenere due contratti senza una scadenza definita.

La deprecazione ha un inizio e una fine verificabili. Si annuncia l'alternativa, si misura l'uso del contratto precedente, si assiste la migrazione e si controllano gli ultimi fruitori. La dismissione avviene quando i criteri concordati sono soddisfatti, non soltanto quando il produttore ritiene vecchia la versione.

L'osservabilità sostiene anche l'evoluzione: tasso di errori per versione, latenze, volumi, retry e operazioni deprecate mostrano l'impatto reale. Le metriche devono evitare dati personali non necessari e restare correlate a versioni e richieste. Senza questa base, una compatibilità presunta può nascondere errori dei fruitori.

6.6 Interoperabilità PA ed e-service

L'interoperabilità consente a organizzazioni e sistemi differenti di scambiare informazioni e usarle correttamente. Può essere letta sui livelli giuridico, organizzativo, semantico e tecnico.

Una API tecnicamente funzionante non risolve da sola gli altri livelli. Due enti possono scambiare un campo chiamato stato ma attribuirgli significati diversi; oppure condividere il significato ma non avere finalità e responsabilità definite.

MODI, PDND ED E-SERVICE

Il Modello di interoperabilità della PA organizza tecnologie, pattern di interazione e sicurezza, profili e regole di governance. Le fonti AgID richiamano API documentate ed evolvibili, catalogazione, tracciabilità, limitazioni d'uso e livelli di servizio.

Nel contesto PDND, un erogatore rende disponibile una capacità; un fruitore ne richiede l'uso. L'e-service descrive il servizio disponibile nel catalogo e collega l'offerta alle API e alle condizioni di fruizione. La finalità esplicita lo scopo dell'accesso. Versioni, richieste di fruizione, attributi, accordi e meccanismi tecnici devono essere verificati sulla documentazione vigente prima di configurare un caso reale.

Il principio once only orienta a non chiedere nuovamente a cittadini e imprese informazioni già legittimamente disponibili. Non autorizza accessi indiscriminati: finalità, minimizzazione, responsabilità, autorizzazioni e tracciabilità restano requisiti del servizio.

Il Regolamento (UE) 2024/903 inquadra l'interoperabilità del settore pubblico a livello europeo. Per una prova generale conta cogliere la continuità fra dimensione tecnica, organizzativa, semantica e giuridica; obblighi e applicazioni puntuali richiedono verifica normativa aggiornata.

CASO GUIDATO: VERIFICA DI UN REQUISITO ANAGRAFICO

Un’amministrazione fruitrice deve verificare un requisito anagrafico durante l’istruttoria di una pratica. L’amministrazione erogatrice possiede il dato autorevole.

- 1. Finalità: usare il dato solo per l’istruttoria prevista.
- 2. Dato minimo: ottenere un esito, non l’intero profilo anagrafico.
- 3. Requisito funzionale: inviare l’identificativo previsto e ricevere esito, data e riferimento correlabile.
- 4. Requisiti non funzionali: tracciare la richiesta, rispettare il livello di servizio e classificare gli errori.
- 5. Contratto: definire input, output, errori, schema e versione.
- 6. Autorizzazione: consentire la fruizione secondo ruoli, finalità e condizioni applicabili.
- 7. Test: verificare esito positivo, requisito assente, input non valido, fruitore non abilitato e indisponibilità.
- 8. Evidenza: correlare richiesta, risposta, versione ed esito del test.

Restituire un esito invece dell’intero record riduce dati e accoppiamento. Se il significato dell’esito cambia, contratto, documentazione e test devono evolvere insieme.

LABORATORIO: DISEGNO ESSENZIALE DI UN’API

Campo	Decisione
bisogno	quale risultato amministrativo serve?
erogatore e fruitore	chi possiede la capacità e chi la usa?
finalità	perché lo scambio è necessario?
operazione	quale capacità viene richiesta?
input minimo	quali dati sono indispensabili?
output	quale esito è utile e comprensibile?
errori	quali condizioni deve distinguere il fruitore?
idempotenza	cosa accade in caso di retry?
versione	quali modifiche sarebbero incompatibili?
qualità	quali obiettivi e metriche servono?
evidenze	come si correlano richieste, test e anomalie?

Un possibile contratto concettuale è POST /verifiche-requisito. La risposta non deve essere inventata a partire dalla tecnologia: va derivata dal bisogno, dallo schema condiviso e dalle condizioni di trattamento.

DOMANDA DA COMMISSARIO

Come passa da un requisito a un e-service interoperabile?

Inizio dal bisogno e chiarisco stakeholder, finalità e vincoli. Traduco il requisito in criteri di accettazione, poi progetto componenti, contratto API, errori e requisiti di qualità. I test devono fornire evidenza per ogni promessa. Nel caso di un e-service individuo anche erogatore, fruitore, dati minimi, condizioni di fruizione e livelli di servizio; versione, documentazione e osservabilità ne accompagnano l'evoluzione.

DOMANDA-TRAPPOLA

Un'API pubblicata è automaticamente accessibile a chiunque?

No. Pubblicare o catalogare una descrizione rende conoscibile il contratto secondo le regole del contesto, ma non elimina autenticazione, autorizzazione, finalità, protezione dei dati o condizioni di fruizione. Interoperabilità e open data non sono sinonimi.

ERRORE TIPICO

Designare subito endpoint e payload. La struttura tecnica finisce così per nascondere decisioni su finalità, significato, errori e qualità. Conviene partire dal bisogno e dal contratto osservabile; soltanto dopo si sceglie la rappresentazione.

PDND: DALL'ADESIONE ALLA CHIAMATA AUTORIZZATA

L'ente aderisce alla PDND sottoscrivendo l'accordo e assegna i ruoli abilitati. L'erogatore pubblica nel catalogo l'e-service con contratto, versione, attributi richiesti e condizioni. Il fruitore presenta la richiesta di fruizione, dimostra gli attributi pertinenti e dichiara finalità e analisi del rischio richieste; l'erogatore valuta/abilita secondo le condizioni del servizio. Si configurano client e chiavi e si ottiene il voucher da usare nella chiamata all'API dell'erogatore. Quest'ultimo verifica voucher e autorizzazioni: la piattaforma abilita uno scambio controllato, non rende pubblici tutti i dati né crea da sola la base giuridica.

Esempio. Un ufficio deve verificare un solo requisito per una pratica. Specifica finalità, attributi minimi e soggetto autorizzato, invece di richiedere l'intero archivio. Prova risposta positiva, requisito assente, voucher non valido ed e-service indisponibile; registra correlazione ed esito senza riversare il contenuto personale nei log. Revoca della fruizione, cambio di versione e variazione della finalità richiedono un riesame. Le schermate correnti sono nella documentazione PagoPA; il flusso appena descritto permette di comprenderle senza dipendere da un pulsante o nome di menu.

▣ Verifica

ESERCIZIO 1 — REQUISITO E TEST

Trasforma «il servizio deve essere veloce» in un requisito verificabile.

Soluzione svolta, valori ipotetici: su ambiente di collaudo dichiarato, con 100 utenti concorrenti e un milione di pratiche, il 95° percentile delle risposte valide di GET /v1/pratiche/{id} deve essere al massimo 800 ms durante 30 minuti a carico stabile; errori inferiori allo 0,5%. Il test genera quel carico, conserva latenza per richiesta ed errori e verifica entrambe le soglie. Non scarta le richieste fallite per migliorare artificialmente il percentile. Questi valori sono requisiti dell'esercizio, non soglie imposte alla PA.

ESERCIZIO 2 — COMPATIBILITÀ

Un campo obbligatorio esito viene rinominato risultato. La modifica è compatibile?

Soluzione: in generale no per i fruitori che cercano esito. Occorre mantenere il campo durante una transizione, introdurre una nuova versione o concordare una migrazione verificabile.

ESERCIZIO 3 — CASO E-SERVICE

Elenca cinque elementi da chiarire prima della fruizione.

Soluzione: erogatore, fruitore, finalità, dato o capacità minima, condizioni di accesso; vanno inoltre definiti contratto, errori, livelli di servizio ed evidenze.

QUIZ 1

Quale affermazione è corretta?

- A. Validazione e verifica sono sempre sinonimi.
- B. OpenAPI implementa automaticamente il servizio descritto.
- C. Un test di regressione controlla funzioni già valide dopo una modifica.
- D. Interoperabilità significa rendere ogni dato pubblico.

Risposta corretta: C. A confonde adeguatezza allo scopo e conformità; B attribuisce alla descrizione il comportamento dell'implementazione; D confonde scambio governato e open data.

QUIZ 2

Quale formulazione è un criterio di accettazione verificabile?

- A. Il servizio deve essere moderno.
- B. Il servizio deve essere facile da usare.
- C. Con input valido, il servizio restituisce esito e identificativo correlabile secondo i casi concordati.
- D. Il gruppo deve adottare un metodo Agile.

Risposta corretta: C. Indica una condizione osservabile e un risultato atteso. A e B sono qualità non misurate; D prescrive un metodo senza collegarlo al bisogno o all'esito.

QUIZ 3

Quale affermazione descrive correttamente un trade-off architetturale?

- A. Più servizi riducono sempre la complessità.
- B. Una cache può ridurre la latenza ma introduce problemi di coerenza e invalidazione.
- C. Un monolite non può avere componenti interni.
- D. La tecnologia determina da sola la qualità.

Risposta corretta: B. La decisione migliora una proprietà e crea nuovi costi da governare. A, C e D trasformano scelte contestuali in regole assolute.

QUIZ 4

Un test verifica l'interazione fra servizio applicativo e database. A quale livello appartiene?

- A. Unitario.
- B. Integrazione.
- C. Accettazione, necessariamente.
- D. Regressione, necessariamente.

Risposta corretta: B. L'oggetto è l'interazione fra elementi. Lo stesso test può entrare in una suite di regressione, ma "regressione" descrive lo scopo della riesecuzione, non sostituisce il livello.

QUIZ 5

Quale modifica presenta il rischio più evidente di rompere i fruitori esistenti?

- A. Correggere un refuso nella documentazione.
- B. Aggiungere un esempio non normativo.
- C. Rinominare un campo obbligatorio della risposta.
- D. Aggiungere una metrica interna non esposta.

Risposta corretta: C. I fruitori che cercano il nome precedente non trovano più il dato atteso. Le altre modifiche non cambiano necessariamente il contratto osservabile.

QUIZ 6

Che cosa rende legittima e governabile la fruizione di un e-service?

- A. La sola presenza dell'API nel catalogo.
- B. La disponibilità tecnica dell'endpoint.
- C. Finalità, ruoli, condizioni di fruizione, dati minimi, autorizzazioni ed evidenze coerenti.
- D. L'uso obbligatorio di un formato aperto per qualunque dato.

Risposta corretta: C. Catalogazione e raggiungibilità tecnica non bastano. La fruizione deve rispettare il contesto giuridico-organizzativo, semantico e tecnico.

Checklist finale

Competenza	Riconosco	Spiego	Applico
Distinguere bisogno, requisito, vincolo e criterio	☐	☐	☐
Collegare requisito, progetto, test ed evidenza	☐	☐	☐
Confrontare modelli di ciclo di vita	☐	☐	☐
Leggere componenti, interfacce e dipendenze	☐	☐	☐
Distinguere verifica, validazione e livelli di test	☐	☐	☐
Spiegare configurazione e controllo di versione	☐	☐	☐
Progettare il contratto essenziale di un'API	☐	☐	☐
Valutare compatibilità, errori e idempotenza	☐	☐	☐
Distinguere i livelli di interoperabilità	☐	☐	☐
Costruire un caso e-service	☐	☐	☐

Da sapere in 5 righe

- Il ciclo di vita collega bisogno, requisiti, progetto, test, rilascio ed evoluzione.
- Architettura e qualità si valutano attraverso trade-off espliciti.
- Un'API è un contratto: operazioni, dati, errori, vincoli e livelli di servizio.
- Interoperabilità tecnica, semantica, organizzativa e giuridica devono convergere.

- Un e-service richiede erogatore, fruitore, finalità, condizioni, evidenze e governo delle modifiche.

Riferimenti normativi e professionali essenziali

- Codice dell'amministrazione digitale, decreto legislativo 7 marzo 2005, n. 82, per il quadro generale dell'interoperabilità pubblica.
- AgID, Linee guida sull'interoperabilità tecnica delle Pubbliche Amministrazioni e relativi allegati.
- AgID, Linee guida sull'infrastruttura tecnologica della PDND per l'interoperabilità dei sistemi informativi e delle basi di dati.
- Piano Triennale per l'informatica nella PA 2024-2026, sezione sugli e-service in interoperabilità tramite PDND.
- Regolamento (UE) 2024/903, Interoperable Europe Act, per il quadro europeo dell'interoperabilità del settore pubblico.
- IETF, RFC 9110, per la semantica HTTP; OpenAPI Initiative, OpenAPI Specification, per la descrizione delle API HTTP.
- IEEE Computer Society, Guide to the Software Engineering Body of Knowledge, per requisiti, progettazione, qualità, test e gestione della configurazione.
- Il Metodo BANDO, capitolo 10, § 16, per i prerequisiti su API, PDND e interoperabilità richiamati nel confine iniziale.

Cloud PA, virtualizzazione, container e DevOps

B
BANDO**A**
AREE**N**
NUCLEI**D**
DIARIO**O**
OUTPUT

Spostare un servizio pubblico «su Internet» non basta a modernizzarlo. Una migrazione al cloud cambia il modo in cui si erogano le risorse, si distribuiscono le modifiche e si affrontano i guasti. Cambia anche la ripartizione delle responsabilità.

In una prova tecnica, elencare IaaS, PaaS e SaaS serve a poco se il ragionamento si ferma lì. La risposta deve collegare il modello cloud alla classificazione dei dati, distinguere macchine virtuali e container e descrivere una pipeline controllata. Osservabilità, backup e continuità operativa completano il quadro.

Obiettivo e confine con il volume base

Il capitolo 10, § 17, del VOL-01 introduce cloud computing, IaaS, PaaS e SaaS e i criteri generali per la PA. Qui quei prerequisiti diventano competenza specialistica.

Al termine del capitolo saprai:

- riconoscere caratteristiche e modelli del cloud;
- spiegare la ripartizione delle responsabilità;
- confrontare virtualizzazione e containerizzazione;
- impostare assessment e strategia di migrazione;
- descrivere DevOps, CI/CD e Infrastructure as Code;
- distinguere monitoraggio e osservabilità;
- collegare resilienza, backup, disaster recovery e continuità;
- costruire una checklist utilizzabile in prova.

Reti e troubleshooting sono nel capitolo 5; ciclo di vita, test e API nel capitolo 6; sicurezza nel capitolo 8; IAM e incident response nel capitolo 9; procurement e SLA contrattuali nel capitolo 12.

Mappa BANDO: dalla classificazione all'esercizio

Nel bando cerca formule come cloud computing, infrastrutture virtuali, container, orchestrazione, DevOps, CI/CD, monitoraggio, backup, disaster recovery e continuità operativa. La profondità dipende dal profilo e dal tipo di prova.

Segnale	Nucleo	Output
IaaS/PaaS/SaaS	modelli e responsabilità	confronto motivato
Cloud PA, ACN	classificazione e qualificazione	piano di migrazione
Virtualizzazione	hypervisor, VM, immagini	schema host-guest
Container	immagini, runtime, orchestrazione	confronto VM-container
DevOps, CI/CD	pipeline, artefatti, rollback	flusso di rilascio
Observability	metriche, log, trace	diagnosi guidata
Backup, DR, BC	copie, RPO, RTO, piani	scenario di continuità

Il metodo segue quattro passaggi: classificare, motivare, progettare e verificare. Prima si chiariscono servizio, dati e impatti. Su questa base si sceglie l'architettura e si dimostra che rilascio, esercizio e ripristino sono governati.

7.1 Quadro, modelli cloud e responsabilità

Il modello cloud va letto come un insieme di caratteristiche operative e di confini di responsabilità. La risposta utile non si limita alle sigle: collega servizio, dati, soggetti coinvolti e conseguenze per l'ente.

CINQUE CARATTERISTICHE ESSENZIALI

La definizione NIST riconosce il cloud attraverso cinque caratteristiche:

1. servizio su richiesta, con provisioning fortemente automatizzato;
2. accesso ampio tramite rete, mediante meccanismi standard;
3. condivisione delle risorse, assegnate dinamicamente a più consumatori;
4. elasticità rapida, per crescere o ridursi al variare del carico;
5. servizio misurato, così da osservare e rendicontare il consumo.

Queste proprietà distinguono il cloud da una semplice macchina remota. Un server accessibile via rete, assegnato manualmente e dimensionato una volta per tutte, può essere infrastruttura esterna senza offrire davvero il modello operativo del cloud.

La scalabilità è la capacità di sostenere più carico aggiungendo risorse. L'elasticità aggiunge l'adattamento dinamico, anche verso il basso. Lo scaling è verticale quando aumenta le risorse di una singola istanza, orizzontale quando aumenta le istanze.

MODELLI DI SERVIZIO

Modello	Il provider offre	L'amministrazione governa soprattutto
IaaS	calcolo, rete, storage e virtualizzazione	guest OS, middleware, applicazioni, dati e configurazioni
PaaS	infrastruttura e piattaforma	applicazioni, dati, configurazioni e uso
SaaS	applicazione completa	utenti, dati, configurazione funzionale e processi

Da IaaS a SaaS diminuisce la gestione tecnica diretta, ma non scompare la responsabilità dell'ente. Restano classificazione dei dati, scelta del servizio, identità, autorizzazioni, configurazione, controllo del fornitore, continuità del processo e possibilità di uscita.

Il modello serverless nasconde provisioning e gestione dei server al consumatore. Il termine non significa che i server non esistano.

MODELLI DI DISTRIBUZIONE

Il cloud pubblico è offerto a una pluralità di clienti: «pubblico» non significa «della pubblica amministrazione». Il cloud privato è destinato a una sola organizzazione. Il cloud ibrido coordina ambienti differenti. Il community cloud è condiviso da organizzazioni con esigenze comuni.

Nelle architetture dei provider ricorrono:

- regione, area geografica in cui sono disponibili risorse;
- zona, dominio infrastrutturale separato;
- tenancy, modalità di assegnazione delle risorse a uno o più clienti.

Distribuire istanze su più zone può ridurre l'impatto di un guasto, ma non crea automaticamente continuità: servono dati coerenti, instradamento, capacità, procedure e test.

RESPONSABILITÀ CONDIVISA

La linea di separazione fra provider e cliente varia con servizio e contratto. In IaaS l'ente governa una parte maggiore dello stack; in SaaS il provider ne gestisce una parte più ampia. Non è mai corretto concludere: «è nel cloud, quindi se ne occupa tutto il fornitore».

Per ogni componente occorre chiedere:

1. chi lo configura?
2. chi lo aggiorna?
3. chi ne osserva lo stato?
4. chi interviene e risponde se fallisce?

La matrice fa emergere le zone grigie: il provider può garantire l'infrastruttura mentre l'ente mantiene account o configurazioni errate; il servizio tecnico può essere disponibile mentre il procedimento amministrativo non dispone di una modalità alternativa.

In sede concorsuale conviene trasformare la responsabilità condivisa in una piccola matrice. Per esempio, in un servizio SaaS il fornitore può curare la disponibilità dell'applicazione, ma l'amministrazione deve decidere chi può accedere, con quali profili, quali dati inserire e come controllare le configurazioni che incidono sul procedimento. Se il servizio usa un'integrazione con un archivio dell'ente, occorre inoltre chiarire chi monitora il collegamento e chi comunica l'interruzione agli uffici. La risposta è più solida quando individua il componente, il responsabile, l'evidenza attesa e il punto di escalation.

La scelta del modello non discende solo dal carico previsto. Un'applicazione legacy con requisiti particolari può richiedere maggiore controllo dell'ambiente; un servizio standard può beneficiare di una piattaforma gestita. In entrambi i casi l'ente deve valutare competenze interne, vincoli sui dati, integrazioni, continuità e possibilità di cambiare soluzione. "Più gestito" non significa automaticamente "più adatto": sposta il confine tecnico, ma non elimina le decisioni organizzative.

7.2 Virtualizzazione, container e orchestrazione

Un candidato deve saper motivare la scelta senza ridurla a una preferenza di prodotto.

Qui la questione decisiva è capire che cosa viene isolato, come viene distribuito e quali dati devono sopravvivere al riavvio. La tecnologia scelta deve rendere l'esercizio più controllabile, non soltanto più rapido da avviare.

HYPERVERSOR E MACCHINE VIRTUALI

La virtualizzazione presenta risorse logiche separate sopra la stessa infrastruttura fisica. L'hypervisor crea e governa macchine virtuali, ripartendo CPU, memoria, storage e dispositivi. Quello di tipo 1 opera direttamente sull'hardware; quello di tipo 2 si appoggia a un sistema operativo host.

Ogni VM possiede normalmente sistema operativo guest, librerie e applicazioni. L'isolamento permette carichi differenti sullo stesso host, ma richiede gestione di immagini, patch, configurazioni e capacità.

Una immagine è una base da cui creare istanze; un template aggiunge spesso configurazioni e metadati. Lo snapshot registra lo stato in un momento dato, ma non è automaticamente un backup indipendente: può condividere storage, amministrazione e dominio di guasto con l'originale.

La sovra-allocazione aumenta l'efficienza quando i carichi non raggiungono insieme il massimo, ma produce contesa se le assunzioni sono errate. Le prestazioni vanno quindi misurate.

IMMAGINI E CONTAINER

Un container esegue un processo isolato con codice e dipendenze, condividendo il kernel dell'host secondo il modello del runtime. Una container image è il pacchetto pronto all'esecuzione; il registry conserva e distribuisce immagini; il runtime ne governa esecuzione e ciclo di vita.

L'immagine deve essere identificabile, versionata e ricostruibile. Modificare manualmente un container in esecuzione crea differenze non tracciate: si produce invece una nuova immagine e si ricrea il workload.

I dati che devono sopravvivere alla sostituzione del container richiedono storage persistente progettato e protetto.

Aspetto	VM	Container
Unità	sistema virtuale con OS guest	processo con dipendenze
Kernel	normalmente proprio guest kernel	normalmente condiviso con l'host
Avvio	in genere più pesante	in genere più rapido
Portabilità	immagine dell'ambiente	immagine applicativa
Uso	OS diversi, legacy, isolamento infrastrutturale	servizi replicabili e distribuibili

VM e container possono convivere: molti cluster di container funzionano sopra VM. La scelta dipende da requisiti, competenze, isolamento e carico.

ORCHESTRAZIONE E STATO DESIDERATO

Un orchestratore assegna workload ai nodi, mantiene repliche, espone servizi, distribuisce configurazioni e reagisce a determinati guasti.

Kubernetes è un esempio. Il suo principio centrale è la riconciliazione: si dichiara lo stato desiderato e il control plane confronta continuamente stato attuale e desiderato, adottando azioni correttive. Un Pod è la minima unità distribuibile e può contenere uno o più container co-localizzati.

L'orchestratore non rende affidabile un'applicazione per magia. Se tutte le repliche dipendono da un unico database fragile, quello resta un punto critico. Vanno progettati anche configurazioni, secret, storage, rete e limiti di risorse.

Un confronto utile parte dall'unità che si vuole governare. La VM incapsula un intero ambiente operativo e rende possibile eseguire sistemi diversi sullo stesso hardware; il container impacchetta soprattutto il processo applicativo con le sue dipendenze. Per questo un container non sostituisce automaticamente una VM e non ne eredita, da solo, le garanzie di isolamento, persistenza o continuità. In un servizio pubblico possono convivere: VM per componenti legacy o per delimitare ambienti, container per servizi che si aggiornano e si replicano con maggiore frequenza.

L'orchestrazione introduce una disciplina ulteriore. Il team descrive il numero di repliche, le risorse richieste, le configurazioni e le condizioni di salute; il sistema tenta di riportare il carico allo stato dichiarato. Questo rende visibile la differenza tra una ripartenza casuale e una procedura ripetibile, ma richiede immagini controllate, registri affidabili, configurazioni separate dal codice e limiti che impediscano a un singolo workload di consumare tutte le risorse. La domanda da evitare è "quale orchestratore usiamo?" prima di aver chiarito disponibilità, dipendenze, dati persistenti e competenze di esercizio.

7.3 Cloud PA e percorso di migrazione

Il piano deve indicare ciò che cambia, ciò che resta e come si controlla il passaggio.

Una migrazione ordinata parte dalla funzione pubblica, non dalla piattaforma. Ogni scelta deve lasciare una motivazione, una prova e una via di ritorno praticabile.

CLASSIFICARE PRIMA DI SCEGLIERE

La Strategia Cloud Italia si fonda su classificazione di dati e servizi, qualificazione dei servizi cloud e Polo Strategico Nazionale. La classificazione valuta l'impatto della compromissione e distingue dati e servizi strategici, critici e ordinari.

La sequenza conta. Prima si esaminano dati, funzioni, utenti, dipendenze, impatti e obblighi; solo dopo si individua una destinazione compatibile. Partire dal prodotto rovescia l'ordine del problema.

La qualificazione dei servizi cloud per la PA è di competenza dell'ACN. Il riferimento è il Regolamento unico ACN adottato con decreto direttoriale 21007/24 del 27 giugno 2024, applicabile nel regime ordinario dal 1 agosto 2024. La qualificazione riguarda lo specifico servizio e il relativo livello, non ogni prodotto del fornitore; la validità e il mantenimento dei requisiti si controllano nel catalogo e nel provvedimento. Poiché disciplina e catalogo evolvono, ogni decisione richiede il controllo della versione vigente.

Il Polo Strategico Nazionale è un'infrastruttura ad alta affidabilità destinata, nel quadro della strategia, anche a dati e servizi critici e strategici. Non è la destinazione automatica di ogni applicazione.

IMPATTO E CLASSE: TRE DISTINZIONI

Classe	Conseguenza della compromissione	Esempio didattico di valutazione
Strategica	Impatto sulla sicurezza nazionale e sulle funzioni essenziali dello Stato	Sistema la cui indisponibilità compromette una funzione essenziale di sicurezza nazionale
Critica	Pregiudizio ai servizi rilevanti per società, salute, sicurezza e benessere economico/sociale	Servizio sanitario la cui interruzione impedisce prestazioni rilevanti
Ordinaria	Dati e servizi non ricadenti nei livelli precedenti secondo la valutazione d'impatto	Informazioni di consultazione con impatto limitato, senza dipendenze critiche nascoste

Gli esempi non assegnano una classe automatica a tutti i sistemi di quel settore. Un portale informativo può dipendere da un servizio critico; un dato già pubblico può richiedere elevata integrità. La classificazione considera riservatezza, integrità, disponibilità e conseguenze, mentre la qualificazione dimostra requisiti dell’offerta cloud. Il responsabile censisce dati e servizi, motiva la classe, controlla la compatibilità della destinazione e conserva l’esito; una generica certificazione commerciale non sostituisce questi passaggi. La strategia, il regolamento e il catalogo svolgono funzioni diverse: il primo orienta, il secondo regola, il terzo permette di verificare l’offerta corrente.

ASSESSMENT

Per ogni applicazione occorre rilevare:

- finalità, utenti e finestre di servizio;
- dati, classificazione e vincoli;
- componenti, versioni e manutenzione;
- dipendenze da database, directory, file, API, rete e dispositivi;
- volumi, picchi, latenza e capacità;
- identità e ruoli operativi;
- disponibilità, RPO e RTO;
- contratti, licenze, competenze e costi;
- backup, ripristino e modalità di uscita.

L’inventario evita di migrare l’applicazione scoprendo solo dopo che una dipendenza rimasta on premise rende il servizio lento o inutilizzabile.

STRATEGIE DI MIGRAZIONE

Le cosiddette «R» sono una tassonomia operativa:

- rehost: trasferimento con modifiche minime;
- replatform: modifica di alcuni componenti per usare servizi gestiti;
- refactor: riprogettazione sostanziale;
- repurchase: sostituzione con una soluzione acquistata come servizio;
- retain: mantenimento temporaneo motivato;
- retire: dismissione dopo aver gestito dati e dipendenze.

La strategia può differire fra componenti dello stesso servizio. Va scelta in base a benefici, rischio, tempi, competenze e reversibilità, non per moda.

PORTABILITÀ, REVERSIBILITÀ ED EXIT STRATEGY

La portabilità consente di spostare applicazioni o dati con sforzo accettabile. La reversibilità permette di ritornare o trasferire il servizio. L'exit strategy specifica:

- formati e modalità di esportazione;
- volumi, tempi e costi;
- dipendenze proprietarie;
- restituzione o cancellazione dei dati;
- trasferimento di configurazioni, log e documentazione;
- responsabilità durante la transizione;
- test della procedura.

Il lock-in può essere tecnologico, contrattuale, organizzativo o legato alle competenze. Deve essere gestito, non semplicemente dichiarato assente.

CUTOVER E ROLLBACK

Il cutover è il passaggio controllato al nuovo ambiente. Richiede criteri di ingresso, sincronizzazione dei dati, test finali, comunicazioni e responsabilità. Il rollback riporta a uno stato precedente quando i criteri non sono soddisfatti.

Un rollback credibile indica fino a quando è possibile tornare indietro, come riallineare i dati e chi decide. Dopo nuove transazioni, non basta riavviare il vecchio ambiente.

La migrazione è quindi un cambiamento di servizio, non una copia di macchine. L'assessment deve produrre decisioni verificabili: quali componenti si spostano, quali restano temporaneamente dove sono, quale dipendenza condiziona il passaggio e quale prova dimostra che il nuovo ambiente funziona. Una scelta di rehost può ridurre il tempo iniziale, ma può conservare vincoli del sistema precedente; un refactor può migliorare portabilità o resilienza, ma aumenta il rischio di modifica. Non esiste una "R" migliore in astratto.

Nel contesto della PA, classificazione e qualificazione vanno lette prima dell'architettura di dettaglio. La disciplina Cloud Italia e il catalogo ACN sono soggetti ad aggiornamento: nella prova si descrive il metodo di verifica della destinazione compatibile, senza trasformare nel testo un catalogo di prodotti o una lista di soglie. L'evidenza finale comprende inventario, esito della classificazione, scelta motivata, piano di test, procedura di cutover, condizioni di rollback e responsabilità durante la transizione.

7.4 DevOps, CI/CD e Infrastructure as Code

La pipeline è una catena di responsabilità, non un semplice automatismo. Il codice viene associato a una richiesta, trasformato in un artefatto identificabile e sottoposto ai controlli adatti al suo impatto. L'artefatto verificato non dovrebbe cambiare mentre attraversa gli ambienti: altrimenti non si sa più se la produzione esegue ciò che è stato provato. Quando un controllo fallisce, il flusso deve fermarsi e produrre un'evidenza utile alla correzione.

Anche il rollback richiede preparazione. Una nuova versione può essere tecnicamente distribuibile ma incompatibile con una modifica del dato o con un'interfaccia già usata da altri sistemi. Per questo si definiscono compatibilità, criteri di annullamento, responsabilità e modalità di comunicazione. Il rilascio progressivo limita l'esposizione iniziale, ma non sostituisce test, osservazione e decisioni sulle conseguenze per gli utenti.

DEVOPS COME MODELLO OPERATIVO

DevOps mette sviluppo e operazioni sullo stesso flusso di lavoro. Riduce i passaggi manuali poco trasparenti, accorcia i tempi di feedback e favorisce modifiche circoscritte, verificabili e recuperabili.

Gli elementi ricorrenti sono collaborazione, responsabilità sul servizio, automazione, versionamento, feedback, misurazione e miglioramento continuo. DevOps non coincide con un team, una qualifica o un prodotto.

PIPELINE CI/CD

Una pipeline tipica comprende:

1. acquisizione del codice;
2. build o packaging;
3. test automatici;
4. controlli di qualità e sicurezza;
5. produzione di un artefatto identificato;
6. promozione fra ambienti;
7. distribuzione;
8. verifica ed evidenze.

La continuous integration integra spesso le modifiche e le verifica automaticamente. La continuous delivery mantiene il software rilasciabile, lasciando possibile una decisione esplicita per la produzione. La continuous deployment distribuisce automaticamente ogni modifica che supera i gate.

L'automazione rende i controlli ripetibili; non li elimina. La pipeline può registrare approvazioni, separazione dei compiti ed evidenze. L'artefatto che supera i test dovrebbe essere lo stesso promosso negli ambienti successivi, perché ricostruirlo introduce differenze.

INFRASTRUCTURE AS CODE

Con l'Infrastructure as Code l'infrastruttura è descritta in file versionati e applicata automaticamente. Le modifiche diventano confrontabili, revisionabili e ripetibili.

Il modello dichiarativo descrive lo stato desiderato; quello imperativo una sequenza di operazioni. IaC non garantisce correttezza: un errore codificato può essere replicato ovunque. Servono revisioni, test, gestione separata dei segreti, accessi controllati e confronto con lo stato reale.

DISTRIBUZIONE E ROLLBACK

Il rolling update sostituisce gradualmente le istanze. Il blue-green prepara un ambiente parallelo e sposta il traffico dopo le verifiche. Il canary release espone inizialmente la versione a una quota limitata.

Queste tecniche non sostituiscono compatibilità dei dati e rollback. È utile separare deployment e release: il codice può essere distribuito senza attivare subito la funzione.

Un processo DevOps maturo non accelera le modifiche saltando i controlli. Li rende osservabili e ripetibili. Ogni passaggio deve poter rispondere a quattro domande: quale modifica è stata richiesta, quale artefatto è stato verificato, chi ha autorizzato il passaggio e quale evidenza dimostra l'esito. Questa catena è utile anche quando lo sviluppo è affidato a un fornitore, perché permette all'amministrazione di governare il servizio senza confondere la consegna del codice con la sua effettiva messa in esercizio.

Dopo il rilascio si verificano funzioni, prestazioni ed effetti sui dati; la configurazione versionata conserva il collegamento tra modifica ed esito.

ESEMPIO DI RILASCIO TRACCIABILE

La richiesta R42 corregge il calcolo di una scadenza. Il commit C7 produce l'artefatto con digest H; i test verificano giorno lavorativo, festivo e fine mese. In collaudo si usa proprio H, insieme alla versione documentata dello schema dati. Se un pacchetto diverso viene ricostruito per la produzione, quei risultati non ne attestano automaticamente la qualità. Il responsabile autorizza la promozione di H e registra ambiente, ora ed esito. Prima del passaggio, una migrazione aggiunge un campo mantenendo compatibilità con la versione precedente; solo un rilascio successivo rimuove il vecchio campo dopo il controllo degli utilizzatori.

Questa sequenza rende possibile un rollback applicativo senza presumere che ogni modifica distruttiva del database sia reversibile. La verifica finale confronta casi reali autorizzati, errori e tempi con la baseline.

7.5 Operabilità, osservabilità, capacità e costi

La capacità non riguarda soltanto il numero di server. Comprende connessioni, code, spazio, limiti imposti da servizi esterni e persone in grado di intervenire. Un piano di capacità confronta il profilo storico con gli eventi attesi, dichiara quale margine è necessario e stabilisce cosa fare quando il margine si riduce. Questo evita sia il sovradimensionamento costoso sia una reazione tardiva durante una scadenza.

L'osservabilità è utile anche dopo una modifica. Si confrontano i segnali prima e dopo il rilascio, si verificano errori, tempi di completamento e comportamento delle integrazioni. Se gli utenti non riescono a concludere una pratica, il fatto che i singoli componenti risultino attivi non basta. Occorre seguire la transazione, individuare il punto di blocco e registrare la decisione presa. Lo stesso ragionamento aiuta a discutere il costo: una spesa va collegata a capacità effettivamente richiesta, finestra di utilizzo, livello di continuità e rischio che si intende ridurre.

La revisione periodica trasforma i dati raccolti in gestione: si eliminano indicatori che non guidano alcuna azione, si affinano le soglie che producono falsi allarmi e si aggiornano le procedure quando il servizio o le sue dipendenze cambiano.

METRICHE, LOG E TRACE

Il monitoraggio verifica condizioni note. L'osservabilità permette di comprendere lo stato interno attraverso i segnali prodotti, anche per problemi non previsti da una soglia.

- metriche: misure numeriche nel tempo;
- log: registrazioni di eventi con contesto;
- trace: percorso di una richiesta fra componenti;
- eventi: cambiamenti significativi, come un deployment.

I segnali sono complementari: la metrica mostra un aumento di latenza; la trace localizza il tratto lento; il log spiega un rifiuto del database; l'evento collega il problema a una modifica.

Pannello operativo e alert devono rappresentare il servizio. Un server può essere attivo mentre l'utente non completa la pratica. Gli alert devono essere azionabili, con impatto, contesto ed escalation.

CAPACITÀ, ELASTICITÀ E COSTI

Il capacity planning stima risorse e margini in base a domanda e obiettivi. L'autoscaling reagisce a segnali e politiche, ma non corregge una query inefficiente o una dipendenza satura.

Il consumo misurato richiede attribuzione dei costi, budget, allarmi e spegnimento delle risorse inutili. Nel governo FinOps il prezzo è uno dei criteri: continuità, sicurezza, competenze e reversibilità conservano il loro peso.

Operare un servizio significa collegare segnali tecnici e risultato per l'utente. Un pannello operativo che mostra solo CPU e memoria aiuta il gestore, ma non basta a capire se una pratica viene protocollata o se un pagamento termina correttamente. Per questo si definiscono indicatori del servizio, finestre temporali, soglie, destinatari degli avvisi e procedure di escalation. Un alert utile contiene almeno il sintomo, l'impatto presunto, il componente coinvolto e il primo controllo da eseguire.

Metriche, log e trace svolgono ruoli diversi. La metrica quantifica un fenomeno nel tempo, il log conserva un evento con il suo contesto e la trace ricostruisce il cammino di una singola richiesta. Durante un rallentamento del portale, l'aumento della latenza può segnalare il problema, una trace può mostrare che il tempo si concentra sull'integrazione esterna e i log possono indicare rifiuti o timeout. Nessun segnale, isolato, sostituisce gli altri.

Capacità e costi richiedono la stessa disciplina: si misurano domanda, picchi, margini e consumo, poi si assegnano responsabilità per le decisioni. Autoscaling e spegnimento programmato sono opzioni, non formule universali. Un aumento automatico delle istanze non risolve una dipendenza saturata, un limite di database o un errore applicativo che moltiplica le richieste. Il candidato deve dimostrare di saper scegliere una misura, osservare l'effetto e riesaminare la decisione.

Esempio di capacità. Una coda riceve 120 richieste al minuto e ne smaltisce 100: in dieci minuti, senza altri cambiamenti, l'arretrato cresce di 200. Aggiungere CPU al frontend non basta se il collo di bottiglia è il limite della dipendenza esterna. Si misura quel tratto e si valuta capacità, ammissione delle richieste e informazione agli utenti.

7.6 Resilienza, backup, disaster recovery e continuità operativa

Ogni scelta resta verificabile nel tempo.

La scelta delle copie e delle architetture deve quindi essere proporzionata: non tutti i servizi richiedono la stessa rapidità di recupero, ma nessun servizio essenziale può basarsi su un ripristino mai eseguito.

La continuità richiede di distinguere ciò che riduce l'interruzione da ciò che permette di recuperare dati e servizi dopo un evento grave. La misura corretta dipende dall'impatto della funzione pubblica e deve essere dimostrata con prove.

RESILIENZA E ALTA DISPONIBILITÀ

La resilienza è la capacità di assorbire guasti, adattarsi e ripristinare. L'alta disponibilità riduce le interruzioni mediante ridondanza, bilanciamento, replica e rimozione dei punti singoli di guasto.

La ridondanza funziona solo se i componenti non condividono lo stesso dominio di guasto. Due istanze sul medesimo host non proteggono dal guasto dell'host. Due zone non proteggono da una configurazione errata distribuita ovunque.

La degradazione controllata mantiene le funzioni essenziali quando una parte non è disponibile. La priorità deriva dall’impatto sul servizio pubblico.

BACKUP, SNAPSHOT E REPLICA

Il backup è una copia destinata al ripristino. Può essere:

- completo, se copia tutto l’insieme previsto;
- incrementale, se copia le modifiche dall’ultimo backup;
- differenziale, se copia le modifiche dall’ultimo completo.

Vanno definite frequenza, conservazione, cifratura, accessi, separazione, immutabilità quando appropriata e cancellazione.

Uno snapshot registra rapidamente uno stato; una replica mantiene una copia aggiornata. Nessuno dei due è automaticamente un backup. La replica può propagare cancellazioni o corruzioni; lo snapshot può condividere il dominio di guasto.

Un job concluso con esito positivo dimostra che la copia è stata eseguita, non che il ripristino funzionerà. La verifica richiede il recupero effettivo dei dati entro l’obiettivo e controlla integrità, dipendenze, credenziali, procedure e tempi.

RPO E RTO

Il Recovery Point Objective (RPO) esprime la perdita temporale massima di dati tollerabile. Il Recovery Time Objective (RTO) esprime il tempo obiettivo entro cui il servizio deve tornare utilizzabile.

RPO e RTO derivano dall’analisi d’impatto. Obiettivi più stringenti richiedono normalmente architetture e costi maggiori; non vanno copiati da valori standard.

DISASTER RECOVERY E BUSINESS CONTINUITY

Il disaster recovery riguarda il ripristino di sistemi e servizi dopo un evento grave: architettura alternativa, copie, priorità, ruoli, failover e failback.

La business continuity è più ampia: mantiene o riprende le attività essenziali mediante persone, sedi, fornitori, comunicazioni e anche procedure manuali temporanee.

Concetto	Domanda
Alta disponibilità	come riduco le interruzioni ordinarie?
Backup	da quale copia recupero dati e configurazioni?
Disaster recovery	come ripristino i sistemi dopo un evento grave?
Business continuity	come mantengo le funzioni essenziali dell’ente?

Un piano acquista valore operativo solo quando viene provato. Simulazioni e prove di ripristino fanno emergere dipendenze dimenticate, accessi scaduti, documentazione incompleta e tempi irrealistici.

RPO e RTO non sono promesse generiche di "ripartenza rapida". Il primo impone di stabilire quale quantità di dati può andare persa fra due punti di ripristino; il secondo impone di stabilire entro quando la funzione deve tornare utilizzabile. Se un servizio acquisisce pratiche fino a una scadenza, un RPO troppo ampio può rendere necessario ricostruire molte operazioni; se la funzione è indispensabile in una finestra amministrativa, un RTO troppo lungo può interrompere il procedimento. Gli obiettivi nascono dall'analisi d'impatto e vanno tradotti in copie, procedure, ruoli e prove.

Una soluzione resiliente deve considerare anche i guasti comuni. Repliche nello stesso dominio di guasto, credenziali disponibili solo a una persona, runbook non aggiornati o un collegamento esterno non alternativo possono vanificare l'architettura. Il test di ripristino non è un adempimento conclusivo: misura tempi, integrità dei dati, sequenza delle dipendenze, comunicazioni e capacità di operare in modalità degradata. La business continuity completa il disaster recovery perché decide come l'ente continua le attività essenziali mentre i sistemi sono ripristinati.

■ Verifica 07.A

CASO RAGIONATO: MIGRAZIONE DI UN SERVIZIO COMUNALE

Un Comune deve migrare il portale per le richieste di occupazione del suolo pubblico. Il servizio comprende front end, applicazione, database, archivio documentale, autenticazione, protocollo e pagamenti. L'obiettivo non è trasferire un server, ma mantenere il procedimento accessibile e recuperabile.

Classificazione e inventario. Il gruppo descrive dati e servizio, applica il percorso vigente di classificazione e definisce le attività essenziali e l'impatto dell'indisponibilità. Rileva versioni, volumi, picchi, API, licenze e dipendenze. Scopre che il protocollo accetta richieste solo da indirizzi autorizzati e che alcuni allegati sono ancora su un file server locale.

Scelta motivata. Front end e applicazione diventano workload containerizzati; il database passa a un servizio gestito compatibile; l'integrazione con il protocollo resta inizialmente ibrida. Questa scelta non elimina il rischio: rende esplicite le dipendenze da testare. Il Comune verifica che la destinazione sia compatibile con la classificazione e con la disciplina vigente, senza assumere che qualunque servizio cloud sia idoneo.

Rilascio ed esercizio. La pipeline costruisce un'immagine identificata, esegue i controlli e distribuisce in preproduzione. Il cutover avviene dopo la sincronizzazione dei dati e una prova completa della pratica. Sono definiti il punto di stop, l'autorità che decide il passaggio e il rollback. In esercizio metriche misurano errori e latenza, trace e log seguono una richiesta, mentre backup e ripristino sono provati. RPO e RTO derivano dall'impatto sulle pratiche e non da valori scelti per consuetudine.

Uscita. Il piano specifica esportazione dei dati, configurazioni, tempi, responsabilità e cancellazione finale. Il caso è ben risolto quando impatti, architettura, responsabilità, controlli e recuperabilità restano collegati. Un elenco di strumenti non dimostra la qualità della soluzione.

LABORATORIO: CHECKLIST DI MIGRAZIONE

Area	Domanda	Evidenza
Servizio	Quale funzione e quali utenti supporta?	Scheda del servizio
Dati	Quali dati tratta e come sono classificati?	Esito della classificazione
Dipendenze	Quali sistemi, reti, identità e fornitori coinvolge?	Mappa delle dipendenze
Carico	Quali volumi, picchi e tempi di risposta sono attesi?	Misure e stime
Strategia	Quale strategia di migrazione e perché?	Motivazione scritta
Destinazione	I requisiti e la qualificazione sono compatibili?	Verifica vigente
Responsabilità	Chi configura, osserva e ripristina?	Matrice dei ruoli
Test	Quali criteri devono essere superati?	Evidenze di prova
Cutover e rollback	Quando si passa e come si torna indietro?	Runbook provato
Esercizio	Quali metriche, log, trace e alert servono?	Pannello operativo e procedura
Continuità	Quali RPO, RTO, backup e alternative sono necessari?	Piano e test
Uscita	Come avvengono esportazione e cancellazione?	Exit plan

DOMANDA DA COMMISSARIO

"Come imposteresti la migrazione al cloud di un servizio pubblico?"

Descriverei servizio, classificazione e impatti; inventarierei componenti e dipendenze; definirei requisiti e continuità; sceglierei strategia e destinazione motivate; assegnerei responsabilità; pianificherei dati, test, cutover e rollback; predisporrei osservabilità, backup, ripristino ed exit strategy. La tecnologia viene dopo il problema.

DOMANDA-TRAPPOLA

"Se il database è replicato in un'altra zona, il backup non serve?"

No. La replica sostiene disponibilità e ripartenza, ma può propagare cancellazioni, corruzioni o errori. Il backup conserva punti recuperabili secondo una retention e deve essere testato. Rispondono a rischi diversi.

ERRORE TIPICO

Scegliere provider o orchestratore prima di classificare servizio, dipendenze e impatti produce una migrazione tecnicamente elegante ma incompatibile con dati, processi, continuità o uscita.

QUIZ 1 — RESPONSABILITÀ IN SAAS

In un servizio SaaS l'ente conserva certamente:

- A. manutenzione dell'hardware;
- B. gestione corretta di utenti, dati e configurazioni proprie;
- C. patch dell'hypervisor;
- D. sostituzione dei dischi.

Risposta corretta: B. Il perimetro tecnico del provider è più ampio rispetto a IaaS, ma l'ente mantiene responsabilità su uso, accessi, dati, configurazioni e continuità del proprio procedimento.

QUIZ 2 — CONTAINER E VM

Un container, rispetto a una VM ordinaria:

- A. non usa CPU;
- B. condivide normalmente il kernel dell'host;
- C. contiene sempre un database;
- D. non usa rete.

Risposta corretta: B. Il container isola il processo e le dipendenze secondo il runtime, mentre una VM include normalmente un sistema operativo guest. Le due tecniche possono convivere.

QUIZ 3 — CONTINUOUS DELIVERY

La continuous delivery:

- A. elimina ogni approvazione;
- B. mantiene il software rilasciabile;
- C. coincide con il backup;
- D. richiede Kubernetes.

Risposta corretta: B. Il software rimane in una condizione verificata e rilasciabile; l'attivazione in produzione può richiedere una decisione esplicita e controlli aggiuntivi.

QUIZ 4 — SEGNALI DI OSSERVABILITÀ

Il percorso di una richiesta fra componenti è mostrato da:

- A. una trace;
- B. uno snapshot;
- C. un template;
- D. l'RPO.

Risposta corretta: A. Le trace permettono di seguire una richiesta tra servizi; metriche e log restano complementari per misurare il fenomeno e leggerne il contesto.

QUIZ 5 — RPO

Un RPO di due ore riguarda:

- A. il tempo di sostituzione di un server;
- B. la perdita temporale massima di dati tollerabile;
- C. la durata del contratto;
- D. la latenza della rete.

Risposta corretta: B. L'RPO esprime quanto indietro nel tempo può collocarsi il punto di ripristino; l'RTTO riguarda invece il tempo entro cui il servizio deve tornare utilizzabile.

QUIZ 6 — SNAPSHOT E BACKUP

Perché uno snapshot non è automaticamente un backup?

Risposta corretta: perché può condividere storage, amministrazione o dominio di guasto con l'originale. Per essere utile al ripristino, una copia deve rispettare la conservazione prevista e la procedura deve essere provata.

QUIZ 7 — REPLICHE E CONTINUITÀ

Indica due motivi per cui tre repliche non garantiscono da sole la continuità.

Risposta corretta: le repliche possono condividere un dominio di guasto e possono dipendere dallo stesso database, collegamento, configurazione o credenziale. La continuità richiede anche procedure, ruoli, dati coerenti e test.

MINI-ESERCIZIO

Per un servizio di prenotazione comunale descrivi, in dieci righe, una dipendenza critica, il segnale che ne mostrerebbe il degrado, il proprietario della correzione e la prova che conferma il ripristino. Poi indica se il caso richiede alta disponibilità, backup, disaster recovery o una combinazione delle tre misure.

CHECKLIST FINALE

Verifica di aver distinto caratteristiche, servizi e modelli di distribuzione; assegnato le responsabilità; separato VM, container, immagine, snapshot e backup; spiegato stato desiderato e orchestrazione; classificato prima della scelta; motivato migrazione e uscita; descritto pipeline, artefatti e controlli; distinto integration, delivery e deployment; collegato IaC a versionamento e review; usato metriche, log e trace insieme; distinto ridondanza, replica, backup, disaster recovery e continuità; ricavato RPO e RTO dall'impatto; previsto cutover, rollback e prove.

DA SAPERE IN 5 RIGHE

Il cloud è un modello elastico e misurato, non un server remoto. IaaS, PaaS e SaaS spostano il confine delle responsabilità senza annullare quelle dell'ente. Container e VM sono diversi e complementari. DevOps rende modifiche e infrastruttura versionate, verificabili e osservabili. Backup, disaster recovery e business continuity rispondono a domande diverse e devono essere provati.

RIFERIMENTI NORMATIVI E PROFESSIONALI ESSENZIALI

- NIST SP 800-145, The NIST Definition of Cloud Computing.
- Strategia Cloud Italia e documentazione ACN su classificazione e qualificazione dei servizi cloud per la PA.
- Documentazione Kubernetes sui container, sugli oggetti e sui Pod.
- Documentazione OpenTelemetry su metriche, log e trace.
- NIST SP 800-34 Rev. 1, Contingency Planning Guide for Federal Information Systems.
- ACN, regolamento 21007/24 del 27 giugno 2024, e catalogo di qualificazione del servizio pertinente.

Cybersecurity operativa: rischio, controlli e vulnerabilità

B
BANDO**A**
AREE**N**
NUCLEI**D**
DIARIO**O**
OUTPUT

Una vulnerabilità critica non coincide automaticamente con il rischio più urgente per un ente. La priorità dipende dal servizio interessato, dall'esposizione, dalla possibilità di sfruttamento, dagli impatti e dai controlli già presenti. La cybersecurity operativa traduce queste informazioni tecniche in decisioni motivate.

Il percorso parte dagli asset. Collega minacce e vulnerabilità a scenari concreti, valuta il rischio e seleziona i controlli. Affronta poi la correzione delle debolezze, lo sviluppo sicuro e la software supply chain, dove possono nascere vulnerabilità già prima del rilascio.

Obiettivo e confine con il volume base

Il VOL-01, capitolo 10, § 6, tratta sicurezza di base, password, MFA, phishing, malware, antivirus, firewall, backup, aggiornamenti e inquadramento generale NIS2. Il capitolo li assume come prerequisiti.

Al termine saprai:

- distinguere asset, minaccia, vulnerabilità, evento, impatto e rischio;
- costruire una valutazione con scale dichiarate;
- distinguere rischio inerente e residuo;
- scegliere e classificare controlli;
- impostare threat modeling e vulnerability management;
- spiegare secure SDLC, secure coding e supply chain;
- produrre una risk matrix e una scheda di trattamento.

Continuità e backup sono nel capitolo 7. IAM, crittografia, logging e risposta all'incidente sono nel capitolo 9. Requisiti contrattuali e governo dei fornitori sono nel capitolo 12.

Mappa BANDO: dal contesto al trattamento

Formula nel bando	Nucleo	Output
rischio cyber, risk assessment	scenario, probabilità, impatto	risk matrix
controlli, misure di sicurezza	natura, funzione, adeguatezza	piano di trattamento
vulnerability management	scoperta, priorità, remediation	registro vulnerabilità
threat modeling	asset, flussi, confini, minacce	modello di minaccia
secure coding, OWASP	debolezze e pratiche preventive	analisi applicativa
supply chain, SBOM	dipendenze, pipeline, artefatti	verifica di filiera

In prova conviene seguire un ordine stabile: contesto, asset, scenario, rischio, trattamento, evidenza e riesame. Lo strumento scelto è secondario rispetto alla coerenza del ragionamento.

8.1 Obiettivi di sicurezza e asset

OBIETTIVI DI SICUREZZA E ASSET

PROPRIETÀ DA PROTEGGERE

La triade CIA riassume tre proprietà:

- riservatezza: informazioni accessibili soltanto ai soggetti autorizzati;
- integrità: dati e sistemi non alterati in modo improprio e modifiche rilevabili;
- disponibilità: informazioni e servizi accessibili quando servono.

Si aggiungono, secondo il contesto, autenticità, tracciabilità e accountability. Il non ripudio riguarda la capacità di impedire che un soggetto neghi in modo plausibile una determinata azione, quando il sistema e il quadro applicabile lo consentono. Queste proprietà non sono intercambiabili: la cifratura può sostenere la riservatezza, ma non garantisce da sola disponibilità o correttezza del processo.

ASSET, VALORE E DIPENDENZE

Un asset è qualcosa che ha valore per l'organizzazione e richiede protezione. Può essere:

- informativo, come un archivio;
- tecnologico, come server, applicazioni e reti;
- umano, come competenze e ruoli;
- fisico, come locali e apparati;
- organizzativo, come procedure e reputazione;
- di servizio, come la capacità di erogare una prestazione pubblica.

L'inventario registra responsabile, collocazione, dipendenze, criticità e ciclo di vita. Un asset isolato sulla carta può dipendere da identità, DNS, connettività, provider e personale reperibile. Senza tali relazioni, l'analisi sottostima i punti di guasto.

VALORE E RESPONSABILITÀ DELL'ASSET

La classificazione non parte dalla tecnologia, ma dal servizio che l'ente deve rendere. Un archivio può essere tecnicamente ordinario e tuttavia essenziale se blocca una graduatoria, un pagamento o una pratica con una scadenza. Per ogni asset occorre quindi chiedersi quale risultato pubblico sostiene, chi lo usa, quali dati tratta, quale proprietà CIA è più esposta e quali altri elementi devono funzionare insieme. Un portale che sembra autonomo può dipendere dal servizio di identità, dal dominio, dalla rete, dall'archivio documentale, dal fornitore e da persone con compiti distinti.

Il responsabile dell'asset non coincide sempre con chi amministra il server. Il primo deve chiarire il valore del servizio, le conseguenze di un fermo e le priorità; il secondo può gestire una configurazione o un controllo. Questa separazione evita una valutazione fatta solo sul valore tecnico del componente. In prova è utile esplicitare sia l'asset sia la dipendenza: "la disponibilità del portale dipende anche dall'identità digitale e dal deposito dei documenti" è una frase più verificabile di "proteggerei il server".

Un inventario utile non è un elenco statico. Collega asset, proprietario, collocazione, versione, dipendenze, dati trattati e criticità. Le informazioni consentono di aggiornare l'analisi quando cambia un'integrazione o un fornitore. L'errore opposto consiste nel classificare tutti gli asset come critici: così la priorità perde significato. La criticità deve derivare dal servizio, dalle persone coinvolte, dalle conseguenze e dalle alternative disponibili.

PRIORITÀ LEGATA AL SERVIZIO

Considera due asset: un archivio di manuali pubblici e il registro che abilita un pagamento dovuto. Entrambi contengono informazioni e richiedono protezione, ma l'interruzione non produce lo stesso effetto. Nel secondo caso la disponibilità può essere decisiva in una data precisa; nel primo può prevalere l'integrità delle versioni pubblicate. Questa differenza deve comparire nella valutazione, altrimenti la scelta dei controlli rimane astratta.

Le dipendenze rendono visibile un altro aspetto: la sicurezza è una proprietà del servizio, non del singolo apparato. Se l'accesso al portale dipende da un componente esterno, la sua indisponibilità può interrompere la prestazione anche con il database integro. Il candidato può rappresentare il legame con una frase semplice: servizio, asset, dipendenza, proprietà da proteggere e conseguenza. La stessa traccia rende verificabile il ragionamento e consente di aggiornare la valutazione quando cambiano architettura o responsabilità.

Infine, valore non significa solo costo di sostituzione. Comprende obblighi di servizio, continuità amministrativa, affidabilità delle decisioni e fiducia degli utenti. Un asset non critico oggi può diventarlo in una finestra concorsuale o in prossimità di una scadenza. Per questo la classificazione deve avere un riesame, non un'etichetta definitiva.

8.2 Minacce, vulnerabilità e scenari

INQUADRAMENTO: UN LINGUAGGIO SENZA AMBIGUITÀ

Una minaccia è una circostanza o causa potenziale capace di produrre un evento dannoso. Il threat actor è il soggetto che può agire intenzionalmente; non tutte le minacce, però, hanno un attore ostile: esistono errori, guasti ed eventi ambientali.

Una vulnerabilità è una debolezza che può essere sfruttata o contribuire al danno. Può trovarsi nel codice, nella configurazione, nel processo, nelle persone o nell'architettura. L'exploit è un mezzo o una tecnica che sfrutta una vulnerabilità. Il vettore di attacco è il percorso usato per raggiungere il bersaglio.

L'evento è ciò che accade; l'impatto è la conseguenza per l'organizzazione. Il rischio combina incertezza, probabilità o verosimiglianza e conseguenze rispetto agli obiettivi.

Esempio: il portale usa un componente vulnerabile (vulnerabilità); un attore invia una richiesta costruita per sfruttarlo (minaccia e vettore); il componente esegue un'operazione non prevista (evento); pratiche e servizio diventano indisponibili (impatto).

SUPERFICIE DI ATTACCO E SCENARIO

La superficie di attacco comprende punti attraverso i quali un sistema può essere raggiunto o influenzato: interfacce, API, account, dipendenze, dispositivi, persone e processi. Ridurla significa eliminare funzioni inutili, limitare esposizioni e controllare interazioni.

Uno scenario di rischio deve indicare:

1. asset e obiettivo compromesso;
2. fonte o attore della minaccia;
3. vulnerabilità e condizioni;
4. evento temuto;
5. conseguenze.

La formula «rischio ransomware alto» è troppo generica. Bisogna precisare il servizio coinvolto, il percorso d'attacco, le dipendenze e l'impatto atteso.

DALLO SCENARIO ALLA DECISIONE

Uno scenario ben scritto rende controllabile il ragionamento. Indica un asset, una condizione di debolezza, una minaccia, un percorso plausibile, un evento e una conseguenza. Non richiede la descrizione di tecniche offensive: basta spiegare perché quella combinazione può compromettere una proprietà del servizio. Per esempio, una funzione esposta e una dipendenza non aggiornata non sono già un incidente. Diventano uno scenario quando si chiarisce che un soggetto esterno può raggiungere la funzione e che l'evento temuto comporta accesso improprio o indisponibilità.

La superficie di attacco include la pagina visibile del portale, API, account, canali di amministrazione, componenti di terze parti, processi di aggiornamento e confini fra ambienti. Ridurla può significare dismettere un'interfaccia non necessaria, limitare un privilegio, separare un ambiente di prova o rendere tracciabile una modifica. Il candidato deve usare questi elementi per motivare una scelta, non per compilare un elenco di parole chiave.

Nel linguaggio della prova, minaccia, vulnerabilità ed evento devono restare distinti. Una vulnerabilità è una debolezza; la minaccia è una causa possibile; l'evento è ciò che accade; l'impatto è la conseguenza. Il rischio esprime la possibilità e la gravità della conseguenza rispetto agli obiettivi. Confondere il punteggio tecnico di una debolezza con il rischio dell'ente porta a una priorità priva di contesto.

CONDIZIONI CHE CAMBIANO LO SCENARIO

La medesima vulnerabilità può produrre scenari diversi. Su un componente non esposto e isolato il percorso di attacco può essere poco plausibile; sullo stesso componente raggiungibile da Internet e collegato a documenti sensibili la priorità cambia. Anche le credenziali, la segmentazione, la validazione lato server e la presenza di un percorso alternativo modificano probabilità e impatto. L'analisi deve dichiarare queste condizioni invece di assumere che un nome tecnico le contenga già tutte.

Una buona descrizione evita formule ambigue. Non dire soltanto "esiste un rischio di accesso". Indica quale attore può raggiungere quale funzione, quale debolezza rende possibile l'evento, quali dati o servizi sono coinvolti e quale controllo limita il danno. Questa impostazione è utile anche quando la minaccia non è intenzionale: una configurazione errata, un aggiornamento difettoso o un guasto possono compromettere lo stesso servizio e richiedere trattamenti diversi.

L'uso concorsuale consiste nel mostrare che la sicurezza parte dal contesto. Dopo aver definito lo scenario, il candidato può proporre un controllo e motivarne il legame con la condizione osservata. Una lista di prodotti o sigle senza scenario non dimostra capacità di analisi.

8.3 Valutazione e trattamento del rischio

VALUTARE E TRATTARE IL RISCHIO

PROBABILITÀ E IMPATTO

La probabilità stima quanto sia plausibile lo scenario, considerando capacità dell’attore, esposizione, frequenza, vulnerabilità e controlli. L’impatto considera conseguenze su servizi, dati, persone, finanze, conformità e fiducia istituzionale.

Una scala qualitativa può usare basso, medio e alto. Ogni livello deve avere criteri. «Impatto alto» potrebbe indicare interruzione di un servizio essenziale o compromissione significativa di dati; la definizione dipende dal contesto.

La matrice incrocia probabilità e impatto:

Impatto \ Probabilità	Bassa	Media	Alta
Alto	Medio	Alto	Alto
Medio	Basso	Medio	Alto
Basso	Basso	Basso	Medio

Questa matrice ordina le priorità, ma non produce una misura assoluta. Scale diverse possono dare risultati diversi; scenari con lo stesso livello possono richiedere decisioni differenti.

INERENTE E RESIDUO

Il rischio inerente è valutato prima di considerare i controlli pertinenti secondo il metodo scelto. Il rischio residuo rimane dopo l’applicazione dei controlli. Dichiarare «residuo basso» richiede evidenze che i controlli siano presenti e funzionino.

Un controllo soltanto previsto non riduce il rischio. Anche un controllo installato offre garanzie deboli se nessuno ne verifica il funzionamento.

PROCESSO DI ASSESSMENT

Un risk assessment comprende:

1. definizione di scopo, contesto, metodo e criteri;
2. identificazione di asset e dipendenze;
3. costruzione degli scenari;
4. analisi di probabilità e impatto;
5. valutazione rispetto a soglie e propensione al rischio;
6. priorità, trattamento e documentazione;
7. monitoraggio e aggiornamento.

Il risk register conserva scenario, asset, livello inerente, controlli, trattamento, responsabile, scadenza, evidenza e rischio residuo. Deve essere aggiornato quando cambiano architettura, minacce, vulnerabilità o impatti.

OPZIONI DI TRATTAMENTO

- Evitare: cessare l'attività che genera il rischio.
- Ridurre o mitigare: intervenire su probabilità o impatto.
- Trasferire o condividere: allocare parte delle conseguenze a un altro soggetto, senza cancellare la responsabilità dell'ente.
- Accettare: assumere consapevolmente il rischio residuo entro autorità e criteri definiti.

L'accettazione non equivale a ignorare. Richiede motivazione, titolare autorizzato, durata e riesame.

CRITERI, SOGLIE E ACCETTAZIONE CONSAPEVOLE

La matrice aiuta a ordinare le decisioni solo se le scale sono dichiarate prima del risultato. "Probabilità alta" può significare esposizione pubblica, sfruttabilità plausibile e assenza di un controllo efficace; "impatto alto" può riferirsi a un'interruzione rilevante del servizio o alla compromissione di informazioni importanti. I criteri cambiano con l'ente, ma non possono cambiare a posteriori per giustificare una scelta già presa.

Il trattamento deve lasciare una traccia verificabile. Per ridurre un rischio non basta scrivere "applicare un controllo": occorrono un'azione, un responsabile, una data, un'evidenza e un criterio di chiusura. Se il rischio viene accettato, l'accettazione riguarda il residuo dopo avere valutato le alternative; richiede un soggetto autorizzato e una data di riesame. Trasferire una parte delle conseguenze a un fornitore non trasferisce automaticamente la responsabilità dell'ente verso il servizio.

DALLA MATRICE AL REGISTRO OPERATIVO

La matrice ordina rapidamente gli scenari, mentre il registro conserva le informazioni necessarie per agire. Una riga utile riporta identificativo, asset, scenario, valutazione inerente, controlli esistenti, scelta di trattamento, responsabile, data prevista, evidenza richiesta, valutazione residua e criterio di riesame. Il registro rende confrontabili decisioni ripetute; la matrice da sola non spiega perché un livello è stato attribuito.

Il rischio inerente aiuta a capire la gravità dello scenario prima di contare i controlli. Il rischio residuo non è un punteggio scelto per chiudere una pratica: deriva dalla verifica che i controlli siano presenti, adeguati e funzionanti. Se manca questa evidenza, il valore residuo è un'ipotesi e deve rimanere trattato con prudenza. Quando una misura riduce l'impatto ma non la probabilità, oppure viceversa, la motivazione deve dirlo.

Per l'orale, una risposta efficace segue un filo costante: definisco criteri e confini, identifico asset e dipendenze, costruisco lo scenario, valuto, scelgo il trattamento e programma il riesame. Questa sequenza è preferibile a una formula numerica priva di assunzioni.

8.4 Controlli e difesa per livelli

CONTROLLI E DIFESA PER LIVELLI

NATURA E FUNZIONE

Per natura, i controlli possono essere organizzativi, tecnici o fisici. Per funzione possono essere:

- preventivi, se riducono la probabilità;
- detective, se individuano eventi o anomalie;
- correttivi, se rimuovono o limitano una condizione;
- di recupero, se ripristinano capacità;
- deterrenti, se scoraggiano;
- compensativi, se offrono protezione alternativa quando il controllo previsto non è praticabile.

Le tassonomie possono variare; la classificazione chiarisce la funzione del controllo, non aggiunge etichette fini a sé stesse.

Esempi: formazione e policy sono organizzativi; segmentazione e protezione endpoint sono tecnici; controllo degli accessi ai locali è fisico. Un medesimo controllo può avere più effetti.

DIFESA IN PROFONDITÀ

La difesa in profondità combina controlli diversi affinché il fallimento di uno non esponga direttamente l'asset. Inventario, hardening, patching, minimo privilegio, segmentazione, monitoraggio e backup agiscono su punti differenti.

I livelli devono essere sufficientemente indipendenti. Due strumenti che dipendono dalla stessa identità amministrativa possono fallire insieme se quell'identità è compromessa.

Il principio Zero Trust, quando applicato, evita di attribuire fiducia implicita soltanto per posizione di rete. Richiede verifica esplicita, minimo privilegio e valutazione continua del contesto; non è un prodotto.

NIST CSF 2.0

Il NIST CSF 2.0 organizza outcome in sei funzioni:

- Govern: strategia, ruoli, politiche e supervisione;
- Identify: asset, contesto e rischi;
- Protect: salvaguardie;
- Detect: individuazione degli eventi;
- Respond: gestione della risposta;
- Recover: ripristino e miglioramento.

Le funzioni sono concorrenti e continue, non fasi rigide. Il CSF aiuta a comunicare outcome e lacune, ma non sostituisce l'assessment né prescrive un singolo insieme di tecnologie.

ADEGUATEZZA E PROVA DEL CONTROLLO

Un controllo è adeguato quando risponde allo scenario e quando la sua efficacia può essere verificata. La stessa misura può avere funzioni diverse: una regola di autorizzazione può prevenire un accesso improprio, mentre il log associato può aiutare a rilevarlo. Classificare per natura e funzione serve a vedere lacune e dipendenze, non a ottenere una tassonomia perfetta.

La difesa per livelli richiede una domanda ulteriore: i controlli falliscono tutti per la stessa causa? Due protezioni amministrate con la stessa credenziale privilegiata possono perdere efficacia insieme. Per questo asset inventory, hardening, segmentazione, aggiornamento, minimo privilegio e monitoraggio assumono valore quando sono collegati a responsabilità ed evidenze. Il NIST CSF 2.0 offre una mappa di outcome per discutere governo, identificazione, protezione, rilevazione, risposta e recupero. È un riferimento tecnico utile, non un obbligo generale della PA italiana e non una sequenza rigida da applicare in ogni caso.

CONTROLLI COERENTI CON IL RISCHIO

La scelta non parte da un catalogo. Si parte dalla condizione da modificare. Se il problema è una funzione esposta non necessaria, la rimozione o limitazione dell'esposizione può essere un controllo preventivo. Se il rischio riguarda modifiche non autorizzate, autorizzazioni separate e registrazioni delle operazioni svolgono funzioni differenti. Se la correzione richiede tempo, una misura compensativa può ridurre temporaneamente l'esposizione, ma deve avere durata definita e non diventare una soluzione invisibile.

Le evidenze sono parte del controllo. Una procedura può dimostrare che una responsabilità è assegnata; una configurazione approvata può dimostrare l'applicazione di una regola; un test può mostrare che il controllo produce l'effetto atteso. Senza evidenza, il controllo è soltanto dichiarato. La verifica periodica intercetta anche controlli che erano corretti quando sono stati installati ma non lo sono più dopo una modifica.

Un candidato non deve promettere sicurezza assoluta. Deve spiegare come più livelli indipendenti riducono il rischio residuo e come l'ente saprà se funzionano. Questa è la differenza fra una risposta tecnica e una lista generica di misure.

La scelta va riesaminata quando cambiano asset, esposizione, minaccia, architettura o efficacia del controllo. Un controllo adeguato ieri può non esserlo dopo un'integrazione, una variazione di servizio o un cambiamento organizzativo. Il riesame mantiene il trattamento collegato alla realtà del servizio.

8.5 Threat modeling e gestione delle vulnerabilità

THREAT MODELING

Il threat modeling anticipa domande di sicurezza durante progettazione ed evoluzione. Il gruppo:

- 1. descrive sistema, attori e obiettivi;
- 2. disegna componenti, flussi e confini di fiducia;
- 3. individua asset e scenari di minaccia;
- 4. seleziona mitigazioni;
- 5. verifica assunzioni e copertura.

STRIDE è una tassonomia possibile: spoofing, tampering, repudiation, information disclosure, denial of service ed elevation of privilege. Non è l'unico metodo e non deve diventare un elenco scollegato dal sistema.

Categoria e significato	Minaccia al portale PA	Controllo motivato
Spoofing: impersonificazione	Accesso come istruttore con credenziali sottratte	MFA e recupero account controllato
Tampering: alterazione	Modifica non autorizzata dell'importo	Autorizzazione e controlli di integrità
Repudiation: disconoscimento	Operazione negata da chi l'ha compiuta	Audit trail integro con identità e tempo
Information disclosure: divulgazione	Fascicolo visibile a un altro utente	Autorizzazione per oggetto e minimizzazione
Denial of service: indisponibilità	Richieste saturano il servizio	Limiti proporzionati e capacità/resilienza
Elevation of privilege: aumento dei privilegi	Utente ordinario diventa amministratore	Separazione ruoli e controllo dei privilegi

Un diagramma dei flussi mostra dove i dati entrano, cambiano fiducia, vengono elaborati e conservati. Ogni confine suggerisce domande su validazione, identità, autorizzazione e protezione.

GESTIONE DELLE VULNERABILITÀ

SCOPERTA E VALIDAZIONE

L'inventario di asset, versioni e dipendenze è il punto di partenza. Le vulnerabilità emergono da scanning, test, segnalazioni, intelligence, advisory e comunicazioni dei fornitori.

Ogni risultato richiede una validazione. Un falso positivo segnala una condizione inesistente o non applicabile; un vero positivo richiede trattamento. La validazione non autorizza prove invasive indiscriminate: scopo, permessi e sicurezza del test vanno definiti.

CVE, CWE E CVSS

- CVE identifica una specifica vulnerabilità pubblicamente nota.
- CWE descrive una classe di debolezze che può generare vulnerabilità.
- CVSS comunica caratteristiche e gravità mediante un punteggio e un vettore.

CVSS 4.0 distingue metriche Base, Threat, Environmental e Supplemental. Il punteggio di base non conosce da solo criticità dell'asset, esposizione, exploit osservati o controlli locali. Perciò il CVSS più alto non è sempre la prima priorità.

PRIORITÀ E REMEDIATION

La priorità considera:

- gravità tecnica;
- sfruttabilità e attività osservata;
- esposizione;
- criticità del servizio;
- presenza di dati sensibili;
- controlli compensativi;
- disponibilità e rischio della correzione.

La remediation assegna responsabile e scadenza, prova la patch, pianifica la distribuzione e verifica la chiusura. Se la patch non è applicabile subito, una mitigazione temporanea può ridurre esposizione o funzionalità. L'eccezione deve indicare rischio residuo, approvazione e data di riesame.

La responsabile disclosure coordina segnalazione e correzione evitando diffusione prematura di dettagli. Nel lavoro pubblico vanno rispettati canali, autorizzazioni e procedure applicabili.

PRIORITÀ CONTESTUALE E CHIUSURA VERIFICATA

La gestione delle vulnerabilità è un ciclo, non la somma dei risultati di uno strumento. Inventario, scoperta, validazione, priorità, trattamento e verifica di chiusura devono rimanere collegati. Un risultato può essere un falso positivo oppure non essere applicabile alla configurazione esaminata; trattarlo come un incidente genera lavoro inutile. All'opposto, ignorare una segnalazione senza motivazione impedisce di spiegare il rischio residuo.

CVE identifica una vulnerabilità pubblicamente nota, CWE descrive una classe di debolezza e CVSS comunica caratteristiche e gravità. Nessuno dei tre sostituisce l'analisi del servizio. La priorità considera anche esposizione, sfruttabilità, valore dell'asset, controlli e rischio della modifica. Una patch può richiedere test e una finestra di rilascio; nel frattempo una mitigazione temporanea può ridurre l'esposizione. L'eccezione deve avere proprietario, motivazione, scadenza e riesame.

MODELLO DI MINACCIA COME STRUMENTO DI PROGETTO

Il threat modeling è utile quando accompagna una decisione concreta. Per un portale, il gruppo può disegnare utenti, front end, servizi applicativi, archivio documentale e sistemi esterni; poi identifica dove il dato attraversa un confine di fiducia. A ciascun passaggio associa una domanda: chi può inviare il dato, come viene verificata l'identità, quale autorizzazione serve, quale trasformazione avviene e quale evidenza resta. Serve a collegare ogni flusso alla minaccia e alla mitigazione che lo riguarda.

STRIDE può aiutare a classificare famiglie di minacce, ma non sostituisce la conoscenza del sistema. Un diagramma incompleto o una lista di categorie non genera da sola un trattamento. Lo stesso vale per la gestione delle vulnerabilità: il risultato di una scansione diventa utile quando è attribuito a un asset, validato e collegato a un'azione verificabile. Il ciclo deve poter spiegare anche perché una segnalazione è stata chiusa, rinviata o dichiarata non applicabile.

Quando una vulnerabilità non può essere corretta immediatamente, la scelta responsabile non è nascondersela. Occorre limitare il rischio, approvare l'eccezione, fissare una data e verificare che la condizione temporanea non diventi permanente.

8.6 Secure SDLC e software supply chain

SECURE SDLC E SECURE CODING

SICUREZZA LUNGO IL CICLO DI VITA

Il NIST SSDP integra pratiche di sicurezza nel ciclo di sviluppo. Le pratiche di sicurezza accompagnano requisiti, progettazione, implementazione, verifica, rilascio, esercizio e dismissione.

Security by design significa considerare minacce e requisiti durante la progettazione. Security by default significa fornire configurazioni iniziali prudenti. Una scansione finale difficilmente corregge una scelta architetturale insicura.

Le attività comprendono requisiti verificabili, threat modeling, ambienti protetti, dipendenze governate, review, test, artefatti controllati, gestione delle vulnerabilità e apprendimento dalle cause.

PRATICHE DI CODICE SICURO

- validare input rispetto a tipo, formato, lunghezza e dominio attesi;
- codificare l'output secondo il contesto;
- usare query parametrizzate invece di concatenare input;
- gestire errori senza esporre dettagli sensibili;
- non incorporare secret nel codice;
- applicare autorizzazioni sul server;
- mantenere dipendenze e configurazioni;
- fallire in modo sicuro quando una condizione non è gestibile.

La validazione client migliora l'esperienza, ma non sostituisce quella lato server. Una denylist di stringhe pericolose è spesso meno robusta della definizione positiva dei valori ammessi.

VERIFICHE DIVERSE

La code review esamina logica e modifiche. La SAST analizza codice o rappresentazioni senza eseguire l'applicazione. La DAST osserva il comportamento di un'applicazione in esecuzione. La software composition analysis esamina componenti e dipendenze. Il penetration test simula scenari entro uno scopo autorizzato.

Le tecniche hanno coperture diverse. I risultati vanno validati e ricondotti allo scenario di rischio.

OWASP Top 10 è un documento di awareness sui rischi delle applicazioni web. L'edizione 2025 include categorie come controllo accessi, configurazione, supply chain, crittografia, injection e design insicuro. Non certifica la sicurezza e non sostituisce requisiti, threat modeling o test basati sul contesto.

SOFTWARE SUPPLY CHAIN

La supply chain comprende codice proprio, librerie, componenti open source e commerciali, repository, tool di build, runner, pipeline, artefatti e canali di distribuzione. L'attaccante può compromettere un componente, una credenziale, il processo di build oppure il canale di aggiornamento.

I controlli includono:

- inventario di componenti e versioni;
- fonti e repository autorizzati;
- protezione di account e pipeline;
- review delle modifiche;
- isolamento e aggiornamento della toolchain;
- artefatti identificati e verificabili;
- provenienza e attestazioni;
- monitoraggio degli advisory;

- capacità di ricostruire e revocare una release.

Una SBOM è un record formale dei componenti e delle relazioni. Aiuta a rispondere «dove usiamo questa libreria?», ma non dimostra che il software sia sicuro, aggiornato o privo di componenti non dichiarati. Deve essere mantenuta, associata all'artefatto e usata nei processi di gestione.

Il rischio di terze parti resta dell'organizzazione anche quando alcune attività sono affidate fuori. Requisiti contrattuali e SLA sono trattati nel capitolo 12.

PREVENZIONE PRIMA DEL RILASCIO

La sicurezza del software non inizia con una scansione finale. Requisiti verificabili, progettazione, revisione, test, rilascio ed esercizio producono evidenze diverse e riducono classi diverse di errore. Validare un input, gestire correttamente gli errori e applicare autorizzazioni sul server sono esempi di principi; non costituiscono una ricetta universale. Una review del codice, SAST, DAST, analisi delle dipendenze e penetration test osservano aspetti differenti e richiedono scopo e autorizzazioni.

La filiera comprende anche repository, account, runner, strumenti di build, artefatti e canali di distribuzione. Una SBOM aiuta a sapere dove una libreria è usata; non dimostra da sola che il prodotto sia sicuro. Per questo la filiera richiede componenti identificati, modifiche revisionate, accessi protetti, provenienza verificabile e capacità di ricostruire una release. I requisiti contrattuali dettagliati restano nel capitolo 12, mentre qui il candidato deve saper riconoscere il rischio tecnico e l'evidenza necessaria.

EVIDENZE DELLA FILIERA

Una catena di rilascio affidabile rende possibile rispondere a domande semplici: quale codice è stato usato, quali componenti erano inclusi, chi ha autorizzato la modifica, quale artefatto è stato distribuito e come si può tornare a una versione precedente. Queste domande non richiedono dettagli riservati o procedure offensive; servono a collegare prevenzione, tracciabilità e recupero.

La sicurezza per impostazione predefinita riduce il numero di decisioni rischiose richieste a chi usa o gestisce il sistema. Non elimina la responsabilità di configurare, testare e aggiornare. Un requisito di sicurezza ben scritto indica comportamento atteso, vincolo, evidenza e criterio di accettazione. Questa logica aiuta anche a distinguere una promessa generica di "software sicuro" da un impegno verificabile.

Verifica

MICRO-VERIFICA: CONFINI DI FIDUCIA

Un portale riceve allegati da utenti esterni e li inoltra a un archivio documentale. Indica un confine di fiducia e una domanda da porre nel threat modeling.

Soluzione: il passaggio dall’input pubblico al servizio che conserva i documenti è un confine di fiducia. Occorre chiedere, per esempio, come il sistema valida il file e quale autorizzazione consente di leggerlo.

CASO GUIDATO: PORTALE PER LE DOMANDE

Un ente gestisce un portale per presentare domande di contributo. L’applicazione usa una libreria con una vulnerabilità pubblica; il servizio è esposto e tratta documenti personali.

Scenario. Un attore remoto sfrutta la libreria tramite una funzione di caricamento e accede a documenti non autorizzati.

Asset. Documenti, profili utenti, applicazione e continuità del procedimento.

Valutazione inerente. Probabilità alta per esposizione e applicabilità; impatto alto per riservatezza e servizio. Nella matrice 3×3 il rischio è alto.

Controlli esistenti. Segmentazione, privilegi limitati e monitoraggio riducono propagazione e tempo di rilevazione, ma non eliminano la debolezza.

Trattamento. Il responsabile applica la patch dopo test; nel frattempo limita la funzione, rafforza le regole applicative e monitora gli indicatori pertinenti. Registra scadenza ed evidenze.

Rischio residuo. Viene rivalutato dopo verifica della versione e test funzionale. Il livello non scende automaticamente: servono evidenze sulla correzione e sull’efficacia dei controlli.

Supply chain. La SBOM consente di individuare altri servizi con la stessa libreria. L’ente traduce quanto appreso in nuovi requisiti di aggiornamento e controlli della pipeline.

LABORATORIO: COSTRUIRE UNA RISK MATRIX

Compila una riga per ciascuno scenario.

Campo	Domanda
Asset	che cosa ha valore e chi ne risponde?
Scenario	quale minaccia sfrutta quale vulnerabilità?
Evento	che cosa può accadere?
Impatto	quali conseguenze e con quale livello?
Probabilità	quali fattori la rendono plausibile?
Inerente	quale livello prima dei controlli?
Controlli	quali esistono e quali evidenze?
Trattamento	evitare, ridurre, trasferire o accettare?

Campo	Domanda
Responsabile	chi attua e chi accetta?
Scadenza	entro quando?
Residuo	che cosa rimane dopo la verifica?
Riesame	quale evento o data riapre la valutazione?

DOMANDA DA COMMISSARIO

«Come imposteresti un risk assessment cyber?»

Definirei scopo, metodo e criteri; identificarei asset e dipendenze; costruirei scenari con minacce, vulnerabilità, eventi e impatti; stimerei il rischio inerente; valuterei poi i controlli; sceglierei il trattamento, con responsabile, scadenza ed evidenza; infine stimerei il rischio residuo e fisserei il riesame.

DOMANDA-TRAPPOLA

«La vulnerabilità con CVSS più alto va sempre corretta per prima?»

No. CVSS comunica la gravità tecnica, ma la priorità dipende anche da esposizione, sfruttabilità, criticità dell’asset, impatto, controlli e rischio della modifica. Il punteggio è un input, non la decisione.

ERRORE TIPICO

L’errore consiste nell’assegnare un «rischio alto» senza descrivere lo scenario e le scale adottate. Una valutazione verificabile esplicita asset, minaccia, vulnerabilità, impatto, probabilità, controlli e assunzioni.

MINI-ESERCIZI E QUIZ

1. Una configurazione predefinita con credenziali note è:

- A. una minaccia
- B. una vulnerabilità
- C. un impatto
- D. un rischio residuo

2. Il rischio dopo i controlli è:

- A. inerente
- B. residuo
- C. assoluto
- D. CVE

3. Quale identificatore riguarda una vulnerabilità specifica?

- A. CWE
- B. CVE
- C. CVSS
- D. CSF

4. Quale tecnica analizza un'applicazione in esecuzione?

- A. DAST
- B. SAST
- C. SBOM
- D. risk register

5. Una SBOM:

- A. certifica che il software è sicuro
- B. elenca componenti e relazioni della supply chain
- C. sostituisce il penetration test
- D. corregge le dipendenze

6. Costruisci uno scenario distinguendo minaccia, vulnerabilità, evento e impatto.

7. Indica un controllo preventivo, uno detective e uno compensativo per lo stesso scenario.

Soluzioni sintetiche: 1-B; 2-B; 3-B; 4-A; 5-B.

Risposta corretta: B. Una configurazione con credenziali note è una debolezza sfruttabile: non descrive ancora un attore, un evento o l'impatto.

Risposta corretta: B. Il rischio residuo è quello che rimane dopo avere considerato controlli pertinenti ed evidenze della loro efficacia.

Risposta corretta: B. CVE identifica una vulnerabilità nota; CWE descrive una classe di debolezza e CVSS ne comunica caratteristiche e gravità.

Risposta corretta: A. DAST osserva il comportamento dell'applicazione in esecuzione. SAST è analisi statica; i due approcci non sono intercambiabili.

Risposta corretta: B. La SBOM è un inventario formale di componenti e relazioni. Non certifica il software e non sostituisce i controlli di sviluppo.

Risposta corretta: un esempio per l'esercizio 6. Asset: fascicoli del portale. Minaccia: un attore tenta di leggere pratiche altrui. Vulnerabilità: l'API controlla il login ma non l'assegnazione della pratica. Evento: l'attore cambia l'identificativo e ottiene un fascicolo non autorizzato. Impatto: divulgazione di dati e danno alle persone; il rischio combina probabilità dello sfruttamento e conseguenze, senza attribuire un punteggio privo di scala.

Risposta corretta: un esempio per l'esercizio 7. Preventivo: autorizzazione per oggetto sul server, con test fra due utenti distinti. Detective: alert su sequenze anomale di richieste, da validare senza registrare il contenuto dei fascicoli. Compensativo, mentre si corregge il difetto: sospendere l'endpoint esposto e gestire le richieste tramite canale autenticato e operatore autorizzato, preservando un servizio minimo. La misura temporanea ha responsabile, termine e prova di chiusura; un semplice banner non compensa l'assenza di autorizzazione.

CHECKLIST FINALE

- ho definito scopo, metodo e scale;
- ho inventariato asset e dipendenze;
- ho distinto minaccia, vulnerabilità, evento e impatto;
- ho costruito scenari verificabili;
- ho valutato inerente e residuo;
- ho associato controlli a evidenze;
- ho assegnato responsabile e scadenza;
- ho prioritizzato vulnerabilità con il contesto;
- ho distinto CVE, CWE e CVSS;
- ho integrato sicurezza nel ciclo di sviluppo;
- ho considerato dipendenze, pipeline e artefatti;
- ho previsto riesame e chiusura.

DA SAPERE IN 5 RIGHE

Il rischio nasce da uno scenario, non da un punteggio isolato. Minaccia, vulnerabilità, evento e impatto sono concetti distinti. I controlli riducono probabilità o conseguenze e devono produrre evidenze. CVE identifica, CWE classifica debolezze, CVSS comunica gravità. Sviluppo sicuro e supply chain portano la prevenzione prima del rilascio.

Riferimenti normativi e professionali essenziali

- NIST Cybersecurity Framework 2.0 e NIST SP 800-30, per outcome e valutazione del rischio.
- NIST Secure Software Development Framework, SP 800-218, per le pratiche lungo il ciclo di sviluppo.
- CVE Program, Common Weakness Enumeration e CVSS v4.0, per distinguere identificazione, classe di debolezza e gravità.
- OWASP Top 10, come documento di awareness sui rischi applicativi.
- CISA, Software Bill of Materials e pratiche per il consumo della SBOM.
- Quadro italiano di cybersicurezza applicabile al bando e all'ente, da verificare al cut-off della procedura.

IAM, crittografia, logging e incident response

B BANDO	A AREE	N NUCLEI	D DIARIO	O OUTPUT
-------------------	------------------	--------------------	--------------------	--------------------

Un servizio pubblico digitale è affidabile quando l'ente sa chi opera, che cosa può fare, quali tracce lascia e come reagire a un'anomalia. IAM, crittografia, logging e incident response formano una catena: l'identità rende attribuibile l'azione, l'autorizzazione la limita, la crittografia protegge proprietà definite, i log rendono osservabile il sistema e la risposta coordina decisioni, continuità ed evidenze.

Obiettivo e confine del capitolo

Il volume base tratta password, MFA, phishing e nozioni di sicurezza. Qui il lettore impara a collegare ciclo di vita degli accessi, privilegi, chiavi, rilevazione e gestione di un incidente. Il capitolo 7 tratta continuità, backup, RPO, RTO e disaster recovery; il capitolo 8 tratta rischio, vulnerabilità e controlli; il capitolo 10 tratta governance del dato. Questo capitolo non sostituisce un playbook locale né stabilisce termini di notifica: tali decisioni dipendono dal soggetto, dal caso e dalle fonti ufficiali vigenti.

Mappa BANDO

Formula nel bando	Domanda utile	Output concorsuale
IAM, accessi, privilegi	chi accede, per quale scopo e per quanto tempo?	matrice ruolo-risorsa-permesso
crittografia, certificati, chiavi	quale proprietà proteggerò e come governo il materiale?	checklist key management
logging, SIEM, monitoraggio	quali evidenze servono per qualificare un segnale?	matrice sorgente-evento-alert
incidente cyber, CSIRT, NIS2	chi decide, contiene, comunica e recupera?	playbook e timeline

9.1 Identità, account e ciclo di vita

INQUADRAMENTO E PRINCIPI

Un'identità digitale rappresenta una persona, un'applicazione, un servizio, un dispositivo o un workload. L'account è il contenitore operativo usato dal sistema; una credenziale è un elemento associato all'identità; un autenticatore dimostra il controllo della credenziale; la sessione conserva l'esito dell'accesso; un attributo descrive soggetto, ruolo o contesto. Distinguere questi elementi evita un errore frequente: disabilitare un account non revoca automaticamente sessioni, token, chiavi applicative o account amministrativi collegati.

L'identity proofing stabilisce con un livello proporzionato di fiducia chi sia il soggetto. L'enrollment registra identità e autenticator. Il provisioning crea account, gruppi e autorizzazioni; il deprovisioning li rimuove o li sospende. Nel modello joiner-mover-leaver, la fonte autorevole comunica l'ingresso, il cambio di funzione o la cessazione. Il passaggio più rischioso è il mover: aggiungere nuovi accessi senza rimuovere quelli precedenti accumula privilegi che non corrispondono più al compito.

Ogni fase deve produrre una richiesta, un'approvazione, un'esecuzione e un'evidenza verificabile. Un responsabile di servizio non dovrebbe scoprire solo dopo un incidente chi avesse accesso a una pratica. La revisione periodica confronta ruoli effettivi, assegnazioni tecniche, eccezioni e account non usati. L'obiettivo non è avere un elenco perfetto, ma rendere governabile la domanda: chi può accedere a quale servizio, per quale motivo e fino a quando?

Gli account nominativi facilitano attribuzione e accountability. Gli account condivisi rendono difficile capire chi abbia compiuto un'azione e, se inevitabili per una transizione controllata, richiedono misure compensative, durata breve e tracciamento rafforzato. Gli account di servizio hanno proprietario, scopo, privilegi minimi, scadenza e meccanismo di rotazione. Gli account di emergenza devono essere pochi, protetti, monitorati e riesaminati subito dopo l'uso. Non sono scorciatoie permanenti per aggirare il processo ordinario.

In prova, una risposta solida distingue identità, account e autorizzazione prima di parlare di strumenti. Per esempio, quando un istruttore cambia ufficio, l'ente aggiorna ruolo e ambito, revoca i diritti non più necessari, controlla deleghe e sessioni ancora attive e conserva l'evidenza. Dire soltanto «disattivare l'utente» lascia irrisolte dipendenze e privilegi indiretti. L'errore tipico è trattare l'IAM come un elenco di account: è invece un processo che lega organizzazione, identità, sistemi e riesame.

Un registro delle identità deve anche distinguere titolare dell'identità, proprietario dell'account e responsabile dell'autorizzazione. Nelle identità tecniche queste figure possono essere diverse: un'applicazione usa l'account, un responsabile di servizio ne giustifica lo scopo e un team operativo ne amministra la configurazione. La separazione rende possibile riesaminare l'accesso quando cambia il servizio o il fornitore. Una scadenza è utile solo se qualcuno riceve l'evento, valuta il rinnovo e può revocare senza lasciare eccezioni invisibili.

La federazione può ridurre la duplicazione delle credenziali, ma introduce dipendenze da asserzioni, attributi, disponibilità e configurazioni tra sistemi. Prima di fidarsi di un'informazione ricevuta occorre definire quali attributi sono necessari, quale sistema li governa e che cosa accade quando sono assenti o obsoleti. Non occorre conoscere un prodotto per esporre questo punto: ogni integrazione di identità deve avere confini, proprietario, evidenze e procedura di revoca. L'esame valuta la capacità di collegare una scelta tecnica all'effetto sul servizio.

Un riesame ben progettato non chiede genericamente «tieni ancora l'accesso?». Confronta ruolo organizzativo, mansione, risorse assegnate, privilegi tecnici, eccezioni e attività recente. Il responsabile conferma o corregge una situazione documentata; il sistema registra la decisione e l'esecuzione. Così l'IAM sostiene anche continuità: in caso di assenza improvvisa, l'ente sa quali ruoli devono essere coperti senza condividere indiscriminatamente credenziali personali.

Una procedura di cessazione efficace include anche deleghe, caselle funzionali, dispositivi e rapporti con fornitori. Ogni elemento deve avere un proprietario che confermi la revoca e un'evidenza dell'esecuzione. In questo modo l'uscita di una persona non lascia capacità tecniche attive senza responsabilità.

9.2 Autenticazione, autorizzazione e privilegi

L'identificazione presenta un'identità; l'autenticazione verifica un autenticatore; l'autorizzazione decide se quell'identità possa eseguire un'azione; l'accounting registra l'attività. Un login riuscito non prova il diritto a consultare una pratica o a disporre un pagamento. Questa sequenza permette di individuare dove nasce il problema: un'identità non verificata, una sessione sottratta, un'autorizzazione eccessiva oppure un'azione non tracciata richiedono risposte diverse.

La MFA combina fattori indipendenti, non due copie dello stesso segreto. Riduce alcuni rischi ma non elimina la necessità di un enrollment affidabile, di un recupero account governato e di revocare sessioni e token quando serve. Anche nella federazione, in cui un servizio accetta un'asserzione di un identity provider, il servizio deve decidere localmente quali operazioni autorizzare. Confondere autenticazione federata e autorizzazione applicativa espone a privilegi concessi per inerzia.

Nel RBAC i permessi sono associati a ruoli; nell'ABAC la decisione usa attributi di soggetto, risorsa, azione e contesto; una ACL associa direttamente soggetti e permessi a una risorsa. Il RBAC è utile per compiti stabili e ripetibili. L'ABAC esprime regole contestuali, ma dipende dalla qualità e dall'aggiornamento degli attributi. Le ACL risolvono eccezioni puntuali, ma diventano difficili da governare se sostituiscono un modello generale. Nessun modello è automaticamente migliore: la scelta dipende da processo, frequenza dei cambiamenti e bisogno di dimostrare la decisione.

Il minimo privilegio assegna solo ciò che serve, per il tempo necessario. La separazione dei compiti evita che una persona svolga da sola attività incompatibili. Privilegi just-in-time e just-enough limitano durata e ampiezza dell'elevazione; il controllo degli accessi amministrativi richiede proprietario, approvazione, sessione tracciata e riesame. Il principio operativo è deny by default: quando nessuna regola consente l'azione, il servizio la nega. Il controllo deve stare lato servizio; nascondere un pulsante nell'interfaccia non impedisce chiamate dirette.

In un ufficio contribuiti, un istruttore può leggere e aggiornare pratiche della propria unità, mentre l'approvazione appartiene a un responsabile diverso. Il ruolo spiega il permesso base; attributi come ufficio, rapporto attivo e stato della pratica possono limitarne l'uso; un'eccezione circoscritta richiede motivazione e scadenza. L'errore tipico è chiamare «sicuro» un accesso perché protetto da MFA: la MFA verifica il soggetto, non dimostra che il privilegio sia appropriato.

L'autorizzazione va verificata vicino alla risorsa e all'azione sensibile. Un portale può mostrare una pratica a un utente, ma il servizio deve controllare di nuovo identità, ruolo, ambito e stato quando riceve la richiesta di modifica o approvazione. Il controllo lato server riduce il rischio che una pagina nascosta o una chiamata costruita dall'esterno aggiri la logica dell'interfaccia. Le decisioni più rilevanti devono lasciare un'evidenza sufficiente a ricostruire regola applicata, esito e soggetto.

La gestione dei privilegi richiede anche una procedura per le eccezioni. Un accesso temporaneo può essere necessario per un'emergenza o per una verifica tecnica; deve però avere motivo, approvazione, durata, risorsa limitata e riesame successivo. Un'eccezione senza scadenza diventa spesso un ruolo parallelo non controllato. La disponibilità del servizio non giustifica il mantenimento indefinito di un accesso amministrativo.

Quando il sistema usa attributi, deve gestire anche la loro qualità. Un attributo di ufficio non aggiornato può autorizzare una pratica fuori contesto; un attributo mancante può bloccare un'operazione legittima. Per questo RBAC e ABAC non si valutano con una sigla: occorre chiedere quale fonte alimenta i dati, chi la mantiene, come si gestiscono conflitti e ritardi e quale sia il comportamento sicuro in caso di dubbio. La risposta concorsuale diventa concreta quando include questa catena di responsabilità.

La scelta del modello deve essere documentata con esempi di operazioni consentite e negate. Test di autorizzazione ripetibili dimostrano che il privilegio atteso funziona e che una modifica non ha ampliato involontariamente l'accesso.

Ogni autorizzazione significativa deve poter essere spiegata, testata e riesaminata in modo proporzionato.

9.3 Crittografia e gestione delle chiavi

La crittografia protegge proprietà specifiche; non sostituisce IAM, hardening, gestione delle vulnerabilità o continuità. La cifratura simmetrica usa una chiave condivisa e tutela soprattutto la riservatezza. Quella asimmetrica usa una coppia di chiavi ed è utile in scenari diversi, ma non è una scelta automaticamente più efficace. Un hash produce un'impronta non reversibile; un MAC usa un segreto per sostenere integrità e autenticità; una firma digitale consente verifiche con la chiave pubblica corrispondente; un certificato collega una chiave pubblica a un soggetto. Confondere questi strumenti porta a promettere proprietà che non offrono.

La cifratura del disco protegge principalmente il supporto quando esce dal sistema autorizzato; non impedisce a un account legittimamente abilitato di leggere il dato. La cifratura del canale protegge il trasferimento; non risolve da sola un endpoint compromesso. Dati in transito, a riposo e in uso hanno esposizioni diverse e richiedono un'architettura proporzionata. La pseudonimizzazione non coincide con la cifratura: riduce la riconducibilità diretta ma può richiedere informazioni aggiuntive governate separatamente. Il capitolo 10 affronta la governance del dato; qui interessa collegare la scelta tecnica alla proprietà da proteggere.

La sicurezza della chiave è parte della sicurezza del dato. Il ciclo di vita comprende generazione, inventario, distribuzione, conservazione, uso, rotazione, revoca, eventuale recupero, archiviazione e distruzione. Per ogni chiave, certificato o secret servono almeno scopo, proprietario, custode, sistemi dipendenti, stato e scadenza. Un servizio HSM o di key management può ridurre errori operativi, ma non decide da sé chi autorizza l'uso, quali copie esistono o quando un materiale deve essere revocato.

Rotazione e revoca non sono sinonimi. La rotazione sostituisce un materiale ancora ritenuto affidabile secondo la politica o il rischio; la revoca dichiara che non deve più essere accettato, ad esempio dopo una compromissione. Distribuire un nuovo secret senza aggiornare tutte le dipendenze può interrompere il servizio. Cambiarlo senza valutare copie, log e possibili usi precedenti può lasciare irrisolto l'incidente. La separazione di ruoli limita il rischio che chi custodisce il materiale possa autorizzare da solo ogni operazione sensibile.

Se un secret applicativo risulta esposto, il primo compito è capire quali servizi, ambienti e dati dipendano da esso. Il team limita l'uso, preserva le evidenze, valuta revoca o sostituzione, distribuisce il nuovo materiale in modo controllato e osserva i log per stimare l'impatto. Non occorrono dettagli di prodotto per esporre il ragionamento. L'errore tipico è credere che un algoritmo robusto compensi un inventario assente o un secret inserito nel codice.

Un inventario non è una lista di valori segreti da copiare in un foglio. Registra riferimenti, proprietà e dipendenze senza esporre il materiale sensibile. Deve consentire di rispondere a domande pratiche: quale servizio usa questa chiave, quale certificato sta per scadere, chi autorizza un rinnovo, quale componente si ferma se il secret viene revocato? Senza queste informazioni, la rotazione diventa una modifica rischiosa e la risposta a una compromissione procede per tentativi.

Proteggere il materiale crittografico comprende controllo degli accessi, separazione di ambienti, registrazione degli usi sensibili e backup coerenti con il rischio. Il recupero di una chiave può essere necessario per continuità o obblighi di conservazione, ma amplia il numero di copie e soggetti da governare. Non è quindi un dettaglio tecnico isolato: è una decisione di processo, con proprietario, autorizzazione e test. Il candidato non deve proporre algoritmi o parametri senza un contesto aggiornato.

Anche il certificato richiede ciclo di vita. Scadenza, rinnovo, sostituzione e revoca possono interrompere integrazioni se non sono pianificati. La continuità non consiste nell'ignorare una scadenza, ma nel predisporre inventario, allerta, prova del rinnovo e rollback controllato. L'errore opposto è conservare lo stesso materiale troppo a lungo per timore del disservizio: in quel caso un'esigenza di esercizio annulla la possibilità di reagire correttamente a una perdita di fiducia.

CHIAVI, IMPRONTE E FIDUCIA

Meccanismo	Chi conserva quale chiave	Proprietà ed esempio
Cifratura simmetrica	Mittente e destinatario condividono un segreto	Protegge un archivio; chi ottiene la chiave può decifrare
Cifratura asimmetrica	Coppia pubblica/privata del destinatario	Si protegge per il destinatario con la pubblica; soltanto la privata consente il recupero previsto dallo schema
Firma crittografica	Il firmatario custodisce la privata; la pubblica verifica	Consente di verificare origine e integrità, non nasconde il documento
Hash	Nessuna chiave per l'hash ordinario	Impronta per confrontare integrità; non dimostra da sola l'identità dell'autore

Nei protocolli reali si combinano meccanismi: la cifratura asimmetrica non è descritta come soluzione efficiente per ogni grande archivio. Le firme non sono una generica “cifratura con la chiave privata”: algoritmi, codifiche e verifiche seguono schemi specifici. Un MAC usa invece un segreto condiviso per autenticità/integrità e non permette di attribuire pubblicamente la firma a uno solo dei titolari del segreto.

Un hash crittografico deve rendere impraticabile ricavare una preimmagine, trovare una seconda preimmagine per un messaggio dato o costruire collisioni. Le collisioni esistono matematicamente perché l'output è finito; resistenza non significa unicità assoluta. Per le password, un hash veloce non basta: si usano funzioni dedicate con salt individuale e costo regolato, senza memorizzare la password reversibile. Il salt non è una password segreta e ostacola il riuso di tabelle precalcolate; non compensa password deboli o controlli di accesso assenti.

Un certificato associa una chiave pubblica a un'identità secondo una politica, con firma dell'emittente e periodo di validità. La PKI gestisce emissione, catena di fiducia, rinnovo e revoca. Il client TLS verifica catena, identità del server atteso, validità e condizioni applicabili, non soltanto la presenza di un certificato. TLS protegge il canale e negozia chiavi di sessione; non prova che ogni contenuto del sito sia corretto né autorizza l'utente sulle pratiche.

Esempio. L'ufficio firma un documento con la propria privata e il destinatario verifica con la pubblica associata al firmatario. Per inviare lo stesso documento riservatamente serve anche cifratura/canale protetto. Un attaccante che sostituisce documento e semplice digest potrebbe far coincidere i due: manca una fonte affidabile dell'impronta. Certificati e firme crittografiche vanno inoltre distinti dalla qualificazione giuridica delle firme elettroniche nel quadro eIDAS.

9.4 Logging, rilevazione e triage

Un evento è un fatto osservabile; un log è la sua registrazione; un audit trail permette di ricostruire attività rilevanti; un alert è una segnalazione prodotta da una regola o da un'analisi; un incidente compromette o minaccia obiettivi di sicurezza o servizio. Un alert apre quindi una verifica: non prova da solo l'esistenza di un incidente. La distinzione evita sia allarmi ignorati sia escalation che paralizzano l'ente senza evidenze sufficienti.

Le sorgenti includono identity provider, sistemi operativi, reti, applicazioni, database, API, cloud e dispositivi. Un record utile contiene timestamp, identità o soggetto tecnico, azione, risorsa, esito, origine, contesto e un identificatore di correlazione. Il dato registrato deve essere pertinente: inserire password, token o dati personali non necessari nei log crea un nuovo rischio. Orologi incoerenti rendono fragile una timeline; accessi, modifica e cancellazione dei log devono essere governati; la conservazione dipende da finalità, rischio e obblighi applicabili, non da una durata universale.

La centralizzazione aiuta a correlare dati che resterebbero isolati. Un SIEM raccoglie, normalizza e mette in relazione eventi, ma non corregge log incompleti né sostituisce l'analista. Una regola troppo sensibile genera falsi positivi e consuma capacità di triage; una regola permissiva produce falsi negativi e ritarda la rilevazione. Baseline, soglie e casi d'uso devono essere legati ai servizi e riesaminati quando cambiano utenti, architettura o processi.

Il triage separa fatti documentati e ipotesi da verificare. Chiarisce attendibilità del segnale, identità coinvolte, privilegi, servizio, possibile diffusione, dati interessati e decisore. Un registro essenziale annota fonte, orario, azione, responsabile e risultato. È preferibile dichiarare «origine da verificare» piuttosto che trasformare una correlazione in una certezza. La qualità del ragionamento conta più della velocità di attribuire una causa.

Per un'elevazione anomala di privilegi, il team confronta log dell'identità, dell'applicazione e della risorsa interessata; verifica orari, sessione, origine, cambiamenti di ruolo e operazioni successive; poi valuta severità e attiva i ruoli necessari. L'errore tipico è installare un SIEM e considerare concluso il problema: servono scopo, sorgenti, campi, integrità, accesso governato, analisi ed esercitazioni.

La progettazione del logging parte dai casi d'uso: accessi privilegiati, modifiche di autorizzazione, operazioni su dati rilevanti, errori ripetuti, integrazioni fallite e azioni amministrative richiedono campi e correlazioni diversi. Registrare tutto senza criterio aumenta costo, rumore e rischio privacy; registrare troppo poco rende impossibile ricostruire una decisione. Una matrice sorgente-evento-campo-responsabile aiuta a rendere esplicita questa scelta prima dell'incidente.

L'integrità non significa soltanto impedire la cancellazione. Occorre definire chi può accedere ai log, chi può configurarli, chi può esportarli e quali tracce restano di queste stesse operazioni. Se chi compie un'azione può alterare liberamente la sua registrazione, l'audit trail perde affidabilità. I privilegi sul sistema di logging devono quindi essere separati, riesaminati e collegati a un processo di conservazione e dismissione.

Un alert utile è spiegabile. La regola indica quali condizioni hanno generato il segnale e quali dati servono per controllarlo; l'analista annota perché lo ha confermato, escluso o elevato. Questo feedback migliora la regola e la baseline. L'errore tipico è misurare il numero di alert invece della qualità delle decisioni: molte segnalazioni non dimostrano una capacità di rilevazione se nessuno può qualificarle entro un tempo coerente con il servizio.

La disponibilità dei log è anch'essa una dipendenza di servizio: una raccolta interrotta nel momento critico impedisce di valutare l'evento. Il monitoraggio deve quindi controllare anche copertura e salute delle sorgenti, non solo il loro contenuto. Il responsabile verifica periodicamente che i campi siano ancora prodotti dopo aggiornamenti o cambi di configurazione.

La verifica periodica deve produrre un esito, una correzione assegnata e una nuova prova di efficacia, non soltanto un controllo dichiarato.

Il controllo periodico mantiene attendibili le fonti e le correlazioni necessarie.

9.5 Incident response e continuità del servizio

La risposta a un incidente inizia prima dell'incidente. Policy, contatti, ruoli, accessi di emergenza, capacità di logging e prove di coordinamento fanno parte della preparazione. Il NIST CSF 2.0 collega la risposta alla governance, all'identificazione e alla protezione: rilevare, rispondere e recuperare non sono una sequenza isolata dal rischio e dalla continuità. Il modello locale può cambiare, ma deve chiarire decisioni, responsabilità, evidenze e condizioni di chiusura.

L'analisi definisce perimetro, vettore plausibile, identità, privilegi, sistemi, dati, dipendenze e timeline. La severità considera servizio, diffusione, impatto, urgenza e incertezza residua. Il contenimento limita danno e propagazione: può includere revoca di sessioni, sospensione di account o isolamento di componenti. Non è una reazione automatica. Isolare un sistema può preservare sicurezza ma interrompere un servizio essenziale o modificare evidenze; la decisione richiede un responsabile e una motivazione registrata.

L'eradicazione rimuove causa e persistenza; il recupero verifica identità, configurazioni, integrità, chiavi, dipendenze e monitoraggio. Le attività forensi non vanno improvvisate: acquisizione, conservazione e trattamento delle evidenze seguono competenze, autorizzazioni e procedure applicabili. Il capitolo 7 tratta backup e disaster recovery; qui il raccordo consiste nel chiedere se il servizio può rientrare in condizioni controllate e se il recupero reintroduce credenziali, configurazioni o dati compromessi.

Il playbook assegna ruoli: coordinatore dell'incidente, analista tecnico, responsabile del servizio, funzione di sicurezza, privacy, comunicazione, vertice e fornitori quando pertinenti. Una buona escalation distingue piano tecnico, decisioni sul servizio, valutazione privacy e interlocuzione istituzionale. La comunicazione non è un messaggio generico: deve dichiarare ciò che è noto, ciò che è ancora un'ipotesi, il canale e il prossimo aggiornamento. La timeline conserva fatti, fonti, decisioni e responsabili.

La chiusura richiede servizio stabile, evidenze custodite, comunicazioni svolte e azioni residue assegnate. Il riesame finale trasforma l'incidente in miglioramenti verificabili: correzione dei privilegi, rotazione di materiali, regole di rilevazione, esercitazioni, formazione o modifica del playbook. L'errore tipico è chiudere appena il servizio torna disponibile: recupero tecnico e apprendimento organizzativo sono entrambi necessari.

La continuità modifica le decisioni di risposta ma non le sostituisce. Se un servizio è essenziale, il coordinatore può scegliere un contenimento graduale, una modalità degradata o un ripristino da un ambiente controllato; deve però dichiarare quale rischio rimane e quali evidenze occorre preservare. Backup e replica possono aiutare a recuperare, ma non garantiscono che dati, configurazioni o credenziali ripristinate siano affidabili. L'analisi dell'incidente precede il rientro pieno in esercizio.

Il rapporto con i fornitori deve essere preparato prima dell'evento. Contatti, ruoli, accessi, evidenze disponibili e canali di escalation devono essere conosciuti senza trasferire automaticamente all'esterno la responsabilità dell'ente. Durante l'incidente il fornitore può fornire dati tecnici o intervenire su un componente, mentre il responsabile del servizio mantiene la visione delle conseguenze amministrative e della comunicazione. Il capitolo 12 approfondisce SLA e requisiti contrattuali; qui basta rendere esplicita la dipendenza.

Le esercitazioni verificano se numeri di reperibilità, privilegi di emergenza, log e ruoli funzionano davvero. Un playbook mai provato tende a contenere passaggi ambigui scoperti quando il tempo è scarso. Dopo l'esercitazione o l'incidente, le azioni di miglioramento devono avere responsabile, priorità ed evidenza di chiusura. «Fare formazione» è troppo generico se non indica a chi, su quale decisione e con quale verifica.

Una decisione di contenimento va riesaminata se cambiano le evidenze. Per esempio, una sospensione inizialmente limitata può diventare più ampia se emergono privilegi aggiuntivi, oppure essere ridotta se una verifica esclude la propagazione. Documentare il perché consente di bilanciare sicurezza, continuità e proporzionalità.

Il piano indica anche chi autorizza il rientro, quali controlli restano rafforzati e quando il riesame conferma la chiusura.

Una procedura aggiornata riduce ritardi e contraddizioni tra team, evidenze e decisioni.

La documentazione consente inoltre un riesame indipendente delle scelte adottate durante l'emergenza.

9.6 PA, ACN, CSIRT, NIS2 e privacy

SOGGETTI, GOVERNANCE E MISURE

Il d.lgs. 138/2024 recepisce NIS2. Il campo si verifica mediante articolo 3, settori degli allegati, dimensioni ed eccezioni: non basta che un'organizzazione usi un computer o sia pubblica. L'articolo 6 distingue essenziali e importanti. Fra gli essenziali rientrano, alle condizioni previste, grandi soggetti dell'allegato I, soggetti critici, determinati operatori digitali e le PA centrali dell'allegato III; gli altri soggetti nel campo che non sono essenziali sono importanti. L'individuazione ACN e la comunicazione di inserimento permettono di fissare gli adempimenti del soggetto. "Importante" non significa facoltativamente protetto.

Gli organi di amministrazione e direttivi approvano le modalità di attuazione delle misure, ne sovrintendono l'implementazione e rispondono delle violazioni nei termini dell'articolo 23. Seguono formazione cyber e promuovono quella del personale. La funzione tecnica realizza e documenta i controlli, ma non assorbe questa responsabilità. L'articolo 24 richiede misure tecniche, operative e organizzative proporzionate: analisi dei rischi, gestione degli incidenti, continuità e backup, fornitori, sviluppo e vulnerabilità, verifica dell'efficacia, igiene/formazione, crittografia, accessi e autenticazione quando appropriata. L'ente conserva decisioni, responsabili ed evidenze di efficacia, non soltanto acquisti di prodotti.

QUANDO E COME NOTIFICARE

Per l'articolo 25 è significativo l'incidente che causa o può causare grave perturbazione operativa/perdite finanziarie, oppure ripercussioni considerevoli materiali o immateriali su altri. La valutazione applica le specifiche ACN pertinenti al soggetto e al servizio; un alert isolato non è automaticamente un incidente significativo. ACN è autorità nazionale competente; CSIRT Italia riceve le notifiche e svolge i compiti di risposta e supporto previsti.

Passaggio ordinario NIS	Termine massimo e contenuto
Pre-notifica	Senza ingiustificato ritardo, entro 24 ore dalla conoscenza dell'incidente significativo; possibile origine malevola e impatto transfrontaliero
Notifica	Entro 72 ore dalla stessa conoscenza; prima valutazione di gravità/impatto e indicatori disponibili
Relazione intermedia	Su richiesta del CSIRT
Relazione finale	Entro un mese dalla notifica: descrizione, causa probabile, mitigazioni e impatto transfrontaliero noto

Se l'incidente è ancora in corso, si trasmette una relazione mensile sui progressi e quella finale entro un mese dalla conclusione della gestione. Per i prestatori di servizi fiduciari interessati dal caso, la notifica ha il termine speciale di 24 ore. I massimi non autorizzano ritardi ingiustificati né si calcolano dalla fine dell'indagine. Informazioni incomplete si aggiornano secondo la procedura, invece di aspettare una ricostruzione perfetta.

DECORRENZE AL 3 OTTOBRE 2026

La determinazione ACN 379907/2025, applicabile dal 15 gennaio 2026, sostituisce la 164179/2025: allegati 1/2 per misure degli importanti/essenziali, 3/4 per incidenti significativi di base. La 127434/2026, applicabile dal 30 aprile, distingue le coorti.

Coorte	Notifiche di base	Misure di base
Inseriti nel 2025 e permanenti nel 2026	Nove mesi dalla comunicazione di inserimento	Diciotto mesi dalla stessa comunicazione
Inseriti per la prima volta nel 2026	Dal 1 gennaio 2027	Entro 31 luglio 2027

Si conserva dunque la data della comunicazione: il reinserimento annuale non riavvia automaticamente i termini. Queste decorrenze non sospendono altri obblighi già applicabili, né cancellano le misure previgenti degli operatori per i quali il regime transitorio ne impone il mantenimento.

LEGGE 90/2024 E PROTEZIONE DEI DATI

La legge 90/2024, articolo 1 aggiornato anche dalla legge 132/2025, ha una platea e una tassonomia proprie: comprende, fra gli altri soggetti indicati, PA centrali, regioni, città metropolitane, comuni oltre 100.000 abitanti e capoluoghi di regione, ASL e determinati trasporti/società in house. Prevede segnalazione entro 24 ore e notifica completa entro 72 dalla medesima conoscenza dell'incidente, tramite le procedure ACN. Campo, esclusioni del comma 7 e coordinamento con NIS/altre discipline vanno verificati: non si sommano moduli indiscriminatamente perché due norme usano numeri uguali.

Un incidente cyber può compromettere un servizio senza dati personali; un data breach può derivare anche da un invio errato non ostile. Il GDPR richiede al titolare notifica al Garante senza ingiustificato ritardo e, ove possibile, entro 72 ore dalla conoscenza, salvo che sia improbabile un rischio per diritti e libertà; se il rischio è elevato, valuta la comunicazione agli interessati secondo l'articolo 34 e le sue eccezioni. Il responsabile del trattamento informa il titolare senza ingiustificato ritardo. I presupposti restano distinti e ogni violazione è documentata; il DPO informa e consiglia, mentre la decisione compete al titolare.

Caso svolto. Un ente già obbligato apprende lunedì alle 10 di un incidente NIS significativo. Senza attendere il massimo se può agire prima, pre-notifica entro martedì alle 10 e notifica entro giovedì alle 10; il mese della relazione finale decorre dalla notifica. Se emerge anche esfiltrazione di dati, il titolare attiva una valutazione GDPR distinta. Il coordinatore conserva timeline, fonti, decisioni, turni e sostituzioni; il ripristino è autorizzato dopo verifica di causa, privilegi e integrità, non solo perché il portale risponde.

▣ Verifica

MICRO-VERIFICA: CAMBIO DI RUOLO

Un istruttore passa a un altro ufficio. Quali due azioni, oltre ad assegnare i nuovi permessi, evitano privilegi residui?

Soluzione: revocare o riesaminare gli accessi legati al ruolo precedente e verificare sessioni, token, deleghe e account tecnici collegati. La procedura conserva l'evidenza della modifica.

CASO GUIDATO: ACCOUNT PRIVILEGIATO COMPROMESSO

Alle 08:20 l'identity provider segnala l'accesso di un amministratore da una sorgente insolita. Poco dopo l'applicazione registra un'esportazione anomala; il titolare nega di essere operativo.

Il team conserva alert e log, controlla l'allineamento temporale e correla identità, sessione, origine, privilegi e azioni. Il coordinatore decide il contenimento: revoca sessioni e sospende l'account, controllando token, recupero e account tecnici collegati. L'analisi cerca modifiche ai privilegi, persistenze ed esportazioni; il responsabile del servizio valuta l'impatto; la funzione privacy valuta separatamente l'eventuale violazione di dati personali. Il recupero include rimozione della causa, rotazione del materiale coinvolto, ripristino con privilegi minimi e monitoraggio rafforzato. Il riesame introduce elevazione temporanea, revoca centralizzata ed esercitazioni.

LABORATORIO: PLAYBOOK E TIMELINE

Campo		Domanda da compilare	
Trigger		quale evidenza avvia il triage?	
Severità		quali servizio, privilegi, dati e diffusione sono coinvolti?	
Coordinatore		chi mantiene decisioni e quadro di situazione?	
Azioni		quali validazioni, evidenze e contenimenti servono?	
Escalation		quali piani tecnico, servizio, privacy e istituzionale vanno valutati?	
Recupero		quali condizioni permettono il rientro?	
Chiusura		quali evidenze e azioni residue confermano la stabilità?	

Ora	Fatto o ipotesi	Fonte	Decisione
08:20	accesso amministrativo anomalo	log IdP	aprire il triage
08:27	esportazione anomala confermata	log applicativo	elevare la severità
08:35	il titolare nega l'accesso	contatto verificato	revocare sessioni e sospendere l'account
da compilare	fatto o ipotesi	evidenza	decisione motivata

Ora	Responsabile	Esito
08:20	analista SOC	caso preso in carico
08:27	coordinatore	referenti attivati
08:35	responsabile IAM	contenimento eseguito
da compilare	ruolo	risultato

SEI QUIZ COMMENTATI

Quiz 1. Un utente autenticato apre la pratica di un altro cittadino cambiando l’ID nell’URL. Quale controllo manca?

- A) Il rinnovo periodico del certificato TLS.
- B) L’autorizzazione sulla singola risorsa.
- C) La compressione della risposta.
- D) La verifica della sintassi JSON.

Risposta corretta: B. L’identità accertata non dimostra il diritto a consultare ogni oggetto. TLS protegge il canale, non sostituisce la verifica della risorsa.

Quiz 2. Quale combinazione esprime due categorie di fattori?

- A) Password e PIN.
- B) Due password di servizi diversi.
- C) Domanda segreta e password.
- D) Password e dispositivo di possesso verificato.

Risposta corretta: D. Conoscenza e possesso sono categorie diverse. Due segreti ricordati rimangono nella stessa categoria; il recupero deve conservare protezione adeguata.

Quiz 3. Vuoi verificare pubblicamente l’origine di un documento e rilevarne modifiche. Quale meccanismo è pertinente?

- A) Firma digitale crittografica verificabile con la chiave pubblica.
- B) Hash pubblicato nello stesso file senza fonte affidabile.
- C) Sola cifratura simmetrica.
- D) Codifica Base64.

Risposta corretta: A. La firma lega l’integrità alla chiave del firmatario secondo lo schema e la fiducia adottati; non offre da sola riservatezza. Un hash sostituibile insieme al documento non autentica l’origine.

Quiz 4. Il SIEM produce un alert insolito. Qual è il primo uso corretto?

- A) Dichiarare concluso l’incidente.
- B) Cancellare i log duplicati prima dell’analisi.
- C) Qualificarlo correlando fonti, contesto e impatto.
- D) Ripristinare subito tutti i sistemi.

Risposta corretta: C. L>alert è un segnale da valutare. La classificazione guida contenimento ed escalation e conserva le evidenze.

Quiz 5. Una chiave è sospettata compromessa. Quale distinzione è corretta?

- A) La rotazione prova che la vecchia chiave non è mai stata usata.
- B) La revoca ne invalida l'uso nel sistema pertinente; la sostituzione non cancella gli effetti pregressi.
- C) Revocare significa soltanto copiarla in un altro archivio.
- D) La scadenza futura rende superfluo intervenire.

Risposta corretta: B. Si revocano o disabilitano le credenziali coinvolte, si sostituisce il materiale e si ricostruisce l'esposizione. La rotazione da sola non chiude sessioni o usi già avvenuti.

Quiz 6. Un soggetto già obbligato apprende lunedì alle 10 di un incidente NIS significativo. Nel regime ordinario quale termine massimo vale per la notifica, distinta dalla pre-notifica?

- A) Martedì alle 10.
- B) Un mese dalla conoscenza.
- C) 72 ore dopo la pre-notifica.
- D) Giovedì alle 10.

Risposta corretta: D. Le 72 ore decorrono dalla medesima conoscenza dell'incidente, non dall'invio della pre-notifica. Rimane l'obbligo di agire senza ingiustificato ritardo e la disciplina speciale dei prestatori fiduciari.

DOMANDA DA COMMISSARIO

«Come imposteresti la risposta alla compromissione di un account privilegiato?»

Validerei il segnale e preserverei le evidenze; identificarei account, sessioni, privilegi, sistemi e dati; valuterei impatto e propagazione; attiveri i ruoli pertinenti. Conterrei senza distruggere evidenze, rimuoveri causa e persistenze, ruoterei il materiale interessato e ripristinerei con monitoraggio rafforzato. Chiuderei con timeline, azioni assegnate e lesson learned.

ERRORE TIPICO

Affidarsi a un solo controllo: una MFA con recupero debole, una cifratura con chiave esposta o un SIEM alimentato da log incompleti danno un falso senso di sicurezza.

DA SAPERE IN 5 RIGHE

IAM governa identità, account e privilegi lungo il ciclo di vita. Autenticare non significa autorizzare. La crittografia protegge proprietà definite solo se chiavi e secret sono governati. I log sono utili quando completi, coerenti e protetti. Incident response coordina triage, contenimento, recupero, comunicazione e apprendimento.

Riferimenti normativi e professionali essenziali

- NIST SP 800-63-4, per identità, autenticazione e federazione.
- NIST SP 800-57 Part 1 rev. 5, per concetti di key management.
- NIST SP 800-92 e progetto di revisione, per log management.
- NIST SP 800-61 rev. 3 e NIST CSF 2.0, per raccordo tra risposta, recupero e gestione del rischio.
- D.lgs. 4 settembre 2024, n. 138 e atti ACN/CSIRT Italia applicabili, da verificare al cut-off sul caso concreto.
- GDPR e fonti del Garante, per la distinta valutazione delle violazioni di dati personali.
- Legge 90/2024, articolo 1 vigente; determinazioni ACN 379907/2025 e 127434/2026, per decorrenze e coorti.

Data governance, open data, interoperabilità e qualità

B
BANDO

A
AREE

N
NUCLEI

D
DIARIO

O
OUTPUT

Un'amministrazione può possedere molti archivi e continuare a non sapere quale sia il dato affidabile, chi ne decida il significato e quale uso sia consentito. Quando il codice di una pratica cambia in un applicativo ma non nell'altro, il problema non è soltanto tecnico: può produrre istruttorie lente, pagamenti errati o risposte incoerenti al cittadino. Governare i dati significa rendere queste decisioni esplicite, verificabili e ripetibili.

Obiettivo didattico

Al termine del capitolo saprai distinguere governo, gestione e architettura del dato; assegnare ruoli e responsabilità; progettare inventario, glossario e scheda dataset; trasformare un difetto in una regola di qualità; distinguere open data, trasparenza, accesso e interoperabilità. Saprai inoltre argomentare perché uno scambio tra enti e una pubblicazione per il riuso richiedono valutazioni diverse.

Mappa BANDO: governare il dato

- B — Bando: individua profilo, programma e formato della prova; ricava il caso d'uso richiesto.
- A — Aree: collega le materie tecniche alle responsabilità dei soggetti coinvolti.
- N — Nuclei: qualità, classificazione, diritti, dipendenze e limiti.
- D — Diario: registra errori, motivazioni e correzioni; conserva i documenti utili a ricostruire il ragionamento.
- O — Output: uso interno, scambio autorizzato, e-service o distribuzione aperta.

Spiegazione teorica

Ogni scelta sul dato deve collegare significato, finalità, responsabilità, controllo e conseguenza. I nuclei del capitolo mostrano come rendere questa catena verificabile nel lavoro dell'ente e nella risposta concorsuale.

10.1 Il dato come asset informativo e le decisioni di governo

Un dato è una rappresentazione elementare: una data, un codice, un indirizzo, un importo. Diventa informazione quando qualcuno conosce il contesto in cui leggerlo. Il valore CHIUSA, per esempio, non è utile se non si sa quale procedimento descriva, chi l'abbia aggiornato e se indichi l'adozione del provvedimento, l'invio della comunicazione o il pagamento concluso. Un dataset è una raccolta organizzata di dati con struttura e significato definiti; un dominio riunisce dataset collegati a un ambito, come tributi, personale o occupazioni di suolo pubblico.

Parlare di dato come asset informativo non significa attribuirgli un prezzo automatico. Significa riconoscere che il dato sostiene decisioni, servizi, controlli e diritti, quindi richiede responsabilità paragonabili a quelle applicate agli altri beni necessari al funzionamento dell'ente. Un dato senza proprietario semantico può essere tecnicamente disponibile e insieme inutilizzabile: due uffici possono impiegare la stessa parola con definizioni incompatibili, oppure una procedura può conservare un identificativo che nessun servizio considera autorevole.

La data governance stabilisce chi decide sul dominio, con quali principi e attraverso quali evidenze. Riguarda finalità, usi ammessi, definizioni, priorità, qualità, classificazione, condivisione e dismissione. La gestione operativa esegue le decisioni: acquisisce, corregge, conserva, distribuisce e controlla. L'architettura descrive sistemi, modelli, flussi, interfacce e dipendenze. Sono piani collegati ma non equivalenti. La governance può stabilire che il codice pratica sia un identificativo unico; la gestione elimina i duplicati; l'architettura realizza il vincolo o il controllo nel servizio.

Un buon governo non impone lo stesso apparato documentale a ogni foglio di calcolo. Parte da rischio, impatto e riuso. Un dataset che alimenta un pagamento, un indicatore pubblico o un servizio rivolto a molte persone richiede definizioni, controlli e responsabilità più robuste di un elenco temporaneo usato da un singolo ufficio. La proporzionalità non è un permesso per restare vaghi: richiede di motivare perché una regola, un controllo o una revisione sono adeguati allo scopo.

Le decisioni diventano affidabili quando lasciano traccia. Una policy, una definizione approvata, un esito di controllo, una richiesta di accesso, una versione della scheda o una decisione di dismettere una distribuzione sono evidenze diverse. Non servono a costruire burocrazia aggiuntiva; permettono di ricostruire perché il dato è stato usato in un certo modo e di correggere un errore senza affidarsi alla memoria delle persone. Il Piano Triennale per l'informatica nella PA richiama proprio il legame tra organizzazione, processi, qualità dei dati e servizi digitali.

DALLA DECISIONE AL SERVIZIO

Un asset informativo va letto anche per le sue conseguenze: se una definizione cambia, possono cambiare calcoli, comunicazioni, indicatori, controlli e servizi collegati. Per questo la governance individua il dominio, stabilisce una fonte autorevole quando occorre risolvere un conflitto e rende visibile la versione valida. L'obiettivo non è centralizzare ogni decisione, ma evitare che ciascun ufficio costruisca una propria verità senza dichiararlo.

Un esempio utile riguarda l'indirizzo del richiedente. Il dato può essere usato per comunicazioni, istruttoria territoriale o statistiche aggregate; questi usi richiedono livelli diversi di aggiornamento, accesso e verifica. Il governo del dato esplicita tale differenza prima che l'applicativo la nasconda dietro un unico campo. Quando la stessa informazione alimenta più servizi, l'ente deve valutare effetto, priorità e dipendenze della modifica.

In una risposta scritta, distingui sempre principio e realizzazione. Dire che serve una piattaforma è insufficiente se non si indica quale decisione la piattaforma debba sostenere e quale evidenza permetta di controllarla. Un candidato mostra maturità quando collega definizione, responsabilità, controllo ed esito per il cittadino.

Errore tipico: confondere la governance con il possesso del database. L'amministratore tecnico custodisce un sistema; non può decidere da solo un nuovo uso di dati o il significato istituzionale di un campo.

Uso in prova: davanti a una domanda generica, parti da finalità e dominio, poi spiega decisione, attuazione ed evidenza. Questa sequenza evita una risposta ridotta a strumenti o sigle.

10.2 Ruoli, decisioni, RACI ed evidenze

I nomi data owner, data steward e data custodian descrivono funzioni organizzative frequenti. Non sono qualifiche giuridiche automaticamente obbligatorie né sostituiscono gli incarichi previsti dalla normativa. Sono utili se chiariscono chi può decidere, chi cura il significato e chi assicura l'attuazione tecnica. In un ente piccolo una persona può svolgere più funzioni; ciò non elimina la necessità di separare le decisioni dalle attività che le eseguono.

Il data owner presidia le decisioni sul dominio: finalità, priorità di qualità, usi consentiti, definizioni rilevanti e autorizzazioni organizzative. Deve conoscere il processo amministrativo cui il dato si riferisce. Il data steward cura il linguaggio comune, i metadati, le regole di qualità, le eccezioni e il dialogo tra uffici. Il custodian assicura disponibilità, backup, protezione, configurazioni e attuazione tecnica degli accessi. Il produttore crea o acquisisce il dato; il fruitore lo utilizza entro le autorizzazioni ricevute; il responsabile del servizio presidia l'erogazione verso utenti e altri uffici.

Queste funzioni si raccordano con RTD, responsabili di processo, sicurezza, DPO, trasparenza, archivio e procurement secondo le rispettive competenze. Il DPO non autorizza per definizione ogni trattamento; la funzione sicurezza non decide da sola la finalità; il tecnico non stabilisce il significato amministrativo. Quando sono coinvolti dati personali, accessi o diffusione, il confronto tra ruoli è necessario, ma ciascuno deve contribuire alla domanda che gli compete.

Una matrice RACI può rendere leggibile la distribuzione. R significa chi svolge l'attività; A chi risponde della decisione; C chi deve essere consultato; I chi deve essere informato. Il modello non vale perché ha quattro lettere: serve quando è collegato a una decisione concreta e a una prova dell'esecuzione. Per la definizione di "occupazione attiva", ad esempio, l'owner approva il significato, lo steward cura il glossario, il custodian applica la modifica nei sistemi e gli uffici interessati sono informati.

Decisione	Responsabilità di governo	Cura operativa	Evidenza utile
definizione di un campo	owner	steward	glossario approvato
regola di completezza	owner	steward	specificazione e risultato del controllo
accesso tecnico	owner autorizza	custodian attua	richiesta, autorizzazione e log
distribuzione aperta	funzione competente	gruppo multidisciplinare	valutazione, metadati e licenza

La responsabilità non si esaurisce con la prima assegnazione. Un cambio di processo, fornitore, sistema o finalità può rendere inadeguata una vecchia matrice. Perciò occorre prevedere riesami e un canale per le eccezioni: un'anomalia segnalata senza assegnatario resta soltanto un'informazione, non diventa un rimedio.

COME ASSEGNARE RESPONSABILITÀ SENZA CREARE ETICHETTE VUOTE

Una mappa di ruoli parte dalle decisioni ricorrenti: chi approva una definizione, chi autorizza un nuovo fruitore, chi valuta un difetto di qualità, chi cambia la frequenza di aggiornamento, chi sospende una distribuzione. Solo dopo si collegano persone o uffici alle funzioni. Questo ordine evita che la matrice RACI diventi un organigramma cosmetico, pieno di sigle ma incapace di risolvere un conflitto reale.

Prendi il caso di un campo usato da tributi e polizia locale. Se il produttore segnala che il valore non è più attendibile, lo steward raccoglie l'anomalia e verifica la regola; l'owner valuta impatto e priorità; il custodian applica la correzione tecnica; i fruitori ricevono la comunicazione sulle conseguenze. Se invece si deve ampliare l'uso del campo a un nuovo servizio, intervengono anche processo, privacy e sicurezza. La stessa persona può partecipare a più passaggi, ma le evidenze devono mostrare quale funzione ha svolto.

Le responsabilità devono essere comprensibili anche in caso di assenza, cambio di personale o affidamento a un fornitore. Perciò la documentazione indica funzione, recapito organizzativo, sostituzione e momento di riesame. Un contratto esterno può affidare attività tecniche, ma non cancella il dovere dell'ente di governare finalità e controlli.

Errore tipico: usare owner e steward come sinonimi. Il primo decide sul dominio; il secondo rende applicabili e controllabili le decisioni. In prova è preferibile descrivere la funzione invece di attribuire titoli non previsti nell'ente.

Applicazione: costruisci una riga “decisione–responsabile–evidenza” prima di disegnare un organigramma. Mostra subito che sai collegare responsabilità e controllabilità.

10.3 Ciclo di vita, classificazione e lineage

Il ciclo di vita del dato inizia prima dell'inserimento nel database. Nella fase di pianificazione si chiariscono finalità, fonte, dati necessari, durata, formati, classificazione e controlli iniziali. Raccogliere informazioni “perché potrebbero servire” aumenta costi, errori e rischi. La domanda corretta è: quali dati servono a questo processo, con quale significato, per quanto tempo e con quale livello di affidabilità?

Nella creazione o acquisizione si registrano origine, data, responsabile e regole di validazione. Nella trasformazione possono cambiare struttura, codici o granularità: una lista di pratiche diventa un indicatore; più fonti vengono riconciliate; un archivio produce una distribuzione. Senza traccia di tali passaggi, un risultato non è verificabile. Il lineage ricostruisce provenienza e trasformazioni: non coincide con un diagramma tecnico completo, ma consente di capire da quale fonte deriva un campo, quale versione è stata applicata e quali dataset dipendono da esso.

Validazione, uso e condivisione sono fasi diverse. Validare controlla che i dati rispettino regole definite; uso interno richiede che gli addetti siano autorizzati e che la finalità sia pertinente; condividere con un altro soggetto richiede di definire destinatari, attributi, responsabilità e condizioni. La disponibilità tecnica di un archivio non rende automatico alcun passaggio. In particolare, il principio *once only* mira a evitare richieste ripetute di informazioni già disponibili, ma non autorizza accessi indiscriminati.

La classificazione accompagna l'intero ciclo. Un dataset può essere pubblico, interno o riservato; può contenere dati personali, informazioni soggette a limitazioni o dati la cui diffusione danneggerebbe la sicurezza o interessi di terzi. La classificazione non è un'etichetta decorativa: orienta accessi, canali, misure, conservazione e valutazione di eventuale pubblicazione. Pseudonimizzazione, anonimizzazione e aggregazione non sono sinonimi. La pseudonimizzazione riduce il collegamento immediato alla persona ma conserva la possibilità di riconnessione con informazioni aggiuntive; l'anonimizzazione richiede una valutazione più rigorosa del rischio di reidentificazione.

La conservazione comprende archivi principali, copie, estratti, cache e dataset derivati. Dismettere una tabella lasciando una replica usata per elaborazioni periodiche non conclude il ciclo. Occorre individuare dipendenze, stabilire cosa archiviare o cancellare secondo la disciplina applicabile e verificare l'esito. Una dismissione ben governata produce un'evidenza: chi ha deciso, che cosa è stato dismesso, quali servizi sono stati aggiornati e quale accesso deve cessare.

IL DATO CAMBIA PRIMA DEL DATABASE E DOPO L'USO

Ogni fase del ciclo deve avere un punto di ingresso e uno di uscita riconoscibile. L'acquisizione registra una fonte; la validazione decide se un record può essere usato; la trasformazione produce una nuova versione; la condivisione individua destinatari; l'archiviazione modifica disponibilità e tempi di recupero; la dismissione interrompe accessi e dipendenze. Questa lettura aiuta a evitare che un cambiamento apparentemente locale si propaghi in moduli, cruscotti e comunicazioni.

Il lineage è particolarmente utile quando due risultati non coincidono. Non si parte accusando l'altro ufficio di avere dati sbagliati: si confrontano definizione, fonte, data di estrazione, trasformazioni e regole applicate. Può emergere che una tabella usi la data di protocollo e l'altra la data di conclusione, oppure che una riconciliazione sia avvenuta solo in uno dei due sistemi. La causa diventa così discutibile e correggibile.

La classificazione va aggiornata quando cambiano contenuto, destinatari o contesto. Un dataset interno può contenere un estratto idoneo alla pubblicazione solo dopo una nuova valutazione; una distribuzione precedentemente aperta può richiedere sospensione se cambia la composizione. La governance registra queste decisioni, ne individua il responsabile e stabilisce come informare i fruitori.

Errore tipico: considerare il lineage una funzione riservata ai data engineer. Anche il candidato che non progetta pipeline deve saper chiedere origine, trasformazioni, versioni e dipendenze prima di fidarsi di un indicatore.

Uso in prova: per descrivere il ciclo di vita, collega ogni fase a una domanda: perché raccogliamo, da dove arriva, chi controlla, chi usa, cosa condividiamo, quanto conserviamo e come dismettiamo.

10.4 Inventario, catalogo, glossario e metadati

Inventario, catalogo, glossario e scheda dataset si sostengono a vicenda, ma rispondono a domande diverse. L'inventario censisce ciò che l'ente possiede o utilizza: dominio, dataset, finalità, fonti, sistemi, responsabili, classificazione e stato. È utile anche per risorse non pubbliche, perché rende visibili archivi dimenticati, repliche e dipendenze. Il catalogo rende le risorse ricercabili e comprensibili attraverso metadati coerenti. Può essere interno o pubblico. Catalogare non significa rendere accessibili i record.

Il glossario stabilisce il significato dei termini. “Pratica chiusa” può indicare un provvedimento adottato, una comunicazione inviata o un pagamento concluso. Se uffici diversi usano la stessa etichetta per eventi diversi, non basta uniformare il formato: occorre decidere la definizione, il dominio di validità e il responsabile del suo aggiornamento. Un vocabolario condiviso, codici stabili e regole di mappatura sostengono anche l'interoperabilità semantica.

I metadati descrivono il dato e il suo contesto. Quelli descrittivi comprendono titolo, descrizione, parole chiave e copertura. Quelli strutturali indicano campi, tipi, relazioni e codifiche. Quelli amministrativi riportano titolare, contatti, condizioni d'uso e licenza; quelli tecnici formato, interfaccia, versione e frequenza; quelli di qualità regole, indicatori, esiti e data dell'ultima verifica. Il profilo nazionale DCAT-AP_IT è un riferimento per descrivere dataset e distribuzioni nei cataloghi pubblici: non sostituisce l'analisi del caso né rende obbligatoria la pubblicazione.

Una scheda dataset minima deve permettere a una persona competente di capire cosa sia la risorsa, a quale processo serva e come trattarla. Include identificativo stabile, finalità, descrizione, fonte, owner e steward, struttura, classificazione, frequenza, lineage, regole di qualità, accessi, conservazione, distribuzioni, licenza se pertinente e contatto. Una distribuzione è una modalità concreta di accesso, per esempio file CSV, JSON, API o download; il dataset è invece la risorsa logica. Confondere i due piani produce cataloghi poco chiari e richieste tecniche mal formulate.

Artefatto	Domanda a cui risponde	Esempio
inventario	quali risorse esistono e chi le presidia?	archivio autorizzazioni e sistemi collegati
catalogo	come trovo e comprendo una risorsa?	descrizione ricercabile di un dataset
glossario	che cosa significa un termine?	definizione di “occupazione attiva”
lineage	da dove proviene e come cambia?	istanza, protocollo, gestionale, mappa
scheda dataset	quali regole valgono per questa risorsa?	frequenza, qualità, accessi e versione

METADATI CHE AIUTANO DAVVERO A DECIDERE

Un metadato utile risponde a una domanda concreta. L'identificativo stabile evita che una risorsa venga confusa con una copia; la frequenza indica se un dato è adatto a un controllo giornaliero; il contatto evita che il fruitore corregga da sé un valore ambiguo; la licenza chiarisce se e come il riuso sia consentito. Anche la data di ultima revisione conta: una descrizione elegante ma obsoleta può essere più dannosa dell'assenza di catalogo.

Nel costruire un inventario, non limitarti ai dataset ufficiali. Chiedi quali estratti, fogli di lavoro, interfacce, file di scambio e copie vengono davvero usati dal processo. L'obiettivo non è censire ogni file personale, ma individuare risorse che condizionano un servizio, un obbligo o una decisione. Poi collega ogni risorsa al dominio, alla fonte e alla funzione che la presidia.

Il glossario richiede una disciplina semplice: definizione, contesto, esempi di inclusione o esclusione, responsabile e versione. Questa struttura rende il termine riusabile da uffici diversi e permette di riconoscere quando una modifica semantica impone aggiornamenti a schemi, istruzioni o indicatori. Senza questa cura, l'interoperabilità tecnica può moltiplicare rapidamente un equivoco invece di risolverlo.

Errore tipico: compilare metadati solo quando si pubblica un file. Se definizioni, frequenza e responsabilità non sono curate durante il ciclo di vita, il catalogo diventa presto un elenco non affidabile.

Applicazione: nel laboratorio finale usa prima l'inventario e poi la scheda. Il primo dà la vista d'insieme; la seconda consente di governare il singolo dataset.

10.5 Qualità del dato: regole, metriche, anomalie e rimedio

La qualità del dato è l'adeguatezza allo scopo. Non coincide con l'assenza di celle vuote, con la pulizia grafica o con un numero molto grande di informazioni raccolte. Un indirizzo può essere formalmente completo ma non aggiornato; un codice può rispettare il formato e riferirsi alla pratica sbagliata. Perciò una valutazione seria sceglie dimensioni e controlli in rapporto all'uso amministrativo del dataset.

Le dimensioni ricorrenti sono accuratezza, completezza, coerenza, tempestività, validità e unicità. L'accuratezza riguarda la corrispondenza alla realtà o a una fonte autorevole. La completezza indica la presenza dei valori necessari, non la raccolta indiscriminata di ogni possibile valore. La coerenza cerca contraddizioni fra campi, record o sistemi. La tempestività valuta se l'aggiornamento avviene entro il tempo utile. La validità verifica formato, dominio e regole; l'unicità individua duplicati non giustificati. Nessuna di queste dimensioni basta da sola.

Una regola di qualità deve essere controllabile. “I dati devono essere buoni” non consente alcuna verifica. “Ogni autorizzazione attiva deve avere codice ufficio valorizzato e appartenente al dominio approvato” permette invece di definire popolazione, controllo e risultato. La metrica può essere percentuale di record conformi, numero di duplicati o tempo tra evento e aggiornamento. La soglia va motivata in base a rischio, volume e conseguenze; non va inventata perché una percentuale sembri rassicurante.

Problema	Regola	Metrica	Presidio
responsabile mancante	pratica attiva con responsabile	percentuale conforme	controllo periodico
codice duplicato	identificativo unico	duplicati rilevati	controllo al salvataggio
stato incoerente	chiusa implica data di chiusura	record incoerenti	riconciliazione

Il controllo non termina quando individua uno scarto. Il registro delle anomalie associa origine, impatto, priorità, assegnatario, rimedio, data di chiusura e verifica successiva. Se la stessa anomalia ricompare, occorre chiedersi se il difetto nasca dall'interfaccia, dalla formazione, dalla regola di processo o da un'integrazione. Correggere un singolo record può essere necessario, ma non risolve una causa sistemica.

Il capitolo sulle basi dati sviluppa vincoli e qualità a livello tecnico. Qui l'attenzione è sul governo: chi definisce la regola, chi verifica l'esito, chi accetta un'eccezione e come l'ente dimostra di aver corretto il problema. Un indicatore senza proprietario e senza azione prevista è una misura descrittiva, non un controllo.

DAL CONTROLLO ALLA RESPONSABILITÀ DI MIGLIORAMENTO

Una regola di qualità deve indicare anche il punto in cui viene applicata. Un controllo al salvataggio previene l'errore; un controllo notturno intercetta dati già caricati; una riconciliazione periodica confronta fonti diverse. La scelta dipende dalle conseguenze: bloccare un'istruttoria per un campo secondario può essere sproporzionato, mentre consentire un pagamento con un identificativo duplicato può essere rischioso. Il candidato deve saper motivare questo equilibrio.

Le anomalie non hanno tutte lo stesso peso. Un valore mancante può impedire una comunicazione ma non la registrazione iniziale; un codice errato può propagarsi a più servizi; un ritardo nell'aggiornamento può essere tollerabile per statistiche mensili e inaccettabile per una situazione operativa. Perciò il registro distingue gravità, impatto, dataset dipendenti, scadenza e verifica di chiusura. La priorità deriva dalla finalità e dal danno possibile.

Il monitoraggio deve produrre apprendimento. Se una regola fallisce spesso, l'ente può rivedere l'interfaccia, aggiornare le codifiche, correggere l'integrazione o formare chi inserisce i dati. In ogni caso conserva l'evidenza della decisione. Limitarsi a inviare un elenco di errori senza assegnazione e riesame equivale a misurare il problema senza governarlo.

Errore tipico: fissare una soglia senza spiegare che cosa accade se non viene raggiunta. Una soglia utile deve attivare un responsabile, un approfondimento o una decisione proporzionata.

Uso in prova: formula sempre la catena regola–metrica–evidenza–responsabile–rimedio. È più convincente di un elenco di dimensioni imparato a memoria.

Quando una metrica peggiora, l'ente confronta periodo, fonte e cambiamenti di processo prima di attribuire la colpa a chi inserisce i dati. Questa analisi distingue un errore occasionale da una regola ormai inadatta e consente di programmare un rimedio verificabile.

10.6 Open data, riuso, distribuzioni e limiti

Gli open data sono dati predisposti per il riuso secondo il quadro applicabile. La definizione richiede i tre requisiti cumulativi dell'articolo 1, lettera l-ter, CAD: una licenza o previsione normativa che permetta a chiunque l'uso anche commerciale in forma disaggregata; accessibilità con tecnologie ICT in formato aperto adatto all'elaborazione automatica e con metadati; gratuità o costi marginali di riproduzione/divulgazione, salvo la disciplina dell'articolo 7 del d.lgs. 36/2006. Si aggiungono aggiornamento e documentazione necessari a usare correttamente la risorsa. Un PDF può essere utile alla consultazione; difficilmente consente elaborazioni automatiche. Un CSV senza descrizione dei campi e senza indicazioni sul riuso lascia invece il fruitore nell'incertezza.

Occorre distinguere situazioni che spesso vengono confuse. La pubblicazione web rende un contenuto disponibile online. La trasparenza risponde a specifici obblighi di conoscibilità previsti dalla disciplina applicabile. L'accesso segue una richiesta, un oggetto e limiti propri. La condivisione mette dati a disposizione di soggetti determinati per una finalità autorizzata. L'open data destina una risorsa al riuso del pubblico con formati, metadati e condizioni adeguati. Le situazioni possono sovrapporsi, ma nessuna si trasforma automaticamente nell'altra.

La direttiva (UE) 2019/1024 disciplina il riutilizzo dell'informazione del settore pubblico. Il regolamento di esecuzione (UE) 2023/138 individua categorie di dataset di elevato valore: geospaziale, osservazione della Terra e ambiente, meteorologia, statistiche, imprese e proprietà delle imprese, mobilità. Per le serie concretamente comprese, il regolamento prevede formati leggibili meccanicamente e disponibilità tramite API, oltre al download massivo dove indicato nell'allegato. Si pubblicano termini d'uso, criteri di qualità e documentazione delle API e un contatto; i metadati identificano il dataset come HVD.

Le condizioni di riuso seguono CC0, CC BY 4.0 o licenza equivalente/meno restrittiva secondo l'allegato, con il regime di gratuità ed eventuali deroghe applicabili. Non basta però riconoscere una parola chiave: occorre verificare allegato, titolare, campo di applicazione e limiti del caso.

Dataset e distribuzione restano distinti. Il dataset "occupazioni di suolo pubblico" può avere una distribuzione CSV per il download, una JSON per elaborazioni e una visualizzazione cartografica. Ogni distribuzione richiede versione, frequenza, descrizione dei campi e condizioni d'uso coerenti. La scelta della licenza e le indicazioni operative devono essere ricontrollate sul quadro vigente prima della pubblicazione: non è prudente trasformare una raccomandazione tecnica in un obbligo universale.

Non tutto ciò che un ente detiene può essere reso aperto. Vanno valutati dati personali, segreti, sicurezza, diritti di terzi, finalità e disciplina settoriale. Una selezione di campi o un'aggregazione può ridurre il rischio, ma non autorizza automaticamente la diffusione. La valutazione dell'apertura è separata da quella dell'uso interno o dello scambio autorizzato; coinvolge competenze amministrative, tecniche, privacy e sicurezza.

RIUSO UTILE, NON SEMPLICE ESPOSIZIONE

Un dataset aperto è utile quando una persona diversa dall'ente può capirlo, ottenerlo e riutilizzarlo senza ricostruirne ogni volta il significato. Per questo le descrizioni dei campi, le codifiche, la frequenza e la versione fanno parte della qualità del riuso. Se un codice territoriale cambia senza avviso, una distribuzione formalmente disponibile può produrre analisi errate. Il governo dell'apertura include quindi un canale per aggiornamenti, correzioni e cessazione della distribuzione.

La valutazione dei limiti non è una formalità da svolgere all'ultimo momento. Si analizzano contenuto, granularità, collegabilità con altre fonti, destinatari prevedibili, finalità e diritti coinvolti. Una tabella apparentemente anonima può diventare problematica se combina luogo, data, evento raro e altri attributi. Aggregare o eliminare un campo può ridurre il rischio, ma occorre verificare che la risorsa resti corretta e non induca interpretazioni sbagliate.

Nelle risposte d'esame, evita formule assolute. Spiega il metodo: prima si determina se il quadro applicabile consente la pubblicazione, poi si definiscono contenuto e distribuzione, infine si curano metadati, condizioni, aggiornamento e controllo degli effetti.

Errore tipico: affermare che un file già online sia automaticamente open data. Senza metadati, condizioni di riuso, qualità e valutazione dei limiti, la sola pubblicazione non basta.

Applicazione: prima di proporre l'apertura, verifica finalità, classificazione, campi, qualità, formato, aggiornamento, licenza, contatto e canale di distribuzione. Solo dopo scegli se il riuso pubblico sia appropriato.

Il riuso richiede anche continuità informativa. Se il dataset cambia struttura, l'ente avvisa i fruitori, conserva una descrizione della versione e valuta tempi ragionevoli di transizione. Pubblicare senza comunicare una modifica può compromettere analisi e applicazioni altrui.

10.7 Interoperabilità, privacy, sicurezza e responsabilizzazione

L'interoperabilità è la capacità di organizzazioni e sistemi di cooperare preservando significato, responsabilità e regole. Non coincide con un'API né con l'accesso libero. Un'API riguarda principalmente il livello tecnico; non decide se lo scambio sia legittimo, chi corregga un errore o quale definizione debba essere usata per un attributo come "residente attivo".

I quattro livelli aiutano a ordinare il ragionamento. Il livello giuridico verifica basi, condizioni e limiti compatibili. Quello organizzativo chiarisce processi, ruoli, accordi e gestione delle eccezioni. Il livello semantico allinea significati, vocabolari, schemi, codici e identificatori. Il livello tecnico riguarda protocolli, formati, sicurezza dell'interfaccia e modalità di scambio. Un progetto può funzionare sul piano tecnico e fallire sugli altri: due sistemi possono scambiarsi il campo stato senza condividere il significato dei codici.

ModI, PDND ed e-service disciplinano aspetti di interoperabilità della PA che il candidato deve saper collocare. Per contratti API, ruoli di erogatore e fruitore e il percorso concettuale di adesione/fruizione il rinvio è al capitolo 06, sezione “N-TR01-06-06 · Interoperabilità PA ed e-service”. Qui resta la domanda di governo: quale finalità giustifica lo scambio, quali attributi sono necessari, chi autorizza, quale versione è valida, come si gestiscono errori e revoche, quali evidenze dimostrano i controlli svolti.

Il regolamento (UE) 2024/903, noto come Interoperable Europe Act, considera l'interoperabilità dei servizi pubblici digitali transeuropei anche sotto profili organizzativi, semantici e giuridici. Le valutazioni previste dal regolamento vanno applicate nel loro preciso campo soggettivo e oggettivo; non è corretto trasferirle automaticamente a ogni progetto locale. In prova, questa cautela è preferibile a dichiarazioni assolute su obblighi e piattaforme.

Privacy e sicurezza accompagnano ogni scelta di uso, scambio o apertura. Si verificano finalità, base giuridica, minimizzazione, classificazione, destinatari, conservazione e misure. Gli accessi seguono necessità e autorizzazione; operazioni e decisioni rilevanti devono lasciare evidenza. Privacy by design e security by design significano che tali requisiti entrano nella progettazione e nei cambiamenti, non che vengano controllati soltanto dopo un incidente. IAM, crittografia, logging e gestione degli incidenti sono sviluppati nel capitolo 09; qui servono a sostenere responsabilizzazione e governo del dataset.

LO SCAMBIO CORRETTO È ANCHE UNA DECISIONE DI PROCESSO

Prima di attivare uno scambio automatico, l'ente descrive evento che lo attiva, finalità, attributi necessari, erogatore, fruitore, risposta attesa, errori possibili e responsabilità di correzione. Questo non è ancora il contratto tecnico dell'API: è il requisito organizzativo che il contratto dovrà rappresentare. Se manca, la tecnologia rischia di trasferire campi superflui o semanticamente ambigui con grande efficienza.

La semantica va verificata con esempi reali. Due enti possono usare la data inizio occupazione con riferimento alla richiesta, al provvedimento o all'effettiva occupazione. Un vocabolario, una regola di codifica e una versione condivisa rendono lo scambio controllabile. Quando il significato cambia, occorre comunicare la modifica, gestire la compatibilità e verificare gli effetti sui fruitori. È il punto in cui governance, interoperabilità e qualità si incontrano.

Responsabilizzazione significa poter spiegare, dopo un controllo o un reclamo, quale dato è stato usato, con quale finalità, da quale fonte, sotto quale autorizzazione e con quali misure. Non promette assenza di errori; rende invece gli errori individuabili, limitabili e correggibili. Questo è il collegamento pratico tra data governance, tutela dei diritti e affidabilità dei servizi digitali.

Errore tipico: applicare il principio *once only* come se autorizzasse ogni amministrazione a interrogare qualsiasi archivio. Evitare richieste ripetute non elimina finalità, autorizzazioni, minimizzazione e tracciabilità.

Uso in prova: descrivi lo scambio in quattro livelli e concludi separando interoperabilità autorizzata e apertura al riuso. Questa distinzione evita di confondere cooperazione amministrativa e pubblicazione indiscriminata.

La sicurezza dello scambio include controllo degli accessi, protezione delle comunicazioni, registrazione delle operazioni e gestione delle anomalie. Il livello di dettaglio tecnico dipende dal servizio, ma la governance deve stabilire chi valuta il rischio, chi approva le eccezioni e come si verifica l'efficacia dei controlli.

Caso guidato: occupazioni di suolo pubblico

Un comune gestisce autorizzazioni per occupazione di suolo pubblico. Le istanze arrivano dal portale, l'ufficio aggiorna il gestionale, polizia locale e tributi usano alcuni dati e l'ente valuta una mappa aperta.

1. Definisce finalità: istruttoria, controllo ed entrate. L'eventuale apertura resta una decisione distinta.
2. Assegna ruoli: il responsabile del processo presidia il dominio; uno steward cura glossario e qualità; la funzione ICT custodisce sistemi e accessi.
3. Registra lineage: portale, protocollo, gestionale e sistema cartografico. Ogni campo rilevante ha fonte e versione.
4. Fissa regole: identificativo unico, data finale non precedente all'inizio, stato coerente con il provvedimento, geometria presente quando necessaria.
5. Classifica i dati: nominativi e contatti non confluiscono automaticamente in una distribuzione aperta.
6. Per lo scambio con altri uffici, definisce finalità, attributi, semantica, responsabilità e gestione degli errori.
7. Per l'open data, valuta separatamente campi, limiti, formato, metadati, licenza, frequenza e contatto.
8. Riesamina scarti, ritardi, reclami, versioni e decisioni di dismissione.

Laboratorio: inventario e scheda dataset

Campo dell'inventario	Compilazione richiesta
dominio e dataset	nome e identificativo stabile
finalità	processi e usi autorizzati
fonti e sistemi	origine, copie e dipendenze
responsabilità	owner, steward, custodian e fruitori
classificazione	pubblico, interno o riservato; dati personali
stato	attivo, in revisione o dismesso

Per la scheda dataset aggiungi descrizione, copertura, struttura e codifiche, glossario, frequenza, lineage, regole e indicatori di qualità, accessi, conservazione, distribuzioni, licenza quando applicabile, contatto, data e versione di revisione. Poi percorri l'albero decisionale: serve al processo interno? Va condiviso con soggetti determinati? Richiede uno scambio automatico? È destinato al riuso pubblico? Ogni risposta apre una valutazione diversa.

Domanda da commissario

Come imposteresti la governance di un dataset destinato allo scambio tra enti e al possibile riuso come open data?

Partirei da finalità, dominio e fonti. Assegnerei responsabilità, costruirei inventario, glossario, lineage e scheda dataset, poi definirei regole e indicatori di qualità. Per lo scambio fisserei base, destinatari, semantica, versione e gestione degli errori. Per l'apertura verificherei separatamente limiti, formato leggibile meccanicamente, metadati, licenza, distribuzioni e aggiornamento. Documenterei decisioni, accessi, anomalie e riesami.

Verifica

SEI QUIZ COMMENTATI

Quiz 1. Chi decide finalità e priorità di qualità del dominio, nel modello organizzativo illustrato?

- A) Il data owner nel perimetro delle proprie competenze.
- B) Il solo amministratore del database.
- C) Qualunque fruitore che possiede una copia.
- D) Il catalogo automatico.

Risposta corretta: A. L'owner presidia le decisioni; steward e custodian curano rispettivamente significato/qualità e attuazione tecnica. I nomi del modello non creano nuovi poteri legali.

Quiz 2. Quale attività descrive meglio lo steward?

- A) Approvare autonomamente qualsiasi nuovo trattamento.
- B) Sostituire il responsabile del procedimento.
- C) Curare glossario, metadati, regole ed eccezioni di qualità.
- D) Amministrare sempre l'infrastruttura fisica.

Risposta corretta: C. Lo steward rende coerenti significati e regole e raccorda gli uffici. La decisione su nuovi usi resta alla funzione competente.

Quiz 3. Due indicatori diversi derivano apparentemente dallo stesso archivio. Quale controllo aiuta a ricostruire la differenza?

- A) Confrontare soltanto il formato dei grafici.
- B) Verificare fonte, estrazione, trasformazioni e versioni tramite lineage.
- C) Scegliere automaticamente il risultato più recente.
- D) Eliminare uno dei dataset.

Risposta corretta: B. Il lineage ricostruisce la provenienza e le trasformazioni; differenze di data, definizione o filtro possono spiegare valori diversi senza presumere un errore dell'altro ufficio.

Quiz 4. Un catalogo descrive un dataset riservato. Quale conclusione è corretta?

- A) I record diventano aperti.
- B) Il dataset perde la classificazione.
- C) Ogni ente può interrogarlo senza finalità.
- D) La descrizione non equivale all'accesso ai record.

Risposta corretta: D. Metadati e accesso sono piani diversi. La catalogazione rende reperibile e comprensibile una risorsa senza cancellarne limiti e autorizzazioni.

Quiz 5. Una regola di qualità prevede un codice ufficio valido in ogni pratica attiva. Quale indicatore è coerente?

- A) Pratiche attive con codice valido diviso tutte le pratiche attive, con periodo, soglia e azione definiti.
- B) Dimensione totale del database in byte.
- C) Numero di utenti registrati.
- D) Percentuale di campi qualsiasi compilati.

Risposta corretta: A. Numeratore e denominatore si riferiscono alla popolazione della regola. Occorre poi assegnare chi interviene sugli scarti; misure di volume non dimostrano quella qualità.

Quiz 6. Un CSV è scaricabile gratuitamente, ma vieta ogni riuso commerciale e non ha metadati. Rispetto già la definizione CAD di dati di tipo aperto?

- A) Sì, basta la gratuità.
- B) Sì, basta l'estensione CSV.
- C) No: mancano condizioni di riuso e metadati richiesti.
- D) Solo se il file supera una dimensione minima.

Risposta corretta: C. I requisiti CAD sono cumulativi: condizioni per l'uso anche commerciale, formato aperto elaborabile con metadati e regime economico previsto. La dimensione non sostituisce tali requisiti.

Da sapere in 5 righe

La data governance rende esplicite decisioni, ruoli, regole ed evidenze sul patrimonio informativo. Inventario, catalogo, glossario, lineage e scheda dataset sono strumenti diversi. La qualità nasce da regole misurabili e da rimedi assegnati. Open data, trasparenza, accesso, condivisione e interoperabilità non sono sinonimi. Uno scambio corretto richiede finalità, semantica, responsabilità, sicurezza e tracciabilità.

Riferimenti normativi e professionali essenziali

- Codice dell'amministrazione digitale, decreto legislativo 7 marzo 2005, n. 82.
- AgID, Linee guida recanti regole tecniche per l'apertura dei dati e il riutilizzo dell'informazione del settore pubblico, versione 1.0.
- Direttiva (UE) 2019/1024 relativa all'apertura dei dati e al riutilizzo dell'informazione del settore pubblico.
- Regolamento di esecuzione (UE) 2023/138 sulle serie di dati di elevato valore.
- Regolamento (UE) 2024/903 sull'interoperabilità dei servizi pubblici digitali nell'Unione.
- AgID, Piano Triennale per l'informatica nella PA 2024-2026, Aggiornamento 2026.

Errori da evitare

Non usare dato aperto, dato online e dato condivisibile come sinonimi. Non assegnare la qualità a una funzione senza darle regole, poteri e tempi. Non confondere la fonte tecnica con la fonte autorevole del significato. Non progettare un'interoperabilità partendo dal payload: prima chiarisci finalità, semantica, responsabilità e limiti.

AI/ML nella PA: modelli, rischi e compliance

B
BANDO

A
AREE

N
NUCLEI

D
DIARIO

O
OUTPUT

Un sistema che assegna una priorità alle segnalazioni può aiutare un comune a mettere prima i casi urgenti. Può anche incorporare dati incompleti, spostare l'attenzione su zone già più segnalate o indurre chi lavora allo sportello a fidarsi di un punteggio senza comprenderne i limiti. Per questo la domanda d'esame non è soltanto «quanto è accurato il modello?», ma «quale decisione supporta, con quali dati, controlli, evidenze e responsabilità?».

Obiettivo e confini

Al termine del capitolo saprai distinguere algoritmo, sistema di IA, modello e output generativo; leggere dati, metriche, errori e rischi in funzione di un servizio pubblico; progettare un controllo umano reale; descrivere il ciclo di vita del sistema e orientarti nel quadro europeo e italiano senza trasformare una classificazione giuridica in uno slogan. Gli algoritmi generali restano nel capitolo 3; la qualità e il governo del dato nei capitoli 4 e 10; lifecycle software, sicurezza, logging e incidenti nei capitoli 6, 8 e 9; contratti e fornitori nel capitolo 12. Qui questi elementi sono ricomposti intorno al caso d'uso AI.

Mapa BANDO del sistema AI

- B — Bando: individua profilo, programma e formato della prova; ricava il caso d'uso richiesto.
- A — Aree: collega le materie tecniche alle responsabilità dei soggetti coinvolti.
- N — Nuclei: dati, errori, bias, sicurezza, diritti, drift e uso improprio.
- D — Diario: registra errori, motivazioni e correzioni; conserva i documenti utili a ricostruire il ragionamento.
- O — Output: previsione, raccomandazione o contenuto inseriti in un procedimento governato.

11.1 Algoritmo, sistema AI, modello e inferenza

Un'automazione deterministica applica regole esplicite: se input e regole non cambiano, il risultato resta uguale. Un algoritmo è una procedura finita che trasforma input in output. L'intelligenza artificiale non è invece il nome di un singolo programma: nel linguaggio tecnico e nel quadro europeo va letta come un insieme più ampio, composto dal modello quando presente, dai dati, dal software, dalle interfacce, dalle istruzioni, dalle persone e dal processo nel quale l'output viene impiegato.

Nel machine learning una parte delle regole viene appresa dai dati. Il modello riceve esempi, modifica parametri interni e tenta di ridurre l'errore rispetto a un obiettivo. Le feature sono le caratteristiche usate in ingresso; la label è il risultato atteso nei problemi supervisionati. Un parametro è appreso durante l'addestramento; un iperparametro è invece deciso nella configurazione del processo. Distinguere i due termini serve a evitare l'errore di trattare il modello come una scatola autonoma, priva di decisioni umane precedenti.

L'addestramento serve a stimare il modello usando esempi; l'inferenza applica il modello a nuovi casi. Una prestazione elevata sui dati già visti non dimostra che il sistema funzioni su situazioni nuove, né prova che sia utile nel procedimento amministrativo. Un servizio può classificare correttamente molte pratiche ma essere poco utile se il suo risultato arriva tardi, se gli operatori non possono contestarlo o se la soglia scelta produce troppe priorità improprie.

Un sistema predittivo stima una classe o un valore; un raccomandatore ordina opzioni; un sistema generativo produce testo, immagini, audio o altro contenuto. Quest'ultimo può formulare un testo plausibile e tuttavia errato: la plausibilità non rende l'output una fonte né trasferisce la responsabilità dell'atto all'applicazione. Nel colloquio è utile dire che l'output AI è un elemento istruttorio o operativo da verificare, non un soggetto che decide.

L'unità di analisi è sempre il caso d'uso: una stessa tecnologia può sostenere una bozza interna a basso impatto o incidere su una scelta che richiede garanzie molto maggiori. Finalità, destinatari, dati, modo d'uso e possibilità di intervento sono quindi più informativi dell'etichetta commerciale del prodotto. Errore tipico: chiamare "AI" qualunque automazione e, al contrario, pensare che l'uso di un modello elimini la necessità di definire regole, ruoli e controlli.

APPLICAZIONE AL PROFILO ICT PA

Quando un ufficio valuta un prodotto, la prima scheda non dovrebbe iniziare dal nome commerciale, ma da cinque domande: quale bisogno soddisfa, quale output produce, chi lo usa, quali dati riceve e quale scelta concreta cambia. Se la risposta è "nessuna", può essere un semplice strumento di ricerca o redazione; se cambia l'ordine delle pratiche, l'accesso a un servizio o l'intensità di un controllo, diventa un caso d'uso da descrivere molto meglio. Questo metodo impedisce due estremi: vietare ogni automazione perché contiene una componente AI, oppure adottarla come se la sua natura tecnica bastasse a renderla affidabile.

Un modello può anche essere aggiornato senza che l'ente se ne accorga, per esempio quando un fornitore sostituisce una versione o modifica un servizio collegato. Perciò la descrizione dell'architettura deve indicare dipendenze, versione e condizioni di impiego. Non serve trasformare il candidato in un programmatore del modello: serve renderlo capace di chiedere evidenze verificabili e di riconoscere che l'inferenza avviene dentro un processo con responsabilità, non in uno spazio neutro. La stessa prudenza vale per la generazione di testo: controllare il contenuto non significa solo correggere un refuso, ma verificare fatti, fonti, destinatario e coerenza con l'atto che si sta preparando.

In pratica, la scheda del caso deve anche indicare ciò che il sistema non fa: usi esclusi, decisioni riservate alla persona e condizioni che obbligano a fermare il flusso. Questa delimitazione protegge utenti e operatori perché trasforma una promessa tecnica vaga in un impiego controllabile. Un output non previsto non va semplicemente accettato o scartato: va ricondotto alla finalità, verificato e, se necessario, registrato come anomalia da riesaminare.

11.2 Paradigmi, partizioni e generalizzazione

Nell'apprendimento supervisionato gli esempi contengono input e risultato atteso. La classificazione assegna una categoria, per esempio “urgente/non urgente”; la regressione stima un valore numerico, come un tempo atteso di lavorazione. Nell'apprendimento non supervisionato non esiste una label-obiettivo e il metodo cerca strutture nei dati, come gruppi con caratteristiche simili. L'apprendimento per rinforzo usa ricompense e penalità per apprendere una strategia: è utile riconoscerne la logica, senza confonderlo con una comune classificazione.

La scelta del paradigma dipende dal problema e dai dati disponibili. Se un ente possiede esempi affidabili di esiti, il supervisionato può essere valutato; se possiede soltanto molte osservazioni non etichettate, un raggruppamento può servire a esplorare fenomeni, non a decidere automaticamente. La domanda corretta è: quale output deve sostenere quale fase del processo, e con quali conseguenze se sbaglia? Un modello non compensa una finalità vaga o etichette prodotte da prassi storiche incoerenti.

Per stimare la capacità di generalizzare, il dataset viene separato. Il blocco di training serve ad apprendere; quello di validation consente di confrontare configurazioni e soglie; il test set misura la prestazione finale su dati non usati nelle decisioni di sviluppo. Se il test è consultato ripetutamente per migliorare il sistema, perde la propria indipendenza e diventa di fatto un altro set di validazione. La separazione può essere temporale quando il servizio cambia nel tempo, oppure deve evitare che pratiche molto simili finiscano contemporaneamente in addestramento e test.

L'overfitting si verifica quando il modello aderisce eccessivamente ai dati di training e rende male su casi nuovi. L'underfitting indica un modello troppo semplice o una configurazione incapace di cogliere relazioni utili. Il data leakage introduce nella previsione informazioni che, al momento reale della decisione, non sarebbero disponibili: usare la data finale di chiusura per prevedere all'apertura un ritardo è un esempio classico. Il leakage può anche derivare da trasformazioni applicate prima della separazione dei dati.

Nella PA la generalizzazione non è soltanto una proprietà statistica. Deve riguardare il periodo, il territorio, le tipologie di domanda e le condizioni operative per le quali il sistema sarà davvero usato. Una procedura modificata, nuovi canali di accesso o una campagna informativa possono cambiare gli input. Errore tipico: dichiarare valido un modello perché supera una prova interna senza spiegare come sono stati separati dati, soglie e casi futuri. In sede d'esame, descrivere il processo di validazione vale più che elencare nomi di algoritmi.

ESEMPIO: ORDINARE NON SIGNIFICA DECIDERE

Immagina che il comune disponga di diecimila segnalazioni storiche. Il team può addestrare un classificatore, ma deve prima chiedersi se l'ordine scelto dagli operatori nel passato sia una label affidabile o il prodotto di carichi, consuetudini e risorse diseguali. Se il sistema propone solo una priorità, l'operatore conserva una fase di valutazione; se invece il punteggio chiude automaticamente una pratica, cambiano rischio, garanzie e controlli richiesti. La stessa differenza vale per un cluster: raggruppare segnalazioni simili può suggerire un'analisi, ma non prova che i gruppi abbiano un significato amministrativo o che sia lecito usarli per trattare le persone in modo diverso.

La verifica deve essere ripetibile. Si annotano data del campione, criteri di inclusione, regole di pulizia, partizioni, modello, metriche e soglia. Se un dato futuro è entrato per errore nel training, non basta correggere una cella: occorre rieseguire la valutazione e aggiornare l'evidenza. Questa disciplina è utile anche nella prova scritta: mostra che il candidato distingue una sperimentazione promettente da un servizio pronto per l'esercizio. L'obiettivo è una decisione proporzionata, non la massimizzazione astratta di una percentuale.

Una separazione corretta richiede inoltre che il team documenti la ragione della scelta: non esiste un unico schema valido per ogni servizio. Se le pratiche appartengono allo stesso evento o alla stessa persona, distribuirle casualmente tra i set può gonfiare artificialmente il risultato. Il controllo metodologico deve perciò essere comprensibile anche a chi governa il processo, non soltanto a chi sviluppa il modello.

DUE ALGORITMI SVOLTI SU PICCOLI DATI

Un albero decisionale divide i dati mediante condizioni e associa una previsione alle foglie. Considera quattro ticket didattici: giorni di attesa 1, 2, 8, 9; etichette storiche «da riesaminare» 0, 0, 1, 1. Una divisione a 5 separa perfettamente i due gruppi. L'impurità di Gini iniziale è $1 - (2/4)^2 - (2/4)^2 = 0,5$; nelle due foglie è zero. Il piccolo albero restituisce 1 per un ticket con attesa 6. La soglia è appresa confrontando separazioni sui dati etichettati: non coincide con una regola normativa scritta da un programmatore.

Quattro osservazioni non provano generalizzazione; un albero molto profondo può memorizzare il training. Si limitano complessità e profondità e si misura su dati separati. L'esempio non autorizza a decidere diritti o priorità amministrative in base alla sola attesa.

Nel k-means, invece, mancano etichette da predire. Su dati monodimensionali 1, 2, 8, 9, scegli $k=2$ e centroidi iniziali 1 e 9. Assegna ciascun punto al centro più vicino: gruppi $\{1,2\}$ e $\{8,9\}$. Ricalcola le medie: 1,5 e 8,5. La successiva assegnazione non cambia; la somma dei quadrati delle distanze è quattro volte $0,5^2$, cioè 1. Il metodo alterna assegnazione e aggiornamento per ridurre tale obiettivo, ma l'esito dipende dall'inizializzazione e non garantisce il minimo globale.

Scale diverse delle variabili richiedono attenzione e spesso standardizzazione; un gruppo numerico non è una categoria giuridica o una classe vera. Il numero k e l'utilità dei gruppi vanno motivati rispetto allo scopo.

11.3 Dati, provenienza, etichette e rappresentatività

Il dato per l'AI non è una materia prima neutra. Ogni dataset ha una provenienza, un periodo, una finalità, una popolazione osservata e una serie di trasformazioni. Una scheda minima deve indicare chi ha prodotto i dati, che cosa rappresentano, come sono state create le etichette, quali campi mancano, quali categorie sono poco rappresentate e quali limitazioni impediscono determinati usi. Questa documentazione rende discutibile e migliorabile il sistema; non è un adempimento decorativo.

Le etichette possono riprodurre errori passati. Se “urgenza” coincide con la priorità storicamente attribuita da un ufficio, il modello può imparare abitudini organizzative e non l'urgenza reale. La qualità va quindi letta rispetto allo scopo: accuratezza, completezza, tempestività, coerenza, validità e rappresentatività possono avere pesi diversi. L'assenza di valori mancanti non dimostra che il dataset sia adeguato, soprattutto se i casi rari o i cittadini che usano canali diversi sono quasi assenti.

Lo sbilanciamento delle classi è frequente: se solo pochi casi sono realmente urgenti, un modello che li ignora può ottenere comunque un'accuracy elevata. Anche una variabile apparentemente neutra può funzionare da proxy di caratteristiche sensibili o di disuguaglianze territoriali. Eliminare il campo più evidente non elimina automaticamente il rischio di disparità: occorre capire quali correlazioni siano rilevanti, testare gli effetti e valutare le conseguenze nel processo.

La raccolta deve essere proporzionata alla finalità e accompagnata da accessi, classificazione, conservazione e sicurezza adeguati. Per inventario, lineage, qualità trasversale e ruoli sul dato si applicano le logiche del capitolo 10; qui conta l'effetto specifico del dato sull'addestramento e sulla decisione. Dati personali, profilazione e decisioni automatizzate richiedono una valutazione nel quadro applicabile e non possono essere risolti con una formula tecnica generica.

Un buon esercizio consiste nel confrontare due campioni: uno ricco di segnalazioni digitali e uno che include anche sportello e telefono. Il primo potrebbe rappresentare meglio chi usa un canale, non l'intera popolazione. La correzione può richiedere nuovi dati, un diverso uso dell'output, un controllo umano più forte o perfino la rinuncia al modello. Errore tipico: scambiare il dataset disponibile per il dataset necessario. La disponibilità tecnica non autorizza da sola una raccolta, un riuso o una conclusione automatizzata.

APPLICAZIONE: UNA SCHEDA DATI PRIMA DEL MODELLO

Per ciascun campo rilevante conviene indicare definizione, fonte, frequenza di aggiornamento, trasformazioni, qualità conosciuta e rischio d'uso. La data di apertura, per esempio, può essere un dato legittimo e utile; un campo raccolto dopo la decisione non lo è per una previsione fatta prima della decisione. Questa semplice tabella aiuta a scoprire leakage, campi duplicati e variabili che sembrano tecniche ma riflettono modalità di accesso al servizio. Aiuta inoltre a distinguere un dato incompleto da un dato deliberatamente non raccolto perché non necessario alla finalità.

L'ente dovrebbe coinvolgere competenze di processo, dati, sicurezza e protezione dei dati prima di trasformare uno storico in addestramento. Non tutti i difetti richiedono la stessa risposta: un errore di battitura può essere corretto; un campione che esclude sistematicamente un canale richiede cautela più profonda; una label priva di significato condiviso può rendere improprio l'intero progetto. Il candidato può esporre questa logica con una formula chiara: “documento il dato, ne verifico l'adeguatezza all'uso, misuro l'effetto dell'errore e definisco chi può correggerlo”.

Le evidenze sui dati devono restare aggiornabili. Quando si aggiunge un nuovo canale, cambia la definizione di una variabile o si corregge un archivio, occorre sapere quali versioni del modello sono state influenzate. Questa catena permette di rispondere a una contestazione con elementi verificabili, anziché con una generica affermazione sulla qualità del sistema. La qualità è quindi una proprietà della relazione tra dato, finalità e contesto; va riesaminata quando cambia uno di questi elementi, non solo quando un controllo automatico segnala un valore anomalo.

11.4 Baseline, metriche e conseguenze degli errori

Prima di misurare un modello occorre fissare una baseline: può essere una regola semplice, il procedimento attuale o una prestazione minima. Senza confronto, un valore percentuale isolato non dice se il sistema migliora davvero il servizio. La baseline obbliga a dichiarare che cosa si vuole superare: tempi, coerenza, capacità di individuare i casi urgenti, carico degli operatori o qualità della motivazione.

In una classificazione binaria la matrice di confusione distingue vero positivo, falso positivo, falso negativo e vero negativo. Se il modello tratta come non urgente una segnalazione effettivamente urgente, produce un falso negativo; se porta in cima alla coda una pratica ordinaria, produce un falso positivo. Queste non sono soltanto caselle: rappresentano possibili ritardi, risorse assorbite o controlli aggiuntivi. La valutazione deve chiedere chi sostiene il costo dell'errore e quale misura può ridurlo.

L'accuracy indica la quota complessiva di classificazioni corrette. La precision dice quanti casi segnalati come positivi lo sono davvero; il recall dice quanti casi realmente positivi vengono intercettati; l'F1 sintetizza precision e recall. Non esiste una metrica migliore in assoluto. Se perdere un'urgenza è molto grave, il recall può diventare decisivo, accettando che aumentino falsi positivi e attività di verifica. La soglia non è quindi un dato puramente matematico: è una scelta di servizio da motivare e riesaminare.

La valutazione non finisce con la media. Occorre osservare periodi, territori, gruppi pertinenti, casi limite e condizioni di uso. La calibrazione verifica se le probabilità stimate corrispondono in modo ragionevole alle frequenze osservate; la robustezza riguarda la tenuta a variazioni degli input e del contesto. Un modello può essere accurato e mal calibrato, o buono in media ma debole proprio nella situazione più delicata.

Per una prova concorsuale, una risposta solida collega metrica, errore e decisione: «Nel caso delle segnalazioni urgenti misuro recall e precision, confronto la baseline manuale, analizzo sottogruppi e scelgo una soglia con operatori capaci di verificare i casi». Errore tipico: presentare l'accuracy come certificato di qualità, equità e conformità. Le metriche descrivono aspetti diversi; nessuna sostituisce l'analisi del servizio e dei suoi destinatari.

LETTURA RAGIONATA DELLE METRICHE

Supponi che cento segnalazioni siano urgenti e il modello ne riconosca novanta: il recall è 90%. Per capire se questa è una buona prestazione servono però anche quanti casi ordinari siano stati dichiarati urgenti, quale era la baseline e quante verifiche aggiuntive l'ufficio può assorbire. Una soglia più bassa può aumentare il recall ma generare una lista troppo lunga; una soglia più alta può ridurre i falsi positivi ma lasciare fuori urgenze reali. La scelta non è un difetto del modello: è una responsabilità organizzativa che deve essere visibile e riesaminabile.

Anche una metrica ben scelta deve essere calcolata su dati coerenti con l'uso futuro. Se il test riguarda un inverno con molte emergenze e il rilascio avviene in un periodo ordinario, la prestazione può essere difficile da interpretare. Per questo report e confronto con la baseline devono indicare condizioni, limiti e intervalli di osservazione. Un buon candidato non promette precisione assoluta: propone un rilascio controllato, un monitoraggio iniziale e criteri per sospendere la raccomandazione se gli errori superano la capacità di controllo umano.

È utile distinguere la metrica scelta per il rilascio da quella usata per sorvegliare il servizio. Una prestazione accettabile al test può deteriorarsi nel tempo; per questo si definiscono frequenza di controllo, soglia di attenzione, responsabile della lettura e azione conseguente. Senza queste decisioni il numero resta una fotografia, non uno strumento di governo. La scelta deve essere annotata con il motivo, il responsabile e il criterio di revisione: così l'ente può verificare se il compromesso tra urgenze intercettate e carico operativo sia ancora proporzionato.

MATRICE DI CONFUSIONE E CALCOLO COMPLETO

Su 1.000 segnalazioni, la verifica umana ne classifica 100 come urgenti. Il modello segnala 120 urgenze, 90 delle quali corrette.

Previsione	Urgente reale	Non urgente reale
Urgente	TP = 90	FP = 30
Non urgente	FN = 10	TN = 870

La precision è $TP/(TP+FP)=90/120=75\%$: tra gli allarmi, quanti sono corretti. Il recall è $TP/(TP+FN)=90/100=90\%$: quante urgenze intercetto. $F1=2TP/(2TP+FP+FN)=180/220\approx81,82\%$; $accuracy=(TP+TN)/1.000=96\%$. Un modello che risponde sempre «non urgente» raggiunge il 90% di accuracy ma recall zero sulle urgenze. Abbassare una soglia può aumentare il recall al costo di più falsi positivi: si sceglie su validation considerando carico di revisione e danno delle omissioni, poi si misura sul test senza riaggiustarla. Precision e recall dipendono dalla classe positiva dichiarata; denominatori nulli vanno gestiti esplicitamente.

11.5 Bias, fairness, spiegabilità e controllo umano

Il bias può nascere nel campionamento, nelle etichette, nelle feature, nell'obiettivo ottimizzato o nel modo in cui il risultato entra nel lavoro quotidiano. Una disparità osservata non prova da sola una discriminazione illecita, ma è un segnale che impone analisi tecnica, organizzativa e giuridica. Il punto di partenza non è scegliere una formula di fairness, ma capire quale effetto il servizio può produrre su persone e gruppi, con quale meccanismo e quali possibilità di correzione.

Le metriche di fairness sono strumenti contestuali. Alcune misurano differenze tra gruppi, altre parità di errore o di trattamento; possono entrare in tensione tra loro. Non esiste un numero universale che certifichi l'equità. Un candidato deve quindi spiegare che una metrica è utile per individuare un problema e confrontare alternative, non per sostituire valutazione del caso, regole di servizio e tutela delle persone.

L'interpretabilità globale aiuta a capire il comportamento generale del modello; una spiegazione locale riguarda un singolo output. La trasparenza comprende anche finalità, dati usati, limiti, ruolo del sistema e possibilità di riesame. Una spiegazione post-hoc può essere utile per l'operatore, ma non dimostra che il risultato sia corretto né rende automaticamente motivata una decisione amministrativa. Errore tipico: confondere una frase convincente prodotta da un sistema con una spiegazione verificata.

Il controllo umano deve essere sostanziale. Human-in-the-loop, human-on-the-loop e human-in-command descrivono modelli di presenza umana, ma da soli non garantiscono nulla. La persona deve comprendere scopo e limiti, ricevere informazioni utili, disporre di tempo e competenza, poter discostarsi dall'output, sospendere il flusso o attivare un'escalation. Deve inoltre essere possibile tracciare quando l'output è stato seguito o corretto e perché.

L'automation bias spinge a privilegiare la raccomandazione automatica anche quando vi sono segnali contrari. Un pulsante “approva” non crea supervisione se chi lo usa non ha dati, autorità o tempo per dissentire. Nel caso delle segnalazioni, l'operatore deve poter cambiare priorità, vedere gli elementi rilevanti e segnalare pattern anomali. Il controllo umano collega qualità tecnica e responsabilità amministrativa: non è un passaggio formale messo alla fine del processo.

DAL PUNTEGGIO ALLA DECISIONE MOTIVATA

L'equità non coincide con trattare ogni caso come identico. Un servizio può dover riconoscere differenze reali, ma deve dimostrare che siano pertinenti alla finalità e che non producano un danno ingiustificato. Per questo una verifica sui sottogruppi non è una sentenza automatica: è un modo per scoprire dove chiedere spiegazioni, controllare dati e modificare il processo. La trasparenza rivolta al cittadino e la spiegazione disponibile all'operatore possono avere forma e dettaglio diversi; entrambe devono essere comprensibili e proporzionate al ruolo.

Un override ben progettato richiede un motivo selezionabile, uno spazio breve per la motivazione e una revisione periodica dei casi in cui l'operatore si discosta dal sistema. Se tutti confermano il modello, l'ente deve chiedersi se l'output sia davvero affidabile o se il flusso impedisca di dissentire. Se tutti lo correggono, può essere il modello a non essere adatto. Il log degli override non serve a sorvegliare chi lavora, ma a rendere osservabile il rapporto tra raccomandazione, competenza umana e risultato del servizio.

Il riesame non deve attendere un danno conclamato. Reclami ripetuti, differenze inattese negli override, difficoltà di comprensione o cambiamenti negli utenti sono segnali da analizzare. Il controllo umano diventa così una fonte di apprendimento organizzativo: rende visibili limiti del modello e limiti della procedura, senza trasformare l'operatore in un semplice esecutore della raccomandazione. Anche la formazione degli operatori è un controllo: chi non sa leggere limiti e incertezze non può svolgere una supervisione sostanziale, per quanto il flusso preveda formalmente un intervento umano. Una supervisione reale rende quindi discutibile ogni output e conserva una via praticabile per correggere l'errore prima che produca effetti non rimediabili.

11.6 Lifecycle, MLOps, rischio e governance

Un sistema AI va governato lungo il suo ciclo di vita: definizione del problema, raccolta e preparazione dei dati, sviluppo, validazione, rilascio, uso, monitoraggio, modifica e dismissione. MLOps porta in questo ciclo pratiche di ingegneria e operations: versionamento di dati, codice, configurazioni e modello; ambienti controllati; approvazioni; test ripetibili; osservazione del comportamento in esercizio. Il risultato deve essere ricostruibile: non basta sapere che “il modello è stato aggiornato”.

Il monitoraggio comprende qualità degli input, prestazioni, errori, sottogruppi, override umani, reclami, incidenti e usi fuori perimetro. Il data drift è un cambiamento nella distribuzione degli input; il concept drift riguarda un mutamento della relazione tra input e risultato. Una nuova procedura amministrativa può modificare entrambi. Anche senza drift, l'efficacia può peggiorare perché cambia l'organizzazione o perché il sistema viene usato per una finalità diversa da quella testata.

Quando emerge un problema, le opzioni non sono soltanto “tenere” o “spegnere” il sistema. Si possono correggere dati e regole, rieseguire i test, cambiare soglia, riaddestrare, limitare l'uso, passare a controllo manuale, eseguire un rollback o dismettere. La scelta richiede evidenze e un responsabile; il rollback non è una formula magica, perché la versione precedente deve essere ancora nota, sicura e adatta al contesto.

La governance parte da un inventario dei casi d'uso, anche quando il modello è acquistato. Per ogni caso occorrono finalità, dati, versione, destinatari, owner, operatori, fornitore, rischi, controlli e evidenze. La valutazione considera effetti su persone, diritti, servizio, finanze, sicurezza e reputazione. Per ogni scenario si descrivono causa, evento, conseguenza, probabilità, gravità, controlli e rischio residuo. Il rischio tecnico e la classificazione giuridica sono collegati ma non coincidono.

Un contratto non trasferisce automaticamente tutte le responsabilità dell'ente. Versioni, prestazioni, dati, audit, subfornitori, incidenti, restituzione delle evidenze, portabilità e uscita devono essere governati insieme a chi acquista e a chi esercisce il servizio. Il capitolo 12 sviluppa gli strumenti di procurement; qui l'errore da evitare è acquistare un sistema come se fosse un risultato già validato. Errore tipico: trattare monitoraggio e revisione come attività successive al rilascio, quando devono essere progettati fin dall'inizio.

GRIGLIA OPERATIVA DI RISCHIO

Una griglia sintetica può riportare, per ogni rischio, causa, evento, conseguenza, evidenza, controllo e responsabile. Per esempio: etichette storiche incoerenti possono produrre una priorità distorta; il controllo è la revisione delle etichette e il test per gruppi pertinenti; il responsabile è il titolare del caso insieme a chi cura i dati. Questa scrittura costringe a collegare una preoccupazione generica a una scelta osservabile. Lo stesso vale per la dipendenza dal fornitore: non basta annotare “vendor risk”, ma occorre indicare versione, accesso alle evidenze, gestione delle modifiche e condizioni di uscita.

Governare significa anche decidere quando non usare l'AI. Se il bisogno può essere risolto con una regola trasparente o se non esistono dati adeguati, una soluzione più semplice può essere preferibile. La proporzionalità riguarda costi, impatti, competenze e reversibilità. Nel tempo, un sistema inizialmente adeguato può non esserlo più: una nuova fonte dati, una modifica contrattuale o un reclamo ricorrente sono segnali che attivano riesame. La documentazione di queste decisioni rende possibile spiegare perché il sistema è stato limitato, corretto o sospeso.

La governance efficace separa chi propone una modifica, chi la valuta e chi autorizza il rilascio, secondo dimensioni e rischi del servizio. Non occorre creare una burocrazia uniforme: occorre rendere chiaro chi decide, su quale evidenza e con quale possibilità di revisione. Anche un piccolo pilota deve avere criteri di successo e di uscita, altrimenti la sperimentazione tende a diventare esercizio permanente senza controllo. La revisione periodica deve includere anche i destinatari del servizio e chi raccoglie reclami, perché le criticità d'uso possono emergere fuori dai soli indicatori tecnici.

LABORATORIO COMPILABILE: GRIGLIA DI RISCHIO DEL CASO AI

Usa questa scheda per il caso delle segnalazioni o per un altro sistema AI descritto nel bando. Compila una riga per ogni rischio rilevante; se non riesci a indicare evidenza, controllo e responsabile, il rischio non è ancora governato. La griglia non assegna automaticamente una classe giuridica: serve a rendere verificabili le decisioni tecniche e organizzative.

Campo	Compilazione
Scenario e causa	
Evento temuto	
Conseguenza per persone o servizio	
Evidenza da raccogliere	
Controllo o misura di trattamento	
Responsabile del controllo	
Rischio residuo e criterio di riesame	

Esempio di lettura: etichette storiche incoerenti possono portare a una priorità distorta; la conseguenza è il ritardo di casi urgenti; l'evidenza comprende il campione etichettato e i risultati per gruppi pertinenti; il controllo combina riesame delle etichette, test e override; il responsabile coordina processo e dati; il rischio residuo va riesaminato quando cambiano dati, soglia o procedura.

11.7 Quadro UE e italiano: ruoli, rischio e confini

Il regolamento (UE) 2024/1689, AI Act, distingue pratiche vietate, sistemi ad alto rischio, obblighi specifici di trasparenza e modelli di IA per finalità generali. Il rischio dipende da uso previsto, contesto e ruolo; non basta il nome commerciale del prodotto. Il provider sviluppa o fa sviluppare un sistema e lo immette sul mercato o mette in servizio sotto il proprio nome; il deployer lo usa sotto la propria autorità nell'attività professionale. Una PA può essere deployer acquistando un servizio, ma anche assumere responsabilità da provider nei casi previsti, per esempio cambiandone la destinazione in modo da renderlo ad alto rischio. Il contratto deve rendere leggibili ruoli, documenti, assistenza e modifiche, senza trasferire artificialmente obblighi fissati dalla legge.

CLASSIFICARE UN CASO DELLA PA

1. Verifica prima le pratiche vietate dell'articolo 5, tra cui forme di manipolazione dannosa e social scoring alle condizioni previste. Non si compensa un divieto con una firma umana finale.
2. Esamina l'articolo 6: prodotti/componenti di sicurezza dell'allegato I soggetti alla valutazione prevista e usi dell'allegato III, tra cui selezione del personale, accesso a determinati servizi essenziali, istruzione e giustizia.
3. L'articolo 6(3) consente, alle condizioni indicate, di non classificare come alto rischio alcuni usi dell'allegato III che non presentano un rischio significativo, per esempio compiti preparatori o procedurali ristretti. Non è un'esenzione generica per qualunque assistente. Un sistema dell'allegato III che effettua profilazione di persone rimane ad alto rischio; la valutazione del provider deve essere documentata.
4. Considera gli obblighi di trasparenza dell'articolo 50 e gli altri regimi applicabili. Un chatbot informativo non diventa automaticamente ad alto rischio, ma l'interazione con AI deve essere resa riconoscibile nei casi previsti; l'uso per selezionare candidati cambia la valutazione.

Per un sistema ad alto rischio, il deployer segue le istruzioni, assegna sorveglianza a persone competenti e dotate di autorità, controlla i dati di input sotto il proprio controllo, monitora e conserva i log automaticamente generati sotto il proprio controllo per il periodo pertinente, almeno sei mesi salvo diversa disciplina applicabile. Gestisce rischi, incidenti e sospensione secondo l'articolo 26. La PA verifica inoltre registrazione e valutazione d'impatto sui diritti fondamentali dell'articolo 27, quando richiede: quest'ultima ha un ambito proprio e si coordina con la DPIA privacy, senza sostituirla automaticamente. La semplice presenza di un operatore non basta: deve poter capire i limiti, contestare il risultato e fermare o modificare l'uso.

CALENDARIO AGGIORNATO AL 3 OTTOBRE 2026

Il regolamento (UE) 2026/1744, in vigore dal 27 luglio 2026, ha modificato il calendario. La tabella distingue regole generali e transitori: il rinvio di una categoria non sospende ogni obbligo AI.

Disposizione o categoria	Applicazione pertinente
Capi I e II, inclusi alfabetizzazione e pratiche vietate	Dal 2 febbraio 2025, con le modifiche successive
Governance, modelli per finalità generali e altre parti anticipate dall'articolo 113	Dal 2 agosto 2025 secondo il rispettivo ambito
Regola generale, incluse disposizioni di trasparenza non differite	Dal 2 agosto 2026
Alto rischio, allegato III: capo III sezioni 1–3, salvo articolo 6(5)	Dal 2 dicembre 2027
Articolo 6(1), prodotti dell'allegato I e obblighi collegati	Dal 2 agosto 2028

Per i sistemi generativi già immessi prima del 2 agosto 2026, il nuovo articolo 111(4) differisce al 2 dicembre 2026 il solo obbligo di marcatura dell'articolo 50(2): non rinvia indistintamente tutta la trasparenza. L'articolo 111(2) disciplina separatamente sistemi ad alto rischio già immessi/in servizio prima delle rispettive date, le modifiche significative successive e il termine del 2 agosto 2030 per provider e deployer di sistemi destinati a essere usati dalle autorità pubbliche. Quest'ultimo non è una proroga generale per ogni nuovo sistema acquistato dalla PA.

Il nuovo articolo 4 richiede misure che sostengano lo sviluppo dell'alfabetizzazione AI, considerando conoscenze, esperienza, formazione e contesto; non impone di garantire un identico livello individuale per tutti. Per l'esame collega quindi formazione, istruzioni e limiti agli usi effettivi del personale.

LA REGOLA ITALIANA PER LA PA E UN ESEMPIO

L'articolo 14 della legge 132/2025 richiede conoscibilità del funzionamento e tracciabilità dell'impiego. L'AI supporta l'attività provvedimentale: autonomia e responsabilità della persona che decide restano ferme. Le amministrazioni adottano misure tecniche, organizzative e formative per un utilizzo responsabile. La legge è distinta dalle strategie e dalle bozze di linee guida; un documento in consultazione non si presenta come obbligo già definitivo.

Caso. Un ufficio sperimenta un sistema che ordina curriculum per convocare candidati. Non lo classifica come semplice chatbot perché genera spiegazioni testuali: la finalità è selezione del personale, quindi verifica l'allegato III, il ruolo dell'ente e il calendario pertinente. Chiede documentazione, dati e limiti delle prestazioni, individua responsabili della sorveglianza, registra motivi delle decisioni e valuta impatti e disciplina privacy. Una firma che conferma sempre il punteggio non è un controllo effettivo. Se invece il sistema prepara solo bozze informative sui bandi, si rivaluta il diverso uso senza importare automaticamente la medesima classe.

Il NIST AI RMF, articolato in Govern, Map, Measure e Manage, offre un riferimento tecnico volontario per il ciclo di vita; non è una norma europea obbligatoria. Prima di una nuova decisione operativa, il calendario e gli atti attuativi vanno controllati sulla fonte ufficiale, conservando versione e data del riscontro.

Verifica

QUIZ 1 — QUAL È LA DIFFERENZA PIÙ UTILE TRA SISTEMA AI E MODELLO?

Risposta corretta: il modello è una componente che elabora input; il sistema comprende anche dati, software, persone, istruzioni e processo d'uso. La distinzione evita di attribuire al solo modello decisioni che dipendono dall'organizzazione.

QUIZ 2 — PERCHÉ IL TEST SET NON VA USATO RIPETUTAMENTE PER REGOLARE IL MODELLO?

Risposta corretta: perché perderebbe indipendenza e diventerebbe un set di validation. Il test deve stimare la prestazione su dati non usati nelle scelte di sviluppo.

QUIZ 3 — UN'ACCURACY ELEVATA PROVA CHE IL SISTEMA INTERCETTA BENE LE URGENZE?

Risposta corretta: no. Con classi sbilanciate può restare elevata anche se molti casi urgenti non sono riconosciuti. Occorre osservare almeno recall, precision e costo dei falsi negativi.

QUIZ 4 — ELIMINARE UN CAMPO SENSIBILE ELIMINA OGNI RISCHIO DI BIAS?

Risposta corretta: no. Altre variabili possono agire da proxy; inoltre il problema può dipendere da etichette, campionamento o uso dell'output.

QUIZ 5 — QUANDO IL CONTROLLO UMANO È EFFETTIVO?

Risposta corretta: quando la persona comprende limiti e finalità, ha informazioni, tempo, competenza e potere di override o escalation, e la decisione è tracciabile.

QUIZ 6 — IL NIST AI RMF È UNA NORMA EUROPEA OBBLIGATORIA?

Risposta corretta: no. È un riferimento tecnico volontario utile per organizzare il rischio; non sostituisce il regolamento europeo o le verifiche sul caso concreto.

CASO RAGIONATO — PRIORITÀ ALLE SEGNALAZIONI DI MANUTENZIONE

Un comune vuole ordinare le segnalazioni su strade ed edifici. La baseline è la coda manuale; i dati includono storico, esiti e tempi. Prima di adottare il sistema, il gruppo definisce “urgenza”, controlla la provenienza delle etichette, separa dati di training, validation e test e osserva recall degli urgenti, precision e risultati per territorio. Gli operatori vedono il punteggio come supporto, possono modificarlo e motivano gli override. Ogni versione di dati e modello, ogni soglia e ogni anomalia sono tracciati.

La decisione ragionata non consiste nel dichiarare subito una classe AI Act. Consiste nel documentare finalità, persone interessate, errori possibili, controlli, ruoli e quadro applicabile; se le prestazioni peggiorano o cambia il processo, il comune rivalida, limita l'uso o torna alla procedura manuale. La griglia di rischio deve collegare dati, modello, impatto, controllo umano, evidenze e responsabile.

MICROCASO — BOZZA GENERATIVA PER IL CITTADINO

Un ufficio usa un modello generativo per preparare una bozza. L'operatore non inserisce dati non autorizzati, verifica fatti e fonti, corregge il testo e approva l'invio secondo il processo previsto. L'output plausibile non viene trattato come atto o fonte: la responsabilità resta nella validazione umana, nelle istruzioni di uso, negli accessi e nella tracciabilità.

Domanda da commissario

Come governeresti un modello usato per dare priorità alle segnalazioni comunali?

Partirei da finalità e baseline, documenterei dati ed etichette, separerei training, validation e test, sceglierei metriche in base al costo degli errori e verificherei sottogruppi. Progetterei controllo umano, versioni, logging e monitoraggio; valuterei rischi, ruoli, fornitore e quadro applicabile sul caso concreto. Prevederei infine condizioni di riesame, rollback e sospensione.

Domande-trappola

- Accuracy elevata significa equità? No: può nascondere errori concentrati.
- Una spiegazione convincente prova che l'output sia corretto? No.
- Il controllo umano coincide con un clic di approvazione? No.
- Il fornitore assorbe tutte le responsabilità? No: ruoli e contratto vanno verificati.

Mini-esercizio

Compila per un caso AI cinque campi: finalità, baseline, dati e loro limite, errore più grave, potere di override. Se non riesci a indicare un'evidenza o un responsabile per ciascun campo, il sistema non è ancora pronto per l'uso.

Da sapere in 5 righe

Il modello è solo una parte del sistema AI. Dati, metriche e soglie vanno letti in relazione agli effetti sugli utenti. Accuratezza, fairness, spiegabilità e conformità non sono sinonimi. Il controllo umano richiede competenza e autorità. Rischio, versioni, monitoraggio e quadro applicabile accompagnano l'intero ciclo di vita.

Riferimenti essenziali

Regolamento (UE) 2024/1689; legge 23 settembre 2025, n. 132; materiali istituzionali della Commissione europea e di AgID sull'intelligenza artificiale; NIST AI Risk Management Framework. Per atti modificativi, linee guida definitive e calendario applicabile verificare sempre il testo ufficiale vigente.

Regolamento (UE) 2026/1744, modifiche all'AI Act e calendario; legge 132/2025, articolo 14. Pat Morin e la documentazione scikit-learn sugli alberi e sul k-means per i metodi, distinti dagli esempi numerici originali.

Procurement ICT e gestione dei fornitori

B
BANDO**A**
AREE**N**
NUCLEI**D**
DIARIO**O**
OUTPUT

Anche un servizio aggiudicato correttamente può fallire durante l'esecuzione. Accade se il bisogno resta vago, i requisiti non sono verificabili o gli SLA misurano ciò che è facile contare invece di ciò che serve agli utenti. In altri casi, l'ente scopre troppo tardi di non poter recuperare dati e configurazioni.

Nel procurement ICT la competenza tecnica non sostituisce quella amministrativa. Entrambe concorrono allo stesso risultato: tradurre il fabbisogno in una prestazione, precisarla con requisiti e verificarla attraverso evidenze utili al governo del contratto.

Obiettivo e confini

Al termine del capitolo il candidato saprà:

- tradurre un fabbisogno in requisiti funzionali e non funzionali;
- distinguere requisito, criterio di accettazione, criterio di valutazione, obbligo e SLA;
- costruire un livello di servizio misurabile;
- collegare ruoli tecnici e contrattuali durante l'esecuzione;
- inserire sicurezza, dati, continuità e supply chain nel governo del fornitore;
- riconoscere lock-in e progettare portabilità, reversibilità ed uscita;
- orientarsi tra ciclo digitale e strumenti di acquisto senza confonderli.

La disciplina generale dei contratti pubblici va studiata insieme alle nozioni comuni sul ciclo contrattuale. Per procedure, soglie e appalti avanzati il lettore può approfondire in VOL-09/M-TR02 — Appalti, PNRR e fondi UE; questo rinvio non è necessario per comprendere il governo tecnico-organizzativo dell'acquisto ICT qui trattato.

Mappa BANDO dell'acquisto ICT

- B — Bando: individua profilo, programma e formato della prova; ricava il caso d'uso richiesto.
- A — Aree: collega le materie tecniche alle responsabilità dei soggetti coinvolti.
- N — Nuclei: requisiti vaghi, misure ambigue, dipendenze, incidenti, modifiche, subfornitori e lock-in.
- D — Diario: registra errori, motivazioni e correzioni; conserva i documenti utili a ricostruire il ragionamento.
- O — Output: servizio accettato e controllato, con evidenze, continuità e possibilità reale di uscita.

12.1 Fabbisogno, baseline e strategia di acquisizione

RISULTATO, PERIMETRO E BASELINE

Il fabbisogno parte dal problema pubblico da risolvere, non dal prodotto desiderato. «Acquistare una piattaforma cloud» indica già una soluzione; «garantire ai cittadini l'accesso continuativo al servizio, riducendo tempi di indisponibilità e oneri di gestione» esplicita invece il risultato.

La baseline fotografa la situazione iniziale: utenti, volumi, tempi, costi, incidenti, tecnologie, competenze e vincoli. Senza baseline non si può dimostrare se l'acquisto abbia migliorato il servizio.

Il perimetro chiarisce ciò che il fornitore deve consegnare e ciò che resta all'ente. Include ambienti, integrazioni, dati, migrazione, assistenza, manutenzione, formazione, documentazione e uscita. Se il confine resta incerto, alcune attività non trovano un responsabile e altri costi emergono soltanto durante l'esecuzione.

MAKE, BUY E REUSE

L'ente può sviluppare internamente, acquistare, riusare soluzioni disponibili o combinare le opzioni. La scelta considera competenze, tempi, criticità, mercato, interoperabilità, manutenzione, sicurezza e sostenibilità nel ciclo di vita.

Anche dopo l'acquisto, l'ente deve conservare competenze interne. Se nessuno nell'ente sa verificare requisiti, dati, configurazioni e livelli di servizio, il fornitore finisce per controllare anche il significato delle misure con cui viene valutato.

COSTO E DIPENDENZE

Il prezzo iniziale è solo una parte del costo. Contano migrazione, integrazioni, licenze, gestione, formazione, crescita dei volumi, assistenza, modifiche, transizione e dismissione. Il costo totale di possesso è una lente decisionale; il calcolo concreto dipende dagli atti e dai dati disponibili.

Il risparmio iniziale può svanire se la soluzione richiede formati proprietari, competenze rare o costi elevati per esportare dati e configurazioni. Per questo il rischio di uscita si valuta prima dell'affidamento.

APPLICAZIONE: DECIDERE PRIMA DI ACQUISTARE

Una baseline non è un elenco di apparati. Deve spiegare quale servizio esiste oggi, chi lo usa, quali tempi di attesa o indisponibilità produce, quali integrazioni lo sostengono e quali vincoli non possono essere rimossi. Per un portale comunale, ad esempio, non basta sapere che è ospitato in cloud: servono i flussi con identità digitale, pagamenti, protocollo, notifiche e archivi. Servono anche le persone che gestiscono il servizio e il momento in cui un fermo diventa un danno concreto per utenti o uffici.

La scelta tra sviluppo, acquisto, riuso o combinazione non è una gara tra tecnologie. Lo sviluppo interno può aumentare controllo e conoscenza, ma chiede capacità di manutenzione. Un prodotto o servizio già disponibile può ridurre il tempo iniziale, ma può imporre interfacce, licenze o modalità operative difficili da cambiare. Il riuso è utile se risolve il bisogno e se l'ente può governarne evoluzione, sicurezza e integrazione. La scelta va quindi motivata con obiettivo, vincoli, competenze, tempi, costo di gestione e possibilità di uscita, non con la popolarità di un fornitore.

Un buon perimetro separa ciò che deve essere consegnato da ciò che rimane in capo all'ente: migrazione, ambienti, dati, integrazioni, formazione, documentazione, assistenza e dismissione. Questa separazione evita il costo nascosto delle attività “incluse a voce”. In prova, una risposta solida dichiara le assunzioni: se il servizio gestisce pratiche dei cittadini, continuità, accessi, tracciabilità e recupero assumono un peso diverso rispetto a un sito informativo. Errore tipico: partire dalla piattaforma desiderata e cercare dopo un bisogno che la giustifichi. Il percorso corretto è inverso: risultato pubblico, evidenze iniziali, requisiti e soltanto allora scelta della soluzione.

Conserva motivazione e alternative scartate: chi subentra deve poter riesaminare la decisione se cambiano fabbisogno o costi.

CONTROLLO DELLA SCELTA E STRUMENTI DI ACQUISTO

La strategia non termina con l'individuazione della soluzione. Prima di definire l'acquisto, l'ente verifica se il servizio deve essere unico o separabile in componenti: per esempio hosting, migrazione, manutenzione applicativa, assistenza e formazione. Separare senza ragione può moltiplicare interfacce e responsabilità; concentrare tutto in un solo affidamento può rendere difficile il subentro. La domanda pratica è quale confine consenta di governare qualità, sicurezza, tempi e continuità senza creare una dipendenza che l'ente non sa più misurare.

Anche gli strumenti di acquisto e negoziazione non sostituiscono questa analisi. Convenzioni, accordi, mercati elettronici e altri canali possono orientare il modo di approvvigionarsi, ma non trasformano un fabbisogno vago in una buona prestazione. Il candidato deve prima saper descrivere l'oggetto, i vincoli e le evidenze; solo dopo colloca la scelta nello strumento e nella procedura che gli atti consentono. Banca dati e piattaforme digitali rendono tracciabile il ciclo, non decidono al posto dell'amministrazione che cosa sia necessario acquistare.

Una scheda iniziale utile mette sulla stessa pagina risultato atteso, utenti, dati coinvolti, integrazioni, volume previsto, indisponibilità accettabile, competenze interne e dipendenze esterne. Aggiunge poi le assunzioni da confermare e le alternative da confrontare. Se l'ipotesi di volume cambia, se un'integrazione non è disponibile o se l'ente non possiede una competenza essenziale, deve cambiare anche la valutazione dell'opzione. In questo senso la baseline è viva: viene aggiornata quando emergono dati nuovi, mantenendo traccia della decisione e del suo motivo.

Per evitare una scelta solo economica, conviene includere fin dall'inizio il costo del non fare e il costo dell'uscita. Un servizio lasciato senza manutenzione può produrre ritardi, errori e perdita di fiducia; una transizione improvvisata può imporre doppi ambienti, estrazioni manuali e supporto urgente. Non servono numeri universali: serve mostrare che la decisione considera l'intero ciclo di vita. Questo è anche il criterio per leggere criticamente una proposta apparentemente conveniente: chiedere quali attività, conoscenze e dati resterebbero all'ente se il fornitore cambiasse domani.

CAD 68–69: CONFRONTO E RIUSO PRIMA DELL'ACQUISTO

L'articolo 68 CAD richiede una valutazione comparativa tecnico-economica: sviluppo per conto dell'ente, riuso di software della PA, software libero/a codice aperto, cloud, software proprietario mediante licenza e combinazioni. Considera costo complessivo, formati e interfacce aperti, interoperabilità, sicurezza, protezione dei dati e livelli di servizio. Prima del software proprietario o dello sviluppo ad hoc deve essere verificata, con motivazione secondo le linee guida AgID, l'impossibilità di ricorrere alle soluzioni in riuso o a codice aperto adeguate alle esigenze. La disponibilità del sorgente non elimina costi di migrazione e gestione.

Confronto didattico su cinque anni. Una soluzione in riuso costa 20 mila euro di adattamento, 5 di migrazione, 30 di manutenzione e 5 di uscita: TCO 60 mila. Un prodotto proprietario costa 25 di licenze, 5 di migrazione, 30 di gestione e 10 di uscita: TCO 70 mila. Se entrambe soddisfano requisiti, sicurezza e servizio, il confronto motiva la scelta del riuso. Se una non soddisfa un requisito necessario, si documenta la verifica e si confrontano alternative: il solo totale non decide. Le cifre sono ipotetiche e non prezzi di mercato.

L'articolo 69 riguarda il riuso delle soluzioni di cui la PA è titolare: codice sorgente e documentazione devono essere resi disponibili con licenza aperta in repertorio pubblico secondo le regole AgID, salvo le eccezioni previste, tra cui sicurezza e difesa. Per software sviluppato su specifica indicazione dell'ente, la titolarità dei diritti necessari al riuso deve essere assicurata contrattualmente, salvo comprovate ragioni tecnico-economiche. Richiedere solo un eseguibile alla consegna può quindi compromettere un obbligo e la capacità di subentro.

VINCOLI SPECIFICI PER L'APPROVVIGIONAMENTO ICT

La legge 208/2015, articolo 1, commi 512 e seguenti, aggiunge una disciplina specifica per beni e servizi informatici e connettività. I soggetti individuati dal comma 512 ricorrono agli strumenti Consip o dei soggetti aggregatori, comprese le centrali regionali, per i beni e servizi disponibili presso tali soggetti. Il comma 516 consente l'approvvigionamento fuori dalle modalità prescritte solo previa autorizzazione motivata dell'organo di vertice amministrativo, se il bene/servizio non è disponibile o idoneo al fabbisogno, oppure in caso di necessità e urgenza funzionale alla continuità amministrativa; prevede comunicazione ad ANAC e AgID. Vanno controllati ambito soggettivo, norme speciali e deroghe applicabili al caso: non si trasforma questa sintesi in un obbligo identico per qualsiasi ente o attività di ricerca.

Il percorso è quindi duplice: documentare la scelta della soluzione secondo CAD e linee guida, poi individuare canale e procedura consentiti. Un confronto tecnico favorevole a un prodotto non autorizza da solo a saltare gli strumenti obbligatori.

12.2 Requisiti verificabili e capitolato tecnico

FUNZIONALI E NON FUNZIONALI

Un requisito funzionale descrive ciò che il sistema deve fare. Esempio: «l'operatore può consultare lo stato della pratica e la cronologia delle modifiche».

Un requisito non funzionale riguarda qualità o vincoli. Può riferirsi a disponibilità, prestazioni, sicurezza, accessibilità, interoperabilità, manutenibilità, capacità o portabilità. Esempio: «la cronologia deve essere esportabile in un formato documentato e leggibile da strumenti indipendenti».

Questa distinzione orienta le verifiche, ma i due gruppi restano collegati: una funzione priva di requisiti di sicurezza e prestazione può risultare inutilizzabile.

SCRIVERE REQUISITI VERIFICABILI

Per essere efficace, un requisito specifica:

1. soggetto o componente interessato;
2. comportamento o qualità richiesta;
3. condizioni operative;
4. risultato osservabile;
5. criterio di accettazione;
6. evidenza attesa.

«Il sistema deve essere veloce» non è verificabile. «Nelle condizioni di carico definite nel piano di test, il servizio risponde entro la soglia stabilita, misurata dal punto concordato e registrata nel resoconto tecnico» indica almeno come completare la specifica. Il valore numerico va motivato e approvato, non inventato.

Errore tipico: descrivere una marca o una tecnologia senza collegarla al bisogno, ai vincoli e alla disciplina applicabile. Le specifiche devono consentire una valutazione tecnica trasparente e gestire correttamente standard, compatibilità ed equivalenze.

REQUISITO, CRITERIO, OBBLIGO E SLA

Quattro elementi spesso confusi hanno funzioni diverse:

Elemento	Domanda
requisito	che cosa deve fare o garantire la soluzione?
criterio di accettazione	come dimostriamo che il requisito è soddisfatto?

Elemento	Domanda
criterio di valutazione	come confrontiamo le offerte secondo gli atti?
SLA	quale livello deve essere mantenuto durante il servizio?

Un requisito può essere verificato, per servizi e forniture ICT, nell'ambito della verifica di conformità oppure monitorato nel tempo. Il collaudo riguarda invece i lavori. Un criterio di valutazione opera nella scelta dell'offerta; non sostituisce l'obbligo di esecuzione. Lo SLA riguarda il servizio erogato e deve collegarsi a misure e conseguenze definite.

TRACCIABILITÀ E ACCETTAZIONE

La matrice requisito-test-evidenza collega promessa e verifica:

ID	Requisito	Verifica	Evidenza
R-01	esportazione dati	prova su campione	file e verbale
R-02	gestione accessi	test dei ruoli	log e resoconto tecnico
R-03	ripristino	esercitazione	rapporto di prova

ID	Esito
R-01	da compilare
R-02	da compilare
R-03	da compilare

Accettare una prestazione non significa esprimere un assenso generico. Registra versione, ambiente, dati di prova, risultato, anomalie, riserve, responsabili e decisione. Se il requisito cambia, devono cambiare in modo controllato anche test ed evidenze.

APPLICAZIONE: DALLA FRASE GENERICA ALLA PROVA

Il capitolato deve consentire a persone diverse di capire la stessa cosa. Per questo un requisito collega un soggetto, un comportamento o una qualità, condizioni d'uso, criterio di accettazione ed evidenza. “Il sistema deve essere sicuro” non permette di decidere se la prestazione è resa; “gli accessi amministrativi devono essere attribuiti a identità individuali, registrati e verificabili nel resoconto tecnico concordato” indica invece una proprietà controllabile. Il valore esatto, il metodo e l'ambito restano da definire negli atti, ma la struttura della verifica è già chiara.

La neutralità tecnica non equivale a vaghezza. È possibile descrivere interfacce, standard, compatibilità, prestazioni, accessibilità e portabilità senza fissare una marca come soluzione obbligata. Se un vincolo tecnico è necessario, deve essere collegato a interoperabilità, continuità, sicurezza o altra esigenza dimostrabile e gestito secondo la disciplina applicabile. Il candidato deve distinguere l'oggetto della prestazione dal modo con cui le offerte sono confrontate: un criterio di valutazione opera nella scelta dell'offerta; un requisito e un obbligo regolano ciò che dovrà essere consegnato; il criterio di accettazione serve a verificare il risultato.

La matrice requisito-test-evidenza evita che la verifica di conformità diventi un giudizio impressionistico. Per ciascun requisito critico annota identificativo, test o ispezione, ambiente, dati di prova, responsabile, evidenza attesa ed esito. Se la migrazione produce un file esportato ma il file non può essere letto nel sistema destinatario, l'evidenza non dimostra ancora portabilità. Se una funzione opera solo su dati preparati dal fornitore, occorre chiarire se quel campione rappresenti l'uso reale. Errore tipico: confondere la documentazione consegnata con l'accettazione. La prima può essere necessaria; la seconda richiede una verifica tracciabile.

DALLA SPECIFICA ALLA VERIFICA DI CONFORMITÀ

Un capitolato robusto non è una lista di desideri. Ordina i requisiti per oggetto, priorità e dipendenza, chiarendo cosa è indispensabile per avviare il servizio e cosa può essere consegnato in una fase successiva. La priorità non riduce l'obbligo di verificare: aiuta a decidere quali prove sono bloccanti, quali anomalie ammettono una correzione e quali condizioni impediscono l'accettazione. Quando un requisito dipende da un'integrazione di terzi, la specifica deve dire chi fornisce l'accesso, quale ambiente viene usato e come si tratta l'indisponibilità del soggetto esterno.

Il criterio di accettazione deve essere proporzionato alla promessa. Per una funzione critica può servire una prova end-to-end con utenti, dati rappresentativi, gestione delle eccezioni e traccia dell'esito. Per un documento tecnico possono bastare completezza, aggiornamento e verificabilità di ciò che descrive. Una dimostrazione guidata dal fornitore è utile, ma non coincide automaticamente con un test indipendente: chi verifica deve poter riprodurre, controllare o almeno comprendere la prova e le sue condizioni.

È utile distinguere tre piani temporali. Nella selezione si valuta, secondo gli atti, la proposta disponibile; prima dell'avvio si accerta che ambienti, configurazioni e consegne iniziali siano pronti; durante l'esecuzione si misura che il servizio resti conforme. Confondere i piani porta a due errori: usare una promessa di offerta come se fosse una prova di esercizio, oppure pretendere, nella verifica di conformità, una qualità che non è stata descritta. La tracciabilità requisito-test-evidenza collega i piani e rende visibili le lacune.

Quando una richiesta cambia, non basta aggiornare una riga di testo. Occorre rivalutare impatto su utenti, sicurezza, dati, tempi, costo, integrazioni, test e documentazione. La nuova versione del requisito deve indicare chi l'ha approvata e quale prova la renderà accettabile. Così il capitolato resta uno strumento di governo e non un documento congelato che viene interpretato solo quando nasce un conflitto.

12.3 SLA, SLI, KPI e misurazione del servizio

SLA, SLI E SOGLIA

Lo SLA è l'impegno contrattuale sul livello di servizio. Lo SLI è l'indicatore osservato. La soglia o obiettivo indica il livello atteso. Il lessico può variare tra documenti; ciò che conta è rendere non ambigue definizione, misura e responsabilità.

Una scheda SLA completa specifica:

- servizio e popolazione osservata;
- formula o regola di misura;
- soglia;
- finestra e periodo di osservazione;
- fonte dei dati e punto di misura;
- esclusioni ammesse;
- severità o classe dell'evento;
- rendicontazione tecnica e responsabile della verifica;
- escalation e conseguenze previste dagli atti.

DISPONIBILITÀ E TEMPI

La disponibilità indica la quota di tempo in cui il servizio è utilizzabile nel perimetro concordato. Senza finestra di servizio, periodo, componenti inclusi ed esclusioni, la percentuale è equivoca.

Il tempo di presa in carico misura quanto passa prima che il fornitore inizi a gestire una segnalazione. Il tempo di ripristino riguarda il ritorno a una condizione operativa; il tempo di risoluzione può richiedere la rimozione definitiva della causa. Un workaround può ripristinare il servizio senza chiudere il problema.

La severità dipende dall'impatto e dall'urgenza. Un malfunzionamento che blocca tutti gli utenti non è equivalente a un difetto cosmetico. Classi e regole devono essere definite prima dell'incidente.

SLA E KPI

Un KPI sostiene il governo del servizio: volume ticket, tasso di riapertura, modifiche fallite, vulnerabilità scadute, soddisfazione o backlog. Può essere utile anche senza costituire uno SLA.

Una penale o un service credit non ripara da solo il servizio. L'ente deve ottenere ripristino, analisi della causa, azione correttiva e prevenzione della ricorrenza. Le conseguenze contrattuali concrete dipendono dagli atti e dal quadro vigente.

APPLICAZIONE: MISURARE CIÒ CHE CONTA

Un livello di servizio diventa utile quando collega una misura a una decisione. L'indicatore osservato può provenire da strumenti del fornitore, dell'ente o da entrambi; questo non basta, perché occorre definire quale componente è osservata, con quale orologio, in quale finestra e come sono trattati eventi eccezionali. La stessa percentuale di disponibilità cambia significato se si riferisce al solo front-end o anche alle integrazioni necessarie per completare una pratica. Senza perimetro e fonte dei dati, un resoconto tecnico può apparire preciso ma non essere contestabile.

Conviene separare gli indicatori di esito dagli indicatori di governo. Disponibilità, presa in carico o ripristino possono essere impegni contrattuali se gli atti li definiscono come tali. Numero di ticket, riaperture, modifiche fallite o arretrati aiutano invece a comprendere tendenze e rischi; possono essere KPI importanti senza diventare automaticamente SLA. Anche le classi di severità devono essere concordate prima del disservizio, usando impatto su utenti, servizio e attività amministrativa, non solo la comodità tecnica di chi riceve la segnalazione.

Una soglia non sostituisce la gestione. Davanti a uno scostamento, l'ente deve ottenere evidenza, valutare l'impatto, chiedere ripristino, analisi della causa e azione preventiva, registrando responsabile e scadenza. Penali o service credit, se previsti, seguono gli atti e non rendono accettabile un servizio che continua a fallire. In un orale, formula una risposta in questo ordine: servizio e utenti, misura e fonte, soglia e finestra, eccezioni, rendicontazione tecnica, escalation e verifica della chiusura. Errore tipico: usare "KPI" come sinonimo di qualsiasi numero del resoconto tecnico.

La scheda SLA deve rimanere leggibile: servizio, indicatore, definizione, soglia, finestra, periodo, fonte, esclusioni, severità, rendicontazione tecnica, escalation e responsabile della verifica. Se uno di questi campi manca, il dato può essere utile come osservazione tecnica ma non basta a sostenere una contestazione o una decisione contrattuale.

LEGGERE UNO SCOSTAMENTO SENZA AUTOMATISMI

La misura va progettata prima che il servizio entri in difficoltà. Per ogni indicatore è necessario stabilire chi raccoglie il dato, con quale strumento, come si gestiscono dati mancanti e differenze di orologio, chi può contestare il calcolo e dove resta la traccia della decisione. Un dato prodotto dal fornitore non è inattendibile per definizione; diventa fragile quando l'ente non può ricostruire il perimetro o confrontarlo con segnali indipendenti, come segnalazioni degli utenti, log applicativi o esiti delle integrazioni.

La disponibilità non esaurisce l'esperienza del servizio. Un portale può essere raggiungibile e insieme impedire di completare una pratica perché l'autenticazione o il pagamento non rispondono. Per questo la scheda distingue componenti, percorsi essenziali e classi di impatto. La stessa cautela vale per presa in carico, ripristino e risoluzione: sono momenti diversi. Dire che un ticket è stato preso in carico non dimostra che l'utente abbia recuperato la funzione; dire che è stato ripristinato non dimostra ancora che la causa non ricorrerà.

Davanti a una deviazione, la sequenza ordinata è: accertare il fatto, delimitare utenti e servizi coinvolti, attivare il canale previsto, registrare le azioni di contenimento, chiedere analisi e piano correttivo, quindi verificare l'esito. Un indicatore può segnalare l'evento, ma non decide da solo la risposta. Le conseguenze economiche o contrattuali, quando previste, dipendono da atti e circostanze; non vanno dedotte da una percentuale isolata. L'obiettivo prioritario resta riportare il servizio in una condizione affidabile e prevenire la ricorrenza.

Un buon cruscotto non contiene solo valori verdi o rossi. Mostra tendenza, cause, rischio residuo, azioni in scadenza e decisioni richieste. Il candidato può dimostrare maturità osservando che una metrica stabile può nascondere un problema se cambia il volume di utenti o si riduce il perimetro osservato. La domanda corretta non è «il numero rispetta la soglia?», ma «il numero descrive ancora il servizio che cittadini e uffici ricevono?».

CALCOLO SVOLTO DELLA DISPONIBILITÀ

Il contratto ipotetico misura il servizio end-to-end per 30 giorni, 24 ore su 24, senza esclusioni: $30 \times 24 \times 60 = 43.200$ minuti. Le indisponibilità registrate e riconciliate durano 90 minuti. $\text{Disponibilità} = (43.200 - 90) / 43.200 \times 100 \approx 99,7917\%$. Con obiettivo 99,9%, l'indisponibilità ammessa è 43,2 minuti: il limite è superato di 46,8 minuti. Il verbale riporta intervalli, fonte di misura e confronto, non soltanto la percentuale del fornitore. Penali o altri rimedi dipendono dalle clausole: lo scostamento numerico non autorizza a inventarne l'importo.

Se il contratto escludesse manutenzioni programmate, occorrerebbe definirne preventivamente autorizzazione, finestra e trattamento del denominatore. Non si cancellano ex post i minuti sfavorevoli. Anche con disponibilità rispettata, latenza o errori applicativi potrebbero violare altri indicatori: uno SLA non rappresenta da solo tutta la qualità.

12.4 Ruoli, evidenze e governo dell'esecuzione

COMPETENZE DIVERSE, UN SOLO SISTEMA DI CONTROLLO

Il RUP, responsabile unico del progetto, presidia il progetto secondo la disciplina applicabile. La funzione di direzione dell'esecuzione deve essere assicurata: per servizi e forniture è normalmente svolta dal RUP; nei casi di particolare importanza previsti dall'articolo 114, comma 8, e dall'allegato II.14, articoli 31–32, del d.lgs. 36/2023 si nomina un DEC distinto. Non confondere la presenza della funzione con l'obbligo di una persona separata, né presumere che qualsiasi acquisto ICT lo richieda automaticamente.

L'owner del servizio presidia risultato e utenti; il referente tecnico verifica architettura e deliverable; sicurezza e DPO intervengono per competenza. Il fornitore organizza le proprie responsabilità e produce le evidenze dovute.

Una matrice RACI può chiarire chi esegue, approva, consulta e riceve informazioni. È uno strumento organizzativo: non modifica competenze stabilite dalla normativa o dagli atti.

AVVIO, RENDICONTAZIONE TECNICA E VERIFICHE

Il kick-off traduce il contratto in un assetto operativo: contatti, canali, calendario, deliverable, ambienti, accessi, rendicontazione tecnica, rischi ed escalation. Le decisioni vanno verbalizzate e collegate ai documenti applicabili.

Un resoconto tecnico periodico serve a decidere e controllare, non a raccogliere numeri senza esito. Deve mostrare livelli raggiunti, scostamenti, incidenti, modifiche, rischi, azioni, responsabili e scadenze. L'ente conserva dati sufficienti per verificare la misura anche quando il calcolo è prodotto dal fornitore.

NON CONFORMITÀ ED ESCALATION

Una non conformità è uno scostamento rispetto a requisito, deliverable o obbligo verificabile. Va descritta con evidenza, impatto, causa nota o da analizzare, azione, termine, responsabile e verifica di chiusura.

L'escalation può essere tecnica, gestionale o contrattuale. Saltare subito al livello direzionale crea rumore; restare nel supporto operativo quando il servizio è critico ritarda la risposta. Trigger e canali devono essere stabiliti prima.

APPLICAZIONE: FAR PARLARE IL RESOCONTO TECNICO

Una riunione periodica, da sola, non governa l'esecuzione. Al kick-off si trasformano documenti e impegni in un calendario operativo: contatti, ambienti, canali, deliverable, accessi, frequenza del resoconto tecnico, registro dei rischi e regole di escalation. Un resoconto tecnico utile confronta il risultato atteso con quello osservato e mostra ciò che richiede una decisione. Ticket, livelli di servizio, vulnerabilità, modifiche e scadenze devono quindi avere un responsabile, una data, un'evidenza e un esito, non soltanto un colore di stato.

Il RUP, il soggetto che svolge la direzione dell'esecuzione, il referente tecnico, l'owner del servizio, sicurezza e DPO svolgono funzioni che devono restare coerenti con normativa e atti. Una matrice RACI può rendere visibile chi lavora, chi approva, chi viene consultato e chi riceve informazioni; non attribuisce però poteri che non esistono. Il fornitore può produrre misure e resoconto tecnico, ma l'ente deve poterli leggere e verificare. Se nessuno sa contestare una misura o ricostruire una configurazione, il controllo è solo formale.

Una non conformità parte da un fatto osservabile: requisito, deliverable o livello non rispettato. Il registro deve indicare impatto, evidenza disponibile, causa nota o da analizzare, azione, termine, responsabile e criterio di chiusura. L'escalation sale quando l'impatto o il ritardo lo richiedono; non è né una punizione automatica né una catena infinita di e-mail. Errore tipico: pensare che la stipula trasferisca al fornitore il risultato pubblico. L'ente conserva il dovere di governare servizio, decisioni ed evidenze.

La chiusura di un'azione deve verificare il risultato e non soltanto la risposta del fornitore. Un ticket “risolto” senza prova, un resoconto tecnico firmato senza confronto con la fonte e una riunione senza decisioni lasciano il rischio invariato. Per questo il registro è anche uno strumento d'esame: obbliga a collegare fatto, criterio, responsabile, azione ed evidenza.

CATENA DECISIONALE E EVIDENZE

Ogni riunione operativa dovrebbe produrre una decisione o una domanda ancora aperta. Per questo il verbale separa fatti osservati, ipotesi, impegni, responsabili e date di riesame. Se il fornitore segnala una difficoltà, l'ente non deve limitarsi a ricevere l'informazione: deve capire quale requisito, servizio o dipendenza è coinvolto, quali utenti sono esposti, quale misura provvisoria è attiva e quale prova chiuderà il problema. Questa disciplina trasforma il confronto periodico in governo del contratto.

Il ruolo tecnico non coincide con il ruolo contrattuale. Il referente applicativo può sapere che una funzione è inutilizzabile; sicurezza può indicare un rischio negli accessi; il DPO può richiedere un esame dei trattamenti; il responsabile unico del progetto e le figure previste dagli atti governano le decisioni nel perimetro delle loro competenze. Collegare questi contributi non significa attribuire a ciascuno poteri inventati. Significa predisporre canali e tempi perché l'evidenza arrivi a chi può decidere.

Una matrice di responsabilità è particolarmente utile nelle attività che attraversano più uffici: attivazione di un account amministrativo, gestione di un incidente, rilascio urgente, verifica di un export dati o passaggio a un nuovo fornitore. Per ciascuna attività occorre distinguere chi prepara il lavoro, chi approva, chi deve essere consultato e chi deve conoscere l'esito. La matrice va poi confrontata con contratto, nomine e procedure interne: se non coincide, prevalgono questi ultimi.

La trasparenza delle evidenze protegge entrambe le parti. Il fornitore sa quale risultato deve dimostrare; l'ente può motivare una richiesta di correzione o la chiusura di un'anomalia. La mancanza di una traccia condivisa, invece, trasforma problemi ordinari in discussioni sulla memoria delle persone. Perciò registri, verbali e prove non sono burocrazia aggiuntiva: sono il ponte tra il servizio realmente erogato e la decisione amministrativa.

12.5 Sicurezza, dati e supply chain contrattuale

SICUREZZA COME PRESTAZIONE VERIFICABILE

La formula «il fornitore garantisce la sicurezza» non definisce una prestazione verificabile. I requisiti possono riguardare accessi privilegiati, segregazione, logging, patching, vulnerabilità, cifratura, incidenti, backup, test di ripristino e conservazione delle evidenze. Ogni controllo deve essere proporzionato al rischio e verificabile.

Il contratto chiarisce tempi e modalità di cooperazione negli incidenti, preservazione dei log, comunicazioni, contenimento e ripristino. Gli obblighi normativi dell'ente e del fornitore vanno verificati sul caso; lo SLA interno non sostituisce eventuali termini legali.

DATI E SUBFORNITORI

Occorre conoscere quali dati sono trattati, dove transitano, chi vi accede, per quanto tempo sono conservati e come vengono restituiti o cancellati. Formati e procedure di esportazione vanno provati, non soltanto dichiarati.

I subfornitori possono introdurre dipendenze tecniche e geografiche. L'ente deve avere visibilità coerente con rischio e disciplina applicabile. Il titolare decide su ruoli privacy, istruzioni, trasferimenti e misure, acquisendo le competenze legali necessarie e il parere/consulenza del DPO nei suoi compiti. Il DPO informa, consiglia e sorveglia: non è un organo che approva i trattamenti al posto del titolare e non assume funzioni operative incompatibili con la propria indipendenza.

SUPPLY CHAIN

Componenti software, librerie, servizi esterni e aggiornamenti appartengono alla supply chain. Quando pertinente, inventario, provenienza, vulnerabilità, supporto ed eventuale SBOM aiutano a governare il rischio. Il capitolo 8 sviluppa la teoria; qui questi elementi diventano requisiti, resoconto tecnico o condizioni di gestione.

APPLICAZIONE: TRASFORMARE IL RISCHIO IN REQUISITI

La sicurezza diventa governabile quando si traduce in prestazioni verificabili e proporzionate. A seconda del servizio, il contratto può richiedere gestione degli accessi privilegiati, registrazione degli eventi rilevanti, trattamento delle vulnerabilità, cooperazione negli incidenti, copie di sicurezza, prove di recupero e conservazione delle evidenze. Non esiste una formula valida per tutti i casi: ciò che è ragionevole per un servizio informativo può non bastare per un servizio che gestisce pratiche, pagamenti o dati sensibili.

Per i dati bisogna sapere quali categorie sono trattate, dove transitano, chi accede, quali integrazioni le ricevono e come saranno restituite o cancellate. Il contratto non può decidere da solo ruoli privacy o trasferimenti: questi aspetti vanno esaminati nel caso concreto con le figure competenti. Il candidato deve però riconoscere l'errore di rinviare tutto a una clausola generica. Anche un'esportazione dichiarata disponibile va provata su dati e formati definiti, verificando metadati, completezza e riuso.

La supply chain comprende componenti, librerie, servizi esterni, aggiornamenti e subfornitori che rendono possibile la prestazione. L'ente non deve trasformarsi nel gestore tecnico di ogni componente, ma deve conoscere le dipendenze rilevanti, le evidenze disponibili e i trigger di escalation. Un inventario, una dichiarazione di provenienza o una SBOM possono aiutare quando sono pertinenti, senza diventare un adempimento decorativo. Errore tipico: scrivere che il fornitore “garantisce la sicurezza” senza dire come l'ente verifica accessi, incidenti, dati, subfornitori e rimedi.

Quando il servizio usa componenti esterni, l'ente deve prevedere come ricevere notifiche, come valutare impatto e chi autorizza le azioni urgenti. Questa preparazione evita due errori opposti: accettare automaticamente ogni aggiornamento del fornitore oppure bloccare ogni cambiamento fino a compromettere sicurezza e continuità. La decisione resta proporzionata, documentata e verificabile.

PRESIDI PROPORZIONATI E VERIFICABILI

La sicurezza contrattuale richiede un equilibrio. Elencare ogni possibile controllo senza collegarlo al servizio genera obblighi difficili da valutare; una clausola astratta lascia invece spazio a interpretazioni incompatibili. Il punto di partenza è il rischio: dati, criticità del servizio, esposizione esterna, utenti, integrazioni, privilegi amministrativi e dipendenze. Da qui si scelgono requisiti osservabili, fonti di evidenza e modalità di riesame. Il lettore non deve memorizzare una lista universale, ma imparare a collegare controllo, minaccia e prova.

Gli accessi amministrativi meritano una particolare attenzione perché consentono modifiche ampie. Devono essere attribuibili, limitati, riesaminabili e tracciati secondo il perimetro definito. Analogamente, log e alert non sono utili se nessuno sa conservarli, leggerli e collegarli a un incidente. Nel contratto contano quindi sia la disponibilità del dato sia la cooperazione: chi avvisa, con quali informazioni, chi decide il contenimento e come viene conservata l'evidenza della chiusura.

La presenza di un subfornitore non equivale automaticamente a una non conformità, ma non può restare invisibile. L'ente deve conoscere, nella misura richiesta dal caso, quale parte del servizio è affidata all'esterno, quali dati o componenti coinvolge, quali controlli si applicano e come viene gestita una variazione. Lo stesso vale per componenti che arrivano tramite una filiera software: provenienza, supporto, aggiornamenti e vulnerabilità diventano rilevanti quando incidono su continuità o sicurezza del servizio acquistato.

Nella pratica la verifica può combinare evidenze diverse: configurazioni, registri di accesso, risultati di test, attestazioni, elenco delle dipendenze, esiti di esercitazioni e verbali di gestione dell'incidente. Nessuna prova isolata basta sempre. L'ente cerca coerenza tra ciò che è promesso, ciò che è configurato e ciò che accade durante l'esercizio. Questo evita sia l'affidamento cieco a una dichiarazione, sia una richiesta di documenti privi di un criterio di lettura.

12.6 Modifiche, rilasci e continuità del servizio

CLASSIFICARE LE MODIFICHE

Correzione di un difetto, manutenzione ordinaria, evoluzione funzionale, variazione del perimetro e modifica contrattuale non sono sinonimi. La classificazione determina analisi, approvazioni, costi, test e documentazione.

Una change request descrive motivo, impatto, dipendenze, rischio, piano di test, rilascio e rollback. Versioni di requisito, configurazione, codice, documentazione e approvazione devono essere riconciliabili.

CUTOVER E ROLLBACK

Il cutover è il passaggio operativo alla nuova soluzione o versione. Richiede prerequisiti, sequenza, responsabili, comunicazioni e criteri di go/no-go. Il rollback riporta a una condizione precedente controllata se il rilascio non soddisfa i criteri.

CONTINUITÀ

Backup, replica, alta disponibilità e disaster recovery rispondono a scopi diversi. RPO e RTO, spiegati nel capitolo 7, diventano qui requisiti da provare. Finché non viene provato, il piano non dimostra la capacità di ripristino.

Il contratto deve considerare anche obsolescenza ed end of support. Un componente non più supportato può trasformare manutenzione e sicurezza in un rischio di continuità.

APPLICAZIONE: NON CONFONDERE IL RILASCIO CON IL RISULTATO

Una modifica può correggere un difetto, aggiornare una componente, estendere una funzione, cambiare un perimetro o incidere sugli impegni contrattuali. La distinzione serve perché non tutte le variazioni chiedono lo stesso controllo. Una change request ben costruita indica motivo, impatto su utenti e dipendenze, versione coinvolta, rischi, test, approvazioni, piano di rilascio e condizioni di rollback. Se cambia un requisito, devono rimanere riconciliabili anche criteri di accettazione, test ed evidenze.

Il cutover è il passaggio alla nuova versione o soluzione. Prima del go/no-go occorrono prerequisiti, responsabilità, comunicazioni e criteri osservabili; dopo il rilascio occorre verificare l'effetto, non soltanto la riuscita tecnica della procedura. Il rollback riporta a una condizione precedente controllata quando non sono rispettati i criteri. Non è un'ammissione di fallimento: è un presidio per limitare l'impatto mentre si analizza il problema.

Backup, replica, alta disponibilità e disaster recovery rispondono a bisogni differenti. RPO e RTO aiutano a esprimere rispettivamente la perdita di dati tollerabile e il tempo di recupero atteso, ma acquistano valore soltanto se sono coerenti con il servizio e vengono provati. Un piano non testato non dimostra capacità di ripristino. Anche l'obsolescenza e la fine del supporto devono entrare nel monitoraggio: una dipendenza non più mantenuta può trasformare un aggiornamento rinviato in un problema di sicurezza e continuità. Errore tipico: trattare ogni rilascio come manutenzione ordinaria e scoprire solo dopo l'impatto su dati, integrazioni o utenti.

Per il candidato, la domanda utile è: quale evidenza dimostra che il servizio può continuare dopo un cambiamento? La risposta combina test, inventario delle dipendenze, documentazione aggiornata, responsabilità chiare e prova di recupero. Un contratto può prevedere questi elementi; l'efficacia dipende però dalla loro applicazione durante l'esecuzione, non dalla sola presenza di un titolo nel capitolato.

CONTINUITÀ COME PROVA, NON COME ETICHETTA

Il piano di rilascio deve descrivere non solo la sequenza tecnica ma anche l'impatto sul servizio. Chi comunica agli utenti? Quali integrazioni possono risentirne? Quale dato deve essere salvaguardato? Quale condizione ferma il passaggio e quale consente il ritorno alla versione precedente? Queste domande aiutano a costruire criteri di go/no-go concreti. Il go non è la semplice autorizzazione a premere un pulsante; è la decisione, basata su evidenze, che prerequisiti e rischi residui sono accettabili.

Prima di un rilascio rilevante sono utili test coerenti con l'uso previsto: funzionali, di integrazione, di sicurezza, di prestazione o di recupero, secondo il caso. L'ambiente di prova e i dati utilizzati devono essere sufficientemente rappresentativi, altrimenti un esito positivo può non dire nulla sul funzionamento reale. Dopo il rilascio, l'osservazione prosegue: un controllo su log, ticket, tempi di risposta e percorsi utente può rivelare una regressione che il test iniziale non aveva intercettato.

RPO e RTO non sono sigle decorative. Esprimono, rispettivamente, quanta perdita di dati può essere tollerata e in quale tempo il servizio deve recuperare una condizione operativa. I valori non sono standard: derivano dal danno che il fermo o la perdita produrrebbero per utenti e amministrazione. Per questo vanno collegati a responsabilità, mezzi tecnici, prova periodica e decisione sulle eccezioni. Dire che esiste un backup non dimostra che il ripristino sia possibile entro il bisogno del servizio.

Anche l'obsolescenza richiede anticipo. Una libreria senza supporto, una piattaforma che cambia condizioni o un'integrazione destinata a chiudere possono produrre un rischio prevedibile. Il registro delle dipendenze, aggiornato nel governo del fornitore, consente di pianificare migrazione, aggiornamento o sostituzione prima che la scelta diventi emergenziale. In prova, il candidato mostra questa continuità: modifica, test, decisione, monitoraggio e aggiornamento dell'inventario.

12.7 Lock-in, portabilità, reversibilità ed exit strategy

FORME DI DIPENDENZA

Il lock-in non riguarda soltanto la tecnologia. Può dipendere da:

- formati o API non trasferibili;
- licenze e condizioni contrattuali;
- costi di estrazione o migrazione;
- processi organizzativi costruiti sul fornitore;
- documentazione incompleta;
- competenze presenti solo nel gruppo esterno;
- dati, configurazioni o log non esportabili.

PORTABILITÀ E REVERSIBILITÀ

La portabilità consente di trasferire dati, applicazioni o carichi in un altro ambiente. La reversibilità comprende il ritorno o il passaggio controllato del servizio, con conoscenze, configurazioni, documenti e assistenza alla transizione.

La consegna finale di un file, da sola, non garantisce la portabilità. Occorre verificare completezza, struttura, metadati, leggibilità, tempi e capacità del destinatario di riutilizzarlo.

EXIT PLAN

L'exit plan definisce:

1. eventi che lo attivano;
2. ruoli e governance della transizione;
3. inventario di dati, configurazioni, integrazioni e documenti;
4. formati e procedure di estrazione;
5. supporto al subentro;
6. continuità durante il passaggio;
7. verifica di restituzione e cancellazione;
8. criteri di chiusura e accettazione finale.

Preparare l'exit plan soltanto alla scadenza è tardivo: l'ente deve progettarlo all'ingresso e provarlo durante l'esecuzione.

USCITA PROGETTATA FIN DALL'INGRESSO

Il lock-in è spesso il risultato di piccole decisioni accumulate. Un formato proprietario senza schema, un'API non documentata, un ambiente configurato solo dal fornitore o una squadra interna che non conosce più il servizio possono rendere impossibile il cambio anche quando il contratto è formalmente concluso. Per questo la dipendenza va osservata in cinque dimensioni: tecnica, dati, contratto, economia e organizzazione. Ciascuna richiede una domanda concreta: cosa bisogna trasferire, chi ne possiede il diritto, quanto costa estrarlo, chi sa usarlo e quale servizio deve rimanere disponibile durante la transizione.

La portabilità riguarda la possibilità materiale di trasferire elementi riutilizzabili; la reversibilità riguarda il percorso controllato che permette di uscire, rientrare o passare a un altro gestore. Un'esportazione è utile solo se include gli oggetti necessari, conserva struttura e metadati, è accompagnata da documentazione e può essere verificata dal destinatario. Per le configurazioni, la domanda non è solo «posso scaricarle?», ma «posso ricostruire un ambiente funzionante senza conoscere segreti non consegnati?».

L'exit plan deve essere trattato come un deliverable vivo. All'avvio raccoglie inventario, ruoli e condizioni di restituzione; durante l'esecuzione viene aggiornato quando cambiano dati, integrazioni, componenti o subfornitori; prima della transizione definisce sequenza, supporto, controlli di continuità e criteri di chiusura. I trigger possono essere scadenza, recesso, insostenibilità tecnica, cambiamento organizzativo o semplice esigenza di verifica periodica. La loro disciplina concreta dipende dal contratto, ma la preparazione tecnica non può aspettare l'evento.

Una prova di uscita non richiede necessariamente di migrare davvero tutto il servizio. Può iniziare con un campione rappresentativo: esportare dati, verificare schema e completezza, ricostruire una configurazione in ambiente controllato, misurare le dipendenze non trasferite e annotare le correzioni. Se il test fallisce, l'esito non è un dettaglio tecnico: segnala che requisito, documentazione, competenze o supporto al subentro sono insufficienti. L'ente deve quindi aggiornare piano e controllo prima che il rischio diventi urgente.

L'uscita ha anche una dimensione di continuità e sicurezza. Durante il passaggio si proteggono accessi, dati, log e servizi essenziali; al termine si verifica che permessi, copie e trattamenti residui siano gestiti secondo il caso. Non basta dichiarare che i dati sono stati cancellati o consegnati: occorre disporre dell'evidenza prevista e verificare che il nuovo assetto sappia svolgere il servizio. Errore tipico: chiamare exit strategy un elenco di file finali. Un'uscita reale trasferisce capacità operativa, non soltanto materiale digitale.

Una verifica di uscita ben documentata riduce anche l'asimmetria informativa: permette all'ente di capire quali attività restano indispensabili, quali dipendenze richiedono un piano e quali conoscenze devono essere trasferite prima della chiusura.

L'esito della prova va condiviso con i responsabili del servizio, così che l'uscita resti una capacità organizzativa e non una conoscenza isolata.

▣ Verifica

Le domande seguenti controllano le distinzioni che rendono governabile un contratto ICT. Rispondi prima a voce, poi confronta la motivazione.

QUIZ 1 — DAL BISOGNO ALLA SCELTA

Un ufficio chiede «un nuovo cloud» senza descrivere utenti, carichi, integrazioni o indisponibilità attuale. Qual è il primo passo?

Risposta corretta: costruire una baseline e definire il risultato pubblico, il perimetro e i vincoli prima di scegliere la soluzione. La tecnologia può essere una risposta, non il bisogno da dimostrare.

QUIZ 2 — REQUISITO E ACCETTAZIONE

«Il portale deve essere sicuro» è sufficiente per accettare la prestazione?

Risposta corretta: no. Occorrono requisiti osservabili, condizioni di prova, criterio di accettazione ed evidenza. Un'affermazione generica non permette di stabilire né il risultato né la chiusura di un'anomalia.

QUIZ 3 — SLA, SLI E KPI

Un cruscotto mostra il numero di ticket aperti: è automaticamente uno SLA?

Risposta corretta: no. Può essere un KPI di governo. Per essere uno SLA occorrono un impegno definito, metrica, soglia, finestra, fonte, esclusioni e responsabilità di verifica.

QUIZ 4 — GOVERNO DELL'ESECUZIONE

Il fornitore invia una misura mensile; l'ente può limitarvisi?

Risposta corretta: no. Deve poter comprendere perimetro e fonte del dato, collegarlo a utenti e requisiti, registrare scostamenti e verificare le azioni correttive. La misura prodotta dal fornitore non sostituisce il controllo dell'ente.

QUIZ 5 — SICUREZZA E FILIERA

Una clausola che impone al fornitore di «garantire la sicurezza» basta a governare la filiera?

Risposta corretta: no. Il contratto deve tradurre il rischio in evidenze e modalità di cooperazione: accessi, log, vulnerabilità, incidenti, dati, dipendenze e subfornitori, in modo proporzionato al servizio.

QUIZ 6 — PORTABILITÀ ED USCITA

Alla scadenza il fornitore consegna un archivio dati. L'exit è dimostrata?

Risposta corretta: non ancora. Occorre verificare completezza, struttura, metadati, configurazioni, documentazione, supporto alla transizione e capacità del destinatario di riutilizzare quanto ricevuto senza interrompere il servizio.

Strumenti di acquisto e ciclo digitale

Consip opera nell'ambito del programma di razionalizzazione degli acquisti. Acquisti in Rete è il canale attraverso cui amministrazioni e imprese accedono agli strumenti; il MEPA è uno di questi. Convenzioni, accordi quadro e sistemi dinamici hanno logiche diverse. Nessuno strumento elimina la necessità di definire fabbisogno, requisiti, copertura e controlli.

La digitalizzazione riguarda tutto il ciclo del contratto. La BDNCP è la banca dati nazionale gestita da ANAC; la PCP e il FVOE svolgono funzioni specifiche nell'ecosistema. Non vanno trattati come sinonimi. Dettagli operativi, procedure e soglie devono essere verificati sulle fonti vigenti e restano nella sede contrattuale generale.

Caso guidato: servizio cloud comunale

Un comune acquisisce migrazione, hosting cloud, manutenzione e assistenza per un portale al cittadino.

1. Bisogno: garantire accesso e continuità; baseline con utenti, carichi e incidenti.
2. Perimetro: migrazione, ambienti, integrazioni, supporto, sicurezza, documentazione ed uscita.
3. Requisiti: funzioni del portale più disponibilità, accessibilità, capacità, logging e portabilità.
4. Accettazione: test funzionali, prestazionali, di accesso, migrazione e ripristino con verbale.

5. SLA: indicatori distinti per disponibilità, presa in carico e ripristino; fonte e finestra concordate.
6. Sicurezza: account privilegiati, log, vulnerabilità, incidenti e subfornitori tracciati.
7. Esecuzione: resoconto tecnico mensile, registro rischi, non conformità e escalation.
8. Change: ogni rilascio ha impatto, test, go/no-go e rollback.
9. Continuità: backup e ripristino sono provati; RPO/RTO coerenti col servizio.
10. Uscita: esportazione di dati e configurazioni, documentazione, supporto al subentro e verifica di cancellazione.

Il caso non determina procedura, soglia o penale: tali elementi dipendono dagli atti e dalla disciplina vigente.

Laboratorio operativo

CHECKLIST DEL CAPITOLATO

- ☐ bisogno, baseline, utenti e perimetro sono chiari;
- ☐ requisiti funzionali e non funzionali sono verificabili;
- ☐ ogni requisito critico ha criterio di accettazione ed evidenza;
- ☐ dati, integrazioni, ambienti e migrazione sono descritti;
- ☐ SLA indicano misura, soglia, finestra, fonte ed esclusioni;
- ☐ ruoli, rendicontazione tecnica, non conformità ed escalation sono definiti;
- ☐ sicurezza, incidenti, continuità e subfornitori sono governati;
- ☐ change, versioni, cutover e rollback sono tracciati;
- ☐ portabilità, reversibilità ed exit plan sono provabili.

SCHEMA SLA

Campo	Compilazione
servizio e indicatore	che cosa misuriamo
soglia e periodo	livello e intervallo
fonte e punto di misura	da quali dati
esclusioni	casi ammessi e motivati
rendicontazione tecnica	frequenza e responsabile
escalation	trigger e canale

REGISTRO FORNITORE

Elemento	Dipendenza	Rischio	Controllo
servizio	da compilare	da valutare	da definire
dati	da compilare	da valutare	esportazione
subfornitore	da compilare	da valutare	monitoraggio

Elemento	Evidenza
servizio	resoconto tecnico
dati	prova
subfornitore	registro

Domanda da commissario

Come imposteresti e controlleresti un contratto per un servizio cloud destinato ai cittadini?

Partirei dal risultato pubblico, dalla baseline e dal perimetro. Tradurrei il bisogno in requisiti funzionali e non funzionali, ciascuno collegato a criteri di accettazione ed evidenze. Definirei SLA misurabili con fonte, finestra, soglia ed esclusioni. Assegnerei ruoli a RUP, direzione dell'esecuzione, owner, tecnici, sicurezza, DPO e fornitore. Governerei resoconto tecnico, non conformità, incidenti, modifiche, continuità e supply chain. Infine predisporrei fin dall'inizio portabilità, reversibilità ed exit plan. Procedura e clausole puntuali sarebbero verificate sugli atti e sul quadro vigente.

Domande-trappola

- Un requisito molto dettagliato è sempre verificabile? No: servono condizione, risultato ed evidenza.
- Disponibilità del 99,9% basta da sola? No: mancano perimetro, finestra, fonte, periodo ed esclusioni.
- SLA e KPI sono sinonimi? No: un KPI può non essere un impegno contrattuale.
- La stipula conclude il controllo dell'ente? No: l'esecuzione richiede verifica e governo.
- La penale risolve il disservizio? No: servono ripristino e azione correttiva.
- Il backup è disaster recovery? No: è solo uno dei possibili presidi.
- Il cloud trasferisce ogni responsabilità al provider? No: i ruoli dipendono dal caso e dagli atti.
- L'exit strategy si prepara alla scadenza? No: si progetta all'ingresso.
- MEPA è Consip? No: MEPA è uno strumento; Consip è il soggetto che opera nel programma.

Mini-esercizi e soluzioni

1. «Il portale deve essere intuitivo» è un requisito verificabile?

Soluzione: no. Occorre tradurlo in condizioni e verifiche osservabili, per esempio compiti, profili di utenti, criteri di completamento e test di accessibilità o usabilità appropriati.

2. Un resoconto tecnico indica soltanto «disponibilità 99,5%». Che cosa manca?

Soluzione: almeno perimetro, finestra, periodo, fonte, regola di calcolo, esclusioni e soglia contrattuale di confronto.

3. Il fornitore propone una nuova versione il giorno prima del rilascio. Quali evidenze chiedere?

Soluzione: impatto, versione, dipendenze, test, approvazione, piano di cutover, criteri di go/no-go e rollback.

4. Alla scadenza l'ente riceve un file senza schema né metadati. È portabilità effettiva?

Soluzione: non ancora. Deve verificare completezza, struttura, documentazione, leggibilità e riuso nel sistema destinatario.

Da sapere in 5 righe

Il fabbisogno deve diventare una prestazione verificabile. Requisito, accettazione, criterio di valutazione e SLA non sono sinonimi. Un livello di servizio vale solo se misura, finestra, fonte ed esclusioni sono chiare. Acquistare non trasferisce all'esterno il governo del servizio. Portabilità ed uscita si progettano prima della stipula e si verificano durante l'esecuzione.

Riferimenti per l'approfondimento

- Codice dei contratti pubblici e relativi allegati, nel testo vigente.
- Indicazioni ufficiali di ANAC sulla digitalizzazione del ciclo di vita del contratto.
- Materiali istituzionali di Consip e Acquisti in Rete sugli strumenti di acquisto e negoziazione.
- Documentazione istituzionale applicabile su cloud della PA, interoperabilità, sicurezza e protezione dei dati.
- Standard e buone pratiche di service management, continuità e portabilità, da applicare in modo coerente con gli atti del caso.

Avvertenze operative

Procedure, soglie, termini, penali, clausole, qualificazione cloud, ruoli privacy e condizioni di subfornitura dipendono dal quadro vigente e dagli atti del caso. Vanno quindi verificati, alla data del bando o della pubblicazione, sulle fonti istituzionali vigenti; questo capitolo non fissa valori o modelli contrattuali universali.

- CAD, articoli 68–69 e linee guida AgID su acquisizione e riuso; legge 208/2015, articolo 1, commi 512 e seguenti.

Laboratorio prove ICT: quiz, scritto tecnico, orale e casi

B
BANDO**A**
AREE**N**
NUCLEI**D**
DIARIO**O**
OUTPUT

13.1 Mappa BANDO: decodifica della prova e della consegna ICT

Conoscere la differenza tra autenticazione e autorizzazione è necessario. In prova, però, potrebbe non bastare. Un quiz può nascondere la distinzione dentro un distrattore plausibile; una domanda aperta può imporre quindici righe; un orale può chiedere un esempio; un caso può obbligare a scegliere un controllo, motivarlo e indicare come verificarne l'efficacia.

In prova la conoscenza deve diventare un output adatto alla consegna. Nei capitoli precedenti hai studiato sistemi, programmazione, dati, reti, software, cloud, cybersecurity, identità, AI e fornitori. Ora devi usare quei nuclei sotto vincoli di tempo, spazio e incertezza.

Il campione ufficiale consolidato comprende prove scritte e orali; per alcune procedure sono disponibili anche questionari, tracce o criteri. Il campione documenta la varietà degli output. Non consente, invece, di ricavare un modello universale. Formato, durata, punteggio, soglia, penalità e strumenti ammessi devono essere ricavati dal bando e dalle comunicazioni ufficiali.

OBIETTIVO DEL CAPITOLO

Al termine saprai:

- trasformare le regole della procedura in un piano di esecuzione;
- riconoscere che cosa chiede una traccia tecnica;
- affrontare quiz ICT distinguendo memoria, calcolo e strategia;
- costruire risposte brevi proporzionate;
- progettare un elaborato tecnico verificabile;
- esporre un nucleo ICT in modo ordinato;
- risolvere un caso separando fatti, ipotesi, evidenze e decisioni;
- correggere una simulazione con rubriche e Diario degli errori.

Il capitolo applica al settore ICT il metodo generale sulle prove del VOL-01. Per la teoria completa usa i capitoli 14-17 di Il Metodo BANDO; per il Diario degli errori, il capitolo 23. Questo capitolo riguarda il delta specialistico: la produzione di una prestazione tecnica.

MAPPA BANDO DELLA PROVA ICT

B — BANDO

Raccogli in una pagina le regole effettive:

Campo	Dato da estrarre
Formato	quiz, domande aperte, elaborato, caso, orale o combinazione
Materie	formule esatte del programma
Durata	tempo comunicato
Valutazione	punteggio, soglia, penalità e criteri pubblicati
Strumenti	materiali e ausili ammessi
Consegna	lunghezza, formato e modalità di risposta
Aggiornamenti	avvisi e istruzioni successive

Non trasferire numeri da un altro concorso. Una penalità diversa incide sulla scelta di rispondere a un quesito incerto; uno spazio diverso impone un altro livello di approfondimento.

A — AREE

Raggruppa il programma in sviluppo, dati, reti e sistemi, cloud, cybersecurity, governo digitale, AI e procurement ICT. Una traccia può attraversare più aree: un disservizio applicativo può coinvolgere DNS, database, logging, continuità e fornitore.

N — NUCLEI

Scomponi ogni area in unità verificabili. «Cybersecurity» non è un nucleo: rischio, vulnerabilità, controllo, IAM, logging e incident response lo sono.

D — DIARIO

Classifica la causa dell'errore: conoscenza, distinzione, lettura, calcolo, assunzione non dichiarata, soluzione non verificabile, lessico o tempo. Ogni errore deve produrre un'azione e una nuova verifica.

O — OUTPUT

Associa a ogni nucleo almeno un prodotto: scelta motivata, definizione, confronto, query, pseudocodice, calcolo, schema, diagnosi, progetto o esposizione orale.

LEGGERE UNA CONSEGNA TECNICA

Prima di rispondere, separa:

- 1. verbo operativo: definire, distinguere, progettare, diagnosticare, calcolare;
- 2. oggetto: concetto o sistema;
- 3. contesto: ente, servizio, utenti e dati;
- 4. vincoli: tempo, spazio, continuità, sicurezza e risorse;
- 5. output: testo, query, schema, piano o soluzione.

«Illustrare il DNS» e «diagnosticare un servizio raggiungibile per IP ma non per nome» non sono la stessa richiesta. Nel primo caso serve una spiegazione di funzione e risoluzione; nel secondo una catena di ipotesi ed evidenze.

CANVAS DELLA TRACCIA TECNICA

Domanda	Nota
Che cosa devo produrre?	
Quali fatti sono dichiarati?	
Quali dati mancano?	
Quali assunzioni devo esplicitare?	
Quali alternative sono plausibili?	
Quale rischio devo controllare?	
Quale evidenza dimostra l'esito?	

Errore tipico. Rispondere alla materia, non alla domanda. Una pagina corretta sul cloud resta fuori traccia se la consegna chiede un piano di migrazione con continuità e portabilità.

PREPARARE IL DOSSIER DI PROVA

La prima lettura non serve a scrivere: serve a evitare di costruire una risposta su una consegna immaginata. Trascrivi in un foglio separato il verbo della traccia, il risultato atteso e i vincoli che modificano la scelta tecnica. Poi distingui i dati già presenti dalle informazioni mancanti. Se il caso non dichiara numero di utenti, classificazione dei dati o finestra di indisponibilità, non trasformare il silenzio in un fatto: annota l'assunzione e il modo in cui la controlleresti.

Distribuisci il tempo in blocchi adattati alla durata effettiva: lettura e piano, sviluppo, controllo e consegna. Non esiste una ripartizione valida per tutte le procedure; il punto è riservare un margine per verificare che ogni decisione risponda alla domanda. Una risposta terminata senza controllo può contenere una buona soluzione per un problema diverso.

Errore da registrare. Cominciare dal diagramma o dal prodotto preferito prima di avere definito il bisogno. La correzione non è aggiungere tecnologia: è tornare a verbo, oggetto, contesto, vincoli e output.

13.2 Quiz tecnico: decisione, distrattori e correzione

PRIMO GIRO

Classifica il quesito come diretto, di distinzione, di calcolo, di scenario, incerto o bloccante. La decisione se rispondere a un quesito incerto dipende dalle regole effettive, non da formule universali.

Per smontare un distrattore chiedi se il termine indica davvero quella funzione, se la proprietà è necessaria o soltanto utile, quale obiettivo di sicurezza è coinvolto e se l'affermazione vale sempre o soltanto a certe condizioni.

ESEMPIO RAGIONATO

Quale affermazione distingue correttamente autenticazione e autorizzazione?

- A. L'autenticazione assegna i permessi; l'autorizzazione verifica l'identità.
- B. L'autenticazione verifica un'identità dichiarata; l'autorizzazione determina le azioni consentite.
- C. Entrambe cifrano le credenziali.
- D. L'autorizzazione precede sempre l'identificazione.

Soluzione: B. A inverte le funzioni; C attribuisce una funzione che non definisce i concetti; D introduce una sequenza assoluta errata. Vedi capitolo 9.

MICRO-QUIZ

1. GROUP BY serve principalmente a: A) cifrare colonne; B) raggruppare righe per aggregazioni; C) creare un indice; D) avviare una transazione. Risposta corretta: B. GROUP BY forma gruppi su cui applicare funzioni di aggregazione. A riguarda la protezione dei dati, C può migliorare l'accesso ma non raggruppa le righe, D governa una transazione.
2. Un backup dimostra da solo il rispetto del tempo di ripristino? A) sempre; B) se remoto; C) no, servono procedure, risorse e prove; D) se cifrato. Risposta corretta: C. Una copia non prova che il servizio torni operativo entro l'obiettivo: servono procedura, risorse disponibili e test di ripristino. La collocazione remota o la cifratura possono essere utili, ma non risolvono da sole il problema.
3. Il rischio residuo è: A) precedente ai controlli; B) rimanente dopo il trattamento considerato; C) una vulnerabilità corretta; D) l'impatto massimo. Risposta corretta: B. Il rischio residuo è quello che rimane dopo i controlli o il trattamento valutati. A descrive il rischio prima del trattamento, C confonde vulnerabilità e rischio, D riduce il rischio al solo impatto.
4. Quale evidenza verifica meglio il tempo di presa in carico? A) brochure; B) organigramma; C) timestamp del ticketing; D) numero di tecnici. Risposta corretta: C. I timestamp collegano apertura e presa in carico e consentono di misurare il tempo effettivo. Brochure, organigramma e numero di tecnici descrivono il contesto, ma non dimostrano l'esito della misura.

Correggi registrando quesito, esito, causa, regola correttiva e nuova prova. La lettera giusta, da sola, non spiega l'errore.

ALLENARE LA DECISIONE, NON LA SOLA MEMORIA

Dopo un primo giro, raccogli i quesiti incerti in tre gruppi: concetto non ricordato, distinzione letta male, calcolo o scenario da ricostruire. Questa classificazione indica un esercizio diverso. Per un concetto serve richiamo attivo; per una distinzione serve un confronto in due colonne; per uno scenario serve esplicitare fatti, ipotesi e conseguenza prima di scegliere.

Quando due alternative sembrano plausibili, chiedi quale risponde al verbo e quale introduce una condizione non dichiarata. Parole come "solo", "sempre", "automaticamente" e "garantisce" meritano un controllo in più: talvolta sono corrette, ma soltanto se il contesto le sostiene. Non cambiare una risposta corretta solo perché sembra troppo semplice; torna invece alla funzione tecnica che la domanda sta verificando.

Nel diario conserva anche la ragione del distrattore: confusione tra disponibilità e ripristino, tra identità e permessi, tra requisito e prova, tra attività svolta ed evidenza dell'esito. Alla simulazione successiva misura se la stessa causa ricompare. La correzione utile è una regola verificabile, non un generico "ripassare".

PROTOCOLLO DI CORREZIONE DEL QUIZ

Correggi il quiz in due tempi. Nel primo, segna soltanto corretto, errato o non risposto. Nel secondo, per ogni errore, ricostruisci il percorso: quale parola della domanda hai trascurato, quale regola tecnica hai applicato, quale alternativa ti ha attirato e quale evidenza avrebbe permesso di escluderla. Questo passaggio impedisce di attribuire tutti gli errori alla memoria quando il problema è invece la lettura della consegna.

Prepara piccole serie con un solo obiettivo: distinguere backup e ripristino, requisito e controllo, vulnerabilità e rischio, autenticazione e autorizzazione. Dopo ogni serie cambia il contesto ma conserva la distinzione. Se rispondi correttamente soltanto quando riconosci la formulazione, il nucleo non è ancora trasferibile in una prova nuova.

La scelta di lasciare una risposta incerta dipende da penalità e istruzioni ufficiali. Prima della prova allena la decisione su scenari dichiarati; durante la prova applica solo la regola ricavata dagli atti della procedura. Un consiglio generale non può sostituire questa verifica.

ULTIMO CONTROLLO PRIMA DELLA SCELTA

Prima di segnare la risposta, rileggi domanda e alternativa scelta come una coppia. Verifica soggetto, condizione, tempo verbale e relazione richiesta. Se l'alternativa risponde a una domanda leggermente diversa, è un distrattore anche quando contiene termini corretti. Questa pausa breve è più affidabile di una riletture indistinta dell'intero quesito.

13.3 Risposta breve e domanda aperta

La qualità di una risposta breve dipende dalla selezione. Il nucleo richiesto deve emergere attraverso un ordine riconoscibile.

Per una domanda definitoria o comparativa usa, quando pertinenti: definizione, funzione, differenza decisiva, conseguenza, esempio e chiusura.

RISPOSTA MODELLO

Traccia: distinguere autenticazione e autorizzazione, indicando una conseguenza operativa.

L'autenticazione verifica che un soggetto o un'identità tecnica corrisponda all'identità dichiarata. L'autorizzazione stabilisce invece quali risorse e operazioni siano consentite. I controlli sono collegati ma non intercambiabili: un utente autenticato può non essere autorizzato a modificare una risorsa. La progettazione deve quindi affiancare a credenziali robuste una gestione dei permessi coerente con minimo privilegio e separazione delle funzioni.

La risposta copre definizione, distinzione e conseguenza operativa, mantenendo fuori la parte di teoria IAM che la traccia non richiede.

REVISIONE 5P

- Pertinenza: ho risposto al verbo?
- Perimetro: ho inserito contenuti estranei?
- Precisione: termini e riferimenti sono sicuri?
- Proporzione: ho distribuito bene lo spazio?
- Punto finale: la chiusura torna alla domanda?

COSTRUIRE UNA RISPOSTA BREVE CONTROLLABILE

Prima di scrivere, prepara una micro-scaletta di quattro righe: tesi o definizione, elemento distintivo, conseguenza operativa, esempio o chiusura. La scaletta evita due errori opposti: una definizione da glossario che non risponde al caso e una risposta ricca di esempi senza nucleo concettuale. Se la domanda chiede di confrontare, mantieni gli stessi assi per entrambi gli oggetti; se chiede di spiegare, rendi esplicito il rapporto di causa o funzione.

Controlla le parole che possono cambiare il perimetro. "Descrivere" richiede ordine e precisione; "valutare" richiede criteri e motivazione; "proporre" richiede una scelta collegata a vincoli e rischi. Non attribuire al bando, al fornitore o a una tecnologia un effetto che la traccia non dichiara. Quando un dato manca, una frase di assunzione motivata vale più di un dettaglio inventato.

La revisione finale può seguire un percorso inverso: leggi prima l'ultima frase, poi verifica se i passaggi precedenti la sostengono e se il primo periodo risponde al verbo. Elimina le frasi decorative, conserva le distinzioni necessarie e chiudi con l'effetto concreto della scelta. In uno spazio limitato, la proporzione è parte della correttezza.

RISPONDERE A UNA DOMANDA APERTA CON VINCOLI

Quando la consegna fissa un limite di righe o caratteri, assegna spazio alla tesi prima di aggiungere dettagli. La prima frase deve rendere riconoscibile il nucleo. Le frasi successive possono spiegare funzione, differenza e conseguenza; l'esempio entra solo se chiarisce un passaggio che altrimenti resterebbe ambiguo. Evita l'apertura con formule vaghe come "è molto importante": non dicono quale relazione tecnica stai sostenendo.

Per allenarti, riscrivi la stessa risposta in tre versioni: una definizione essenziale, una risposta comparativa e una mini-applicazione a un ente. Confronta ciò che rimane invariato, ciò che dipende dal contesto e ciò che non puoi affermare senza dati aggiuntivi. La variazione fa emergere le frasi di repertorio che sembrano corrette ma non portano evidenza.

Se citi una misura, un obbligo o una soglia, assicurati che derivi dal testo fornito o da una fonte da verificare. In mancanza, descrivi il criterio di scelta e l'evidenza richiesta. Precisione non significa riempire la risposta di riferimenti: significa non attribuire certezza a ciò che è solo una possibilità.

CONTROLLO FINALE DELLA FORMULAZIONE

Prima di consegnare, sostituisci le parole generiche con il termine tecnico necessario e controlla che ogni pronome abbia un referente chiaro. Una risposta breve può essere rigorosa senza diventare telegrafica: il lettore deve poter vedere la distinzione e la conseguenza senza ricostruirle da solo.

ESERCIZIO DI SINTESI

Prendi una definizione studiata nei capitoli tecnici e scrivila prima in due frasi, poi in una risposta che includa una conseguenza per un servizio. Confronta le due versioni: la seconda deve aggiungere funzione o effetto, non soltanto più parole. Se non riesci a indicare la conseguenza, torna al nucleo teorico prima di ampliare il testo.

13.4 Elaborato tecnico: progetto, rischi ed evidenze

Un elaborato tecnico collega bisogno, vincoli, soluzione, rischi ed evidenze. Un elenco di tecnologie non svolge questa funzione.

Una scaletta robusta comprende obiettivo e perimetro; fatti e assunzioni; requisiti; opzioni; soluzione motivata; rischi e controlli; sequenza operativa; metriche, test ed evidenze; limiti e chiusura.

Se la traccia chiede una migrazione in cloud, il nome di un prodotto non è un buon punto di partenza. Chiarisci dati, utenti, dipendenze, carico, disponibilità, sicurezza, portabilità e continuità. Distingui assessment, preparazione, test, migrazione, cutover, rollback e verifica.

Decisione	Motivo	Rischio	Controllo
migrazione per fasi	ridurre l'impatto	doppia gestione	sincronizzazione
finestra di cutover	limitare il disservizio	ritardo	criterio di rollback

Decisione	Evidenza
migrazione per fasi	test e riconciliazione
finestra di cutover	log e verbale

Domanda-trappola. Più dettagli significano una risposta migliore? No. Un dettaglio serve quando è corretto, pertinente e collegato a una decisione verificabile.

DALLA RICHIESTA ALLA SOLUZIONE MOTIVATA

Un elaborato non deve dimostrare di conoscere tutte le tecnologie disponibili: deve rendere controllabile una scelta. Parti da servizio, utenti, dati, dipendenze e vincoli; separa ciò che la traccia dichiara da ciò che ipotizzi. Trasforma poi ogni requisito in una decisione osservabile. Per esempio, un requisito di continuità conduce a una sequenza di prevenzione, ripristino, rollback e prova; non si esaurisce nell'etichetta "alta disponibilità".

Per ogni opzione significativa indica il trade-off. Una soluzione più semplice può ridurre il tempo di esercizio ma aumentare una dipendenza; una soluzione più distribuita può migliorare la resilienza ma richiedere osservabilità e competenze ulteriori. Non serve quantificare valori non presenti nella traccia: spiega quali metriche, test o evidenze permetterebbero all'ente di decidere.

VERIFICARE LA COERENZA DELL'ELABORATO

Una buona scaletta può fallire se le sue parti non si parlano. Dopo la prima stesura, evidenzia i requisiti e cerca per ciascuno una scelta, un controllo e un test. Poi percorri la catena al contrario: ogni componente proposta deve avere una funzione rispetto a un requisito o a un rischio dichiarato. Se una tecnologia compare senza questo legame, può essere un dettaglio superfluo.

Distingui i livelli: esigenza dell'ente, requisito del servizio, decisione progettuale, attività operativa ed evidenza. Confondere questi piani produce risposte persuasive ma non verificabili. Per esempio, "proteggere i dati" è un obiettivo; la scelta di gestione degli accessi è una decisione; i log di riesame e di revoca possono essere evidenze. La traccia determina quali elementi approfondire.

Concludi indicando come la soluzione sarebbe sottoposta a test, accettazione e miglioramento. Non promettere risultati assoluti. Una proposta professionale mostra anche i confini: dati necessari, dipendenze da confermare, criteri per valutare l'esito e condizioni che richiederebbero una diversa scelta.

GESTIRE LE ASSUNZIONI

Le assunzioni utili sono poche, esplicite e collegate a una decisione. Scrivi che cosa manca, quale ipotesi adottata consente di proseguire e quale dato la renderebbe da rivedere. In questo modo l'elaborato rimane concreto senza trasformare l'incertezza in una falsa certezza.

REVISIONE PER SCENARI

Immagina che cambi un solo vincolo: più utenti, minore tolleranza al fermo, dati più sensibili o maggiore dipendenza da un soggetto esterno. Indica quale requisito verrebbe rivalutato e quale prova aggiungerei. L'esercizio serve a controllare che la proposta sia motivata dal contesto e non da una soluzione standard ripetuta.

TRACCIA DI CONTROLLO

Prima della consegna, sottolinea una frase per requisito e una per verifica. Se non trovi entrambe, integra solo il passaggio necessario: aggiungere pagine non sostituisce il collegamento tra scelta ed esito atteso.

ELABORATO MODELLO: PORTALE PER CONSULTARE PRATICHE

Assumo un ente con 100 operatori e accesso esterno dei soli interessati, un archivio di un milione di pratiche e un servizio di sola consultazione nella prima fase. Prima della gara confermo volumi, categorie di dati, integrazioni e competenze interne. Il risultato atteso è consultare stato e documenti autorizzati senza recarsi allo sportello, mantenendo un canale assistito per chi ne ha bisogno.

Separerei interfaccia, API applicativa e database, con ambienti di sviluppo, collaudo e produzione distinti. L'accesso esterno passa dal meccanismo di identificazione previsto; l'applicazione verifica poi l'autorizzazione sulla singola pratica. Un token valido non autorizza a leggere quella di un altro cittadino. Gli operatori ricevono privilegi coerenti con funzione e ufficio; le attività amministrative privilegiate sono tracciate. Nel modello dati, chiavi e vincoli impediscono riferimenti incoerenti, mentre minimizzazione e tempi di conservazione sono definiti con i responsabili competenti.

Per la consultazione propongo un obiettivo didattico: 95° percentile entro 800 ms a 100 utenti concorrenti nell'ambiente e nel carico definiti, con errori sotto lo 0,5%. Il collaudo comprende accesso consentito e negato, input errati, pratiche assenti, indisponibilità di una dipendenza e verifica di accessibilità. Testo anche ripristino di backup e coerenza dei dati: la presenza di un file di backup non dimostra che il servizio riparta. Disponibilità, RTO e RPO sono concordati dopo l'analisi d'impatto; non li ricavo da un'etichetta commerciale.

Confronterei riuso, codice aperto e altre opzioni secondo CAD 68–69, includendo adattamento, migrazione, formazione, gestione e uscita. Per il cloud verifico classificazione di dati/servizi e qualificazione pertinente del servizio concreto. Contratto e piano operativo attribuiscono gestione degli accessi, patch, log, incidenti, supporto e comunicazioni; conservano all'ente evidenze e capacità di verifica.

Il rilascio avviene dopo migrazione di prova, riconciliazione dei record, test accettati e decisione autorizzata. Un piano indica finestra, responsabili, soglie di stop e rollback compatibile con eventuali nuove scritture. In esercizio misuro successo delle consultazioni, errori, tempi, incidenti e richieste allo sportello: se il canale digitale sposta soltanto il lavoro sugli operatori, il risultato va riesaminato. Preparo fin dall'inizio esportazione documentata, prova di subentro e gestione delle credenziali alla fine del contratto.

Questo elaborato rende esplicite assunzioni, architettura, controlli, test e governo. Una soluzione diversa è valutabile se collega altre scelte agli stessi bisogni e produce evidenze verificabili.

13.5 Prova orale ICT: struttura e controllo

All'orale devi rendere udibile la struttura. Apri delimitando il concetto, sviluppa due o tre passaggi e chiudi sulla conseguenza.

Per confrontare due tecnologie usa quattro assi: funzione, oggetto gestito, vantaggio e limite. Alla domanda «container e macchina virtuale» non basta dire che il container è più leggero: distingui livello di virtualizzazione, isolamento, portabilità e conseguenze operative.

Se non ricordi un dettaglio, non inventarlo. Delimita ciò che sai e riparti dal principio sicuro. Non usare questa prudenza per evitare la domanda: ricostruisci un ragionamento pertinente.

RUBRICA ORALE

Criterio	0	1	2
Pertinenza	fuori domanda	parziale	centrata
Correttezza	errori decisivi	imprecisioni	corretta
Struttura	frammentaria	riconoscibile	chiara
Applicazione	assente	debole	concreta
Lessico	improprio	comprensibile	preciso
Durata	non gestita	quasi adeguata	proporzionata

È una rubrica di allenamento, non un criterio ufficiale.

RENDERE UDIBILE IL RAGIONAMENTO

All'orale la commissione deve poter seguire la struttura senza leggere una scaletta. Comincia con una frase che definisce il perimetro, sviluppa pochi passaggi collegati e chiudi con la conseguenza per servizio, dati, utenti o rischio. Una sequenza breve ma ordinata consente anche di gestire le domande di approfondimento: ogni passaggio lascia un punto da chiarire senza contraddirne un altro.

Esercitati su due durate compatibili con gli atti della procedura, non su una durata astratta. Registra una prova e ascolta dove perdi il filo: termini non definiti, esempi che arrivano prima della tesi, collegamenti causali mancanti. La rubrica del capitolo serve a rendere visibili questi aspetti durante l'allenamento; non predice la valutazione ufficiale.

Se il commissario cambia scenario, ripeti prima i fatti ricevuti e dichiara che cosa cambierebbe nella soluzione. Se non ricordi un dettaglio, separa il limite dal principio certo e non riempire il silenzio con riferimenti non verificati. La prudenza tecnica è utile quando permette di continuare con un ragionamento pertinente e falsificabile.

GESTIRE DOMANDE E CONTRODOMANDE

Dopo un'esposizione preparata, esercitati con una seconda persona o con una lista di domande che sposti il punto di vista: utente, responsabile del servizio, sicurezza, dato, fornitore, continuità. Non memorizzare una risposta aggiuntiva per ogni domanda; torna invece alla struttura di base e spiega quale fatto nuovo modifica priorità, rischio o verifica. Questa tecnica limita le risposte rigide.

Usa esempi brevi e dichiaratamente esemplificativi. Un esempio non deve simulare una prova reale né trasformarsi in una prescrizione universale: serve a rendere osservabile un concetto. Se introduci un dato quantitativo, chiarisci che è un dato della traccia o una variabile da definire, non una soglia applicabile a ogni ente.

Alla fine dell'esercitazione valuta non solo l'esattezza ma anche l'ascoltabilità: il destinatario capisce quale problema stai risolvendo, quale scelta proponi e come ne controlleresti l'esito? Se manca uno di questi tre punti, riduci l'elenco di nozioni e ricostruisci il filo logico.

CHIUSURA ORALE

Chiudi richiamando il criterio con cui controlleresti il risultato: disponibilità osservata, correttezza dell'accesso, evidenza di ripristino o coerenza con il requisito. Una chiusura di questo tipo mostra che la spiegazione non è soltanto descrittiva, ma orientata a una decisione verificabile.

ALLENAMENTO A VOCE

Esponi una risposta, fermati dopo ogni passaggio e formula la domanda che un ascoltatore potrebbe porre. Rispondi collegando il chiarimento alla struttura iniziale, non aggiungendo un elenco autonomo. Se una controdomanda mette in luce un'assunzione, dichiarala e spiega quale informazione permetterebbe di confermarla. Questo rende l'esposizione più solida anche quando la conversazione cambia direzione.

PRECISIONE SENZA ECCESSO

Una risposta orale efficace non accumula definizioni. Scegli il concetto che governa la domanda, esplicita il nesso principale e usa un esempio solo per rendere osservabile la conseguenza. L'ascoltatore deve distinguere ciò che sai con certezza da ciò che richiederebbe un dato ulteriore.

La struttura rende l'esposizione controllabile anche sotto pressione.

13.6 Caso tecnico: diagnosi, decisione e verifica

Per una diagnosi usa sintomo → ipotesi → evidenza → test → esito → azione. Per un progetto usa bisogno → requisiti → opzioni → trade-off → scelta → test. Per un incidente usa rilevazione → triage → contenimento → recupero → verifica → lezione appresa. Per un fornitore usa obbligo → indicatore → evidenza → scostamento → escalation → rimedio.

La soluzione deve indicare come sarà verificata. «Alta disponibilità», se mancano architettura, metrica e prova, resta un'etichetta.

CASO GUIDATO: SERVIZIO DIGITALE INTERMITTENTE

TRACCIA

Un ente gestisce un portale per le istanze. Da tre giorni alcuni utenti segnalano lentezza e brevi indisponibilità. Il servizio è ospitato in cloud e gestito da un fornitore. Il monitoraggio registra picchi di latenza senza indicare una causa. Imposta diagnosi, contenimento, ripristino e verifica, considerando sicurezza e continuità.

DELIMITARE

Sappiamo che il problema è intermittente e che esistono segnalazioni e picchi. Non sappiamo se la causa sia applicativa, database, rete, configurazione, dipendenza esterna o sicurezza.

Chiedi quali utenti e funzioni sono colpiti; orari, durata e frequenza; modifiche recenti; dipendenze esterne; metriche e log; soglie e fonti dello SLA.

FORMULARE E VERIFICARE IPOTESI

Le ipotesi comprendono saturazione, query lente, lock, dipendenza esterna, rilascio recente, problema DNS, scalabilità inefficace o traffico anomalo. Ordinale per probabilità, impatto e costo della verifica. Associa a ciascuna un'evidenza attesa.

Correla timestamp delle segnalazioni con metriche, log applicativi, database, rete e piattaforma. Verifica se il problema interessa tutte le funzioni o un'operazione. Se emergono indicatori di incidente, attiva il triage e preserva le evidenze.

CONTENERE E RIPRISTINARE

Le azioni possono includere rollback, riallocazione controllata di risorse, isolamento di una componente o modalità degradata. Ogni intervento deve avere responsabile, rischio, criterio di avvio, criterio di rollback e metrica di verifica.

Il fornitore va coinvolto secondo ruoli ed escalation. L'ente non trasferisce al fornitore la responsabilità di governare il servizio.

VERIFICARE E CHIUDERE

Confronta le metriche con la baseline, prova le funzioni critiche e osserva il servizio per un periodo adeguato. Registra causa, impatto, decisioni, evidenze, azioni correttive e prevenzione.

Elencare strumenti non basta. La risposta deve costruire ipotesi e test, poi collegarli a evidenze e priorità. Sicurezza, continuità, rapporto con il fornitore, rollback e verifica finale completano il ragionamento.

USARE LE EVIDENZE PER SCEGLIERE

Nel caso tecnico, una lista di possibili strumenti non è una diagnosi. Scrivi prima il sintomo osservato, gli utenti o le funzioni coinvolte e le evidenze disponibili. Formula quindi ipotesi concorrenti: applicazione, dati, rete, configurazione, dipendenza esterna, rilascio o sicurezza. A ogni ipotesi associa il test meno invasivo che può confermarla o ridurre la probabilità, e indica chi può svolgerlo.

Le azioni di contenimento devono essere reversibili quando il contesto lo consente. Prima di intervenire, chiarisci effetto atteso, rischio dell'intervento, criterio di rollback e misura di verifica. Dopo il ripristino, non confondere il ritorno apparente del servizio con la chiusura: confronta metriche e funzioni critiche con una baseline, conserva le evidenze e registra il rischio residuo.

Questo schema vale anche quando il problema coinvolge un fornitore. Il fornitore può fornire dati, eseguire attività e partecipare all'escalation; l'ente deve comunque mantenere la capacità di governare priorità, impatto, comunicazione e accettazione dell'esito.

SCRIVERE IL CASO COME UNA SEQUENZA DI DECISIONI

Una risposta al caso guadagna chiarezza quando separa osservazione e interpretazione. "Il portale è lento" è un sintomo; "il database è saturo" è un'ipotesi da provare. Tenere distinte le due frasi evita di scegliere subito un rimedio e rende possibile spiegare perché un test è prioritario. Anche l'assenza di evidenza è un dato: segnala che la causa non è ancora confermata.

Organizza la consegna con blocchi visibili: perimetro e impatto, ipotesi e prove, contenimento, ripristino, verifica, prevenzione. Se devi scegliere tra azioni, motiva la priorità con impatto, reversibilità, sicurezza e continuità. Se il caso riguarda dati personali o funzioni essenziali, segnala che servono ruoli e procedure coerenti con il contesto, senza inventare l'esito di valutazioni non fornite.

CASO AUTONOMO: INDISPONIBILITÀ SELETTIVA DOPO UN RILASCIO

Traccia. Dopo un rilascio serale, un servizio per le istanze funziona per gli utenti interni ma restituisce errori intermittenti a una parte degli utenti esterni. Il monitoraggio mostra un aumento dei codici di errore, ma non identifica la componente responsabile. Sono disponibili log applicativi e di accesso; il fornitore gestisce una dipendenza esterna. Devi proporre la gestione iniziale del caso senza assumere cause non documentate.

Output atteso. Produci una pagina o uno schema con: perimetro e impatto; fatti e assunzioni; almeno tre ipotesi con l'evidenza o il test che le distingue; misura di contenimento proporzionata; criterio di rollback; verifica del ripristino; informazioni da richiedere al fornitore e registrazione dell'esito.

Tempo e criteri di autocorrezione. Usa il tempo indicato dagli atti della procedura; se ti alleni senza un limite pubblicato, definiscilo prima e registralo nel foglio di esito. Assegna 0, 1 o 2 a pertinenza alla traccia, separazione tra fatti e ipotesi, qualità dei test, sicurezza e continuità, motivazione della priorità, rollback ed evidenze finali. La griglia serve a confrontare le simulazioni, non sostituisce i criteri della commissione.

SOLUZIONE COMMENTATA DEL CASO AUTONOMO

La correlazione con il rilascio è un indizio, non una causa dimostrata. Delimito orari, versioni, percentuale di errori e differenza fra canale interno ed esterno usando identificativi di correlazione, senza esportare dati personali non necessari.

Ipotesi	Evidenza da cercare	Esito che cambia la decisione
Configurazione del proxy o timeout diverso all'esterno	Confronto configurazioni/versioni, tempi a ogni tratto e codici gateway	Errore nasce al proxy prima della risposta applicativa
Nuova verifica del token incompatibile con alcuni accessi	Log di validazione, scadenza/audience e prova con account di test autorizzati	Fallisce una classe definita di token, non tutte le richieste
Dipendenza esterna lenta o limite di richieste	Latenza e stato del fornitore, correlazione con timeout e carico	Errori seguono saturazione o limiti della dipendenza

Contengo il problema limitando la funzione coinvolta o attivando il canale alternativo previsto; non disabilito l'autorizzazione per far sparire gli errori. Coinvolgo responsabile del servizio e fornitore con orari, versioni e richieste riproducibili. Decido il rollback solo se versione precedente e dati sono compatibili, gli effetti sono reversibili e la procedura autorizzata è pronta. Se vi è sospetto di incidente di sicurezza, attivo il percorso dedicato preservando evidenze e valutando notifiche secondo il regime applicabile.

Dopo il ripristino confronto errori e latenza su entrambi i canali, provo permessi positivi e negativi, riconcilio operazioni rimaste pendenti e osservo una finestra significativa. Registro causa accertata o ancora da confermare, correzione, responsabile e prevenzione. La rubrica premia la sequenza ipotesi–evidenza–decisione: non richiede di indovinare una delle tre cause con i soli dati della traccia.

13.7 Simulazione, autocorrezione e diario degli errori

Assegna 0, 1 o 2 a pertinenza, perimetro, teoria, applicazione, alternative, rischi, evidenze, struttura, lessico e chiusura. Il totale permette di confrontare le tue simulazioni. Non può prevedere il voto della commissione.

Correggi in quattro passaggi: contenuto falso o mancante; metodo saltato; tempo distribuito male; forma che ostacola la valutazione.

DIARIO DEGLI ERRORI ICT

Data	Prova	Nucleo	Errore
Data	Causa	Regola correttiva	Esercizio
Data	Nuova verifica		

«Ripassare reti» è vago. «Confondo DNS e raggiungibilità IP: svolgere tre diagnosi sintomo-ipotesi-test entro venerdì» è controllabile.

- conoscenza → teoria e richiamo attivo;
- distinzione → confronto e domande-trappola;
- applicazione → caso graduato;
- lettura → canvas della traccia;
- tempo → micro-simulazione;
- forma → riscrittura con rubrica.

SIMULAZIONE MISTA FINALE

Adatta quantità e tempo al tuo bando: questo laboratorio non replica una procedura.

QUIZ

Svolgi prima i quattro micro-quiz del capitolo, poi completa la batteria con questi otto quesiti. I dodici quesiti coprono definizioni, distinzioni, conseguenze e scenari.

5. In una ricerca lineare su una lista non ordinata di n elementi, il numero di confronti nel caso peggiore cresce: A) in modo costante; B) in modo logaritmico; C) linearmente con n ; D) indipendentemente da n .
6. Quale vincolo impedisce che una chiave esterna punti a una riga inesistente? A) integrità referenziale; B) disponibilità; C) segregazione; D) indicizzazione.
7. Un client raggiunge un server tramite indirizzo IP, ma non tramite nome. Quale controllo è più pertinente come primo approfondimento? A) algoritmo di ordinamento; B) risoluzione DNS; C) normalizzazione del database; D) modello di autorizzazione.
8. Un requisito stabilisce che l'API restituisca sempre un identificatore della richiesta. Si tratta soprattutto di: A) requisito funzionale verificabile; B) penalità contrattuale; C) criterio di normalizzazione; D) misura di rischio residuo.

9. Nel modello di responsabilità condivisa del cloud: A) il provider assume ogni responsabilità; B) il cliente assume ogni responsabilità; C) la ripartizione dipende dal servizio e dal contratto; D) la sicurezza non riguarda la configurazione.
10. Una vulnerabilità critica è stata corretta su un server. Quale evidenza sostiene meglio la chiusura? A) l'intenzione di applicare la patch; B) il ticket aperto; C) la verifica della versione e un nuovo controllo tecnico; D) la data della segnalazione.
11. Un log utile alla ricostruzione di un incidente deve permettere almeno di collegare: A) evento, tempo e identità o componente; B) colore, marca e costo; C) organigramma e ferie; D) solo il testo libero dell'operatore.
12. Un dataset contiene valori formalmente validi ma riferiti a soggetti sbagliati. La dimensione di qualità più direttamente compromessa è: A) accuratezza; B) portabilità; C) disponibilità; D) compressione.

SOLUZIONI RAGIONATE DEI QUIZ 5-12

5. C. Nel caso peggiore la ricerca lineare può esaminare tutti gli elementi; il lavoro cresce con n. Rinvio: capitolo 3.
6. A. L'integrità referenziale mantiene coerenti i riferimenti tra tabelle. Un indice può accelerare l'accesso, ma non definisce da solo il vincolo. Rinvio: capitolo 4.
7. B. La raggiungibilità IP riduce la probabilità di un guasto generale di connettività; la differenza introdotta dal nome orienta la diagnosi verso il DNS, senza escludere verifiche successive. Rinvio: capitolo 5.
8. A. Il requisito descrive un comportamento osservabile dell'API e può essere verificato tramite test. Non è, da solo, una penalità o una misura del rischio. Rinvio: capitolo 6.
9. C. La ripartizione cambia con IaaS, PaaS, SaaS e con gli accordi applicabili. Il cliente mantiene le responsabilità sulle attività che gli competono. Rinvio: capitolo 7.
10. C. L'attività dichiarata non prova l'esito. La versione rilevata e un nuovo controllo forniscono evidenze della correzione. Rinvio: capitolo 8.
11. A. Timeline, soggetto o componente ed evento consentono correlazione e ricostruzione. Il solo testo libero rende più fragile l'analisi. Rinvio: capitolo 9.
12. A. Il dato non rappresenta correttamente il soggetto a cui è associato. Il problema riguarda l'accuratezza, anche se il formato è valido. Rinvio: capitolo 10.

RISPOSTE BREVI

1. Distingui processo e thread, indicando una conseguenza sulla gestione delle risorse.
2. Spiega perché portabilità ed exit strategy vanno considerate prima della scadenza.

RISPOSTE MODELLO

1. Un processo è un programma in esecuzione con uno spazio di indirizzamento e risorse proprie; un thread è un flusso di esecuzione interno al processo e condivide con gli altri thread parte delle risorse. La condivisione rende la comunicazione più diretta, ma richiede sincronizzazione per evitare accessi concorrenti incoerenti. La distinzione incide quindi su isolamento, consumo di risorse e gestione della concorrenza.
2. Portabilità ed exit strategy devono essere progettate all'inizio perché dipendono da formati, API, documentazione, licenze, configurazioni, competenze e obblighi del fornitore. Se l'ente le affronta soltanto alla scadenza, può scoprire dipendenze che rendono costosi o rischiosi il trasferimento e la continuità del servizio. Il contratto e il governo dell'esecuzione devono quindi prevedere esportazione, assistenza alla transizione, verifica dei dati e prova periodica dell'uscita.

ELABORATO

Un ente vuole rendere disponibile un servizio a utenti interni ed esterni. Proponi un'impostazione che consideri requisiti, architettura, dati, identità, disponibilità, monitoraggio, continuità e fornitore.

La scaletta deve coprire perimetro, assunzioni, requisiti, opzioni, soluzione, accessi, dati, osservabilità, rischi, continuità, SLA, evidenze e test. Non esiste una sola architettura modello.

SCALETTA-SOLUZIONE COMMENTATA

1. Perimetro e assunzioni. Identificare utenti interni ed esterni, funzioni critiche, dati, carico, dipendenze e vincoli. Dichiarare ciò che la traccia non specifica.
2. Requisiti. Separare funzioni del servizio da disponibilità, prestazioni, accessibilità, interoperabilità, sicurezza, portabilità e manutenibilità.
3. Architettura. Proporre componenti applicative, dati, API, identità e infrastruttura coerenti con i requisiti. Motivare le alternative senza vincolarsi a un marchio.
4. Identità e dati. Definire autenticazione, autorizzazione, minimo privilegio, protezione, qualità, conservazione e responsabilità sul ciclo di vita.
5. Osservabilità e sicurezza. Collegare metriche, log, alert, vulnerabilità e risposta agli incidenti a ruoli ed evidenze.
6. Continuità. Definire backup, ripristino, rollback e prove coerenti con gli obiettivi dell'ente, senza inventare valori assenti dalla traccia.
7. Fornitore. Tradurre requisiti e livelli di servizio in indicatori, fonti di misura, reporting, escalation, portabilità ed exit strategy.
8. Verifica. Concludere con test funzionali, prestazionali, di sicurezza, ripristino e accettazione.

La rubrica verifica pertinenza, coerenza tra requisiti e architettura, gestione dei rischi, motivazione delle scelte e qualità delle evidenze.

ORALE

1. Che cosa distingue vulnerabilità e rischio?
2. Come colleghi logging e incident response?
3. Come valuti un sistema di AI acquistato da un fornitore?

Prepara una versione di due minuti e una di quattro. Registra dove perdi struttura.

NUCLEI ATTESI E RISPOSTE ESSENZIALI

1. Vulnerabilità e rischio. La vulnerabilità è una debolezza che può essere sfruttata; il rischio riguarda uno scenario in cui una minaccia sfrutta quella debolezza producendo conseguenze sugli asset o sui servizi. La gravità della vulnerabilità contribuisce alla valutazione, ma non coincide automaticamente con il rischio: contano esposizione, probabilità, impatto e controlli.
2. Logging e incident response. Il logging produce eventi utili a rilevazione, triage e ricostruzione. Servono sorgenti pertinenti, timestamp coerenti, protezione dei log, accessi controllati e regole di correlazione. I log alimentano la decisione; non sostituiscono procedure, ruoli e azioni di contenimento o recupero.
3. Sistema di AI acquistato. La valutazione parte da scopo, dati, prestazioni attese e impatto sul processo. Considera metriche adatte, errori, bias, spiegabilità quando pertinente, controllo umano, sicurezza, versioni, monitoraggio e responsabilità del fornitore. Requisiti, test, documentazione ed evidenze devono permettere all'ente di controllare il sistema durante l'esecuzione e dopo le modifiche.

Per la simulazione autonoma, usa la traccia, la rubrica e la soluzione commentata del caso autonomo nel nucleo «Caso tecnico: diagnosi, decisione e verifica».

PROVA PRATICA AGGIUNTIVA: DATI, ALGORITMO E METRICA

Tempo didattico: 25 minuti. Tabella Ticket(id, ufficio, minuti, chiuso) con righe (1,A,20,true), (2,A,40,true), (3,B,30,true), (4,B,90,false). Scrivi una query con numero e tempo medio dei soli ticket chiusi per ufficio; indica output. Poi descrivi un algoritmo per il massimo tempo dei ticket chiusi e la sua complessità. Infine interpreta un monitor che segnala 8 anomalie vere, 2 false e ne perde 4.

Soluzione SQL: `SELECT ufficio, COUNT(*) AS numero, AVG(minuti) AS media FROM Ticket WHERE chiuso = TRUE GROUP BY ufficio ORDER BY ufficio;` Output: A, 2, 30; B, 1, 30. La riga 4 è esclusa prima dell'aggregazione. Un ufficio senza chiusure non compare: se la traccia li richiedesse tutti, servirebbe l'anagrafica degli uffici con join e gestione esplicita dei null, senza spostare impropriamente il filtro in WHERE.

Algoritmo: inizializza trovato=false; scorri le righe; per ogni ticket chiuso, se è il primo o supera il massimo, aggiorna massimo e trovato. Restituisci «nessun ticket chiuso» se trovato rimane falso. Il risultato è 40. Per n righe il tempo è $\Theta(n)$, spazio ausiliario $\Theta(1)$; non ordinare tutto per ottenere un solo massimo. Metriche: $\text{precision} = 8/(8+2) = 80\%$; $\text{recall} = 8/(8+4) \approx 66,67\%$; $F1 = 16/(16+2+4) \approx 72,73\%$. Senza il numero di veri negativi non è calcolabile l'accuracy.

Assegna 2 punti a filtro/gruppi corretti, 2 all'output, 2 all'algoritmo e caso vuoto, 2 alla complessità dichiarata e 2 alle metriche con limite sull'accuracy. Ripeti correggendo l'errore specifico, senza memorizzare solo i numeri.

FOGLIO DI ESITO

Parte	Esito	Tempo	Errore dominante
Quiz			
Risposte			
Elaborato			
Orale			
Caso guidato			
Caso autonomo			

Parte	Azione	Scadenza
Quiz		
Risposte		
Elaborato		
Orale		
Caso guidato		
Caso autonomo		

Una simulazione termina quando produce una decisione di studio.

Verifica

QUIZ 1

Un servizio è raggiungibile per indirizzo IP ma non per nome. Quale verifica viene prima?

Risposta corretta: la risoluzione DNS, senza escludere controlli successivi sulla catena del servizio.

QUIZ 2

In una risposta tecnica compare l'avverbio "sempre". Che cosa va controllato?

Risposta corretta: che la regola valga nelle condizioni dichiarate; le assolutizzazioni non motivate sono un segnale di distrattore.

QUIZ 3

Una domanda aperta chiede un confronto. Quale struttura è proporzionata?

Risposta corretta: definizione dei due oggetti, asse di distinzione, conseguenza operativa ed esempio essenziale.

QUIZ 4

La traccia non indica il numero di utenti. Come si procede?

Risposta corretta: si dichiara l'assunzione, la si collega ai requisiti e si spiega quale evidenza la confermerebbe o la cambierebbe.

QUIZ 5

All'orale non ricordi un dettaglio normativo. Che cosa fai?

Risposta corretta: delimiti il punto non certo e sviluppi il principio tecnico sicuro, senza inventare dati.

QUIZ 6

Un intervento di recupero è stato eseguito ma non produce misure. La chiusura è completa?

Risposta corretta: no; occorrono una verifica rispetto alla baseline, evidenze registrate e la valutazione del rischio residuo.

DOMANDA DA COMMISSARIO

Come imposterebbe la diagnosi e il ripristino di un servizio intermittente, garantendo sicurezza e continuità?

Delimiterei servizio, utenti e impatto; raccoglierei timeline, modifiche, metriche e log; formulerei ipotesi su applicazione, dati, rete, piattaforma e sicurezza; assocerei ogni ipotesi a un test. Darei priorità in base a impatto e reversibilità, procedendo con contenimento e ripristino assistiti da un criterio di rollback. Coordinerei fornitore, ruoli tecnici e comunicazione. Infine verificherei il servizio rispetto a baseline e livelli concordati, registrando causa, rischio residuo e azioni preventive.

DOMANDE-TRAPPOLA

Una risposta lunga è necessariamente completa? No: può essere fuori perimetro.

Posso assumere ciò che la traccia non dice? Sì, se l'assunzione è ragionevole, dichiarata e trattata come variabile.

Il backup garantisce continuità? No: servono ripristino, procedure, risorse, tempi e prove.

La rubrica coincide con i criteri della commissione? No: è uno strumento editoriale.

Il punteggio dimostra che il nucleo è acquisito? È un indizio, da leggere insieme alla difficoltà della prova, alle condizioni di esecuzione e agli errori commessi.

MINI-ESERCIZI

LEGGI LA TRACCIA

«Confrontare database relazionali e documentali in relazione a schema, interrogazione e coerenza.»

Soluzione: il verbo è confrontare; gli assi sono schema, interrogazione e coerenza. Un tutorial SQL è fuori perimetro.

DICHIARA L'ASSUNZIONE

Un caso parla di «servizio critico» senza indicare RTO.

Soluzione: non inventare il valore. Dichiarare che la soluzione dipende dall'obiettivo definito dall'ente e spiega come ricavarlo e verificarlo.

TRASFORMA LA NOZIONE

Rendi verificabile il minimo privilegio.

Soluzione possibile: definire ruoli, assegnare accessi necessari, approvare eccezioni, riesaminare privilegi e verificare tramite account, log e revoche.

CHECKLIST OPERATIVA

Prima della prova:

- ☐ ho verificato bando, avvisi e istruzioni;
- ☐ conosco formato, durata, punteggio e strumenti;
- ☐ ho associato materie, nuclei e output;
- ☐ ho simulato i formati previsti;
- ☐ so dove recuperare gli errori.

Prima di consegnare:

- ☐ ho risposto al verbo;
- ☐ ho distinto fatti e assunzioni;
- ☐ ho motivato le decisioni;
- ☐ ho indicato rischi, controlli ed evidenze;
- ☐ ho eliminato riferimenti insicuri;
- ☐ ho chiuso sulla consegna.

DA SAPERE IN 5 RIGHE

La prova ICT valuta conoscenza e capacità di produrre un output. Formato, tempo e criteri derivano dalla procedura concreta. Una soluzione forte dichiara assunzioni, rischi ed evidenze. Le rubriche allenano, ma non sostituiscono la commissione. Ogni simulazione deve terminare con una decisione di recupero.

Riferimenti per l'approfondimento

- Bando, avvisi, istruzioni e criteri della procedura a cui partecipi: sono la fonte per formato, durata, strumenti e valutazione.
- D.P.R. 9 maggio 1994, n. 487, per la disciplina generale delle procedure concorsuali, da leggere insieme agli atti della singola selezione.
- Materiali di metodo sulle prove a quiz, scritte, orali, sui casi e sul diario degli errori: usati per allenare il processo, non per sostituire le regole della procedura.
- Capitoli tecnici 2-12 del volume: usati per consolidare i contenuti da trasformare in risposta, diagnosi o proposta progettuale.

Avvertenze operative

Quantità delle prove, tempi, punteggi, soglie, penalità e strumenti ammessi dipendono dagli atti pubblicati per la procedura concreta. Le rubriche, i canvas e le simulazioni del capitolo sono strumenti di allenamento: non equivalgono ai criteri della commissione.